

The Machine (Full Book 1)

August 28, 2026

THE MACHINE

Technology, शून्यापासून: एका माणसासाठी

हे Book 1. Machine म्हणजे नक्की काय: switch पासून cloud पर्यंत. पुढे Book 2 (The Thinking Machine): याच machine वर चालणारा AI.

अनुक्रम

PART 1 पैसा आणि MACHINE

- 0.1 गाव: पैसा exist का करतो
- 0.2 तास सारखे, कमाई वेगळी
- 0.3 समाजाचे चार levels
- 0.4 Technology चा मजला दिसत का ना...
- 0.5 मग आता आपल्याला काय उघडावं ला...
- 1.1 एक switch दाबल्यावर काय होतं
- 1.2 मी हे गृहीत धरून चाललोय
- 1.3 Computer खरं आहे तरी काय
- 1.4 Switches आकडे कसे ठेवतात
- 1.5 Switches ठरवतात कसे
- 1.6 एकच machine सगळं का करू शकते
- 2.1 एक गोष्ट किती जागा घेते
- 2.2 Text, photo, आवाज, video: सगळं...
- 2.3 मोठ्या आकड्यांचा अर्थ
- 2.4 एक instruction चालते कशी
- 2.5 लक्षात ठेवणं आणि विसरणं

PART 2 MACHINE शी बोलणं

- 3.1 भाषांतराची problem
- 3.2 जवळजवळ-English मध्ये लिहिणं
- 3.3 भाषांतरकाराचे दोन प्रकार
- 3.4 इतक्या साऱ्या languages का आह...
- 3.5 तो manager जो machine वाटतो
- 3.6 "चालू आहे" चा अर्थ काय
- 3.7 फुकट गोष्टी exist का करतात
- 4.1 Software मध्ये bugs असतातच का
- 4.2 चूक आधी शोधणं
- 4.3 प्रत्येक बदलाची नोंद
- 4.4 "माझ्या machine वर तर चालत हो...
- 4.5 Estimate नेहमी चुकीचा का असतो

PART 3 MACHINES चं जाळं

- 5.1 एक machine पुरी पडत नाही
- 5.2 प्रत्येक machine ला एक address
- 5.3 Message चे तुकडे करणं
- 5.4 नियमांवर आधीच सहमती
- 5.5 Numbers ऐवजी नावं
- 5.6 Public रस्त्यावर private बोलणं
- 5.7 Internet म्हणजे नक्की काय
- 5.8 Internet चे नियम कोण बनवतं

PART 4 DATA आणि आठवणी

- 6.1 विसरण्याची problem
- 6.2 10 million मधून एक शोधणं
- 6.3 आठवणींचे वेगवेगळे आकार
- 6.4 दोघं एकाच क्षणी लिहितात तेव्हा
- 6.5 कामाला रांगेत लावणं
- 6.6 जे पुन्हा पुन्हा लागतं ते जवळ...
- 6.7 Data हरवला तर?

PART 5 जगाला SERVE करणं

- 7.1 एका user पासून लाखांपर्यंत
- 7.2 कधीही बंद न होणारी machine
- 7.3 दिसणारा भाग, न दिसणारा भाग
- 7.4 दोन programs आपसात बोलतात कसे
- 7.5 विकत घेण्याऐवजी भाड्याने
- 7.6 सगळे एकदम आले तर काय तुटतं
- 7.7 गोष्टी हळू का वाटतात
- 7.8 हे सगळं चालवायचा खर्च किती
- 7.9 Part 0 चा दावा, तपासलेला
- 8.1 एक खरं product उघडून बघणं
- 8.2 काय बनवणं सोपं, काय अवघड
- 8.3 Spec कसा दिसतो
- 8.4 खऱ्या आयुष्यात गोष्टी कशा तुट...
- 8.5 एक technology जिंकते, दुसरी म...
- 8.6 Book 1 ची परीक्षा

सूचना: Part 2 पासून प्रत्येक भाग मागच्याच्या उजळणी-परीक्षेने सुरू होतो; एका बैठकीत वाचायचं असेल तर त्या भराभर ओलांडा.

THE MACHINE

PART 1 OF 5: पैसा आणि MACHINE

हे पुस्तक पाच छोट्या पुस्तकांमध्ये आहे. हे पहिलं.

पाच PARTS चा नकाशा

[तुम्ही इथे आहात]

PART 1	पैसा आणि MACHINE	पैसा का आहे, computer म्हणजे काय, सगळं numbers कसं आहे
PART 2	MACHINE शी बोलणं	languages, operating system, software बनतं कसं
PART 3	MACHINES चं जाळं	network, address, internet
PART 4	DATA आणि आठवणी	storage, database, शोधणं
PART 5	जगाला SERVE करणं	scale, cloud, खर्च, पूर्ण जोड

हे पुस्तक कशासाठी आहे

तुम्हाला पैसे कमवायचे आहेत. तुम्ही जगाला चार levels मध्ये बघितलं आहे:

LEVEL 1	माणसाची गरज	भूक, आरोग्य, ज्ञान, सुरक्षा
LEVEL 2	BUSINESS	जो गरज पूर्ण करून पैसे घेतो
LEVEL 3	TECHNOLOGY	जी business ला ताकद देते
LEVEL 4	AI	जी technology ला ताकद देते

तुम्हाला Level 4 वर काम करायचं आहे, म्हणजे खालचे सगळे levels serve करता येतील. पण Level 4 हवेत तरंगत नाही. तो Level 3 वर उभा आहे. म्हणून आधी machine समजून घ्यावी लागेल. हे पाच parts तेच करतात.

वाचून झाल्यावर तुम्ही चार गोष्टी करू शकाल:

1. कुठल्याही business problem साठी tech solution चं design स्वतः विचार करणं.
2. AI tools कडून काम काढून घेणं, जसं एक engineer काढतो.
3. कुठलाही business बघून सांगणं की tech ने पैसा कुठे बनेल.
4. Engineers आणि tech बातम्यांची भाषा पूर्ण समजणं. कोणी तुम्हाला मूर्ख बनवू शकणार नाही.

हे पुस्तक कसं लिहिलं आहे

सरळ बोलणं, तुमच्याशी. Lecture नाही. प्रत्येक गोष्टीसाठी आधी problem, मग समज, मग नाव. नाव नेहमी शेवटी येतं, कारण नाव आधी आलं की डोकं विचार करणं बंद करतं.

आणि प्रत्येक मोठ्या कल्पनेला रोजच्या जगातली एक उपमा आहे: truck आणि रस्ते, ढाबा आणि recipe, गाव आणि बाजार, post office आणि bank. Machine नवीन आहे; तिच्या आतल्या कल्पना हजारो वर्ष जुन्या आहेत.

प्रत्येक chapter चा ढाचा

CHAPTER	आधी problem, मग समज; नावं आणि खरी उदाहरणं मजकुरातच, ठळक अक्षरात
इथे लोक काय चुकीचं समजतात	जिथे लोक नेहमी घसरतात
MAP वर स्वतः बघा	ही गोष्ट Level 1-4 मध्ये कुठे, पैसा कुठे
विचार करा	5 मिनिटांचं काम, तुमच्या phone/laptop वर एक प्रश्न, उत्तर लगेच खाली

प्रत्येक chapter च्या नावासोबत एक खूण आहे. **[SPINE]** म्हणजे कणा, वगळू नका. **[DEPTH]** म्हणजे खोली, पहिल्या वेळेस वगळू शकता, नंतर परत या.

एकच सूचना

घाई करू नका. दिवसाला दोन-तीन chapters पुरे. प्रश्नाचं उत्तर आधी स्वतः विचार करा, मग खाली वाचा. जी गोष्ट समजली नाही, तिथे थांबा, पुढचं सगळं तिच्यावरच उभं असेल.

SECTION 0: पैसा, VALUE आणि SOCIETY

पाच chapters. हा section machine बदल नाही. ज्या गोष्टीसाठी तुम्ही machine शिकताय, त्या गोष्टीबद्दल आहे: पैसा.

हे तुमच्या business decision मध्ये कुठे येईल: जेव्हा तुम्ही कुठलीही कल्पना उचलाल, पहिला प्रश्न असेल “पैसा कुठून येणार.” याचं उत्तर नेहमी Level 1 पासून सुरू होतं: कोणाची गरज, किती मोठी, किती लोक. हा section तो प्रश्न विचारायला शिकवतो. याच्याशिवाय बाकीचे चार parts फक्त ज्ञान आहेत, कमाई नाही.

Chapter 0.1 [SPINE]: गावः पैसा exist का करतो

शंभर लोकांचं एक गाव विचार करा. पैसा नावाची गोष्ट अजून नाही. शेतकऱ्याकडे धान्य आहे, मिस्त्रीकडे हुनर, वैद्याकडे इलाज, चांभाराकडे जोडे.

पहिली व्यवस्था सरळ आहे: वस्तूच्या बदल्यात वस्तू. शेतकरी धान्य देईल, मिस्त्री छप्पर बांधेल. चाललं.

आता एक प्रसंग बघा. वैद्याने चांभाराचा हात बरा केला. तीन महिन्यांनी वैद्याला जोडे हवेत. पण यावेळेस चांभार एकदम ठणठणीत आहे, त्याला वैद्याकडून काहीच नको. मग वैद्य काय करणार? वाट बघणार की चांभार आजारी पडेल?

इथे व्यवस्था तुटली. सरळ देवाण-घेवाण तेव्हाच चालते जेव्हा दोन माणसांना एकाच वेळेस एकमेकांचं काम हवं असतं. या पहिल्या तुटीचं नाव economists ठेवतात **double coincidence of wants**: दोघांच्या गरजा एकाच क्षणी जुळल्या पाहिजेत. शंभर लोकांच्या गावातही हे क्वचित जुळतं. ही तूट लक्षात ठेवा.

दुसरी व्यवस्था स्वतः काढा. थांबा, खरंच विचार करा. काय कराल?

बरेच लोक म्हणतात: लिहून ठेवा. एक वही बनवा. वैद्याने चांभाराचं काम केलं, लिहून ठेवा, नंतर वसूल करेल. चांगलं उत्तर आहे, जगाने हे केलंही आहे; अशा वहीचं नाव **ledger**. पण दोन नवे प्रश्न उभे राहतात.

पहिला: वही ठेवणार कोण? कोणी एक माणूस. आणि ज्या दिवशी त्याने स्वतःच्या नावापुढे थोडं जास्त लिहिलं? दुकानाच्या गल्ल्यावर बसलेला माणूसच हिशोब लिहितो, तेव्हा हिशोबावर विश्वास म्हणजे माणसावर विश्वास. दुसरा: शंभर लोकांत एक वही चालली. पण दहा हजार लोक, वेगवेगळी गावं, येणारे-जाणारे लोक? किती वह्या? आणि कोणाचा कोणावर विश्वास?

म्हणजे वहीची कल्पना बरोबर आहे, तिचं रूप चुकीचं आहे. आता तिसरी व्यवस्था विचार करा. अशी कुठली गोष्ट असेल जी चांभार घेईल, त्याला इलाज अजिबात नको असला तरी? तो ती का घेईल? फक्त एवढ्यासाठी, की त्याला खात्री आहे की बाकी सगळेही ती घेतील. ती गोष्ट त्याच्या स्वतःच्या कुठल्याच कामाची नाही. तिची पूर्ण किंमत एकाच गोष्टीत आहे: पुढे चालवता येईल हा विश्वास.

आता स्वतः काढा, त्या गोष्टीत काय-काय हवं:

1. सडू नये, नाहीतर ठेवता ठेवता संपेल.
2. उचलायला सोपी हवी, जड दगडाने काम चालणार नाही.
3. नकली बनवणं अवघड हवं, नाहीतर कोणीही न देता सगळं घेईल.
4. तुकड्यांत वाटता यावी, छोटी गोष्टही विकत घ्यावी लागते.

5. सगळ्यांनी ती मानली पाहिजे. एकाने जरी नाकारली तरी साखळी तुटते.

तुम्ही आत्ता **पैसा** invent केला. पैसा म्हणजे चालती-फिरती वही: असा ledger ज्यात विश्वास कोणा एका माणसाच्या हातात नाही, सगळ्यांमध्ये वाटलेला आहे. तुमचा **UPI** बघा: 50 रुपये पाठवता तेव्हा हातातून हातात काहीच जात नाही; एका वहीत एवढंच लिहिलं जातं की तुमच्या नावापुढे 50 कमी, त्याच्या नावापुढे 50 जास्त. ती वही banks आणि NPCI ठेवतात. जग वहीच्या कल्पनेवर परत आलं; फरक एवढाच की वही ठेवणारा आता एक माणूस नाही, एक system आहे जिला सगळे मानतात.

इथे लोक काय चुकीचं समजतात

लोकांना वाटतं पैसा म्हणजे value आहे. नाही. पैसा value ची नोंद आहे. नोट स्वतः कागद आहे; तिची किंमत या विश्वासात आहे की पुढचा माणूस ती घेईल. Post office चं stamp आठवा: तो छोटा कागद पत्र पोहोचवत नाही, तो फक्त “फी भरली आहे” ची नोंद आहे; system मेली तर stamp रद्दी. तसंच ज्या देशांमध्ये लोकांचा विश्वास तुटला, तिथे लोक पोत्यांमध्ये नोटा भरून भाकरी विकत घ्यायला गेले. कागद कागदच राहिला.

MAP वर

हा पूर्ण chapter Level 1 वर उभा आहे: माणसाची गरज आणि त्यांच्यामधली देवाण-घेवाण. पैसा इथेच जन्मतो, आणि लक्षात ठेवा, पैसा **फक्त इथेच** घुसतो, जेव्हा एक माणूस आपल्या गरजेसाठी देतो. Level 2, 3, 4 फक्त ही देवाण-घेवाण मोठी, वेगवान आणि स्वस्त करतात. जो कोणी कमवू इच्छितो, त्याची साखळी कुठेतरी Level 1 च्या एका माणसावर संपते.

स्वतः बघा (5 मिनिटं)

तुमचं UPI app उघडा. शेवटचे 5 व्यवहार बघा. प्रत्येकासाठी विचारा: कुठल्या गरजेसाठी पैसा गेला? खाणं, प्रवास, ज्ञान, अजून काही? प्रत्येक व्यवहार कुठल्यातरी गरजेवर संपेल. एकही असा मिळणार नाही जो कुठल्याही माणसाच्या गरजेवर संपत नाही.

विचार करा

1. (derivation) जर जगात फक्त एकाच वस्तूची देवाण-घेवाण असती, समजा फक्त धान्य, तर पैसा तरीही बनला असता का?

उत्तर: नाही, किंवा खूप उशिरा. पैसा त्या problem चा उपाय आहे जिथे खूप वेगवेगळ्या वस्तूंची देवाण-घेवाण आहे आणि वेळा जुळत नाहीत. एकच वस्तू असेल तर सरळ देवाण-घेवाण पुरते, double coincidence ची तूट होतच नाही. यातून एक खोल गोष्ट दिसते: पैसा तेव्हा गरजेचा होतो जेव्हा समाजात कामं वाटली जातात. जेवढी जास्त कामांची वाटणी, तेवढा गरजेचा पैसा. याच गोष्टीवर पुढचा chapter उभा आहे.

Chapter 0.2 [SPINE]: तास सारखे, कमाई वेगळी

दोन माणसं, एकच गाव, एकच वय, दोघेही मेहनती.

रवी मिस्त्री आहे. रोज 12 तास काम करतो. प्रत्येक घर त्याच्या हातांनी बनतं. ज्या दिवशी तो काम करत नाही, त्या दिवशी कमाई शून्य. वीस वर्षांनंतरही त्याचा दिवस 12 तासांचाच आहे.

अनिलने गावासाठी पाण्याच्या नळाचं design बनवलं. सहा महिने लागले. त्यानंतर नळ-योजना बांधली गेली आणि चालू झाली. आता पाणी रोज प्रत्येक घरी पोहोचतं, अनिलने हात न लावता. गाव अनिलला दर महिन्याला हिस्सा देतं. अनिल आता दुसऱ्या गावासाठी design बनवतोय.

रवी जास्त तास लावतो. अनिल जास्त कमावतो. का?

डोकं पहिलं उत्तर देतं: अनिल शिकलेला आहे. पण हे उत्तर काहीच सांगत नाही. वैद्यही शिकलेला आहे आणि रवीएवढंच कमावतो. खरं उत्तर तीन भागांत आहे. तिन्ही स्वतः काढा:

Problem किती मोठी आहे? रवी एका घराची problem सोडवतो, एका कुटुंबासाठी. अनिलने पूर्ण गावाची पाण्याची problem सोडवली, शंभर कुटुंबांसाठी. जेवढे जास्त लोक, जेवढी खोल गरज, तेवढा मोठा पैसा त्या problem च्या मागे उभा असतो. याचं नाव **market size**.

हे किती लोकांना जमतं? भिंत बांधणं गावातले वीस जण जाणतात. नळाचं design एकच जाणत होता. जी गोष्ट कमी लोकांना जमते, तिचा भाव वर जातो. बाजाराचा सरळ नियम: टंचाई. याचं नाव **scarcity**. Auto-rickshaw सगळ्यांना चालवता येते, म्हणून भाडं ठरलेलं; crane फार कमी लोकांना, म्हणून crane वाल्याचा दर तोच सांगतो.

काम एकदा, फायदा किती वेळा? हा सगळ्यात लपलेला भाग आहे. रवीची बांधणी त्या दिवशी संपते. अनिलचं design एकदा बनलं आणि आता रोज काम करतंय, अनिलशिवाय. रवी आपला वेळ विकतोय. अनिलने अशी गोष्ट बनवली जी तो झोपल्यावरही चालते. याचं नाव **leverage**.

मग सूत्र हे निघालं:

$$\text{कमाई} = \text{problem चा SIZE} \times \text{SCARCITY} \times \text{LEVERAGE}$$

या सूत्रात नीट बघा: **तास कुठेच नाहीत**. म्हणूनच 2 तासवाला billionaire असू शकतो आणि 12 तासवाला तंगीत. मेहनत चुकीच्या गोष्टीवर लागली असेल तर मेहनतीचा भाव मिळत नाही.

खऱ्या जगात हे असं दिसतं: ँक surgeon घ्या आणि ँक app बनवणारा. Surgeon ची skill जास्त दुर्मिळ, शिक्षण जास्त लांब. तरी जगातले सगळ्यात श्रीमंत लोक software वाले आहेत, surgeons नाहीत. का? Surgeon चं प्रत्येक operation त्याच्या स्वतःच्या हातांनी होतं, दिवसाला दोन-तीन. Leverage चा काटा शून्यावर अडकलेला. App ँकदा बनते आणि 10 million (1 कोटी) लोक ँकाच वेळेस वापरतात. WhatsApp विकली गेली तेव्हा (19 billion dollars) तिच्याकडे फक्त 55 कर्मचारी होते, आणि users 400 million. हे leverage आहे.

इथे लोक काय चुकीचं समजतात

लोकांना वाटतं जास्त कमवायचा रस्ता जास्त मेहनत आहे. सूत्र म्हणतं: मेहनतीचा गुणाकार तेव्हाच चालतो जेव्हा बाकीचे तीन काटे वर असतात. Rickshaw चालवणारा दुप्पट तास चालवून दुप्पट थकतो, दुप्पट श्रीमंत होत नाही; पण दहा rickshaw भाड्याने देणारा झोपेतही कमावतो. दुसरी चूक: लोक scarcity ला degree समजतात. Scarcity म्हणजे certificate नाही; त्या वेळेस, त्या जागी, ते काम किती लोकांना जमतं, ँवढंच.

MAP वर

या सूत्राचा तिसरा काटा, leverage, हेच खरं कारण आहे की Level 3 (technology) ँवढा पैसा बनवतो. Machine हीच ती गोष्ट आहे जी ँकदा बनते आणि पुन्हा पुन्हा चालते, न थकता. म्हणूनच तुम्ही हे पुस्तक वाचताय: Level 3 आणि 4 वर leverage चा काटा सगळ्यात वर फिरतो.

स्वतः बघा (5 मिनिटं)

जगातल्या 10 सगळ्यात श्रीमंत लोकांची यादी उघडा (search: richest people). प्रत्येक नावापुढे विचारा: हा माणूस तास विकून इथे पोहोचला का? ँकही मिळणार नाही. प्रत्येकाकडे अशी गोष्ट आहे जी त्याच्याशिवाय चालते: company, software, brand, factory.

विचार करा

1. (derivation) ँक शिक्षक 30 मुलांना शिकवतो आणि महिन्याला X कमावतो. त्याच ज्ञानातून तो करोडो कसा कमवू शकतो? सूत्राचे तिन्ही काटे फिरवून सांगा.

उत्तर: Size: 30 मुलांऐवजी देशभरची मुलं, online. Scarcity: सगळे विषय नाहीत; ती एक गोष्ट जी फक्त तोच सगळ्यात चांगली समजावतो. Leverage: रोज class घेण्याऐवजी एकदा recorded course किंवा एक पुस्तक, जे एकदा बनतं आणि लाखो वेळा विकलं जातं. Physics Wallah हीच गोष्ट आहे: एक शिक्षक, YouTube वर recorded classes, आज billion- dollar company. ज्ञान तेच होतं; काटे फिरले.

Chapter 0.3 [SPINE]: समाजाचे चार levels

Chapter 0.1 च्या गावात परत चला, पण आता पूर्ण जग विचार करा. Billions लोक. या गर्दीत पैसा कुठून सुरू होतो आणि कुठे जातो? एक नकाशा बनवू, खालून वर. इमारतीसारखा विचार करा: प्रत्येक मजला खालच्या मजल्यावर उभा असतो, आणि पाया नेहमी जमिनीत असतो.

Level 1: माणसाची गरज. भूक, तहान, छप्पर, आरोग्य, ज्ञान, सुरक्षा, प्रवास, जोडलेपण, करमणूक, इज्जत. ही यादी हजारो वर्षांपासून जवळजवळ तीच आहे. हीच पूर्ण economy ची जमीन आहे. जगात फिरणारा प्रत्येक रुपया शेवटी कोणाच्यातरी कुठल्यातरी गरजेतून निघाला आहे.

Level 2: Business. एका माणसाची गरज एकदा पूर्ण करणं म्हणजे काम. लाखो लोकांची गरज पुन्हा पुन्हा पूर्ण करणं व्यवस्था मागतं: माणसं, सामान, हिशोब, विश्वास. त्या व्यवस्थेचं नाव business. एक बाई घरात स्वयंपाक करते ते काम; तीच बाई रोज शंभर डबे पोहोचवते ती व्यवस्था, म्हणजे business. Restaurant भुकेवर उभं आहे, hospital आरोग्यावर, school ज्ञानावर. Business म्हणजे गरजेला पैशात बदलणारी व्यवस्था.

Level 3: Technology. Business ला नेहमी तीन गोष्टींची भूक असते: जास्त लोक serve करा, कमी खर्चात, कमी चुकांसह. जी गोष्ट हे करते, ती technology. Tractor शेतकऱ्यासाठी, fridge restaurant साठी, software सगळ्यांसाठी. ढाब्याचा तंदूर पण technology आहे: एक cook च्या जागी एका वेळेस वीस रोट्या. Technology म्हणजे business ची ताकद वाढवणारा मजला.

Level 4: AI. आता technology स्वतः एक गरजवाली गोष्ट झाली आहे: तिच्यासाठी विचार करणं, लिहिणं, ठरवणं, हे आत्तापर्यंत माणसं करत होती. जी machine ही कामं करू लागली आहे, ती technology ची technology. AI Level 3 ला तेच देतो जे Level 3 ने Level 2 ला दिलं: जास्त, स्वस्त, कमी चुका.

आता सगळ्यात महत्वाचा बाण काढा:

LEVEL 4	AI	^	
LEVEL 3	TECHNOLOGY		पैसा वरच्या
LEVEL 2	BUSINESS		दिशेने वाहतो
LEVEL 1	माणसाची गरज		

पैसा **फक्त Level 1 वर** घुसतो, जेव्हा एक माणूस आपल्या गरजेसाठी देतो. Level 2 त्यातला हिस्सा घेतो. Level 3 ला Level 2 देतो. Level 4 ला Level 3 देतो. वरच्या मजल्यांचा प्रत्येक रुपया खालून चढत आलेला आहे. अशा एकावर-एक रचनेला **engineers stack** म्हणतात; पुढे “tech stack” ऐकाल तेव्हा हाच अर्थ: कोण कोणावर उभा आहे, कोण कोणाला पैसे देतो.

हे खऱ्या जगात असं दिसतं: **Zomato** घ्या. Level 1: भूक. Level 2: restaurant, आणि स्वतः Zomato चा business. Level 3: app, servers, नकाशा, payment system. Level 4: कोणतं खाणं कोणाला दाखवायचं, delivery चा रस्ता, मागणीचा अंदाज, हे AI करतो. तुम्ही 300 रुपयांचं जेवण मागवता: restaurant हिस्सा घेतं, Zomato हिस्सा घेतं, Zomato आपल्या cloud आणि software वाल्यांना देतं, ते AI चालवणाऱ्यांना देतात. एक भूक, चार मजल्यांची कमाई.

तीन गोष्टी तपासून घ्या, कारण स्वच्छ नकाशा त्या लपवतो: प्रत्येक गरज business ने पूर्ण होत नाही (आई मुलाला जेवू घालते, कुठेही पैसा फिरत नाही). प्रत्येक business ला जास्त technology लागत नाही. आणि प्रत्येक technology ला AI चांगली करत नाही (पाण्याच्या pipe मध्ये विचार करणाऱ्या machine साठी काही कामच नाही). हा नकाशा कल दाखवतो, कायदा नाही.

इथे लोक काय चुकीचं समजतात

लोकांना वाटतं पैसा technology मध्ये आहे, म्हणून tech शिका. उलटं आहे. पैसा **गरजेत** आहे. Technology फक्त ती गरज पकडण्याचा सगळ्यात लांब हात आहे. विहीर पाण्याने भरलेली असते; दोरी-पोहरा फक्त पाणी काढण्याचं साधन आहे. दोरी कितीही उत्तम असो, विहीर कोरडी असेल तर हाती काही येत नाही. जो माणूस Level 4 ची skill घेऊन बसला आहे पण कुठल्याही Level 1 गरजेशी जोडलेला नाही, तो काही कमावत नाही.

MAP वर

हा chapter स्वतःच नकाशा आहे. फोटोसारखा डोक्यात ठेवा. पुढे प्रत्येक chapter च्या शेवटी “MAP वर” याच नकाशावर बोट ठेवेल: ही गोष्ट कुठल्या level वर आहे, आणि पैसा हिच्यापर्यंत कसा पोहोचतो.

स्वतः बघा (5 मिनिटं)

Phone ची home screen उघडा. कुठलीही 3 apps निवडा. प्रत्येकासाठी लिहा: कुठली Level 1 गरज? कुठला Level 2 business? WhatsApp: जोडलेपणाची गरज, Meta चा business. असं app शोधा जे कुठल्याच गरजेवर उभं नाही. मिळणार नाही.

विचार करा

1. (derivation) AI companies (Level 4) billions कमावतायत. पण पैसा फक्त Level 1 वर घुसतो. मग OpenAI चा पैसा कुठल्या रस्त्याने, कुठल्या levels मधून येतो? साखळी लिहा.

उत्तर: एक साखळी अशी असू शकते: तुम्ही भुकेने जेवण order करता (L1 पासून पैसा घुसला). Zomato कमावतं (L2). Zomato आपल्या software आणि cloud वर खर्च करतं (L3). ते software बनवणारी team ChatGPT चं subscription घेते, code वेगाने लिहिण्यासाठी (L4). AI चा प्रत्येक रुपया अशा कुठल्यातरी साखळीने खालच्या levels मधून आलेला असतो. याचं उलटंही खरं आहे: जर एखादं AI product कुठल्याच साखळीने Level 1 पर्यंत पोहोचत नसेल, तर त्याच्याकडे येणारा पैसाच नाही.

Chapter 0.4 [SPINE]: Technology चा मजला दिसत का नाही

आज रात्री तुम्ही phone वरून जेवण order करता. तुम्हाला काय दिसतं? एक button: “Place Order.” बस.

आता मोजू, त्या एका button च्या मागे काय चालतं. तुमचा phone एक message बनवून हवेत पाठवतो. तो message शहरापलीकडच्या कुठल्यातरी building मधल्या machine पर्यंत जातो. ती machine तुमचं नाव, पत्ता, order, पैशाचा हिशोब तपासते. Bank च्या machine शी बोलून 300 रुपये इकडून तिकडे करते. Restaurant च्या screen वर order चमकतो. अजून एक machine विचार करते की कुठला delivery वाला कुठे आहे आणि कोणाला पाठवणं स्वस्त पडेल. तुमच्या नकाशावर तो चालताना दिसतो.

हे सगळं 5 seconds मध्ये झालं. आणि तुम्हाला यातलं **काहीही** दिसलं नाही.

ही चूक नाही. हे design आहे. का? दोन कारणं, दोन्ही पैशाची:

पहिलं: तुम्ही भुकेले आहात. तुम्ही जेवणाचे पैसे देताय, machine बघण्याचे नाही. ढाब्यावर जाता तेव्हा तुम्हाला गरम रोटी हवी असते; मालक तुम्हाला kitchen मध्ये नेऊन तंदूर, पीठाची गोणी आणि gas चं bill दाखवू लागला तर तुम्ही उठून दुसऱ्या ढाब्यावर जाल. जी गोष्ट ग्राहकाला दिसते आणि अडवते, ती पैसा घालवते. म्हणून जी technology जेवढी चांगली, तेवढी गायब.

दुसरं: जी गोष्ट गायब आहे, ती कोणी शिकू शकत नाही. जी शिकता येत नाही, ती बनवता येत नाही. मग दिसणारा भाग (button) स्वस्त झाला, सगळे बनवतात. न दिसणारा भाग (मागची व्यवस्था) महाग राहिला, कारण कमी लोक जाणतात. Chapter 0.2 चा scarcity चा काटा आठवा? Technology ची scarcity तिच्या न-दिसण्यातून येते.

या लपवण्याच्या कामाचं नाव engineers ठेवतात **abstraction**: आतली गुंतागुंत झाकून वर एक सोपं handle देणं. Button abstraction आहे. गाडीचं steering wheel abstraction आहे: engine मध्ये हजार भाग फिरतात, तुमच्या हातात एक चाक. UPI चा QR code abstraction आहे: banks चं पूर्ण जाळं झाकलेलं. हा शब्द या पुस्तकात शंभर वेळा येईल, कारण पूर्ण technology म्हणजे abstraction वर abstraction ची शिडी.

खऱ्या जगातलं सगळ्यात मोठं उदाहरण: **Jio** ने 2016 मध्ये फुकट data दिला आणि देश बदलला. लोकांना काय दिसलं? एक स्वस्त SIM. काय दिसलं नाही? समुद्राखाली पडलेल्या fibre च्या तारा, प्रत्येक शहरात towers, त्यांच्यामधली व्यवस्था, या सगळ्यावर लावलेले 2.5 लाख कोटी रुपये (~30 billion

dollars). Level 3 इतका मोठा होता की देशाची सगळ्यात मोठी company बनला, आणि सामान्य माणसाने तो कधी बघितलाच नाही. तुमच्या शिकण्यासाठी हीच सगळ्यात मोठी बातमी आहे: **सगळ्यात मोठ्या पैशांची कामं डोळ्यांपासून लपलेली आहेत.**

इथे लोक काय चुकीचं समजतात

लोक “technology” ऐकताच gadget विचार करतात: phone, laptop, robot. Gadget म्हणजे technology चा फक्त तो कोपरा जो विकण्यासाठी दाखवला जातो. भाजी-मंडईचं वजन-काटा दिसतो; पण मंडई चालवणारी खरी व्यवस्था म्हणजे पहाटेचे trucks, लिलाव, उधारीचे हिशोब, ते कोणाला दिसत नाहीत. खरी technology व्यवस्था आहे: numbers ठेवण्याची, पाठवण्याची, तपासण्याची, ठरवण्याची. म्हणून “tech शिकणं म्हणजे coding शिकणं” ही पण अर्धीच गोष्ट आहे. Coding एक हत्यार आहे. समजणं हे आहे की पूर्ण व्यवस्था कशी उभी राहते.

MAP वर

रुपयाचा रस्ता बघा: तुमच्या 300 रुपयांतले restaurant ला ~200, Zomato ला ~100 मिळाले. त्या 100 मधून Zomato cloud, payment gateway, नकाशा- सेवा, SMS पाठवणाऱ्यांना देतं. हे सगळे Level 3 च्या आतले business आहेत ज्यांची नावं तुम्ही कधी ऐकली नाहीत: Razorpay, AWS, Twilio. न-दिसणाऱ्या मजल्याच्या आतही पूर्ण बाजार आहेत. जिथे डोळा जात नाही, तिथे स्पर्धाही कमी असते.

स्वतः बघा (5 मिनिटं)

कुठलंही online payment करा (किंवा जुनं आठवा). कागदावर लिहा: या 5 seconds मध्ये किती वेगळ्या companies नी काम केलं असेल? मोजा: तुमचा phone बनवणारी, network वाली, app वाली, तुमची bank, payment gateway, समोरच्याची bank, NPCI. एक tap, सातपेक्षा जास्त companies. प्रत्येकीला त्या tap मधून पैसा मिळतो.

विचार करा

1. (derivation) एखादा business आपली technology लपवण्याऐवजी पूर्ण दाखवू लागला तर (प्रत्येक order वर: “आता server X शी बोलणं चालू आहे...”) त्याचं काय होईल? आणि या उत्तरातून काढा: engineer चा पगार जास्त का असतो?

उत्तर: ग्राहक पळून जातील, कारण त्यांना गरज पूर्ण करायची होती, machine समजायची नव्हती. Business दाखवण्यासाठी नाही, लपवण्यासाठी पैसे देतो. आणि जी गोष्ट सगळ्यात लपलेली आहे, ती समजणारे सगळ्यात कमी लोक आहेत. कमी लोक = scarcity = उंच पगार. Engineer ला या गोष्टीचे पैसे मिळतात की तो त्या खोलीत काम करू शकतो जिथे बाकीचं जग घुसूच शकत नाही. तुम्ही हे पुस्तक वाचून त्याच खोलीची किल्ली बनवत आहात.

Chapter 0.5 [SPINE]: मग आता आपल्याला काय उघडावं लागेल

चार chapters मध्ये तुमच्याकडे आता हे आहे:

1. पैसा म्हणजे विश्वासाची व्यवस्था, आणि **फक्त** Level 1 वर घुसतो. (0.1)
2. कमाई = size x scarcity x leverage. तास सूत्रात नाहीत. (0.2)
3. जग चार levels चा stack आहे, पैसा खालून वर वाहतो. (0.3)
4. Level 3 मुद्दाम लपवलेला आहे, आणि लपलेलं असणं हाच त्याचा भाव. (0.4)

आता एक प्रामाणिक प्रश्न: Level 1 आणि 2 तुम्हाला आधीच समजतात. गरज म्हणजे काय, business कसा चालतो, हे तुम्हाला दिसतं. जे दिसत नाही, ते Level 3 आणि 4. आणि तुमचा plan Level 4 वर काम करण्याचा आहे.

मग सरळ AI च का शिकू नये? हा प्रश्न प्रत्येकजण विचारतो, आणि इथेच सगळे घसरतात. उत्तरासाठी एक प्रश्न पुरे: AI चालतो कशावर? Computer वर. आणि computer तुम्हाला समजत नसेल, तर AI तुमच्यासाठी काय असेल? जादू. जादूच्या प्रेक्षकाला काय जमतं? बघणं, थोडं वापरणं. काय जमत नाही? Design करणं, विकणं, सुधारणं, आणि तुटल्यावर सांगणं की काय तुटलं. जादूचा प्रेक्षक बनता येतं, जादूगार नाही.

Driver आणि mechanic चा फरक हाच आहे. दोघेही गाडी चालवतात. पण गाडी रस्त्यात बंद पडली की driver फक्त उभा राहतो; mechanic bonnet उघडतो. आणि गाड्यांचा **धंदा** कोण करतं? ज्याला आतलं कळतं तो. AI च्या जगात तुम्हाला driver नाही, mechanic-मालक बनावचं आहे.

म्हणून रस्ता खालून आहे. Level 4 Level 3 वर उभा आहे, मग आधी Level 3: machine. हा Part 1 ते Part 5 चा प्रवास आहे:

PART 1 (हा)	machine म्हणजे काय, सगळं numbers कसं आहे
PART 2	machine ला सांगणं की काय करायचं: software
PART 3	machines चं आपसात बोलणं: internet
PART 4	लक्षात ठेवणं: data, database
PART 5	लाखो लोकांना serve करणं: scale, cloud, खर्च

यानंतर Book 2 येते: AI. तेव्हा ती जादू वाटणार नाही, machine वाटेल.

आणि एक दावा लिहून ठेवा, आपण तो Part 5 च्या शेवटी तपासू:

दावा: Machine समजल्यावर तुम्ही कुठलाही business बघून सांगू शकाल की त्यात technology ने पैसा कुठे बनेल, काय बनवणं स्वस्त आहे, काय महाग, आणि AI त्यात कुठे घुसेल.

इथे लोक काय चुकीचं समजतात

"Basics नंतर बघेन, आधी जलद रस्ता." असा रस्ता असता तर सगळे त्यावर असते. जे लोक AI tools सगळ्यात चांगले चालवतात, ते तेच आहेत ज्यांना आत काय चालतं ते माहीत आहे; म्हणून ते बरोबर गोष्ट मागू शकतात आणि चुकीचं उत्तर पकडू शकतात. Machine ची समज हीच AI ची steering आहे.

MAP वर

तुम्ही या क्षणी नकाशावर स्वतःला ठेवताय: Level 1-2 ची समज तुमच्याकडे आहे, Level 3 चा दरवाजा हे पुस्तक आहे, Level 4 पुढचं पुस्तक. ज्या दिवशी तुमच्याकडे चारही levels ची समज असेल, तुम्ही त्या छोट्या गर्दीत असाल जिला पूर्ण साखळी दिसते. साखळी बघणाराच साखळी design करू शकतो.

स्वतः बघा (5 मिनिटं)

एका कागदावर स्वतःच्या हाताने हे लिहा आणि photo काढून ठेवा: "मी हे यासाठी शिकतोय की _____." आपल्या शब्दांत भरा. Part 5 च्या शेवटच्या chapter मध्ये ही ओळ परत मागितली जाईल; तेव्हा बघा उत्तर बदललं की तेच आहे.

विचार करा

1. (derivation) वरचा तर्क उलटा करा: एका माणसाने **फक्त** Level 3 शिकला, machine चा पूर्ण expert, पण Level 1-2 (गरज, business) कधी बघितलंच नाही. त्याच्या कमाईचं काय होईल?

उत्तर: तो कमावेल, पण नेहमी दुसऱ्यासाठी. पगार मिळेल, कारण त्याची skill दुर्मिळ आहे. पण size आणि leverage चे काटे दुसरा कोणी फिरवेल: founder, ज्याला माहीत आहे कुठल्या गरजेवर काय बनवायचं. जगातले लाखो उत्तम engineers पगारावर आहेत आणि त्यांचे founders billionaires आहेत. फरक skill चा नाही, साखळीच्या पूर्ण समजेचा आहे. म्हणून या पुस्तकात प्रत्येक chapter पैशाच्या ओळीसह येतो: तुम्हाला दोन्ही डोळे हवेत, machine वालाही, पैसा वालाही.

SECTION 1: पहिला प्रश्न

सहा chapters. इथून machine उघडते: भिंतीवरच्या switch पासून या गोष्टीपर्यंत की एकच machine सगळं का करू शकते.

हे तुमच्या business decision मध्ये कुठे येईल: जेव्हा तुम्ही एखाद्या engineer कडून किंवा AI कडून product बनवून घ्याल, तेव्हा “काय शक्य आहे, काय नाही, काय स्वस्त, काय महाग” याचा अंदाज याच section च्या समजेतून येईल. जो founder machine ला जादू समजतो, तो चुकीची गोष्ट मागतो, चुकीच्या भावात, चुकीच्या माणसाकडून. जो समजतो, तो नेमकं तेच मागतो जे machine देऊ शकते.

Chapter 1.1 [SPINE]: एक switch दाबल्यावर काय होतं

भिंतीचा switch दाबा. Bulb लागला. हे पूर्ण पुस्तक याच एका घटनेच्या आत लपलं आहे, म्हणून हळूहळू उघडू.

वीज घरात तारांच्या रस्त्याने येते. तार म्हणजे एक बंद रस्ता: power station पासून निघून, bulb मधून जाऊन, परत. जोपर्यंत रस्ता पूर्ण जोडलेला आहे, वीज वाहते आणि bulb जळतो. अशा बंद रस्त्याचं नाव **circuit**. Switch त्या रस्त्यातली एक कडी आहे जी तुटते आणि जुळते. दाबला: रस्ता जुळला, वीज वाहिली, उजेड. परत दाबला: रस्ता तुटला, अंधार.

मग switch खरं काय आहे? **एका प्रश्नाचं उत्तर, जे दोनपैकी एकच असू शकतं: जुळलेला, की तुटलेला. हो, की नाही. 1, की 0.**

आता तीन उड्या मारा, प्रत्येक छोटी आहे:

पहिली: Switch बोटाने दाबला जातो. विजेने दाबता येईल का? हो, असा switch बनतो ज्याला दुसऱ्या तारेतला current दाबतो. याचा अर्थ नीट ऐका: **एक switch दुसऱ्या switch ला दाबू शकतो.** बोटाची गरज फक्त सुरुवातीला आहे.

दुसरी: एक switch दुसऱ्याला दाबू शकतो, तर हजार switch मिळून एक साखळी बनू शकते: हा दाबला तर तो दाबेल, तो दाबला तर हा उघडेला. रेल्वेचं signal-yard आठवा: एक lever ओढला की पुढे रूळ बदलतो, रूळ बदलला की पुढचा signal बदलतो; stationmaster एका जागी बसून पूर्ण yard चालवतो. साखळी design करणं म्हणजे आधीच ठरवणं की काय झाल्यावर काय होईल. हे “आधीच ठरवणं” पुढे खूप मोठं होणार आहे.

तिसरी: आजच्या switch मध्ये हलणारा भाग नाही. तो वाळूच्या धातूचा (silicon) एक तुकडा आहे जो विजेच्या इशान्यावर रस्ता उघडतो-बंद करतो. या विजेने-दाबल्या-जाणाऱ्या switch चं नाव **transistor**: माणसाचा कदाचित सगळ्यात महत्वाचा शोध. झीज नाही, आवाज नाही, आणि आकार? तुमच्या phone च्या आतल्या chip मध्ये असे **15 billion** (1,500 कोटी) transistors आहेत. नखाएवढ्या जागेत.

मग हे पुस्तक जी machine उघडतंय, ती ही आहे: **billions switches, एकमेकांना दाबत, एका design केलेल्या साखळीत.** तुमचा प्रत्येक tap, प्रत्येक message, प्रत्येक video याचाच निकाल आहे. Phone चा torch लावा: बोटाने काच दाबली, काचेखालच्या sensor ने छोटा current पाठवला, त्याने साखळी सुरू केली, साखळीच्या शेवटी एका transistor ने LED चा रस्ता जोडला. उजेड. तुम्ही भिंतीचा switch च दाबला, फक्त मध्ये billions switches होते. फरक फक्त मोजणीचा आहे, कल्पना तीच.

इथे लोक काय चुकीचं समजतात

लोकांना वाटतं computer ही कुठलीतरी विचित्र, समजण्यापलीकडची गोष्ट आहे. खरं हे आहे की तिचा प्रत्येक भाग भिंतीच्या switch इतकाच सरळ आहे. डोकं चक्रावतं ते **मोजणीने**: billions switches, billion वेळा प्रति second. पण मोजणीला घाबरणं म्हणजे समुद्राला घाबरणं कारण त्यात थेंब खूप आहेत. थेंब समजला की समुद्र समजतो.

MAP वर

Level 3 ची पूर्ण इमारत या एका भागावर उभी आहे, आणि transistor बनवणं जगातल्या सगळ्यात मोठ्या धंद्यांपैकी आहे. एक company, **TSMC** (Taiwan), जगातले सगळ्यात उत्तम transistors बनवते, आणि Apple पासून Nvidia पर्यंत सगळे तिच्या दारात रांगेत उभे आहेत. जी गोष्ट दुसऱ्या कोणालाच बनवता येत नाही = scarcity चा काटा पूर्ण वर. एक factory एकदा उभी राहते आणि billions chips काढते = leverage पण पूर्ण वर. सूत्र काम करतंय.

स्वतः बघा (5 मिनिटं)

घरचा कुठलाही switch दाबताना हा विचार करा: मी एका प्रश्नाचं उत्तर set करतोय, हो की नाही. मग phone चा torch लावा आणि स्वतःला सांगा: आता मी billions switches च्या साखळीच्या टोकावर बोट ठेवलं. दोन्ही एकच गोष्ट आहेत. हे जाणवणं हेच या chapter चं खरं काम.

विचार करा

1. (derivation) “एक switch दुसऱ्या switch ला दाबू शकतो.” हे शक्य **नसतं**, बोट नेहमी लागलं असतं, तर computer चं काय झालं असतं?

उत्तर: Computer बनूच शकला नसता. प्रत्येक पावलावर माणूस लागला असता, मग machine चा वेग माणसाच्या वेगावर अडकला असता: second ला billion पावलांऐवजी second ला एक. आणि leverage शून्य झाला असता: machine ही “एकदा बनवा, चालत राहील” वाली गोष्ट राहिली नसती. पूर्ण Level 3 या एका गोष्टीवर उभा आहे की साखळी एकदा design करा, मग ती माणसाशिवाय चालते. अनिलची नळ-योजना आठवा (Chapter 0.2): हे तेच आहे, फक्त पाण्याऐवजी वीज वाहते.

Chapter 1.2 [SPINE]: मी हे गृहीत धरून चाललोय

प्रत्येक पुस्तक काही गोष्टी गृहीत धरतं आणि सांगत नाही. मग वाचणारा कुठेतरी अडतो आणि त्याला वाटतं आपणच कमी आहोत. चूक. पुस्तकाने गृहीत धरलं होतं. हे पुस्तक तसं करणार नाही. मी काय-काय गृहीत धरतोय त्याची ही पूर्ण यादी. जे माहीत आहे त्यावर खूण करा; जे नाही, त्याचं एका ओळीचं उत्तर इथेच आहे.

वीज म्हणजे काय? धातूच्या आत छोटे कण (electrons) असतात, जे धक्का मिळाल्यावर एका दिशेला सरकतात. त्या सरकण्याचं नाव current. पाण्याच्या pipe मधलं पाणी असा विचार करा, काम चालेल. (ही उपमा आहे, पूर्ण सत्य नाही; पण या पुस्तकासाठी पुरेशी आहे.)

तार current का नेते आणि plastic का नाही? काही वस्तूंमध्ये कण सहज सरकतात (धातू: **conductor**), काहींमध्ये अजिबात नाही (plastic, rubber: **insulator**). म्हणून तार धातूची आणि तिचं आवरण plastic चं: आत रस्ता, बाहेर भिंत. रस्ता आणि रस्त्याच्या कडेचा कठडा.

Battery काय करते? धक्का देते. तिची दोन टोकं असतात, आणि ती कणांना एका टोकाकडून दुसऱ्याकडे ढकलते. जोपर्यंत रस्ता जोडलेला आहे, धक्का चालू. हातपंपासारखी: दांडा मारत राहा, पाणी फिरत राहतं.

”Signal” म्हणजे काय? Current चं असणं किंवा नसणं, बस. पुढे जेव्हा म्हणू “phone ने signal पाठवला,” तेव्हा अर्थ: कुठल्यातरी तारेत किंवा हवेत current/लहरीचा एक क्रम गेला: आहे, नाही, आहे, आहे, नाही. Post office ची जुनी tar (telegraph) हेच करायची: टक-टक चा क्रम, आणि पलीकडे अर्थ.

Chip म्हणजे काय? Silicon च्या छोट्या तुकड्यावर छापलेले billions transistors आणि त्यांच्यामधल्या तारा, एका डबीत बंद. Computer च्या आतल्या हिरव्या पट्टीवर (circuit board) याच डब्या बसवलेल्या असतात.

बस. या पुस्तकाला विजेबद्दल यापेक्षा जास्त काहीही लागत नाही. वरच्या पाच गोष्टी ठीक वाटत असतील, तर पूर्ण पुस्तकासाठी पुरेसं physics तुमच्याकडे आहे.

इथे लोक काय चुकीचं समजतात

लोकांना वाटतं technology समजण्यासाठी आधी physics चा डोंगर चढावा लागेल, मग maths चा. नाही. प्रत्येक मजला खालच्या मजल्याचा फक्त छोटा हिस्सा वापरतो. Bus driver ला engine ची पूर्ण रचना लागत नाही; त्याला एवढंच लागतं: accelerator, brake, आणि हे की गाडी भरवशाची आहे.

Computer बनवणाऱ्यालाही electron ची पूर्ण physics लागत नाही; त्याला एवढंच लागतं: switch उघडतो, बंद होतो, भरवशाचा आहे. ही गोष्ट पुढे सगळीकडे लागेल: **प्रत्येक मजला खालच्याचं फक्त handle धरतो, पूर्ण engine नाही.** (हे abstraction आहे, Chapter 0.4 वालं.)

MAP वर

इथे कोण कमावतं: जे हा खालचा मजला पूर्ण जाणतात: electrical engineers, chip designers. त्यांची दुनिया या पुस्तकाच्या बाहेर आहे, आणि तिथेही billions चा बाजार आहे (power grids, batteries, solar). आपण त्या दरवाजासमोरून जातोय, आत घुसत नाही. लक्षात एवढंच ठेवा की तो दरवाजा आहे, आणि त्याच्या आतही Level 3 च आहे: कुठल्यातरी business ची ताकद वाढवणं.

स्वतः बघा (5 मिनिटं)

तुमचा phone charger बघा. त्यावर बारीक लिहिलेलं असेल: “5V 2A” सारखं काही. V धक्क्याचं माप आहे (किती जोरात), A प्रवाहाचं माप (किती कण प्रति second). आता charger ची तार बघा: आत धातू, बाहेर plastic. Conductor, insulator. तुमच्या हातात पूर्ण Chapter 1.2 आहे.

विचार करा

1. (derivation) पाण्याची उपमा कुठे तुटू शकते? एक जागा विचार करा जिथे वीज पाण्यासारखी नाही.

उत्तर: बऱ्याच जागी. सगळ्यात मोठी: पाणी pipe मधून गळालं तर फरशी ओली होते; पण current ला पूर्ण जोडलेला रस्ता लागतो, परत जाण्याच्या रस्त्याशिवाय current वाहतच नाही. म्हणून चिमणी एका तारेवर बसू शकते (रस्ता पूर्ण झाला नाही), पण दोन तारांना एकदम शिवली तर रस्ता पूर्ण होतो. उपमा हत्यार आहे, सत्य नाही. हा धडा AI पर्यंत जाईल: प्रत्येक model, प्रत्येक उपमा कुठेतरी तुटते, आणि expert तोच जो जाणतो कुठे.

Chapter 1.3 [SPINE]: Computer खरं आहे तरी काय

आतापर्यंतचे तुकडे जोडा: switch हो/नाही ठेवतो, switch switch ला दाबतो, साखळी आधीच design होते. मग ही पूर्ण गोष्ट मिळून करते काय?

एका कामापासून सुरुवात करा जे तुम्ही स्वतः करता: $47 + 38$ जोडणं. तुम्ही काय करता? तुमच्याकडे एक **पद्धत** आहे, लहानपणी पाठ केलेली: आधी एक जोडा, $7 + 8 = 15$, 5 लिहा, 1 हातचा. मग दहं जोडा, हातचा 1 पण. उत्तर 85. लक्ष द्या: तुम्ही **विचार** केला नाही, तुम्ही एक पाठ केलेली पद्धत **चालवली**. कुठलेही दोन आकडे द्या, पद्धत तीच राहील.

आता विचारा: switches च्या साखळीतून अशी पद्धत चालू शकेल का? हो. आकडे हो/नाही मध्ये ठेवता येतात (पुढचा chapter). जोडण्याची पद्धत switches च्या साखळीत आधीच design करता येते (Chapter 1.5). मग एक डबा बनू शकतो: या बाजूने दोन आकडे घाला, त्या बाजूने बेरीज निघेल. कुठल्याही माणसाशिवाय.

आणि आता खरी उडी. जोडण्याचा डबा बनू शकतो, तर: तुलना करण्याचा डबा (कुठला मोठा?), copy करण्याचा डबा, “जर हे तर ते” चा डबा. आणि हे डबे रांगेत लावले, एकामागून एक, तर लांबलचक recipe चालू शकते: “आधी जोडा, मग तुलना करा, मोठं असेल तर हे copy करा...”

मग computer हे आहे: एक machine जी आकड्यांवर, आधीच लिहिलेली recipe, न थकता, billion पावलं प्रति second च्या वेगाने चालवते.

या व्याख्येतले तीन शब्द नीट बघा. **आकड्यांवर**: computer ला फक्त numbers कळतात; photo, गाणं, message, सगळं आधी number बनेल (Section 2 चं काम). **आधीच लिहिलेली**: computer च्या प्रत्येक पावलामागे कुठल्यातरी माणसाचा आधीच घेतलेला निर्णय आहे; machine निर्णय घेत नाही, निर्णय **चालवते**. **न थकता**: हीच तिची पूर्ण किंमत; recipe एकदा लिहा, machine billion वेळा चालवेल. Leverage, शुद्ध रूपात.

या recipe चं नाव **program**. Recipe लिहिण्याचं नाव **programming**. सगळ्या recipes मिळून **software**, आणि हाताला लागणारी machine **hardware**. जोडणारा-तुलना करणारा मुख्य डबा **processor** किंवा **CPU**.

ढाब्याच्या भाषेत: मालकीण एकदा recipe लिहून देते; नवा cook रोज तीच चालवतो, विचार न करता, आणि चव तीच येते. Recipe लिहिणं वेगळं काम, चालवणं वेगळं. आणि खऱ्या जगात हे असं दिसलं: bank चा clerk 1960 मध्ये interest मोजायचा: प्रत्येक खात्यावर तीच पावलं, दिवसभर, चुकांसह. आज

तीच recipe software मध्ये लिहिलेली आहे आणि रात्री 2 वाजता कोट्यवधी खात्यांवर एकदम चालते, शून्य थकवा. Clerk ची **पद्धतच** machine मध्ये गेली. माणसाचं काम recipe लिहिणं उरलं, चालवणं machine घेऊन गेली. जिथे जिथे जगात एखादी “पाठ केलेली पद्धत” पुन्हा पुन्हा चालते, तिथे तिथे computer घुसेल. ही ओळ business शोधण्याचं हत्यार आहे, जपून ठेवा.

इथे लोक काय चुकीचं समजतात

“Computer विचार करतो.” नाही. तो recipe चालवतो. Computer “चूक” करतो तेव्हा ती recipe ची चूक असते, कुठल्यातरी माणसाने लिहिलेली. Computer “ठरवतो” तेव्हा तो माणसाचा आधीच लिहिलेला निर्णय असतो. ही गोष्ट AI पर्यंतही जाईल; तिथे recipe विचित्र पद्धतीने लिहिली जाते, पण machine तीच आहे. ज्याला हे समजलं, तो machine ला कधी घाबरणार नाही आणि तिच्यावर कधी आंधळा विश्वासही ठेवणार नाही.

MAP वर

या chapter ने Level 3 ला दोन भागांत कापलं: hardware आणि software. दोघांचे बाजार वेगळे. Hardware मध्ये पैसा factory आणि scarcity तून येतो (TSMC, Samsung, Nvidia). Software मध्ये पैसा या गोष्टीतून येतो की लिहिण्याचा खर्च **एकदाच** लागतो: recipe एकदा लिहिली, आता प्रत्येक copy फुकट. म्हणून जगातल्या सगळ्यात जाड नफ्याच्या companies software च्या आहेत: Microsoft चा माल बनवण्याचा खर्च जवळजवळ शून्य आहे. तुम्ही जे शिकताय, ते याच बाजूचं दार आहे.

स्वतः बघा (5 मिनिटं)

Phone चा calculator उघडा. $47 + 38$ करा. आता स्वतःला विचारा: या काचेखाली काय झालं? तुमचे taps numbers बनले, numbers CPU च्या जोडणाऱ्या डब्यात गेले, उत्तराचे numbers screen चे ठिपके बनले. पूर्ण Chapter 1.3 एका second मध्ये चाललं. आणि ही calculator-recipe कोणी तरी **एकदा** लिहिली होती; आता billions phones वर फुकट चालते.

विचार करा

1. (derivation) “Machine निर्णय घेत नाही, निर्णय चालवते.” मग bank चं software तुमचं loan नाकारतं तेव्हा खरा निर्णय कोणी घेतला? आणि या उत्तरातून business ची एक जबाबदारी काढा.

उत्तर: निर्णय त्या माणसाने घेतला ज्याने recipe लिहिली किंवा लिहून घेतली: “जर income इतक्यापेक्षा कमी, तर नाही.” Machine ने फक्त तो न थकता चालवला. जबाबदारी: business जेव्हा आपला निर्णय recipe मध्ये घालतो, तेव्हा चुकीलाही leverage मिळतो; एक चुकीचा नियम लाखो लोकांवर एकदम चालतो. म्हणून recipe लिहिणाऱ्याची किंमतही जास्त आहे आणि जबाबदारीही. AI च्या जमान्यात हा प्रश्न अजून मोठा होईल; Book 2 मध्ये भेटेल.

Chapter 1.4 [SPINE]: Switches आकडे कसे ठेवतात

एक switch दोन गोष्टी सांगू शकतो: 0 किंवा 1. पण जगातले आकडे तर 0 ते 9 च्या अंकांनी बनतात, आणि मोठे आकडे, जसे 300 रुपये किंवा 15 billion, त्यांचं काय?

स्वतः काढू. एक switch: दोन स्थिती (0, 1). दोन switch: चार स्थिती (00, 01, 10, 11). तीन switch: आठ. प्रत्येक नवा switch स्थितींची मोजणी **दुप्पट** करतो, कारण जुनी प्रत्येक स्थिती आता दोन रूपांत असू शकते: नवा switch 0 सोबत, किंवा 1 सोबत.

आता या स्थितींना नावं द्या, एवढंच करायचं आहे:

00 ला म्हणा "शून्य"	01 ला म्हणा "एक"
10 ला म्हणा "दोन"	11 ला म्हणा "तीन"

थांबा, हे तर तेच आहे जे आपण दहा बोटानी करतो! आपण 9 नंतर नवा अंक सुरू करतो (10), machine 1 नंतर नवा switch सुरू करते (10). आपला प्रत्येक अंक दहापट वाढतो, machine चा प्रत्येक switch दुप्पट. बस, एवढाच फरक. मोजणीची कल्पना तीच, फक्त बोटं 10 ऐवजी 2.

एका switch च्या स्थितीचं नाव **bit** (binary digit). आठ bits च्या गटाचं नाव **byte**. दुप्पट-वाल्या मोजणीचं नाव **binary**. हे तीन शब्द आता तुमचे झाले, आणि पूर्ण digital जग यांनीच मोजलं जातं.

मग कामाच्या आकड्यांसाठी किती switch लागातात?

8 switch	=	256 स्थिती	(0 ते 255)
16 switch	=	65,536	
32 switch	=	4 billion+	(430 कोटी)
64 switch	=	इतकं मोठं की जगातले सगळे रुपये, सगळे phones, सगळ्या files मोजा, संपत नाही	

तुमच्या phone चा processor एकदम 64 switches च्या गटावर काम करतो. म्हणून खोक्यावर “64-bit phone” लिहिलेलं असतं. आता तुम्हाला त्या ओळीचा अर्थ माहीत आहे; जगातल्या 99% लोकांना नाही.

एक शेवटचा प्रश्न स्वतःला विचारा: machine 2 वर का थांबली, 10 स्थितीवाला switch का नाही बनवला? बनू शकला असता. पण विजेचा current थोडा वर-खाली होत राहतो (noise). दोन स्थितींमध्ये फरक इतका मोठा आहे (पूर्ण चालू, पूर्ण बंद) की छोट्या-मोठ्या कंपनेने चूक होत नाही. दहा स्थिती जवळजवळ

असत्या, आणि प्रत्येक हलकी कंपनीची अंक बनली असती. Post office चा शिक्का आठवा: तारीख थोडी तिरकी उमटली तरी चालतं, कारण पर्याय मोजके आणि एकमेकांपासून दूर आहेत. **2 निवडला कारण 2 सगळ्यात भरवशाचा आहे.** Machine च्या प्रत्येक design मध्ये हा प्रश्न भेटेल: वेगवान हवं, स्वस्त हवं, की भरवशाचं हवं? इथे भरवसा जिंकला.

खऱ्या जगात हे कुठे आहे: तुमचा bank balance कुठल्यातरी machine मध्ये 64 switches ची एक रांग आहे. UPI ने 50 रुपये पाठवले तर त्या रांगेतले काही switches पलटले, आणि दुसऱ्या machine मध्ये दुसऱ्या रांगेतले. पूर्ण Indian economy, प्रत्येक खातं, प्रत्येक देवाण-घेवाण, चालू/बंद च्या रांगांमध्ये ठेवलेली आहे. म्हणून त्या रांगांची राखण (Part 3 मधलं encryption) इतका मोठा business आहे: switches मध्ये ठेवलेला पैसा खरा पैसा आहे.

इथे लोक काय चुकीचं समजतात

लोक binary बघून विचार करतात की machine कुठलीतरी वेगळी, परकी भाषा बोलते जी माणसाच्या आवाक्याबाहेर आहे. तुम्ही आता 5 मिनिटांत पूर्ण कल्पना काढली, कारण नवीन काही नव्हतंच: ही तीच मोजणी आहे जी तुम्ही लहानपणापासून करता, कमी बोटांवर. Technology मध्ये “अवघड” हे बहुतेक वेळा “अनोळखी” चं दुसरं नाव असतं.

MAP वर

रुपयाचा रस्ता: तुम्ही जे काही साठवता ते bits मध्ये मोजलं जातं, आणि bits ठेवायच्या जागेचं भाडं लागतं. गोदामासारखं: पोती जास्त, भाडं जास्त. Google तुम्हाला 15 GB फुकट देतो आणि त्यावर महिन्याचं भाडं घेतो (Google One, iCloud). ते भाडं billions चा business आहे. पुढे Chapter 2.1 मध्ये हे माप पूर्ण शिकू, कारण जे मोजलं जातं, तेच विकलं जातं.

स्वतः बघा (5 मिनिटं)

एका हाताची पाच बोटे घ्या. मिटलेलं बोटे 0, उघडं 1. अंगठ्यापासून सुरू करा: फक्त अंगठा उघडा = 1, फक्त दुसरं बोटे = 2, दोन्ही = 3. असं मोजत जा. पाच बोटांवर तुम्ही 31 पर्यंत मोजू शकता. हे एकदा केलंत की binary हाताने चालवली; आता ती कधीच विसरणार नाही.

विचार करा

1. (derivation) 8 switches 256 स्थिती देतात. एक जुना game console “8-bit” होता: screen वरचा एक रंग निवडायला 8 bits. त्या console वर किती रंग शक्य होते, आणि यातून काढा: जुने games तसे का दिसायचे?

उत्तर: 256 रंग, बस. म्हणून जुन्या games चे रंग स्पष्ट तुकड्यांत दिसतात; संध्याकाळच्या आभाळाचा हळू बदलणारा रंग त्यांच्यात बनूच शकत नव्हता. आजचे screens प्रत्येक ठिपक्यासाठी 24+ bits ठेवतात, 16 million (1.6 कोटी) रंग, म्हणून photo खरा वाटतो. धडा: bits ची मोजणी थेट ठरवते की machine किती बारकावा बघू शकते. हा नियम AI पर्यंत जाईल: तिथेही बारकावा bits मध्ये विकला जातो.

Chapter 1.5 [DEPTH]: Switches

ठरवतात कसे

(हा DEPTH chapter आहे. पहिल्या वेळेस जड वाटला तर पुढे जा, Part 1 संपवून परत या. पण परत नक्की या, कारण इथेच “machine विचार करते” वाली जादू पूर्ण तुटते.)

Chapter 1.3 मध्ये आपण मानलं होतं की “जोडण्याचा डबा” आणि “तुलनेचा डबा” बनू शकतात. आता बनवू. फक्त तीन छोटे भाग लागतात, तिन्ही switches पासून बनतात. दरवाजांचा विचार करा:

AND: दोन दरवाजे एकामागून एक, रांगेत. पलीकडे जायचं तर **दोन्ही** उघडे लागतील. म्हणजे: “दोन्ही हो, तर हो.” सरकारी कामासारखं: form वर दोन साह्या लागतात, एक जरी नसली तरी file पुढे जात नाही.

OR: दोन दरवाजे शेजारी-शेजारी, दोन वेगळे रस्ते. **कुठलाही एक** उघडा असला तरी पलीकडे जाता येतं. म्हणजे: “कोणीही हो, तर हो.” गावात जायला दोन रस्ते: पूल बंद असला तरी वळणाचा रस्ता चालतो.

NOT: उलट करणारा. आत 1 तर बाहेर 0, आत 0 तर बाहेर 1.

बस. हे तीनच आहेत. यांचं नाव **logic gates**. आता यांच्याकडून काम करून घेऊ.

तुलना बनवा: “A आणि B वेगळे आहेत का?” विचार करा: वेगळे म्हणजे (A हो, B नाही) **किंवा** (A नाही, B हो). हे तर AND, OR, NOT ची जोडणी आहे: (A AND (NOT B)) OR ((NOT A) AND B). झालं. Machine आता “वेगळं आहे का” सांगू शकते. आणि “सारखं आहे का?” त्याचं NOT आहे.

बेरीज बनवा: एक bit जोडा: $0+0=0$, $0+1=1$, $1+0=1$, $1+1=10$ (दोन bits चं उत्तर: “0 लिहा, 1 हातचा”). बघा, उत्तराचा पहिला भाग “वेगळे आहेत?” वालाच भाग आहे, आणि हातच्याचा भाग AND आहे (दोन्ही 1 असतील तेव्हाच हातचा जातो). शाळेतली बेरजेची पद्धत switches मध्ये उतरली. असे 64 जोडा, आणि 64-bit आकड्यांची बेरीज तयार.

”जर” बनवा: “जर हे 1 असेल तर या तारेचा signal पुढे जाऊ दे.” हे फक्त एक AND आहे: signal AND अट. Recipe चं प्रत्येक “जर असं तर तसं” शेवटी याच AND/OR/NOT च्या साखळ्या आहेत. ATM चा निर्णय बघा: “पैसे द्या **जर** PIN बरोबर **AND** balance पुरेसा **AND** आजची limit शिल्लक.” तीन अटी, एक AND-साखळी. तुम्ही ATM मधून पैसे काढता तेव्हा gates चा एक गट तुमच्या बाजूने 1 म्हणतो.

आता पूर्ण मनोरा खालून वर बघा:

transistor -> AND/OR/NOT -> बेरीज/तुलना/जर -> recipe ची पावलं
 -> पूर्ण program -> WhatsApp

प्रत्येक मजला खालच्या मजल्याच्या भागांनी बनला आहे. कुठेही, कुठल्याही मजल्यावर, “विचार” नावाचा भाग नाही. फक्त switches आहेत, design मध्ये जडवलेले.

इथे लोक काय चुकीचं समजतात

”Machine ठरवते, म्हणजे तिच्या आत कुठेतरी छोटा मेंदू आहे.” आता तुम्हाला “ठरवणं” चा अर्थ माहीत आहे: आधीच design केलेली AND/OR/NOT ची साखळी, जिच्यात प्रत्येक शक्य input चं output **आधीच** ठरलेलं आहे. Machine कडे surprise नसतं. Machine चं उत्तर तुम्हाला surprise करतं, तेव्हा अर्थ: तुम्हाला design माहीत नाही; machine काही नवीन विचार करत नाहीये.

MAP वर

कोण कमावतं: chip design करणारे. Gates जोडून वेगवान, छोटी, कमी वीज खाणारी साखळी बनवणं जगातल्या सगळ्यात दुर्मिळ skills पैकी आहे. **Nvidia** याच खेळात जगातल्या सगळ्यात मौल्यवान companies मध्ये पोहोचली: त्यांच्या gates ची जुळणी AI च्या कामासाठी सगळ्यात वेगवान निघाली. जी गोष्ट तुम्ही आज 15 मिनिटांत समजलात, तिचीच उत्तम जुळणी trillion-dollar company बनवते. फरक खोलीचा आहे, कल्पनेचा नाही.

स्वतः बघा (5 मिनिटं)

कागदावर हे table स्वतः बनवा, न बघता: A आणि B च्या चार जोड्या (00, 01, 10, 11), आणि प्रत्येक जोडीसाठी AND, OR, आणि “वेगळे आहेत?” चं उत्तर. शेवटचा रकाना स्वतः विचार करावा लागेल. बनला की समजा: आज तुम्ही gate design केला.

विचार करा

1. (derivation) NOT gate काढून टाका. फक्त AND आणि OR उरले. “वेगळे आहेत?” वाला भाग अजूनही बनू शकेल का? का किंवा का नाही?

उत्तर: नाही बनू शकत. AND आणि OR दोन्ही “मिळवणारे” आहेत: सगळे inputs 1 असतील तर दोघांचंही output 1 च असतं; उलटवणारा कोणी नाही. पण “वेगळे आहेत?” ला 11 वर 0 द्यावं लागतं. उलटवणाऱ्या भागाशिवाय 1 पासून 0 बनतच नाही. म्हणून NOT गरजेचा आहे. धडा: छोटासा भागही पूर्ण system ची ताकद बदलतो, आणि design म्हणजे हेच जाणणं की कुठला भाग काय जोडतो.

Chapter 1.6 [SPINE]: एकच machine सगळं का करू शकते

एका विचित्र गोष्टीकडे कधी लक्ष दिलंय? Mixer फक्त वाटतो. Fridge फक्त थंड करतो. गाडी फक्त चालते. प्रत्येक machine **एक** काम करते.

पण तुमचा phone? सकाळी alarm, मग camera, मग bank, मग game, मग video call. एकच machine, हजार कामं. जगातल्या बाकी प्रत्येक machine पेक्षा वेगळी. असं काय आहे हिच्यात?

Chapter 1.3 मधून recipe ची गोष्ट उचला. Machine recipe चालवते. आता एक प्रश्न, जो पूर्ण इतिहासातला सगळ्यात सुंदर प्रश्न आहे:

Recipe स्वतः काय आहे?

Recipe म्हणजे पावलांची यादी: “जोडा,” “तुलना करा,” “जर असं तर तिकडे जा.” आता या पावलांना नंबर द्या: जोडा = 1, तुलना = 2, copy = 3... मग पूर्ण recipe बनली numbers ची रांग: 1, 3, 2, 7...

थांबा. Numbers ची रांग? ती तर machine switches मध्ये ठेवू शकते (Chapter 1.4). म्हणजे:

Recipe त्याच जागी ठेवता येते जिथे data ठेवला जातो. आणि machine अशी बनवा की ती आपल्या memory मधून पुढचा number वाचेल, आणि त्या number चं काम करेल. बस.

याचा अर्थ उलगाडा, कारण हा Level 3 चा जन्म आहे:

नवं काम हवं? नवी machine बनवू **नका**. फक्त memory मध्ये नवी recipe घाला. Machine तीच, काम नवं. Camera app घातलं तर phone camera झाला, bank app घातलं तर bank झाला. Machine बदलली नाही, फक्त numbers बदलले.

ढाब्याची उपमा परत: mixer म्हणजे एकाच पदार्थाचा cook. पण एक cook जो **कुठलीही** लिहिलेली recipe वाचून बनवू शकतो, तो एकटा पूर्ण hotel आहे. आज पंजाबी, उद्या चायनीज; cook तोच, कागद बदलला. तुमचा phone हा तो cook आहे. या design चं नाव **stored-program computer** (recipe data च्या जागी ठेवणं), आणि “एक machine जी कुठलीही recipe चालवू शकते” या कल्पनेचं नाव **universal machine**. Alan Turing ने 1936 मध्ये, machine बनण्याआधी, कागदावर हे सिद्ध केलं होतं.

आणि आता पैसा बघा. Machine बनवणं: factory, सामान, मजूर, प्रत्येक copy चा खर्च. Recipe बनवणं: एकदा डोकं लावा, मग प्रत्येक copy **जवळजवळ फुकट**, कारण copy म्हणजे फक्त numbers ची नक्कल. Chapter 0.2 चा leverage चा काटा आठवा. Software ही पहिली गोष्ट आहे जिथे leverage चा काटा टोकापर्यंत फिरतो: एकदा लिहा, billion वेळा चालवा, copy चा भाव शून्य.

याच एका कल्पनेने जगाच्या श्रीमंतीचा नकाशा बदलला. **Nokia** जगातली सगळ्यात मोठी phone company होती. तिचे phones **एका** कामासाठी बनले होते: call आणि SMS. iPhone आला आणि त्याने phone ला universal machine बनवलं: app store म्हणजे “कुठलीही recipe घाला.” Nokia hardware चांगलं करत राहिली; Apple ने recipes चा बाजार नेला. दहा वर्षांत Nokia संपली. Machine विरुद्ध universal machine च्या लढाईत universal नेहमी जिंकते, कारण तिच्याकडे जगभरच्या recipe लिहिणाऱ्यांची फौज असते.

इथे लोक काय चुकीचं समजतात

“Software company काढायला factory सारखा मोठा पैसा लागतो.” उलटं. Software ची factory नाहीच, म्हणून सुरुवातीचा खर्च इतिहासातला सगळ्यात कमी आहे: एक laptop, एक डोकं. म्हणूनच दोन मुलं garage मध्ये Google सुरू करू शकली. आणि म्हणूनच स्पर्धाही भयंकर आहे: जी गोष्ट तुम्ही स्वस्तात सुरू करू शकता, ती कोणीही स्वस्तात सुरू करू शकतो. Scarcity आता recipe च्या कल्पनेत आणि **quality** त आहे, सुरू करण्यात नाही.

MAP वर

India चं उदाहरण: **Zerodha**. दोन भावांनी stock broking ची recipe software मध्ये लिहिली. Branch नाही उघडली, agents नाही ठेवले. एक recipe, 10 million+ (1 कोटी+) ग्राहक, आणि India च्या सगळ्यात नफ्यातल्या startups पैकी एक, बाहेरचा पैसा न घेता. Universal machine वर लिहिलेली एक चांगली recipe काय करू शकते, याचा देशी पुरावा. तुमचा Level 4 वाला plan याच जमिनीवर उभा राहील.

स्वतः बघा (5 मिनिटं)

Phone च्या Settings मध्ये जा, Storage बघा. तिथे apps ची यादी असेल, प्रत्येकीपुढे MB/GB. आता विचार करा: प्रत्येक ओळ एक recipe आहे, numbers ची रांग, जी याच एका machine ला शंभर वेगळ्या machines बनवते. WhatsApp ~100 MB ची recipe आहे. तिची प्रत्येक copy फुकट बनते, आणि 3 billion (300 कोटी) लोक चालवतात. आता सूत्र स्वतः लावा: $\text{size} \times \text{scarcity} \times \text{leverage}$.

विचार करा

1. (derivation) Recipe data च्या जागी राहते. मग एक recipe दुसऱ्या recipe ला **वाचू** किंवा **लिहू** शकते, कारण ती फक्त numbers आहे. या गोष्टीमुळे एक खूप मोठी गोष्ट शक्य होते जी तुम्ही रोज वापरता. कुठली?

उत्तर: Update. WhatsApp स्वतःला update करतं तेव्हा एक recipe (updater) दुसऱ्या recipe ला (जुनं WhatsApp) नव्या numbers नी बदलत असते. Machine ला हात न लावता, जगभरच्या machines रातोरात नवं काम शिकतात. आणि याच दरवाजाने virus पण घुसतो: तोही एक recipe आहे जी दुसऱ्या recipes ना लिहिण्याचा प्रयत्न करते. एकच ताकद, चांगल्या आणि वाईट दोन्ही हातांत. Part 3 मध्ये हिची राखण भेटेल.

SECTION 2: NUMBERS आणि SIZE

पाच chapters. Section 1 ने दाखवलं की machine numbers वर recipe चालवते. आता दोन प्रश्न उरले: जगातल्या गोष्टी (text, photo, आवाज) numbers कशा बनतात, आणि numbers किती जागा घेतात.

हे तुमच्या business decision मध्ये कुठे येईल: प्रत्येक tech product चा खर्च तीन गोष्टींनी बनतो: data किती मोठा, किती वेगवान हवा, किती साठवायचा. हा section तुम्हाला ते माप देतो. ज्या founder ला GB आणि millisecond चा अर्थ माहीत आहे, तो engineer च्या estimate वर प्रश्न विचारू शकतो आणि cloud चं bill वाचू शकतो. जो जाणत नाही, तो अंधारात सही करतो.

Chapter 2.1 [SPINE]: एक गोष्ट किती जागा घेते

कापड meter मध्ये मोजतात, दूध litre मध्ये, आणि digital गोष्टी? नावं तुम्ही ऐकली आहेत: MB, GB. आता ती कमावण्यालायक समजून घेऊ.

मापाचं एकक तेच आहे जे Chapter 1.4 मध्ये बनलं: **bit**, एका switch ची स्थिती. आठ bits चा गट **byte**. एका byte मध्ये 256 स्थिती मावतात; एक अक्षर ठेवायला एवढं पुरे. मग एक ढोबळ गोष्ट पकडा:

1 byte = साधारण 1 अक्षर.

आता सरळ मोजणी, प्रत्येक पायरीवर हजाराचा गुणाकार:

1 KB (kilobyte)	= ~1 हजार bytes	= एक परिच्छेद
1 MB (megabyte)	= ~1 million bytes	= एका जाड पुस्तकाचा text
1 GB (gigabyte)	= ~1 billion bytes	= 1,000 जाड पुस्तकं
1 TB (terabyte)	= ~1 trillion bytes	= एक छोटी library

आता खऱ्या जगाचे anchors; हे पाठ करण्यालायक आहेत, कारण प्रत्येक बोलण्यात येतील:

WhatsApp message (फक्त text)	~ 100 bytes
एक photo (phone camera)	~ 3 MB
एक गाणं	~ 5 MB
एक तास video call	~ 500 MB - 1 GB
एक movie (चांगली quality)	~ 2-4 GB
तुमच्या phone चा पूर्ण storage	~ 128-256 GB

एक pattern दिसतोय? Text चिमूटभर आहे. Photo त्याच्या हजारपट. Video photo च्याही शंभरपट. **Text : photo : video = 1 : हजार : लाख.** हे प्रमाण या पूर्ण पुस्तकातला सगळ्यात कामाचा आकडा आहे, कारण जिथे size आहे तिथे खर्च आहे.

असं का? Text मध्ये प्रत्येक अक्षर एक byte. Photo मध्ये प्रत्येक ठिपक्याचे (pixel) तीन bytes, आणि ठिपके millions मध्ये. Video मध्ये प्रत्येक second ला 30 photos. मोजणी स्वतःच गुणाकार होत जाते. बाजाराच्या भाषेत: चिठ्ठी, पोतं आणि truck. चिठ्ठी खिशात मावते, पोत्याला हमाल लागतो, truck ला रस्ता आणि diesel.

एक बारीक गोष्ट, जिच्यात दुकानदाराची चलाखी लपली आहे: internet चा वेग **bits** मध्ये मोजला जातो (Mbps = megabits per second), storage **bytes** मध्ये (MB). Bit हा byte चा आठवा हिस्सा. म्हणून “100 Mbps” चं connection एका second मध्ये फक्त 12.5 MB आणतं. Telecom companies bits मध्ये बोलतात कारण आकडा आठपट मोठा दिसतो. आता तुम्ही ही चाल पकडू शकता.

आणि हे कमाईशी असं जोडलेलं आहे: WhatsApp वर पाठवलेला photo हलका का होतो? कारण WhatsApp तो पाठवण्याआधी दाबतं (compress): 3 MB वरून ~100 KB, तीसपट छोटा. रोज billions photos जातात; full size गेले असते तर WhatsApp चा data खर्च तीसपट असता, आणि तुमचा data pack पण. एका compression recipe ने company चे billions वाचवले. Size ची व्यवस्था हाच पैसा आहे.

इथे लोक काय चुकीचं समजतात

लोक KB, MB, GB ला नुसतं “छोटा, मोठा, अजून मोठा” समजतात. नाही: प्रत्येक पायरी **हजाराचा** गुणाकार आहे. MB आणि GB मधला फरक तोच आहे जो 1 रुपया आणि 1,000 रुपयांमधला. Engineer म्हणतो “ही file GB मध्ये आहे,” तेव्हा तो म्हणतोय “ही MB वालीपेक्षा हजारपट जड आहे,” आणि तिचा खर्चही तसाच असेल.

MAP वर

रुपयाचा रस्ता: तुम्ही Google ला storage चं भाडं देता (Google One: 100 GB साठी महिना ~130 रुपये). Google तो data ज्यांच्या machines वर ठेवतो त्यांच्या disks Samsung/Seagate कडून येतात. खाली उतरत जा, एक गरज मिळेल: “माझ्या आठवणी हरवू नयेत” (Level 1). त्या गरजेचा पूर्ण stack: Photos app (L2 सेवा), cloud storage (L3), disk factory (L3 चा hardware कोपरा). प्रत्येक GB भाडं देतोय.

स्वतः बघा (5 मिनिटं)

Phone मध्ये एक photo उघडा, त्याची details/info बघा (size मिळेल, ~2-4 MB). आता एका text message चा size आठवा (~100 bytes). आता स्वतः मोजा: एक photo किती messages च्या बरोबर आहे? (~30,000.) आता समजेल की data pack video बघण्यानेच का संपतो, text ने कधीच नाही.

विचार करा

1. (derivation) दोन startup कल्पना: एकीत लोक 200 शब्दांचे reviews लिहितात, दुसरीत लोक 1 मिनिटाचे video reviews बनवतात. दोन्हीकडे 1 million users. फक्त size च्या ज्ञानाने सांगा: कोणाचा खर्च किती जास्त, आणि याचा business वर काय परिणाम?

उत्तर: Text review ~1 KB, video review ~50-100 MB: फरक जवळजवळ **लाखपट**. मग video वाल्याचा साठवण्याचा आणि पाठवण्याचा खर्चही हजारो-लाखोपट. परिणाम: text वालं app फुकट users वरही जगू शकतं; video वाल्याला पहिल्या दिवसापासून पैशाचा रस्ता लागतो (ads, subscription), नाहीतर प्रत्येक नवा user त्याला गरीब करतो. Product चा आकार त्याच्या business model चा आकार ठरवतो. हे सूत्र आता तुमच्याकडे आहे; कुठल्याही कल्पनेवर लावा.

Chapter 2.2 [SPINE]: Text, photo, आवाज, video: सगळे numbers

Machine ला फक्त numbers कळतात. मग तुमचा लग्नाचा photo, आईच्या आवाजाची recording, या पुस्तकाचा text, हे सगळं machine मध्ये कसं घुसलं? प्रत्येकाचं उत्तर एकाच चालीने निघेल. चाल ही आहे:

गोष्टीचे छोटे-छोटे तुकडे करा, प्रत्येक तुकड्याला एक number द्या, आणि numbers चा अर्थ आधीच सगळ्यांनी मिळून ठरवा.

Pincode ची व्यवस्था आठवा: 400001 म्हणजे Mumbai GPO, हे कुठल्या कागदात लपलेलं सत्य नाही; post खात्याने **ठरवलं** आणि सगळ्यांनी मानलं. आता हीच चाल चारही गोष्टींवर लावा:

Text. तुकडा = अक्षर. सगळ्यांनी मिळून ठरवलं: A = 65, B = 66, space = 32. मग "HI" machine मध्ये 72, 73 आहे. बस. तुमच्या नावाचं spelling numbers ची रांग आहे. या ठरवलेल्या table चं नाव **encoding**; जुनं table **ASCII** (1963), आजचं पूर्ण table **Unicode**, ज्यात जगातली प्रत्येक भाषा आहे: अ = 2309, आणि emoji सुद्धा (हसणारा चेहरा = 128514). तुम्ही वाचताय ती मराठी अक्षरंही याच क्षणी numbers म्हणूनच साठवलेली आहेत.

Photo. तुकडा = ठिपका (**pixel**). Photo ला जाळीत कापा, प्रत्येक चौकोनाचा एक रंग. आणि रंग? प्रत्येक रंग लाल, हिरवा, निळा यांच्या मिश्रणाने बनतो, म्हणून तीन numbers (0-255) प्रत्येक ठिपक्यासाठी: (255, 0, 0) = पूर्ण लाल. 4 million ठिपके x 3 numbers = तुमचा photo. रांगोळीसारखं: दुरून चित्र, जवळून फक्त ठिपके.

आवाज. आवाज म्हणजे हवेचं कंपन. तुकडा = वेळेचा क्षण. प्रत्येक second चे ~44,000 क्षण करा, प्रत्येक क्षणी कंपनाची उंची मोजा, तो number लिहा (एका मापाचं नाव **sample**). Numbers ची रांग परत वाजवा, आवाज परत. आईचा आवाज म्हणजे 44,000 numbers प्रति second.

Video. तुकडा = **frame**. प्रत्येक second ला 30 photos, अधिक आवाज. म्हणून video सगळ्यात जड: photo चा खर्च x 30 x seconds.

तीन पावलांमध्ये सगळ्यात महत्वाचं कुठलं? कापणं? मोजणं? नाही: "अर्थ आधीच सगळ्यांनी मिळून ठरवा." 72 चा अर्थ H तेव्हाच आहे जेव्हा पाठवणारा आणि घेणारा दोघे एकच table मानतात. जुन्या websites वर मराठी लिहिल्यावर ??? किंवा चौकोन दिसायचे, ते हेच: numbers पोहोचले, पण त्या

बाजूचं software Unicode चा मराठी भाग जाणत नव्हतं. Numbers बरोबर, अर्थ हरवला. ही गोष्ट, की **numbers चा अर्थ करार असतो, numbers मध्ये नसतो**, पुढे internet (Part 3) आणि AI (Book 2) दोन्हीचा पाया आहे.

इथे लोक काय चुकीचं समजतात

लोकांना वाटतं machine photo “बघते” किंवा गाणं “ऐकते”. नाही. Machine साठी photo म्हणजे 12 million numbers आणि गाणं म्हणजे 15 million. तिला ना रंग माहीत ना राग. आणि तरीही (हे पुढे Book 2 चं बीज आहे) numbers च्या pattern वरून machine सांगते की photo त मांजर आहे. बघणं नाही, मोजणीतून ओळख. जेव्हा ती गोष्ट येईल, तेव्हा हा chapter आठवा: सगळं numbers च होतं, नेहमी.

MAP वर

Company case: **JioCinema/Hotstar** cricket दाखवतात तेव्हा 40 million लोक एकाच वेळेस frames चे numbers मागत असतात. त्यांचा पूर्ण engineering खर्च याच chapter च्या प्रमाणातून येतो: video = सगळ्यात जड numbers. आणि त्यांचं business model पण तेच ठरवतं: ads आणि subscription गरजेचे, कारण प्रत्येक प्रेक्षकाचा data खर्च खरा आहे. Level 1 गरज: करमणूक. तिचे numbers: सगळ्यात महाग.

स्वतः बघा (5 मिनिटं)

कोणाला तरी हा message पाठवा: “72 69 76 76 79”. आणि सांगा: प्रत्येक number मधून 64 वजा करा आणि A=1, B=2 च्या मोजणीने अक्षर बनवा. उत्तर येईल HELLO. तुम्ही आत्ता स्वतःचं encoding table बनवलं आणि एका माणसाकडून decode करून घेतलं. Machine बस हेच करते, billion पट वेगाने.

विचार करा

1. (derivation) WhatsApp voice message पाठवताना “पाठवण्याआधी दाबा” (compress) कसं शक्य आहे? आवाज तर मोजलेलाच होता, 44,000 numbers प्रति second. कुठले numbers फेकता येतात?

उत्तर: ते, जे माणसाचे कान तसेही पकडत नाहीत. खूप उंच कंपन, मोठ्या आवाजामागे दबलेला हलका आवाज, दोन क्षणांमधला फरक जो अंदाजाने परत बनवता येतो. Recipe ते फेकून देते, size दहापट घटतो, कानाला फरक पडत नाही. धडा: compression म्हणजे “अर्थ वाचवा, numbers फेका.” आणि लक्ष द्या: काय फेकायचं हा एक **निर्णय** आहे, आणि प्रत्येक निर्णय कोणा माणसाच्या recipe त लिहिलेला आहे. Machine मध्ये सगळीकडे असे लपलेले निर्णय भेटतील.

Chapter 2.3 [SPINE]: मोठ्या आकड्यांचा अर्थ

या पुस्तकात आतापर्यंत आले: 15 billion transistors, 44,000 samples प्रति second, billion पावलं प्रति second. पुढे अजून मोठे येतील. Problem ही आहे की माणसाचं डोकं लाखाच्या पुढचे सगळे आकडे एकसारखे “खूप मोठे” मानतं. ते चालणार नाही, कारण या आकड्यांमधल्या **फरकातच** सगळी समज आणि सगळा पैसा आहे.

आधी भाषांतराचं table, कारण India मोजणी आपली बोलतो आणि technology इंग्रजी:

1 million	(M)	= 10 लाख
10 million		= 1 कोटी
1 billion	(B)	= 100 कोटी = 1 अब्ज
1 trillion	(T)	= 1 लाख कोटी

आता फरक जाणवून घ्या, घड्याळाने. एका second ला एक अशी मोजणी करा:

1 million पर्यंत मोजायला	~ 11 दिवस लागतील
1 billion पर्यंत	~ 31 वर्ष
1 trillion पर्यंत	~ 31,000 वर्ष

Million आणि billion “जवळजवळ” नाहीत. त्यांच्यामध्ये हजारपट अंतराचा समुद्र आहे. बातमी म्हणते “company X billion dollars ची झाली,” तेव्हा आता तुमच्याकडे मोजपट्टी आहे.

आता उलट्या दिशेला, छोट्या वेळाची मोजणी, कारण machine तिथे जिंकते:

1 second	तुमची एक पापणी लवते
1 millisecond (1/1000 sec)	चांगल्या internet चा एक फेरा
1 microsecond (1/1 million)	machine ने memory तून एक गोष्ट उचलणं
1 nanosecond (1/1 billion)	CPU चं एक पाऊल

एक जाड ओळ काढा: **CPU च्या एका पावलात (nanosecond) आणि internet च्या एका फेऱ्यात (millisecond) 1 million पटीचा फरक आहे.** म्हणजे एक message शहरापलीकडे जाऊन परत येईपर्यंत CPU 1 million पावलं चालून झालेला असतो. ढाब्याची उपमा: cook च्या हातात मीठ शेजारीच आहे (nanosecond), पण कांदे संपले तर मंडईची फेरी (millisecond) म्हणजे अर्धा दिवस. हुशार cook काय करतो? मंडईला रोज शंभर फेऱ्या मारत नाही; सकाळी एकदाच सगळं आणून ठेवतो. Machine चं

पूर्ण design याच शहाणपणावर चालतं: **बाहेर जाऊ नका, जोपर्यंत गरज नाही.** ही एक ओळ Part 3, 4, 5 मध्ये पुन्हा पुन्हा कामी येईल. (या फेऱ्याच्या वेळेचं नाव **latency**; हा शब्द पुढे पूर्ण chapter आणेल.)

आणि हा वेळ थेट पैसा आहे: **Google** ने मोजलं की search 400 milliseconds ने हळू झाली तर लोक कमी search करतात. अर्ध्या पापणीपेक्षा कमी वेळ, आणि करोडोंचा फरक. **Amazon** ची मोजणी: प्रत्येक 100 milliseconds ची उशीर = ~1% कमी विक्री. म्हणून वेग ही engineering ची हौस नाही, थेट पैसा आहे. जी company milliseconds मोजू शकते, ती ते रुपयांत बदलू शकते.

इथे लोक काय चुकीचं समजतात

”Computer वेगवान आहे, बस.” नाही: computer **कुठे** वेगवान आहे, हे जाणणं हीच खरी विद्या. आत nanosecond, बाहेर millisecond, 1 million पटीचा फरक. जो हे जाणत नाही, तो असं product बनवतो जे प्रत्येक पावलावर internet ला विचारतं, आणि मग चकित होतो की सगळं हळू का आहे. वेगवान machine वरही हळू design शक्य आहे, आणि सर्रास आहे.

MAP वर

या chapter मधून कोण कमावतं: जो scale वाचू शकतो. Investor साठी “1 million users” आणि “10 million users” मध्ये दहापट फरक आहे; जो founder दोन्हींना “खूप users” म्हणतो, त्याच्यापासून पैसा दूर राहतो. आणि जो engineer millisecond वाचवू शकतो, तो Google/Amazon सारख्या business मध्ये थेट revenue चा काटा फिरवतो; म्हणून अशा skill चा पगार कोटी पार जातो. आकडे जाणवणं ही Level 2, 3, 4 तिन्हींची सामायिक भाषा आहे.

स्वतः बघा (5 मिनिटं)

Phone वर stopwatch लावा आणि 100 पर्यंत मोठ्याने मोजा (~100 seconds). आता हिशोब: 1 million पर्यंत असं मोजायला किती दिवस? (~11 दिवस, न झोपता.) 1 billion पर्यंत? (~31 वर्ष.) एकदा स्वतः हिशोब केलात की billion हा शब्द कायमचा जड होईल, जसा त्याने असायला हवं.

विचार करा

1. (derivation) तुमच्या phone ची chip 15 billion transistors ची आहे. प्रत्येक transistor तांदळाचा एक दाणा असता, तर किती तांदूळ झाला असता? (एका किलोत ~50,000 दाणे.) आणि यातून समजून घ्या: chip बनवणारी factory जगात तीन-चारच का आहेत?

उत्तर: $15 \text{ billion} / 50,000 = 3 \text{ लाख किलो}$, म्हणजे 300 tonnes तांदूळ, पंचवीस trucks भरून. एवढ्या गोष्टी नखाएवढ्या जागेवर, एकही चुकीची न लागता, छापायच्या आहेत. म्हणून ही जगातली सगळ्यात अवघड manufacturing आहे: एक factory उभारायचा खर्च ~20 billion dollars (1.7 लाख कोटी रुपये), आणि हुनर फक्त TSMC (Taiwan), Samsung (Korea), Intel (US) कडे. Scarcity इतकी उंच की देश यासाठी युद्धाची तयारी करतात. आता समजेल “chip war” बातम्यांमध्ये का असतं.

Chapter 2.4 [DEPTH]: एक instruction चालते कशी

(DEPTH chapter. वगळू शकता, पण हा छोटा आहे आणि यानंतर “processor” हा शब्द कायमचा उघडतो.)

Chapter 1.6 मधून: recipe numbers बनून memory त ठेवली आहे. Machine ती चालवते कशी? पूर्ण रहस्य एका मोजणाऱ्या खणात आहे.

CPU च्या आत एक छोटा खण आहे ज्यात फक्त एक number असतो: **”आता मी recipe च्या पाऊल क्रमांक ___ वर आहे.”** या खणाचं नाव **program counter**. ढाब्याचा cook आठवा, ज्याचं बोट recipe च्या कागदावर असतं: बोट ज्या ओळीवर, तेच काम चालू आणि CPU रात्रंदिवस फक्त हे तीन काम गोल-गोल करतो:

1. उचला (fetch): खणात जो number आहे, memory च्या त्या जागेवरून पाऊल उचला. समजा तिथे लिहिलंय: “3” (म्हणजे: जोडा).

2. समजा (decode): Number 3 चा अर्थ काय? Chapter 1.5 वाली gates ची साखळी number वाचून बरोबर भाग निवडते: जोडणारा डबा चालू.

3. करा (execute): जोडा. उत्तर एका खणात ठेवा. आणि मोजणाऱ्या खणात 1 वाढवा, म्हणजे पुढचा फेरा पुढचं पाऊल उचलेल.

बस. या तीन पावलांचं चक्र (**fetch-decode-execute cycle**) म्हणजेच “computer चालू आहे” चा पूर्ण अर्थ. तुमच्या phone मध्ये हे चक्र एका second ला ~3 billion वेळा फिरतं. “3 GHz processor” चा अर्थ हाच: GHz = billion फेरे प्रति second.

एक प्रश्न स्वतःला विचारा: “जर असं तर तिकडे जा” वालं पाऊल कसं चालेल? सोपं: ते पाऊल फक्त मोजणाऱ्या खणात **नवा number** लिहितं. आता पुढचा fetch तिथूनच उचलेल. Recipe ने उडी मारली. Loop पण हेच आहे: मागच्या number वर उडी, पुन्हा पुन्हा. Cook चं बोट कागदावर मागे सरकलं, बस. Programming चं पूर्ण नियंत्रण याच एका खणातून होतं.

आता रोजची एक गोष्ट या नजरेने बघा: phone कधीकधी “hang” होतो. आत काय झालं: program counter अशा जागी अडकला जिथे recipe गोल-गोल फिरते आहे आणि बाहेर पडायची अट कधीच खरी होत नाही (infinite loop), किंवा अशा number वर गेला जिथे recipe नव्हतीच. Machine बिघडली नाही; recipe ने तिला खड्ड्यात फिरवलं. Restart काय करतो? Program counter सकट सगळं सुरुवातीच्या जागेवर. म्हणून restart एवढे आजार बरे करतो.

इथे लोक काय चुकीचं समजतात

”जास्त GHz = वेगवान computer,” बस एवढंच. अर्धवट आहे. GHz फक्त चक्राचा वेग; काम किती झालं हे यावरही आहे की प्रत्येक फेऱ्यात किती होतं, पावलं किती हुशार आहेत, आणि memory किती वेगाने पावलं पुरवते आहे (पुढचा chapter). दोन processors एकाच GHz वर दुप्पट वेगळं काम करू शकतात. Truck आणि गाडीचा rpm सारखा असू शकतो; ओझं किती नेलं हे rpm सांगत नाही. दुकानदार तुम्हाला एक आकडा विकतोय; machine अनेक आकड्यांचा खेळ आहे.

MAP वर

रुपयाचा रस्ता: Apple आपल्या chips स्वतः design करते (M-series), म्हणजे प्रत्येक फेऱ्यात जास्त काम व्हावं. चांगलं चक्र = कमी battery त जास्त काम = महाग machine विकली जाते. जे चक्र तुम्ही आत्ता 10 मिनिटांत समजलात, त्याच्या बारकाव्यावर जगातल्या सगळ्यात मौल्यवान company चं margin टिकलेलं आहे. Level 3 च्या आतही मजले आहेत, आणि खालचा जेवढा चांगला, वरचं सगळं तेवढं चांगलं.

स्वतः बघा (5 मिनिटं)

ही recipe कागदावर चालवा; तुम्ही स्वतः CPU व्हा. खण K मध्ये 5 आहे. Recipe: पाऊल 1: K मध्ये 3 जोडा. पाऊल 2: जर $K < 15$, पाऊल 1 वर जा. पाऊल 3: थांबा. बोटाने program counter बना आणि चालवा. किती फेरे लागले? (K: 5, 8, 11, 14, 17 -> 4 बेरजा, मग अट खोटी, थांबलात.) तुम्ही आत्ता fetch-decode-execute आणि loop दोन्ही चालवले.

विचार करा

1. (derivation) Program counter मध्ये चुकीचा number आला, अशा जागेचा जिथे recipe नाही, कचरा numbers आहेत. CPU काय करेल? त्याला कसं कळेल की हे “recipe नाही”?

उत्तर: कळणारच नाही; हीच भीतीची गोष्ट आहे. CPU साठी प्रत्येक number पावलासारखाच दिसतो; तो कचऱ्यालाही decode करून चालवायचा प्रयत्न करेल. कधी निरुपयोगी काम होईल, कधी crash. म्हणून भोवतीची व्यवस्था (operating system, Part 2) भिंती बांधते: कुठली recipe कुठे चालू शकते, कुठली memory शिवू शकते. आणि hackers चा अर्धा खेळ हाच आहे: कसंही करून आपले numbers program counter पर्यंत पोहोचवणं. एक छोटा खण, पूर्ण security युद्धाचं मैदान.

Chapter 2.5 [SPINE]: लक्षात ठेवणं आणि विसरणं

एका कोड्याने सुरुवात. Phone वर game खेळत होतात, battery अचानक संपली. Charge करून चालू केला: game जिथे होता तिथे नाही, परत उघडावा लागला; पण तुमचे photos सगळे सुखरूप. एकाच machine मध्ये काही गोष्टी वीज गेल्यावर पुसल्या गेल्या, काही नाही. का?

कारण machine मध्ये स्मरणशक्ती **एक** नाही, **दोन** आहेत, आणि दोघींचा सौदा वेगळा:

वेगवान, विसराळू. Switches पासून बनलेली (Chapter 1.4 वाली). वीज आहे तोवर स्थिती आहे; वीज गेली, सगळं 0. पण वेग? CPU च्या पावलांसोबत धावू शकते. इथे ते राहतं जे **आत्ता** चालू आहे: उघडलेलं app, game ची आत्ताची स्थिती. हिचं नाव **RAM** (नुसतं memory पण म्हणतात).

हळू, पक्की. अशी जागा जिथे bits विजेशिवायही टिकतात. वीज जा किंवा या, data तिथेच. पण हळू: वेगवानपेक्षा शंभर-हजारपट. इथे photos, apps, files राहतात. हिचं नाव **storage** (disk, SSD).

ढाब्याची उपमा सगळं सांगते: **मेज आणि गोदाम.** Cook च्या मेजावर आत्ताच्या order चं सामान असतं: चिरलेला कांदा, हाताशी मसाले. मेज छोटं आणि महाग (जागा मर्यादित), पण हात लगेच पोहोचतो. गोदाम मोठं आणि स्वस्त, पण फेरी लागते. रात्री ढाबा बंद झाला की मेज पुसलं जातं; गोदाम तसंच राहतं.

आता विचारा: दोन्हींचे फायदे असलेली एक गोष्ट का नाही बनवत? वेगवान पण, पक्की पण, स्वस्त पण? Engineers 70 वर्षं प्रयत्न करतायत. Physics चा सौदा आहे: **वेगवान, स्वस्त, पक्की: तिन्हीपैकी दोनच निवडा.** म्हणून machine दोन्ही ठेवते आणि मध्ये data ची ने-आण चालू राहते:

App “उघडणं” = गोदामातून मेजावर recipe copy करणं. म्हणून उघडायला वेळ लागतो. File “save करणं” = मेजावरून गोदामात लिहिणं. म्हणून save चा अर्थ: “आता वीज गेली तरी वाचेल.” Phone “hang” = मेज भरलं, आता machine प्रत्येक पावलाला गोदामाच्या फेऱ्या मारते आहे.

खोक्यावरची ओळ आता पूर्ण उघडते: “8 GB RAM, 128 GB storage” म्हणजे 8 GB चं कामाचं मेज, 128 GB चं पक्कं गोदाम. आणि दुकानदाराची चाल पण दिसते: स्वस्त phone वर तो storage चा मोठा आकडा दाखवतो (128 GB!) आणि RAM लपवतो (4 GB). Phone अडखळतो तो RAM कमी म्हणून, storage मुळे नाही. आता तुम्ही खोका वाचून खरा प्रश्न विचारू शकता: मेज केवढं आहे? हे छोटं ज्ञान हजारो रुपयांच्या चुकीच्या खरेदीपासून वाचवतं.

इथे लोक काय चुकीचं समजतात

”Memory full झाली, जुने photos delete करा.” दोन वेगळ्या गोष्टी मिसळल्या: photos गोदामात (storage) आहेत, hang होणं मेजाचं (RAM) आहे. Photos हटवल्याने hang बरा होत नाही (गोदाम अगदी ठासून भरलं असेल तरच थोडा फरक). आणि उलटंही: “8 GB RAM आहे तर 8 GB ची file आरामात उघडेल” पण सरळ नाही; मेजावर आधीच इतर गोष्टी मांडलेल्या असतात. दोन नावं, दोन कामं; आता कायम वेगळी ठेवा.

MAP वर

Company case: memory chips चा बाजार (Samsung, SK Hynix, Micron) ~150 billion dollars चा आहे, आणि AI ने त्याला आग लावली आहे, कारण AI recipes चं मेज (RAM) राक्षसी आकाराचं लागतं. 2024-25 मध्ये AI च्या memory chips ची (HBM नावाच्या) इतकी टंचाई झाली की भाव आभाळाला गेले. Chapter 0.2 चं सूत्र: scarcity चा काटा. जेव्हा नवी technology येते, तेव्हा तिच्या खालच्या कुठल्यातरी भागाची टंचाई कोणाचं तरी सोनं बनते. हा pattern ओळखणं म्हणजेच संधी ओळखणं.

स्वतः बघा (5 मिनिटं)

एखादं जड app उघडा (game किंवा editor): किती seconds लागले? हा गोदाम-ते-मेज copy चा वेळ होता. आता ते बंद करून **लगेच** परत उघडा: यावेळेस पटकन उघडलं ना? कारण system ने ते मेजाजवळच ठेवलं होतं. तुम्ही आता दोन्ही स्मरणशक्तींमधला फरक स्वतःच्या घड्याळाने मोजला.

विचार करा

1. (derivation) या chapter मधून एका गोष्टीचं उत्तर काढा: “Save” button जगातून हळूहळू गायब का होतोय? (Google Docs मध्ये save button नाहीच.)

उत्तर: आधी save यासाठी होतं की मेजावरून गोदामात लिहिणं महाग काम होतं; मग machine तुम्हाला विचारायची कधी लिहू. आता storage वेगवान आणि स्वस्त झाला, मग software दर काही seconds नी चुपचाप स्वतःच लिहितं (auto-save), आणि internet वरही copy पाठवतं. जेव्हा एखाद्या कामाचा खर्च कोसळतो, तेव्हा त्याच्याशी जोडलेला button पण मरतो. Design ला technology ची economics चालवते; product बनवताना ही गोष्ट नेहमी सोबत ठेवा.

PART 1 चा शेवट: नकाशा आणि पुढचं पाऊल

जे तुमच्याकडे आता आहे

पैसा	विश्वासाची व्यवस्था; फक्त Level 1 वर घुसतो
कर्माईचं सूत्र	size x scarcity x leverage (तास नाहीत)
चार LEVELS	गरज -> business -> technology -> AI; पैसा खालून वर वाहतो
ABSTRACTION	प्रत्येक मजला खालचा लपवतो, आणि लपलेला हिस्साच महाग असतो
MACHINE	billions switches (transistors), design केलेल्या साखळ्यांमध्ये
BIT / BYTE	switch ची स्थिती / 8 switches चा गट
GATES	AND, OR, NOT: यांतूनच बेरीज, तुलना, "जर"
COMPUTER	numbers वर आधीच लिहिलेली recipe, billion पावलं प्रति second, न थकता
UNIVERSAL MACHINE	recipe पण number आहे, म्हणून एक machine सगळं काही: हाच software चा जन्म
सगळं NUMBER	text/photo/आवाज/video: कापा, मोजा, अर्थ आधीच ठरवा (encoding)
माप	KB->MB->GB->TB, प्रत्येक पायरी हजारपट; text : photo : video = 1 : हजार : लाख
आत VS बाहेर	CPU चं पाऊल nanosecond, internet चा फेरा millisecond: 1 million पटीचा फरक
दोन स्मरणशक्ती	RAM (मेज: वेगवान, विसराळू) vs storage (गोदाम: हळू, पक्कं)

एक परीक्षा, स्वतःसाठी

पुस्तक न उघडता, कोणाला तरी (किंवा स्वतःला, बोलून) ही 4 उत्तरं द्या:

1. पैसा फक्त Level 1 वर का घुसतो?
2. Surgeon पेक्षा app वाला जास्त का कमावतो?
3. एकच phone camera पण, bank पण, game पण कसा बनतो?
4. "8 GB RAM, 128 GB storage" चा अर्थ काय?

चारही आली, तर Part 2 साठी तयार आहात. जे अडलं, त्याचा chapter number वरच्या नकाशात मिळेल; तिथे परत जा.

Part 2 मध्ये काय आहे

Machine recipe चालवते, हे कळलं. आता पुढचा प्रश्न: recipe लिहिली कशी जाते? माणसाच्या भाषेपासून machine च्या numbers पर्यंत भाषांतर कोण करतं? Languages इतक्या साऱ्या का आहेत? तो manager कोण जो सगळ्या recipes ना एका machine वर भांडण्यापासून रोखतो (operating system)? आणि software मध्ये bugs असतातच का? Part 2: MACHINE शी बोलणं.

PART 1 चा MINI-GLOSSARY

फक्त ते शब्द जे या part मध्ये आले. प्रत्येकापुढे त्याचा chapter, म्हणजे परत जाणं सोपं.

abstraction	आतली गुंतागुंत झाकून सोपं handle देणं (0.4)
AND / OR / NOT	तीन मूळ gates: दोन्ही/कोणीही/उलट (1.5)
ASCII / Unicode	अक्षरांच्या numbers चा करार, जुना / आजचा (2.2)
binary	दुप्पट-वाली मोजणी, फक्त 0 आणि 1 (1.4)
bit	एका switch ची स्थिती: 0 किंवा 1 (1.4)
byte	8 bits; साधारण एक अक्षर (1.4)
circuit	विजेचा बंद रस्ता (1.1)
CPU / processor	recipe चालवणारा मुख्य भाग (1.3)
double coincidence of wants	दोघांना एकाच वेळेस एकमेकांची गोष्ट हवी (0.1)
encoding	numbers चा अर्थ ठरवणारं table (2.2)
fetch-decode-execute	CPU चं तीन-पावलांचं चक्र: उचला, समजा, करा (2.4)
frame	video चा एक photo; ~30 प्रति second (2.2)
GHz / clock speed	चक्राचा वेग: billion फेरे प्रति second (2.4)
hardware	हाताला लागणारी machine (1.3)
KB / MB / GB / TB	size ची शिडी, प्रत्येक पायरी हजारपट (2.1)
latency	काम सुरू होऊन उत्तर येईपर्यंतचा वेळ (2.3)
ledger	देवाण-घेवाणीची वही; पैसा चालती वही आहे (0.1)
leverage	काम एकदा, फायदा पुन्हा पुन्हा (0.2)
logic gates	switches पासून बनलेले ठरवणारे भाग (1.5)
market size	problem किती लोकांची, किती खोल (0.2)
pixel	photo चा एक ठिपका; तीन numbers चा रंग (2.2)
program	machine साठी लिहिलेली recipe (1.3)
program counter	"आत्ता पाऊल क्रमांक __ वर" वाला खण (2.4)
RAM	वेगवान, विसराळू स्मरणशक्ती; कामाचं मेज (2.5)
sample	आवाजाच्या एका क्षणाचं माप; ~44,000/second (2.2)
scarcity	हे किती कमी लोकांना जमतं (0.2)
software	सगळ्या recipes, मिळून (1.3)
stack	मजल्यांचा मनोरा: कोण कोणावर उभा (0.3)
storage	हळू, पक्की स्मरणशक्ती; गोदाम (2.5)
stored-program computer	recipe ला data च्या जागी ठेवणारं design (1.6)
transistor	विजेने दाबला जाणारा switch (1.1)
universal machine	एक machine जी कुठलीही recipe चालवते (1.6)

THE MACHINE

PART 2 OF 5: MACHINE शी बोलणं

पाच PARTS चा नकाशा

PART 1 पैसा आणि MACHINE [तुम्ही इथे आहात]	झाला
PART 2 MACHINE शी बोलणं	languages, operating system, software बनतं कसं
PART 3 MACHINES चं जाळं	network, address, internet
PART 4 DATA आणि आठवणी	storage, database, शोधणं
PART 5 जगाला SERVE करणं	scale, cloud, खर्च, पूर्ण जोड

आधी Part 1 ची परीक्षा

पाच प्रश्न, Part 1 मधले. आधी स्वतः उत्तर द्या, मग खाली बघा. जिथे अडाल, तिथला chapter number सोबत आहे; तिथे परत जा.

1. पैसा फक्त Level 1 वर का घुसतो? (0.3)
2. कमाईच्या सूत्रात तास का नाहीत? (0.2)
3. "Computer विचार करतो" म्हणणं चुकीचं का आहे? (1.3, 1.5)
4. एकच machine camera, bank आणि game कशी बनते? (1.6)
5. Text पेक्षा video लाखपट जड का आहे? (2.1, 2.2)

उत्तर: 1. कारण गरज फक्त माणसाची असते; Level 2, 3, 4 आपसात जो पैसा देतात, तो सगळा खालून चढत आलेला असतो. 2. कमाई = size x scarcity x leverage; तास विकणं म्हणजे leverage चा सगळ्यात खालचा काटा. 3. Machine आधीच design केलेली gates ची साखळी चालवते; प्रत्येक input चं output आधीच ठरलेलं; surprise फक्त आपल्यासाठी असतं. 4. कारण recipe पण number आहे; नवी recipe घाला, machine तीच, काम नवं. 5. अक्षर 1 byte; photo त millions ठिपके x 3 numbers; video त प्रति second 30 photos. मोजणी स्वतःच गुणाकार होत जाते.

या part मध्ये काय आहे

Part 1 ने दाखवलं की machine numbers वर recipe चालवते. आता दोन प्रश्न उघडायचे आहेत:

Section 3: Recipe लिहिली कशी जाते? माणसाच्या शब्दांपासून machine च्या numbers पर्यंत भाषांतर कोण करतं? Languages इतक्या सान्या का? आणि तो manager कोण जो हजार recipes ना एका machine वर भांडू देत नाही?

Section 4: Software बनतं कसं? Bugs असतातच का? बदलांची नोंद कशी राहते? आणि engineer चा estimate नेहमी चुकीचा का असतो?

शेवटचा chapter तुमच्यासाठी सगळ्यात महाग आहे: जेव्हा तुम्ही कोणाकडून software बनवून घ्याल, तेव्हा हा part सांगेल की काय normal आहे, काय हलगर्जीपणा, आणि पैसा कशाचा द्यायचा.

SECTION 3: MACHINE ला सांगणं की काय करायचं

सात chapters. माणसाच्या विचारापासून machine च्या numbers पर्यंतचा पूर्ण रस्ता: languages, भाषांतर करणारे, आणि operating system.

हे तुमच्या business decision मध्ये कुठे येईल: “कुठल्या language मध्ये बनवायचं,” “कुठला platform धरायचा,” “हे फुकट tool भरवशाचं आहे का?” हे सगळे founder चे निर्णय आहेत, engineer चे नाही, कारण यांतून hiring, खर्च आणि वेग ठरतो. या section नंतर तुम्ही हे निर्णय समजून घेऊ शकाल, ऐकून नाही.

Chapter 3.1 [SPINE]: भाषांतराची problem

Part 1 चं शेवटचं सत्य हे होतं: CPU ला फक्त numbers वाली पावलं समजतात. 3 म्हणजे जोडा, 7 म्हणजे उडी. बस. या numbers वाल्या भाषेचं नाव **machine code**.

मग 1940s च्या पहिल्या programmers नी काय केलं? खरंच numbers च लिहिले. कागदावर पावलं विचार करायचे, मग प्रत्येक पावलाचा number ठरवायचे, मग switches आणि cards ने machine मध्ये भरायचे. एका छोट्या recipe ला आठवडा लागायचा, आणि एक number चुकला तर सगळा कचरा; आणि चूक **शोधणं** लिहिण्यापेक्षाही अवघड.

आता तुम्ही 1950 चे engineer आहात. हा त्रास रोज सहन करताय. काय कराल?

पहिलं पाऊल स्वतःच दिसतं: numbers ना **नावं** द्या. “3” ऐवजी लिहा “ADD”, “7” ऐवजी “JUMP”. माणूस नाव वाचू शकतो, चूक दिसते. पण machine ला तर नावं समजत नाहीत... मग एक छोटा program लिहा जो नावाला परत number बनवेल. हे भाषांतर इतकं सरळ आहे (प्रत्येक नाव = एक number) की program सोपा होता. या नावांवाल्या भाषेचं नाव **assembly**.

थांबा. इथे जे झालं ते नीट बघा, कारण हा पूर्ण software दुनियेचा जन्म आहे: **भाषांतराचं काम स्वतःच एक recipe बनलं**. Machine आपल्याच भाषेचा पूल स्वतः बांधते.

आता पुढचा त्रास. नावं numbers पेक्षा बरी, पण विचार अजूनही machine च्या पातळीवर करावा लागतो: “हे उचला, तिथे ठेवा, जोडा, उडी मारा.” माणूस असा विचार करत नाही. माणूस विचार करतो: “प्रत्येक ग्राहकाचं bill जोडा आणि सगळ्यात मोठं दाखवा.” एक ओळ, जिच्यात machine ची शंभर पावलं लपली आहेत.

मग स्वप्न बनलं: अशी भाषा जी माणसाच्या विचाराच्या **जवळ** असेल, आणि एक जाडजूड भाषांतरकार जो त्या एका ओळीला machine च्या शंभर पावलांमध्ये उघडेल. हा भाषांतरकार आता सरळ नाव-बदल नाही; हे खरं भाषांतर आहे: समजणं, तोडणं, जुळवणं. अवघड program आहे. पण एकदा बनला (leverage आठवतंय?), की प्रत्येक programmer कायमचा माणसाच्या पातळीवर लिहू शकतो. अशा भाषांना **programming languages** म्हणतात (Python, Java, C: पुढच्या chapters मध्ये भेटतील).

हे बनलं (1957 पासून), आणि तेव्हापासून पूर्ण industry याच शिडीवर चढते आहे: machine पासून दूर, माणसाच्या जवळ. Post office ची शिडी आठवा: तुम्ही फक्त पत्ता लिहिता; postman, sorting केंद्र, गाडी, हवाई थैली, हे सगळं खाली आपापलं काम करतं. तुम्हाला त्यातलं काही दिसत नाही. आणि शिडीचं

शेवटचं पाऊल तुम्ही रोज बघता: तुम्ही Claude ला **मराठीत** सांगता आणि code बनतो. ते याच 70 वर्ष जुन्या शिडीचं सगळ्यात नवं पाऊल आहे.

इथे लोक काय चुकीचं समजतात

”Computer आता English समजायला लागला आहे.” नाही. CPU आजही **तेच** numbers समजतो जे 1950 मध्ये समजायचा. बदललं ते **भाषांतर**: मधले मजले इतके चांगले झाले की माणसाला machine दिसणं बंद झालं. AI तुमच्या मराठीतून code बनवतो, तेव्हाही शेवटी तेच 3 आणि 7 वाले numbers चालतात. शिडी लांब झाली; जमीन तीच.

MAP वर

कोण कमावतं: प्रत्येक मजल्यावर वेगळे लोक. Machine code च्या जवळ जगातले सगळ्यात कमी लोक (chip companies मध्ये, प्रति माणूस सगळ्यात उंच पगार). वरच्या मजल्यांवर crores लोक (सामान्य software jobs). आणि आता सगळ्यात वरचा मजला (AI कडून code बनवून घेणं) सगळ्यात नवा आहे; तिथे गर्दी अजून कमी आहे आणि नियम बनत आहेत. नवं पाऊल उघडताच आधी चढणारे सगळ्यात जास्त कमावतात; प्रत्येक वेळेस हेच झालं आहे.

स्वतः बघा (5 मिनिटं)

Laptop वर कुठलीही website उघडा, रिकाम्या जागेवर right-click करा, “View Page Source” दाबा. जे दिसलं, ती त्या page ची recipe आहे (HTML). घाबरू नका, वाचायचं नाही. फक्त हे बघा की प्रत्येक सुंदर page च्या खाली हा text बसलेला आहे, आणि हा text ही शेवटी numbers बनूनच चालला. पडद्यामागे डोकावण्याची सवय लावून घ्या.

विचार करा

1. (derivation) Machine ला सरळ मराठी किंवा English का नाही शिकवत, मधल्या भाषा काढून? अडचण काय आहे?

उत्तर: माणसाच्या भाषेत अर्थ अर्धा बोललेला असतो, अर्धा समजलेला. “थोडं मीठ टाका” मध्ये किती? Machine ला प्रत्येक पाऊल **exact** लागतं, कारण तिच्याकडे “समजून घे ना” वाला mode नाही (Chapter 1.5: सगळं आधीच ठरलेलं). म्हणून मधली भाषा लागते जिच्यात अर्थाचं एकच रूप असतं. AI इथे पहिला खरा प्रयत्न आहे: तो तुमच्या अर्धवट बोलण्याचा अर्थ **अंदाजाने** काढतो, मग exact code बनवतो. पण अंदाज कधीकधी चुकतो; म्हणून AI च्या जमान्यातही शेवटची जबाबदारी तीच आहे: exact काय हवंय, हे **तुम्हाला** माहीत पाहिजे. म्हणूनच तुम्ही हे पुस्तक वाचताय.

Chapter 3.2 [SPINE]: जवळजवळ-English मध्ये लिहिणं

आता ती गोष्ट समोरून बघून घ्या जिला जग एवढं घाबरतं: code. हा घ्या एक पूर्ण, खरा program (Python भाषा):

```
price = 300

if price > 500:
    print("mahag aahe")
else:
    print("theek aahe")
```

वाचा. तुम्ही हे **जवळजवळ** English सारखं वाचू शकता: “price नावाच्या खणात 300 ठेवा. जर price 500 पेक्षा मोठा असेल, तर ‘mahag aahe’ दाखवा, नाहीतर ‘theek aahe’ दाखवा.” चालवलात तर screen वर येईल: theek aahe.

बस. हे coding आहे. ना maths चा डोंगर, ना जादू. कडक व्याकरणात लिहिलेली recipe. या लिहिलेल्या recipe चं नाव **code**, आणि कडक व्याकरणाचं नाव **syntax**.

आता एक गोष्ट जी coding बाहेरच्या लोकांना कोणी सांगत नाही: **प्रत्येक program, WhatsApp पासून ChatGPT पर्यंत, फक्त तीन गोष्टींनी बनला आहे:**

- 1. ठेवा:** नावाच्या खणात value ठेवणं (`price = 300`). खणाचं नाव **variable**. खण म्हणजे तेच RAM चं मेज (Chapter 2.5); नाव माणसाच्या सोयीसाठी.
- 2. विचारा:** अटीवर रस्ता निवडणं (`if ... else ...`). अटीचं नाव **condition**. हे तेच “जर” आहे जे gates पासून बनलं होतं (Chapter 1.5), फक्त आता माणसाच्या रूपात.
- 3. पुन्हा करा:** एक काम पुन्हा पुन्हा करणं (**loop**):

```
for grahak in grahak_yadi:
    bill = bill_kadha(grahak)
    print(bill)
```

“प्रत्येक ग्राहकासाठी: bill काढा, दाखवा.” यादीत तीन असोत की 30 million, recipe त्याच तीन ओळी. **हीच** ती जागा जिथे leverage code मध्ये घुसतो: माणूस 30 million वेळा करू शकत नाही, loop करून टाकतो.

ठेवा, विचारा, पुन्हा करा. ढाब्याची पूर्ण व्यवस्था हीच आहे: डब्यांमध्ये सामान (ठेवा), “order मध्ये तिखट कमी लिहिलंय का?” (विचारा), आणि tandoor मध्ये एकामागून एक रोट्या (पुन्हा करा). आता कुठलंही software बघा आणि स्वतःला विचारा: यात काय ठेवलं जातंय, काय विचारलं जातंय, काय पुन्हा-पुन्हा होतंय. Instagram feed: posts च्या यादीवर loop, प्रत्येक post वर “याला दाखवायची का?” वाला if, आणि बघितलेल्यांचा हिशोब variables मध्ये. पूर्ण app तीन गोष्टींचं जाळं.

आणि bank चं खरं उदाहरण: दर महिन्याच्या 1 तारखेला लाखो खात्यांवर interest पडतो. आत काय आहे? एक loop (“प्रत्येक खात्यासाठी”), एक condition (“जर savings account असेल”), आणि काही variables (rate, balance, दिवसांची मोजणी). जो clerk 1960 मध्ये महिना लावायचा (Chapter 1.3), तो आज 20 ओळींचा loop आहे जो रात्री चालतो.

इथे लोक काय चुकीचं समजतात

“Coding साठी maths genius लागतो.” तुम्ही आता एक program वाचला आणि एकाचा अंदाज केला, कुठल्याही maths शिवाय. सामान्य software मध्ये बेरीज-वजाबाकीपेक्षा जास्त maths नसतंच. खरी skill maths नाही, **साफ विचार** आहे: कामाला इतक्या छोट्या, exact पावलांत तोडणं की एक मूर्ख पण वीज-वेगवान नोकर (machine) ती करू शकेल. ज्या माणसाला business ची प्रक्रिया साफ विचार करता येते, त्याला coding ची विचारसरणी आधीच येते; फक्त व्याकरण येत नाही.

MAP वर

रुपयाचा रस्ता: जगात ~30 million लोक code लिहून कमावतात. पण आता खरी गोष्ट: AI नंतर **व्याकरण स्वस्त** होतंय (Claude ला syntax येतं) आणि **साफ विचार महाग** होतोय. ज्या माणसाला माहीत आहे **काय** बनवायचं आणि ते exact पावलांत तोडता येतं, तो आता व्याकरण न शिकताही बनवून घेऊ शकतो. तुमच्या या पुस्तकाची पूर्ण bet याच ओळीवर आहे.

स्वतः बघा (5 मिनिटं)

वरच्या पहिल्या program मध्ये 300 चं 800 करा (डोक्यात). आता काय छापेल? आता 500 चं 800 करा, price 800 च राहू द्या. आता काय? (लक्ष द्या: प्रश्न “मोठा आहे का” आहे, “बरोबर आहे का” नाही.) दोन्ही उत्तरं न अडता दिलीत (पहिलं: mahag aahe; दुसरं: theek aahe, कारण $800 > 800$ खोटं आहे), तर तुम्ही आज code **trace** केला; हीच debugging ची पहिली skill आहे.

विचार करा

1. (derivation) एक bracket किंवा comma सुटला तर पूर्ण program का थांबतो? माणूस तर अर्ध्या चुकीच्या spelling ची चिठ्ठीही वाचतो.

उत्तर: माणूस अर्थाचा अंदाज लावतो कारण त्याच्याकडे जगाची समज आहे. भाषांतरकाराकडे फक्त व्याकरणाचे नियम आहेत; bracket शिवाय त्याला कळतच नाही की “विचारा” कुठे संपलं आणि “करा” कुठे सुरू झालं. आणि अंदाज लावणं **मुद्दाम** ठेवलेलं नाही: bank च्या code मध्ये भाषांतरकाराचा चुकीचा अंदाज करोडोंचा पडू शकतो. बरं आहे तिथेच थांबवा, माणसाला विचारा. कडक व्याकरण शिक्षा नाही; ती exactness ची किंमत आहे, आणि exactness च ती गोष्ट आहे जिच्यामुळे machine वर विश्वास ठेवता येतो.

Chapter 3.3 [DEPTH]: भाषांतरकाराचे दोन प्रकार

(DEPTH chapter. वगळू शकता; परत आलात की “compiled vs interpreted” वाली प्रत्येक tech चर्चा उघडेल.)

एक पुस्तक मराठीत आहे, तुम्हाला ते English वाचकांपर्यंत पोहोचवायचं आहे. दोन रस्ते आहेत, दोन्ही तुम्ही स्वतः विचार करू शकता:

पहिला: पूर्ण पुस्तकाचं भाषांतर आधी करून घ्या. वेळ लागेल, पण त्यानंतर प्रत्येक English वाचक सरळ वाचेल, पूर्ण वेगाने. आणि भाषांतरकार पूर्ण पुस्तक बघतो, म्हणून चुका आधीच सापडतील, छापण्याआधी.

दुसरा: वाचकाशेजारी एक दुभाषी बसवा. सुरुवात लगेच होईल: एक ओळ ऐका, एक ओळ बोला. पण प्रत्येक वाचनाला भाषांतर पुन्हा होईल, म्हणून वेग कमी. आणि चूक तेव्हाच सापडेल जेव्हा ती ओळ येईल; पुस्तकाच्या मध्यात अडकू शकता.

Code चेही हेच दोन रस्ते आहेत:

Compiler = पूर्ण भाषांतर आधी. Code एकदा machine code मध्ये बदलला, आता ती file सरळ CPU वर धावते. वेगवान. Games, browsers, operating systems असे बनतात (भाषा: C, C++, Rust). तयार file चं नाव **binary** किंवा **executable**: .exe वाली file हीच आहे, भाषांतर झालेली recipe.

Interpreter = शेजारी बसलेला दुभाषी. लिहा आणि लगेच चालवा, बदला आणि पुन्हा चालवा. आराम आहे, पण चालताना प्रत्येक ओळीचं भाषांतर होतं, म्हणून हळू (10-100 पट पर्यंत). Python असं आहे.

मग सौदा साफ आहे: **compiler = मेहनत आधी, वेग नंतर. Interpreter = आराम आधी, वेगाची किंमत नंतर.** स्वयंपाकाची उपमा: लग्नाच्या पंगतीसाठी सगळी तयारी आदल्या दिवशी करून ठेवता (चिरून, वाटून, मोजून), मग वाढताना विजेचा वेग. घरच्या रोजच्या जेवणात चालता-चालता ठरवता, आज काय बनवायचं. दोन्ही बरोबर; प्रसंग वेगळे.

कुठला बरोबर? चुकीचा प्रश्न. **कुठल्या कामासाठी**, हा प्रश्न विचारा. Game ला वेग हवा: compiler. रोज बदलणारं छोटं काम: interpreter. (जगात मधले रस्तेही आहेत; Java आणि JavaScript अर्थ आधी, अर्थ चालता-चालता भाषांतर करतात. पण मूळ कल्पना याच दोन.)

Android phone वर कधी update नंतर लिहून येतं “Optimising apps 1 of 132...” ते हेच आहे: phone आपल्या CPU साठी apps चं भाषांतर आधीच करून ठेवतोय, म्हणजे नंतर प्रत्येक app वेगाने उघडेल. तुम्ही compiler ला काम करताना बघितलं आहे; फक्त नाव माहीत नव्हतं.

इथे लोक काय चुकीचं समजतात

”Python हळू आहे, मग AI सारखी जड कामं Python मध्ये का होतात?” चाल अशी आहे: Python फक्त **steering** आहे. AI चा खरा जड हिशोब (numbers चा गुणाकार, billions वेळा) compiled code मध्ये लिहिलेला आहे (C/C++ च्या libraries); Python फक्त त्याला बोलावतो. हळू भाषेतून वेगवान engine चालवलं जातंय. Abstraction पुन्हा: आराम वर, वेग खाली, दोन्ही एकत्र.

MAP वर

रुपयाचा रस्ता: server वर प्रत्येक second च्या CPU चं भाडं लागतं (Part 5 मध्ये पूर्ण हिशोब). हळू code = जास्त seconds = मोठं bill. म्हणून companies मोठ्या झाल्या की आपले सगळ्यात जास्त चालणारे भाग compiled भाषांमध्ये **पुन्हा** लिहितात, फक्त bill घटवण्यासाठी. Founder साठी धडा: सुरुवातीला आरामवाली भाषा (वेगाआधी जिवंत राहणं गरजेचं), scale वर वेगवाली. चुकीच्या वेळी चुकीची निवड, दोन्ही बाजूंनी महाग.

स्वतः बघा (5 मिनिटं)

Laptop वर एक .exe file शोधा (कुठल्याही app च्या folder मध्ये) आणि एक .txt file. .txt उघडा: माणसाला वाचता येतं. .exe ला Notepad मध्ये उघडायचा प्रयत्न करा: विचित्र अक्षरांचा कचरा दिसेल, कारण ते machine code आहे. दोन दुनियांचा फरक स्वतःच्या डोळ्यांनी: एक माणसासाठी लिहिलेली, एक CPU साठी.

विचार करा

1. (derivation) Compiler चुका आधी पकडतो, interpreter चालताना. Bank चं core system कशाने बनावं, आणि दर आठवड्याला बदलणारी नवी कल्पना कशाने? कारणही द्या.

उत्तर: Bank: compiler वाली भाषा. तिथे चूक चालल्या **नंतर** सापडणं करोडोंचं पडू शकतं; आधी पकडणं हाच बचाव, आणि वेगही हवा. नवी कल्पना: interpreter वाली. तिथे सगळ्यात मोठा धोका हळू code नाही, **हळू शिकणं** आहे: दर आठवड्याला बदल करायचाय, लगेच चालवून बघायचाय. सौदा नेहमी तोच: चुकीची किंमत vs बदलाचा वेग. ही एक ओळ कुठल्याही tech निर्णयावर लावा; अर्ध उत्तर मिळेल.

Chapter 3.4 [SPINE]: इतक्या साऱ्या languages का आहेत

Python, Java, C, JavaScript, Swift, Kotlin, Go, Rust... शंभरपेक्षा जास्त आहेत. नव्या माणसाला वाटतं हा वेडेपणा आहे, किंवा fashion. नाही. कारण तेच आहे जे गाड्यांचं आहे.

Truck, bike, car, tractor: सगळे “जाण्याचं साधन” आहेत, तरी चार का? कारण प्रत्येक design एक **सौदा** आहे: truck सामान जास्त, वेग कमी; bike स्वस्त, पावसात भिजाल. एक साधन सगळं best करू शकत नाही, कारण खुब्या एकमेकींशी भांडतात.

Languages चे सौदेही मोजके आहेत, चार मुख्य:

वेग vs आराम. Machine च्या जवळ लिहा (C) तर वेगवान पण प्रत्येक गोष्ट स्वतः सांभाळा; दूर लिहा (Python) तर आराम पण हळू. (मागच्या chapter चा सौदा.)

स्वातंत्र्य vs सुरक्षा. काही भाषा तुम्हाला काहीही करू देतात (C: memory पर्यंत थेट हात); ताकद मिळते, पण चुकीचं दारही उघडं. काही बंधनं घालतात (Rust, Java); चुकांची पूर्ण जातच संपते.

सर्व कामं vs खास काम. काही प्रत्येक कामासाठी (Python, Java). काही एकाच कामासाठी बनल्या, पण त्या कामात राणी: SQL फक्त data ला प्रश्न विचारण्यासाठी (Part 4 मध्ये भेटेल), HTML फक्त page चं रूप सांगण्यासाठी.

जागेचा कब्जा. काही भाषा यासाठी चालतात कारण त्यांचा एका जागेवर कब्जा आहे: browser च्या आत फक्त JavaScript चालते, iPhone app साठी Apple ने Swift ठेवली आहे, Android साठी Kotlin. तिथे काम करायचं तर तीच भाषा.

बस. प्रत्येक language या चार सौद्यांची एक जोडणी आहे. “सगळ्यात चांगली language कुठली” हा प्रश्न “सगळ्यात चांगली गाडी कुठली” सारखाच आहे: काम सांगितल्याशिवाय उत्तर नाहीच.

एका project मध्ये वापरलेल्या सगळ्या भाषा आणि tools च्या जोडणीला **tech stack** म्हणतात (Chapter 0.3 वाला शब्द परत, आता छोट्या आकारात). Zomato च्या आत एकाच वेळेस किमान पाच भाषा जिवंत आहेत: app Kotlin/Swift मध्ये (phones चा कब्जा), मागचे servers Java/Go मध्ये (वेग + भरवसा), data चे प्रश्न SQL मध्ये, AI चं काम Python मध्ये, website JavaScript मध्ये. वेडेपणा नाही: प्रत्येक जागी तीच भाषा जी त्या जागेचा सौदा जिंकते. मंडईसारखं आहे: भाजीवाला हातगाडी वापरतो, घाऊक व्यापारी tempo, आणि बाहेरचा माल train ने येतो; कोणी “एकच साधन” वापरत नाही.

इथे लोक काय चुकीचं समजतात

लोक language ला धर्म बनवतात (“मी Python वाला”). Engineers चे हे वाद खरे असतात, पण तुम्ही founder च्या नजरेने बघा: language ही एक **hiring** decision आहे. Python/JavaScript वाल्यांची गर्दी मोठी (स्वस्त, लवकर मिळतो), Rust वाल्यांची छोटी (महाग, शोधावा लागेल). नव्या startup चा अर्धा निर्णय इथेच होतो, technology च्या आधी.

MAP वर

Scarcity चा उलटा खेळ बघा: COBOL 1959 ची भाषा आहे, कोणी शिकत नाही. पण जगातल्या banks चे जुने core systems तिच्यावरच उभे आहेत, आणि ते बदलणं इतकं धोक्याचं आहे की कोणी बदलत नाही. निकाल: उरलेल्या म्हाताऱ्या COBOL वाल्यांना banks जाड fees देतात. धडा: पैसा नव्यात पण आहे, आणि त्या जुन्यातही ज्याला सगळे सोडून गेले पण जग अजून ज्याच्यावर उभं आहे. Scarcity दोन्ही टोकांना बनते.

स्वतः बघा (5 मिनिटं)

कुठल्याही job site वर (LinkedIn/Naukri) search करा: “Python developer” आणि मग “COBOL developer”. मोजणी बघा, आणि जिथे salary दिसली तिथे तीही. आता जे दिसलं ते Chapter 0.2 च्या सूत्रात ठेवा: मागणी किती, पुरवठा किती. Language चा बाजार तुम्ही आत्ता स्वतः मोजला.

विचार करा

1. (derivation) Browser मध्ये फक्त JavaScript चालते; हा “जागेचा कब्जा” आहे. या कब्जाचा JavaScript वाल्यांच्या कमाईवर आणि गर्दीवर काय परिणाम झाला असेल? सूत्राने विचार करा.

उत्तर: कब्जाने मागणी पक्की केली: प्रत्येक website बनवणाऱ्याला JavaScript लागतेच, म्हणजे कामाचा size खूप मोठा. पण याच कारणाने **crores** लोकांनी ती शिकली, म्हणजे scarcity कोसळली. निकाल: काम सगळ्यात जास्त, सरासरी पगार मधला. उलट, ज्या skill ची मागणी मोठी **आणि** शिकणारे कमी (आज: AI systems नीट चालवणं आणि जोडणं), तिथे दोन्ही काटे वर. गर्दी जिथे पोहोचली तिथला पैसा average झालेला असतो; सूत्र नेहमी पुढची रिकामी जागा शोधायला सांगतं.

Chapter 3.5 [SPINE]: तो manager जो machine वाटतो

आता तुमच्या phone वर काय-काय चालू आहे? WhatsApp, गाणं, background मध्ये email check, screen चा touch, network ची ऐकीव. पण Part 1 मधून तुम्हाला माहीत आहे: CPU एका वेळेस **एका** recipe चं **एक** पाऊल चालवतो. मग हे सगळं **एकाच वेळेस** कसं?

आणि एक खोल अडचणही आहे. Chapter 2.4 च्या प्रश्नात दिसली होती: एका recipe ने दुसऱ्या recipe च्या memory त लिहिलं तर? शंभर recipes, एक मेज (RAM), कुठलीही रोकटोक नाही: एक खराब app पूर्ण phone पाडेल.

मग स्वतः design करा. तुमच्याकडे एक वेगवान CPU आहे आणि शंभर recipes आहेत ज्यांना चालायचं आहे. काय कराल? एका ढाब्याचा विचार करा ज्यात एकच चूल आहे आणि दहा cooks:

पाळ्या वाटा. प्रत्येक recipe ला थोडा वेळ द्या: 10 milliseconds WhatsApp, 10 गाण्याला, 10 screen ला, मग परत. CPU इतका वेगवान आहे (billion पावलं/second) की प्रत्येक recipe ला वाटतं machine फक्त तिचीच आहे. “एकाच वेळेस” हा भास आहे; खरं म्हणजे वीज-वेगाची पाळी-पाळी. (या पाळ्या वाटण्याचं नाव **scheduling**.)

भिंती बांधा. प्रत्येक recipe ला मेजाचा आपला हिस्सा द्या, आणि नियम: आपल्या हिश्याबाहेर हात घातला तर तिथेच मारून टाका (हेच “app crash” आहे). एक app मेलं, बाकीची वाचली.

सामायिक गोष्टींचा दरबान व्हा. Screen एक आहे, speaker एक, network एक, storage एक. Recipes ना थेट हात लावू देणार नाही; त्या विनंती करतील, दरबान करेल. Files ची व्यवस्थाही हाच दरबान ठेवतो (**file system**), आणि प्रत्येक hardware शी बोलणारा त्याचा तुकडा म्हणजे **driver**.

आता बघा तुम्ही काय बनवलंत: एक **मालक** recipe, जी बाकी सगळ्या recipes च्या वर बसते, वेळ वाटते, भिंती ठेवते, दरबानी करते. हेच **operating system (OS)**. Windows, Android, iOS, Linux: हीच चार नावं जगातल्या जवळजवळ प्रत्येक machine चे मालक आहेत.

आणि आता ती गोष्ट जी याची खरी ताकद आहे: app लिहिणारा screen, touch, network, file, **काहीही** थेट शिवत नाही. तो फक्त OS कडे मागतो. म्हणजे app **OS साठी** लिहिली जाते, machine साठी नाही. म्हणूनच iPhone ची app Android वर चालत नाही: machine जवळजवळ सारखीच आहे, **मालक** वेगळा आहे, आणि app मालकाच्या भाषेत लिहिली होती.

इथे लोक काय चुकीचं समजतात

”Android/iOS म्हणजे phone चा brand-type.” नाही, तो मालक आहे, आणि मालक असणं हे जगातलं सगळ्यात मोठं business-पद आहे. जो OS चा मालक, तो ठरवतो की त्या machine वर कुठलं app येईल, कसं येईल, आणि (खरी गोष्ट) पैशाचा किती हिस्सा रस्त्यात कापला जाईल. Apple प्रत्येक app च्या कमाईतले 15-30% घेतो, फक्त दरवाजाचा मालक म्हणून. या पदाचं नाव **platform**, आणि त्या कापलेल्या हिश्याचं टोपणनाव platform tax. जकात-नाक्यासारखं: रस्ता ज्याचा, जकात त्याची.

MAP वर

साखळी बघा: तुम्ही 100 रुपयांची in-app purchase करता (Level 1 पासून पैसा घुसला). App वाल्याला ~70-85 मिळाले, Apple/Google ने 15-30 रस्त्यात घेतले. App वाला आपल्या cloud आणि tools वाल्यांना देतो. OS चा मालक प्रत्येक व्यवहारात बसलेला आहे, ते app न बनवता. म्हणून companies OS/platform ची लढाई मरेपर्यंत लढतात: Microsoft (Windows), Google (Android फुकट वाटला, मालक होण्यासाठी), Apple. जो दरवाजा जिंकला, तो जकात वसूल करतो.

स्वतः बघा (5 मिनिटं)

Phone च्या Settings मध्ये Battery उघडा: कुठल्या app ने किती खाल्लं. मग Apps मध्ये जाऊन एखाद्या app च्या “Permissions” बघा: camera, location, contacts. हे दोन्ही पाने OS च्या manager-वह्या आहेत: कोणाला किती पाळी मिळाली, कोणाला कुठली किल्ली. मालकाचा हिशोब तुम्ही स्वतः वाचू शकता.

विचार करा

1. (derivation) Google, Android बनवायला billions खर्च करून, तो फुकट का वाटतो? (Chapter 0.1 ची विचारसरणी लावा: पैसा कुठे घुसतो, कुठे वाहतो.)

उत्तर: कारण Android विकून कमावणं छोटा खेळ होता; **मालक** होणं मोठा. फुकट OS मुळे जगातले 70% phones Google चे दरवाजे बनले: त्यांची search, त्यांचं Play Store (15-30% जकात), त्यांच्या ads, त्यांचा data. एकदा बनवा, billions phones वर चाले, प्रत्येक phone कमाईचा रस्ता: leverage चा पूर्ण काटा. Founder साठी धडा: कधीकधी product फुकट देणं हाच सगळ्यात मोठा business असतो, **जर** फुकट गोष्ट तुम्हाला दरवाजाचा मालक बनवत असेल. (फुकटचा पूर्ण खेळ Chapter 3.7 मध्ये.)

Chapter 3.6 [DEPTH]: “चालू आहे” चा अर्थ काय

(DEPTH chapter. छोटा आहे. यानंतर “app बंद करा, restart करा” वाल्या प्रत्येक गोष्टीचा खरा अर्थ दिसू लागेल.)

एक प्रश्न जो सरळ वाटतो पण नाही: WhatsApp तुमच्या phone मध्ये “आहे” याचा अर्थ काय? दोन अगदी वेगळ्या गोष्टी आहेत:

Recipe ची file. Storage मध्ये (पक्कं गोदाम) पडलेली, ~100 MB. झोपलेली. वीज गेली तरी काही बिघडत नाही. ही **app** आहे.

चालती copy. तुम्ही icon दाबलात तेव्हा OS ने recipe ला RAM च्या मेजावर उतरवलं, तिचा program counter set केला, तिला पाळी द्यायला सुरुवात केली. आता ती **जिवंत** आहे: तिची स्वतःची memory, तिची आत्ताची अवस्था (कुठली chat उघडी, कुठपर्यंत scroll केलं). हिचं नाव **process**, आणि त्या आत्ताच्या पूर्ण अवस्थेचं नाव **state**. (हा शब्द मोठा आहे; database आणि AI पर्यंत पुन्हा पुन्हा येईल.)

नाटकाची उपमा: script ची वही कपाटात पडलेली असते (file). प्रयोग रंगमंचावर चालू असतो (process). एकाच script चे दोन प्रयोग दोन गावांत एकाच वेळेस चालू शकतात, वेगवेगळ्या प्रेक्षकांसह, वेगवेगळ्या जागी थांबलेले. Script सामायिक; जिवंतपण वेगळं. Laptop वर दोन browser windows = एक program, दोन processes, प्रत्येकीची आपली state. दोन tabs मध्ये दोन वेगळे Gmail खाते उघडतात ते म्हणूनच.

आता रोजची भाषा या नव्या चष्म्याने वाचा:

”App उघडणं” = process जन्मणं (गोदामातून मेजावर). **”App not responding”** = process जिवंत आहे पण कुठल्यातरी कामात अडकली आहे; पाळी मिळते आहे पण उत्तर देत नाहीये. **”Force stop”** = OS process ला मारून टाकतो. File सुखरूप. **”Restart करा, बरं होईल”** = जुनी गोंधळलेली state वाली process मेली, नवी स्वच्छ state ने जन्मली. म्हणून restart एवढे आजार बरे करतो: आजार **state** मध्ये होता, file मध्ये नाही. **”Background apps”** = process जिवंत आहे पण screen वर नाही; OS तिला कमी पाळी देतो, किंवा झोपवून टाकतो.

इथे लोक काय चुकीचं समजतात

”App बंद केलं म्हणजे phone मधून गेलं.” आणि उलटही: “app delete केलं म्हणजे त्याचं सगळं गेलं.” दोन्ही चूक, कारण लोक **file** आणि **process** एक समजतात. बंद करणं process चा शेवट; file राहिली. Delete करणं file चा शेवट; पण app चा data (chats, settings) बहुतेक वेगळ्या जागी असतो, आणि cloud वरही. दोन नावं वेगळी ठेवा, अर्धा गोंधळ संपतो.

MAP वर

रुपयाचा रस्ता: cloud मध्ये तुम्ही **process च्या वेळाचं** भाडं देता. “Serverless” नावाच्या गोष्टीत हिशोबच हा आहे: तुमचा code जितके milliseconds process बनून चालला, तितका पैसा. Process = meter चालू, process संपली = meter बंद. Taxi सारखं: गाडी उभी आहे तर भाडं नाही, चालली तर meter फिरतं. Part 5 मध्ये cloud चं bill वाचाल, तेव्हा प्रत्येक ओळीमागे हाच प्रश्न असेल: किती processes, किती वेळ, केवढ्या मेजासह (RAM).

स्वतः बघा (5 मिनिटं)

Phone वर recent apps चं button दाबा (किंवा laptop वर Task Manager: Ctrl+Shift+Esc). ही जिवंत processes ची यादी आहे. एखाद्या app ला swipe करून मारा, मग पुन्हा उघडा: थोडं हळू उघडलं ना, आणि मागच्या जागेवर नाही? तुम्ही एका process चा पूर्ण जन्म-मृत्यू स्वतः घडवलात: state गेली, file मधून नवा जन्म झाला.

विचार करा

1. (derivation) Game मध्येच बंद पडला (process मेली). परत उघडला तर score जिथल्या तिथे मिळाला. State तर मेली होती; score कसा वाचला? Part 1 च्या कुठल्या chapter ची व्यवस्था आहे ही?

उत्तर: Game ने मरण्याआधी score storage मध्ये लिहिला होता (save), म्हणजे वेगवान-विसराळू RAM मधून हळू-पक्क्या गोदामात (Chapter 2.5). Process ची state नेहमी मरणारी आहे; जे वाचवायचं ते पक्क्या जागी लिहिणं हाच एकमेव रस्ता. याच एका कामाचं मोठं रूप म्हणजे **database**, जो Part 4 चा मुख्य नायक आहे: state ला अशा जागी ठेवणं जिथे process मेल्याने, machine जळाल्याने, काहीही बिघडत नाही.

Chapter 3.7 [SPINE]: फुकट गोष्टी exist का करतात

एक विचित्र यादी बघा: Android फुकट. WhatsApp फुकट. Google search फुकट. Python फुकट. Linux फुकट (आणि जगातले जवळजवळ सगळे servers त्याच्यावर चालतात). हे सगळं बनवायला billions लागले. Business च्या जगात कोणी billions लावून गोष्ट फुकट का वाटेल?

Chapter 1.6 मधून किल्ली उचला: software ची **copy** फुकट असते. मग फुकट वाटणं **शक्य** आहे. पण शक्य असणं कारण नाही. कारण चार आहेत, आणि चारही मध्ये पैसा कुठून तरी दुसरीकडून येत असतो. प्रत्येक फुकट गोष्टीवर हे चार प्रश्न चालवा:

- 1. फुकट गोष्ट कुठल्या दरवाजाची किल्ली आहे का?** Android फुकट, कारण प्रत्येक Android phone Google च्या ads आणि Play Store चा दरवाजा आहे (मागचा chapter). मंडईबाहेर फुकट चव देणारा तेच करतो: चव फुकट, पोतं विकत. या चालीचं business-नाव **loss leader**.
- 2. फुकट अर्ध आहे, पूर्ण विकलं जातं का?** फुकट app, पैशाने extra features. फुकट 15 GB, पैशाने 100. नाव: **freemium**. लाखो फुकट users पैकी 2-5% नी विकत घेतलं तरी business चालतो, कारण (पुन्हा तीच गोष्ट) फुकटवाल्यांना देण्याचा खर्च जवळजवळ शून्य.
- 3. तुम्हीच माल आहात का?** Facebook/Instagram/YouTube फुकट आहेत कारण खरा ग्राहक advertiser आहे, आणि विकली जाणारी गोष्ट तुमचं **लक्ष** आहे. जिथे गोष्ट फुकट आहे आणि विकायला काही दिसत नाही, तिथे बहुतेक तुम्हीच माल असता.
- 4. सगळे मिळून हत्यार बनवतायत का?** हा सगळ्यात वेगळा आहे: **open source**. Linux, Python, हजारो tools: यांचा code उघडा आहे; कोणीही वाचा, सुधारा, फुकट वापरा. का? कारण हत्यार सगळ्यांना लागतं आणि कोणाचा business नाही. Companies आपल्या engineers ना हे सुधारण्याचे **पगार** देतात, कारण त्यांचा पूर्ण business या हत्यारांवर उभा आहे आणि एकट्याने बनवणं महाग पडतं. गावाचा रस्ता सगळे मिळून बांधतात, दुकानं आपापली; रस्ता फुकट म्हणून कोणी गरीब होत नाही, रस्त्यामुळेच सगळी दुकानं चालतात.

खऱ्या जगातली सगळ्यात मोठी case: **Jio** ने 2016 मध्ये data जवळजवळ फुकट केला. तोटा? नाही; चाल क्रमांक 1: आधी पूर्ण देश दरवाजाच्या आत, मग हळूहळू भाव, आणि वर apps/platforms चा महाल. तोच खेळ Google ने Android शी खेळला, Amazon delivery च्या तोट्याने खेळतो. फुकट हे हत्यार आहे, आणि ते तोच चालवू शकतो ज्याच्याकडे लांब श्वास (पैसा) आणि पुढचा नकाशा आहे.

इथे लोक काय चुकीचं समजतात

दोन उलट्या चुका. पहिली: “फुकट आहे म्हणजे भिकार असेल.” Linux हे जगातलं सगळ्यात तपासलेलं software आहे; तुमची bank, तुमचा phone (Android आत Linux च आहे), सगळं त्याच्यावर. दुसरी: “फुकट आहे म्हणजे फुकट आहे.” नाही; किंमत दुसरीकडे आहे: तुमचं लक्ष, तुमचा data, किंवा दरवाजाची जकात. फुकट दिसलं की नेहमी विचारा: या साखळीत पैसा कुठे घुसतोय? (Chapter 0.3 चा प्रश्न; सगळीकडे कामी येतो.)

MAP वर

तुमच्यासाठी याचा सरळ अर्थ: तुमच्या startup चं पूर्ण हत्यार-घर **फुकट** आहे. Linux, Python, VS Code, Git: billions dollars चं tool-stack, किंमत शून्य. पन्नास वर्षांपूर्वी business काढायला factory लागायची; आज Level 3 चा पूर्ण कारखाना फुकट मिळतो, आणि Level 4 (AI) चं भाडं महिन्याला काही हजार. इतिहासात इतकी स्वस्त entry कधीच नव्हती. Scarcity आता हत्यारात नाही; ती **कल्पनेत** आणि **execution** मध्ये सरकली आहे. हत्यार सगळ्यांकडे आहे; चालवता सगळ्यांना येत नाही.

स्वतः बघा (5 मिनिटं)

तुमच्या phone ची कुठलीही 5 फुकट apps घ्या. प्रत्येकीवर चार-प्रश्न test चालवा: दरवाजा? freemium? तुम्ही-माल? open source? प्रत्येक app कुठल्या तरी एका (किंवा दोन) खणात बसेल. जी कुठल्याच खणात बसत नाही, तिच्यापासून सावध: एकतर तुम्हाला साखळी दिसत नाहीये, किंवा ती app असा data विकते आहे जो दिसत नाही.

विचार करा

1. (derivation) WhatsApp ने सुरुवातीला वर्षाला 1 dollar भाव ठेवला होता. मग Facebook ने 19 billion dollars ना विकत घेऊन तोही काढून टाकला. एवढे पैसे देऊन फुकट का केलं? साखळी लिहा.

उत्तर: Facebook चा माल लक्ष आणि data आहे, आणि त्याची सगळ्यात मोठी भीती होती की messaging चा दरवाजा दुसरा कोणी (Google, किंवा स्वतः WhatsApp मोठा होऊन) घेईल. 19 billion ही दरवाजाची किंमत होती; 1 dollar/वर्ष हा अडथळा होता जो प्रत्येक नव्या user ला थांबवायचा, म्हणून काढला: users जेवढे जास्त, जाळं तेवढं मोठं, दुसऱ्याचं घुसणं तेवढं अवघड. (याचं नाव network effect; Part 5 मध्ये पूर्ण भेटेल.) धडा: मोठी company एखाद्या फुकट गोष्टीवर billions उधळते, तेव्हा ती गोष्ट खर्च नाही, **किल्ला** आहे.

SECTION 4: SOFTWARE बनतं कसं

पाच chapters. आतापर्यंत बघितलं की code काय आहे. आता बघायचंय की खरं software **बनतं** कसं: चुका, बदल, आणि नेहमी-चुकणाऱ्या estimates च्या मधून.

हे तुमच्या business decision मध्ये कुठे येईल: एक दिवस तुम्ही कोणाकडून (माणूस किंवा AI कडून) software बनवून घ्याल. त्या दिवसासाठी हा section आहे: bug आल्यावर किती राग योग्य आहे, “झालं तर होतं” चा अर्थ काय, deadline दुप्पट का होते, आणि पैसा कशाचा द्यायचा: वचनाचा की चालत्या तुकड्याचा. जो founder हे जाणत नाही, तो एकतर लुटला जातो किंवा चुकीच्या माणसावर ओरडतो.

Chapter 4.1 [SPINE]: Software मध्ये bugs असतातच का

पूल पडत नाहीत (बहुतेक). घड्याळं चालत राहतात. मग software, जे जगातले सगळ्यात शिकलेले लोक बनवतात, रोज कुठे ना कुठे का तुटतं? हा हलगर्जीपणा आहे का?

नाही. ही **मोजणी** आहे. स्वतः बघा.

Chapter 1.3 आठवा: machine तेच करते जे **लिहिलं** आहे, ते नाही जो तुमचा **अर्थ** होता. मग bug ची व्याख्या एका ओळीत: **जे लिहिलं आणि जे हवं होतं, यांच्यामधली फट**. आता प्रश्न हा की एवढे हुशार लोक फट का सोडतात?

कारण recipe लिहिणाऱ्याला **प्रत्येक** परिस्थितीचं उत्तर आधीच लिहावं लागतं. आणि परिस्थिती मोजणीत bomb सारख्या फुटतात. एक छोटा form विचार करा: नाव, वय, pincode. नाव रिकामं असेल तर? 500 अक्षरांचं असेल? Emoji असेल? वय 0 असेल? Minus 5? 200? Pincode मध्ये अक्षरं असतील? User ने अर्धा भरला आणि network गेलं? दोनदा submit दाबलं? तीन fields चे दहा-दहा प्रकार घेतले तरी हजार जोड्या झाल्या. खऱ्या app मध्ये अशा **लाखो** जोड्या असतात, आणि लिहिणारा माणूस प्रत्येक जोडी डोक्यात खेळवू शकत नाही. जी जोडी त्याच्या डोक्यात कधी आलीच नाही, तीच एका दिवशी कुठल्या user च्या हाती लागते. त्या दिवशी “bug सापडला.” (अशा कोपऱ्यातल्या परिस्थितीचं नाव **edge case**, आणि program चं अचानक मरणं म्हणजे **crash**. Bug शोधून दुरुस्त करणं: **debugging**. नाव कुठून आलं: 1947 मध्ये एक खरा पतंग, moth, machine मध्ये अडकलेला सापडला होता; नाव चिकटलं.)

आता पुलाशी तुलना करा, फरक दिसेल. पुलाकडे परिस्थिती कमी आहेत: गाडी जड किंवा हलकी, वारा जोराचा किंवा हळू, सगळं एकाच physics च्या आत. Software कडे परिस्थितींचा स्फोट आहे, आणि प्रत्येक नवं feature जुन्या सगळ्यांसोबत नव्या जोड्या बनवतं. म्हणून 10 पट मोठं software 10 पट नाही, 100 पट जास्त जोड्यांचं असतं.

मग सत्य हे आहे: **मोठ्या software मध्ये bugs असणं पक्कं आहे; प्रश्न फक्त हा की किती लवकर सापडतात आणि किती नुकसान करण्याआधी पकडले जातात**. चांगली company ती नाही जिचे bugs शून्य आहेत (अशी company नाहीच); चांगली ती जिची पकडण्याची आणि दुरुस्तीची व्यवस्था वेगवान आहे. पुढचा chapter तीच व्यवस्था आहे.

खऱ्या जगात हे असं दिसतं: 31 December च्या रात्री किंवा cricket final च्या वेळेस UPI/payment apps का अडखळतात? कारण “एकाच वेळेस इतके लोक” हीच एक edge case आहे जी सामान्य दिवशी कधी येत नाही. किंवा calendar चे bugs: 29 February दर चार वर्षांनी येतो आणि दर चार

वर्षांनी कुठे ना कुठे कुठलं तरी system पडतं, कारण कुठल्या तरी लिहिणाऱ्याच्या डोक्यात ती जोडी नव्हती.

इथे लोक काय चुकीचं समजतात

”Bug आला म्हणजे engineer भिकार.” कधीकधी खरं, पण बहुतेक नाही. नवा founder पहिल्या bug वर ओरडतो; समजूतदार founder विचारतो: किती वेळात कळलं, किती users पर्यंत पोहोचला, पुन्हा होऊ नये यासाठी काय व्यवस्था बनली? Team चा दर्जा bugs च्या मोजणीने नाही, **उत्तराच्या** वेगाने मोजला जातो. (आणि जो विकणारा म्हणतो “आमच्या software मध्ये bugs नसतात,” तो खोटं बोलतोय किंवा त्याने मोठं software बनवलंच नाही.)

MAP वर

Bug चा स्वतःचा बाजार आहे. Companies आपलं system तोडणाऱ्यांना **बक्षीस** देतात: bug bounty. Google/Apple/Microsoft एका गंभीर bug चे लाखो-कोटी रुपयांपर्यंत देतात, कारण चोरीच्या आधी चौकीदाराला मिळणं स्वस्त पडतं. India तली मुलं bug bounty तून घरबसल्या dollars कमावतात: scarcity (तोडण्याची नजर सगळ्यांकडे नसते) x size (system billions लोकांचं) = बक्षीस मोठं. Level 3 च्या आत हा एक पूर्ण पेशा आहे ज्यात degree कोणी विचारत नाही, फक्त हुनर.

स्वतः बघा (5 मिनिटं)

कुठल्याही app च्या search box मध्ये टाका: काहीच नाही (रिकामा search), फक्त space, एक emoji, 200 अक्षरांचा कचरा. बघा काय होतं. बहुतेक सगळं ठीक असेल (कोणीतरी ही जोडी विचारली होती), कधी विचित्र उत्तर मिळेल (विचारली नव्हती; तुम्ही edge case पकडलात). तुम्ही आत्ता तेच करताय जे tester पगार घेऊन करतो: त्या जोड्या शोधणं ज्या लिहिणाऱ्याच्या डोक्यात आल्या नाहीत.

विचार करा

1. (derivation) दोन features वाल्या app मध्ये features च्या आपसातल्या जोड्या मोजा: 1. आता 10 features वर किती जोड्या? आणि यातून काढा: “बस एक छोटंसं feature अजून टाका ना” ऐकल्यावर engineer चा चेहरा का पडतो?

उत्तर: 2 features = 1 जोडी. 10 features = प्रत्येकीची प्रत्येकीशी = 45. Features 5 पट वाढले, जोड्या 45 पट. प्रत्येक नवं feature जुन्या **सगळ्यांसोबत** नव्या जोड्या आणतं, आणि प्रत्येक जोडी ही नवी जागा आहे जिथे bug लपू शकतो. म्हणून “छोटं feature” छोटं कधीच नसतं: त्याची खरी किंमत त्याच्या जोड्यांमध्ये आहे. Founder साठी सूत्र: feature ची किंमत = बनवायचा वेळ + कायमचं वाढलेलं जोड्यांचं ओझं. म्हणूनच उत्तम products features **टाकल्याने** नाही, feature न टाकण्याच्या हट्टाने बनतात.

Chapter 4.2 [DEPTH]: चूक आधी शोधणं

(DEPTH chapter. याच्याशिवायही गोष्ट पुढे जाईल, पण जो founder “testing” चा अर्थ जाणतो, तो कधी कच्चा माल विकत घेत नाही.)

मागच्या chapter चं सत्य मान्य करा: bugs येणारच. मग आता engineer च्या खुर्चीत बसा आणि विचार करा: user पर्यंत पोहोचण्याआधी ते कसे पकडायचे?

पहिलं उत्तर सगळ्यांचं तेच असतं: “चालवून बघा.” ठीक आहे. चालवलं, form भरला, काम झालं. पाठवायचं?

थांबा. तुम्ही **एक** रस्ता चालवलात. मागच्या chapter च्या हजार जोड्यांपैकी एक. आणि पुढच्या आठवड्यात code बदलले (code रोज बदलतो); मग पुन्हा सगळं हाताने चालवणार? प्रत्येक बदलानंतर, प्रत्येक रस्ता, कायम? माणसाकडून हे होत नाही. आणि जिथे माणसाकडून होत नाही, तिथे काय येतं? (Part 1 चं उत्तर: recipe.)

मग चाल ही आहे: **तपासण्याचं कामही code बनवा**. अजून एक recipe लिहा जी पहिल्या recipe ला चालवते आणि उत्तरं तपासते:

```
tapasa: beriz(2, 2) == 4
tapasa: beriz(-5, 5) == 0
tapasa: bill_kadha(rikami_yadi) == 0
```

अशा शंभर-दोनशे तपासण्या लिहून ठेवा (डोक्यात येईल ती प्रत्येक edge case: रिकामं, शून्य, minus, खूप मोठं). आता प्रत्येक बदलानंतर machine **सगळ्या** तपासण्या seconds मध्ये चालवते. कालचा code आजही बरोबर आहे की कोणी तोडला, याचं उत्तर रोज, फुकट, लगेच. या छोट्या तपासण्यांचं नाव **unit tests**; प्रत्येक बदलावर आपोआप सगळ्या चालण्याच्या व्यवस्थेचं नाव **CI**; आणि नव्या बदलाने जुनी गोष्ट तुटण्याचं नाव **regression** (engineers च्या तोंडी रोज असतो हा शब्द).

हेच testing चं खरं रूप: एकदा लिहा, कायम तपासा. (Leverage, पुन्हा तेच. चांगल्या engineer चं प्रत्येक हत्यार शेवटी leverage चं निघतं.) Hotel च्या kitchen ची उपमा: चांगला ढाबेवाला प्रत्येक ताट पाठवण्याआधी चव घेतो; पण मोठं hotel दर तासाला ठरलेली यादी तपासतं: gas, फ्रिजचं तापमान, तेलाचा वास. यादी एकदा बनते, रोज चालते. तीच यादी software मध्ये code बनून बसते.

पण एक प्रामाणिक गोष्ट: tests लिहिणं वेळ खातं; आजचा वेग कमी करतं, उद्याचा वाचवतं. मग किती लिहायच्या? हा engineering चा नाही, **business** चा प्रश्न आहे: चुकीची किंमत काय? Bank चा पैसे मोजणारा code: चूक करोडोंची; tests वर कंजूषपणा वेडेपणा. नव्या कल्पनेचं पहिलं रूप जे 50 लोक

बघतील: कमी tests, वेग जास्त, चालेल. जाणीवपूर्वक निवडलेला सौदा शहाणपण; न बघता सोडलेली testing हलगर्जीपणा. फरक तोच “जाणीवपूर्वक” आहे.

खऱ्या जगात: UPI किंवा bank चं नवं version सरळ तुमच्यापर्यंत येत नाही. आधी machine वर लाखो नकली व्यवहार चालतात (प्रत्येक edge case: शून्य रुपये, उलटी वेळ, दुहेरी नोंद), मग थोड्या खऱ्या users वर, मग सगळ्यांवर. म्हणून तुमचे पैशाचे apps इतके कमी पडतात: त्यांच्या मागे तपासण्यांची फौज रोज रात्री धावते. जो विश्वास तुम्हाला वाटतो, तो खरं म्हणजे tests ची मोजणी आहे.

इथे लोक काय चुकीचं समजतात

”Testing = launch च्या आधी एकदा सगळं check करणं.” नाही. ते फक्त एका रात्रीचं पहारा आहे. खरी testing **कायमची** आहे: code सोबत तपासण्यांची फौजही वाढत राहते, आणि प्रत्येक बदल त्यांच्यामधून जाऊनच बाहेर पडतो. म्हणून software बनवून घेताना विचारा: “tests किती आहेत, आणि प्रत्येक बदलावर आपोआप चालतात का?” या एका प्रश्नाने अर्थ कच्चे vendors गळून पडतात.

MAP वर

कोण कमावतं: QA/test engineers चा पूर्ण पेशा (India त लाखो लोक), आणि test-automation च्या tools चा आपला बाजार. पण नवं वळण बघा: AI आता tests **लिहिण्यात** सगळ्यात चांगला आहे, कारण edge cases विचार करणं pattern चं काम आहे. मग “हाताने test चालवणारा” पेशा आकसतोय, आणि “तपासण्यांची व्यवस्था design करणारा” महाग होतोय. तीच शिडी, पुन्हा: काम machine कडे, विचार माणसाकडे, पैसा विचारवाल्याकडे.

स्वतः बघा (5 मिनिटं)

Login form साठी 5 अशा तपासण्या कागदावर लिहा ज्या त्याला तोडू शकतील (रिकामा password? Emoji वाला email? 1000 अक्षरं? दोनदा submit? मध्येच net गेलं?). तुम्ही आता test plan लिहिलात, एका ओळीच्या code शिवाय. बनवून घेताना हीच यादी vendor ला द्याल, तर तो समजेल की समोर कच्चा ग्राहक बसलेला नाही.

विचार करा

1. (derivation) समजा tests लिहिणं पूर्ण फुकट झालं (AI लिहून देतो). मग bugs शून्य होतील का?

उत्तर: नाही. Tests फक्त ते तपासतात जे कोणाच्या तरी **डोक्यात** आलं. Bug तिथे असतो जिथे कोणाचंच डोकं गेलं नाही (Chapter 4.1 ची फट). फुकट tests ती फट छोटी करतील, संपवणार नाहीत: जी जोडी सुचलीच नाही, तिची तपासणीही लिहिली जाणार नाही; AI कडूनही तेव्हाच लिहिली जाईल जेव्हा त्याला विचारायचा रस्ता मिळेल. म्हणून zero-bug चं वचन physics नाही, marketing आहे. शक्य असलेलं best: फट छोटी करा, पकडण्याचा वेग वाढवा, नुकसानाची मर्यादा बांधा.

Chapter 4.3 [SPINE]: प्रत्येक बदलाची नोंद

तुम्ही कधी अशा files बघितल्या आहेत? `project_final.docx`, `project_final_v2.docx`, `project_FINAL_kharach.docx`. हसू येतं, पण हा एका खऱ्या problem चा कच्चा इलाज आहे: गोष्ट बदलत राहते, आणि आपल्याला जुनी रूपं लागतात.

Software च्या जगात ही problem राक्षस बनते. Code रोज बदलतो. दहा (किंवा दहा हजार) लोक **एकाच** code वर काम करतात. कालचं version चालत होतं, आजचं तुटलंय: **कोणी, काय, का** बदललं? आणि दोघांनी एकच file एकाच वेळेस बदलली तर कोणाचा बदल राहणार?

स्वतः व्यवस्था design करा. काय-काय लागेल?

1. प्रत्येक बदलाचा photo: काय बदललं, कोणी, कधी, आणि **का** (एका ओळीची नोंद).
2. कुठल्याही जुन्या photo वर परत जाण्याचं button.
3. एकत्र काम: मी माझ्या वेगळ्या copy वर नवं feature बनवीन, तू तुझ्यावर bug दुरुस्त कर, आणि नंतर दोघांची जोडणी होईल.

ही व्यवस्था बनलेली आहे; नाव **version control**, आणि हत्याराचं नाव **Git** (2005; त्याच माणसाने बनवलं ज्याने Linux बनवलं, स्वतःच्याच कामासाठी). त्याची भाषा तीन शब्दांत:

Commit = बदलाचा photo + नोंद. “Payment चा bug दुरुस्त केला” लिहून photo घेतला. इतिहासात कायमचा.

Branch = तुमची वेगळी copy जिथे तुम्ही कोणाला हात न लावता प्रयोग करता. मुख्य copy सुरक्षित राहते.

Merge = branch चं काम परत मुख्य copy त जोडणं. दोघांनी एकच ओळ बदलली असेल तर Git थांबवून विचारतो: कुठली ठेवू? (याचं नाव **merge conflict**.)

आणि **GitHub**? तो या सगळ्याचा अड्डा आहे: internet वर ठेवलेला सामायिक इतिहास, जिथे जगभरच्या teams (आणि open source चं पूर्ण जग) आपला code ठेवतात.

एक गोष्ट नीट समजून घ्या: हे फक्त “backup” नाही. ही **हिमतीची** व्यवस्था आहे. जेव्हा प्रत्येक बदल परत फिरवता येतो, तेव्हा प्रयोगाची भीती संपते. Git शिवाय engineer हात लावायला घाबरतो (“तोडून बसलो तर”); Git सोबत तो तोडतो, शिकतो, परत आणतो. वेगाचं खरं रहस्य safety net आहे. Bank ची जुनी रोजकीर्द (रोजमेळ) हीच शहाणपण होती: प्रत्येक नोंद कायम, खाडाखोड नाही; चूक झाली तर पुढची नोंद तिला उलटवते, पण इतिहास पुसला जात नाही. Git म्हणजे code ची रोजकीर्द.

सगळ्यात जवळचं उदाहरण तुम्ही स्वतः आहात: हे पुस्तक एका Git repo मध्ये बनतंय. प्रत्येक chapter चा प्रत्येक बदल commit आहे, नोंदीसह. जुनं Hinglish version पूर्णच्या पूर्ण archive मध्ये जिवंत आहे, एका command वर परत येऊ शकतं. जी व्यवस्था Microsoft च्या दहा हजार engineers ना सांभाळते, तीच तुमचं पुस्तक सांभाळते आहे, फुकट. (Chapter 3.7: हत्यार सगळ्यांसाठी सारखं फुकट आहे; फरक वापरण्यात आहे.)

इथे लोक काय चुकीचं समजतात

”हे engineers चं आतलं हत्यार आहे; founder ला काय.” चूक. GitHub हा कुठल्याही engineer किंवा team चा X-ray आहे. तिथे दिसतं: काम किती नियमित आहे (commits), नोंदी किती साफ आहेत, खरंच किती लोक काम करतायत. आणि तुम्ही AI कडून code बनवून घ्याल तेव्हा (आजची tools सरळ Git सोबत चालतात), commit इतिहास हाच तुमचा हिशोब आहे: काय बनवून घेतलं, कधी, कुठल्या नोंदीसह. वही वाचता आली पाहिजे, लिहिणारा कोणीही असो.

MAP वर

Company case: Microsoft ने 2018 मध्ये GitHub 7.5 billion dollars ना विकत घेतला. एका “code ठेवण्याच्या जागे”साठी एवढं का? कारण तिथे जगभरच्या engineers चं काम, सवयी आणि येणारे projects दिसतात: हा developers चा दरवाजा आहे (Chapter 3.5 च्या भाषेत: platform). आणि आता बघा: त्याच GitHub च्या वर Microsoft ने AI coding tools विकायला सुरुवात केली. आधी दरवाजा, मग दुकान. Pattern प्रत्येक वेळेस तोच.

स्वतः बघा (5 मिनिटं)

github.com उघडा (account शिवाय चालेल). Search मध्ये “linux” लिहा, पहिला result उघडा. तुम्ही जगातल्या सगळ्यात महत्वाच्या software चा खरा code बघताय. “Commits” वर click करा: लाखो बदलांचा इतिहास, प्रत्येकावर नाव, तारीख, नोंद. सगळ्यात नवा commit बहुधा कालचा असेल. हे 30 वर्षांपासून रोज बदलतंय आणि कधी तुटलं नाही: हीच version control ची ताकद.

विचार करा

1. (derivation) एकटा माणूस काम करतोय, team नाही. त्याला branch/merge ची काय गरज? (विचार करा: हिमतीची गोष्ट इथे कशी लागू होते.)

उत्तर: गरज तीच आहे, रूप छोटं. Branch ही त्याची प्रयोगशाळा आहे: “नवं design करून बघतो” वाली copy, जिच्यात fail होणं फुकट आहे, खरी गोष्ट सुरक्षित आहे. आणि इतिहास हा त्याचा दुसरा मेंदू: सहा महिन्यांनी “ही विचित्र ओळ का लिहिली होती” चं उत्तर commit च्या नोंदीत मिळतं. एकट्या माणसाकडे विचारायला दुसरं कोणी नाहीच; म्हणून त्याला नोंदीची गरज team पेक्षा **जास्त** आहे. (तुमचा second-brain repo पण हेच आहे: बदलत्या विचारांचं version control.)

Chapter 4.4 [DEPTH]: “माझ्या machine वर तर चालत होतं”

(DEPTH chapter. ही ओळ software जगातला सगळ्यात जुना विनोद आहे. हिच्यामागचं सत्य समजलं की “deploy” आणि “DevOps” सारखे जड शब्द हलके होतील.)

प्रसंग: engineer ने feature बनवलं, आपल्या laptop वर शंभर वेळा चालवलं, सगळं perfect. Server वर टाकलं: crash. User च्या phone वर: रिकामी screen. Engineer ची पहिली ओळ: “माझ्या machine वर तर चालत होतं...”

तो खोटं बोलत नाहीये. मग चूक कुठे आहे?

Chapter 3.5 मधून विचार करा. Recipe कधीच एकटी चालत नाही: ती OS कडे मागते, दुसऱ्या recipes ना (libraries) बोलावते, settings वाचते, files शोधते. म्हणजे recipe च्या भोवती एक पूर्ण **माहोल** आहे ज्याच्यावर ती टेकलेली आहे:

code स्वतः	(हा तर दोन्हीकडे सारखाच होता)
+ OS आणि त्याचं version	(laptop: Windows 11; server: Linux)
+ libraries चे versions	(laptop वर नवे, server वर जुने)
+ settings, passwords	(laptop वर test वाले, server वर खरे)
+ दुसऱ्या services	(laptop वर नकली bank; server वर खरी)

Code सारखा, माहोल वेगळा = निकाल वेगळा. “चालत होतं” चं पूर्ण सत्य: “**माझ्या** माहोलात चालत होतं.” या भोवतीच्या पूर्ण दुनियेचं नाव **environment**.

घरची डाळ आठवा. तुमच्या घरी दरवेळेस perfect बनते. तीच recipe मावशीच्या घरी बनवा: cooker वेगळा, आच वेगळी, मिठाचा डबा वेगळा. चव बदलली. Recipe खोटी बोलली नाही; **kitchen** बदललं होतं.

मग इलाज स्वतःच सुचतो: माहोलही code सारखा बनवा. लिहून ठेवा की या recipe ला काय-काय लागतं (OS, libraries चे exact versions, settings), आणि अशी व्यवस्था करा की प्रत्येक जागी, laptop पासून server पर्यंत, **तोच** माहोल बनेल. Recipe सोबत पूर्ण kitchen च pack करून द्या.

ही व्यवस्था बनली; डब्याचं नाव **container**, सगळ्यात प्रसिद्ध हत्यार **Docker**. एक डबा ज्यात code + त्याचा माहोल एकत्र बंद आहे; डबा जिथे उघडेल, आतली दुनिया सारखीच असेल. नाव ship च्या container वरूनच घेतलं: सामान कुठलंही असो, डबा एकसारखा, म्हणून प्रत्येक बंदराची crane तो

उचलते. McDonald's चं रहस्यही हेच: त्यांनी recipe नाही, **kitchen** standard केलं, म्हणून प्रत्येक शहरात चव तीच. आणि code ला laptop वरून खऱ्या server पर्यंत पोहोचवण्याच्या कामाचं नाव **deploy**; हे सगळं सांभाळणाऱ्या पेशाचं नाव **DevOps**.

इथे लोक काय चुकीचं समजतात

”Software = code ची file. File पाठवली, काम संपलं.” आता तुम्हाला माहीत आहे: software = code + माहोल. म्हणून “app बनली, आता live करणं म्हणजे बस एक button असेल ना” हा founder चा सगळ्यात महाग भ्रम आहे. Deploy हे स्वतःचं पूर्ण काम आहे: माहोल बनवणं, तपासणं, जुन्या version वरून नव्यावर विना-थांबा जाणं, चूक झाल्यास परत फिरणं. Estimate मध्ये याचा वेळ वेगळा विचारा, नाहीतर “झालं” आणि “चालतंय” मधले आठवडे तुम्हाला surprise करतील.

MAP वर

रुपयाचा रस्ता: हा त्रास इतका खरा आहे की त्याच्या इलाजाच्या companies billions च्या आहेत. Docker ने डबा दिला; मग प्रश्न आला “हजार डबे कोण सांभाळणार” तर Google ने Kubernetes उघडं केलं (open source: Chapter 3.7 ची चाल, सगळे त्याच्या cloud च्या पद्धतीवर यावेत म्हणून); आणि cloud companies चा (Part 5) अर्धा business हाच आहे: “माहोलाची कटकट आम्हाला द्या, भाडं द्या.” जिथे प्रत्येक team चं दुखणं एकसारखं असतं, तिथे platform बनतो, आणि platform जकात वसूल करतो. दुखणं शोधा; दुकान तिथेच उघडा.

स्वतः बघा (5 मिनिटं)

Play Store/App Store मध्ये कुठल्याही app चं page उघडा, खाली “Requires Android 12+” किंवा “Requires iOS 16+” शोधा. हे environment चं सगळ्यात सरळ रूप आहे: app म्हणते आहे “माझं kitchen किमान इतकं नवं हवं.” आता समजेल जुन्या phone वर नवी app का चालत नाही: code नाही, माहोल छोटा पडला.

विचार करा

1. (derivation) दोन मित्र, अगदी सारखा phone model, सारखं app version. एकाच्या phone वर app रोज crash, दुसऱ्याच्या कधीच नाही. Code सारखा, phone सारखा. मग काय वेगळं असू शकतं? किमान तीन गोष्टी मोजा.

उत्तर: माहोल अजूनही वेगळा आहे: (1) OS चं छोटं version/update वेगळं असू शकतं, (2) एकाचा storage जवळजवळ full आहे (लिहायला जागा नाही = विचित्र crashes), (3) permissions वेगळ्या (एकाने location नाकारलं; app चा तो रस्ता कधी test झालाच नव्हता), (4) भाषा/region setting वेगळं (तारखेचं रूप वेगळं = चूक), (5) background apps आणि battery-saver वेगळे. धडा: “सारखा phone” कधीच सारखा नसतो. म्हणून मोठे apps हजारो खऱ्या devices वर test होतात, आणि तरीही Chapter 4.1 चा नियम लागू राहतो: कुठली तरी जोडी सुटतेच.

Chapter 4.5 [SPINE]: Estimate नेहमी चुकीचा का असतो

जगातला प्रत्येक founder एकच गोष्ट सांगतो: “Engineer म्हणाला दोन आठवडे. दोन महिने झाले.” घर बांधणारेही हीच गोष्ट सांगतात, पण software मध्ये हे इतकं **पक्कं** का आहे? सगळे engineers खोटेवरडे आहेत का?

नाही. कारण software च्या स्वभावात आहे, आणि तुम्ही ते Part 1 च्या एका सत्यातून स्वतः काढू शकता.

Chapter 1.6: **copy फुकट आहे**. मग जे software आधीच बनलेलं आहे, ते पुन्हा कोणी बनवत नाही; copy करतात (किंवा विकत घेतात, किंवा open source उचलतात). याचा उलटा अर्थ नीट विचार करा: **software बनवण्याचं काम फक्त तेव्हा होतं जेव्हा ती गोष्ट आधी कधी बनलीच नाही**. प्रत्येक software project, व्याख्येनेच, काहीतरी **नवं** बनवणं आहे.

आता तुलना करा. Contractor शंभरावं घर बांधतोय: प्रत्येक भिंत आधीसारखी, surprises कमी, estimate बसतो. Engineer प्रत्येक वेळेस आपलं **पहिलं** “हे वालं” बनवतोय. आणि नव्या कामात surprises कुठे लपले आहेत, हे आधी दिसत नाही (दिसलं असतं तर surprise नसतं). Estimate चुकतो कारण तो त्या गोष्टीचं माप आहे जी अजून अंधारात आहे.

तीन गोष्टी अंधार अजून गडद करतात:

”90% झालं” चा भ्रम. दिसणारा भाग (screens, buttons) आधी बनतो आणि पटकन बनतो. न-दिसणारा (edge cases, deploy, दुसऱ्या systems शी जोड: Chapter 4.1 आणि 4.4 चे सगळे राक्षस) नंतर येतो. म्हणून “90% done” बहुतेक अर्ध्या रस्त्याचं नाव असतं: उरलेला 10% दिसतो छोटा, असतो मोठा.

बदलती मागणी. अर्थ बनल्यावर founder ते पहिल्यांदा चालवतो आणि त्याला समजतं की खरं तर काय हवं होतं. मागणी बदलली, माप जुनं; estimate चा काय दोष?

जोडणीचं दुखणं. दोन बनलेले तुकडे जोडणं “बस जोडून टाका” नसतं (Chapter 4.4: प्रत्येक तुकड्याचा आपला माहोल). जोडणी नेहमी मापापेक्षा जास्त खाते. लग्नाच्या स्वयंपाकासारखं: भात वेगळा perfect, रस्सा वेगळा perfect; पण दोनशे लोकांना एकाच वेळेस गरम वाढायचं म्हणजे वेगळंच युद्ध.

मग इलाज काय? Estimate सुधारणं नाही (तो कधीच पूर्ण सुधारणार नाही); **कामाची पद्धत** बदलणं:

छोटे तुकडे, चालता माल. सहा महिन्यांचं वचन घेऊ नका; दोन आठवड्यांचा असा तुकडा मागा जो **चालून** दाखवेल. चालता तुकडा खोटं बोलत नाही. प्रत्येक तुकड्यानंतर अंधार थोडा कमी, पुढचा estimate थोडा बरोबर. आणि सगळ्यात आधी तो तुकडा बनवा जो कल्पनेची सगळ्यात मोठी शंका test करतो, सगळ्यात सुंदर screen नाही. या पद्धतीचं नाव **agile** (विधी खूप आहेत; खरी गोष्ट हेच तुकडे), आणि पहिल्या छोट्या चालत्या रूपाचं नाव **MVP** (minimum viable product).

खऱ्या जगाची दोन्ही टोकं: सरकारी software चे मोठे प्रकल्प वर्षानुवर्ष उशीर आणि कधीकधी पूर्ण रद्द (billions बुडतात), कारण वचन मोठं आणि तुकडे नाहीत. उलट WhatsApp, Instagram, Zerodha: सगळ्यांनी आधी एक **छोटा** चालता तुकडा काढला (Instagram तर एका मोठ्या app मधलं कापलेलं एकच feature होतं: फक्त photo, filter, share). तुकडा चालला, मग वाढवला. मोठी वचनं बुडतात; छोटे तुकडे तरंगतात.

इथे लोक काय चुकीचं समजतात

”उशीर = team नालायक किंवा चोर.” आता तुम्हाला माहीत आहे: नव्या कामाचं माप हेच अंधाराचं माप. नालायकपणाच्या खऱ्या खुणा वेगळ्या आहेत: चालता तुकडा कधीच न दाखवणं, “सगळं एकदम शेवटी येईल” म्हणणं, दर आठवड्याला तेच “almost done.” आणि प्रामाणिक team ची खूण: छोटे तुकडे, प्रत्येक चालता, आणि वाईट बातमी **लवकर** सांगणं. आता तुम्ही दोन्ही ओळखू शकता.

MAP वर

हा chapter थेट तुमच्या खिशाशी जोडलेला आहे. Software बनवून घेताना (माणसाकडून किंवा AI कडून): (1) पैसा वचनावर नाही, चालत्या तुकड्यावर द्या; (2) पहिला तुकडा तो मागा जो सगळ्यात मोठी शंका test करतो; (3) “90% done” ऐकलं की demo मागा, screen नाही, **खरं** चालतं system; (4) estimate ला 2-3 ने गुणून आपला plan बनवा, आणि जी team स्वतःहून असं करून सांगते तिला मान द्या; ती प्रामाणिक आहे. या चार ओळी तुम्हाला त्या founders पासून वेगळं करतात जे लाखो घालवून शिकतात.

स्वतः बघा (5 मिनिटं)

तुमचा कुठलाही जुना अंदाज आठवा: लग्नाची तयारी, घर shift करणं, कुठलंही नवं काम. किती वाटलं होतं, किती लागलं? गुणोत्तर काढा (बहुतेक 1.5x-3x). आणि लक्ष द्या: जे काम **पहिल्यांदा** करत होतात, तेच सगळ्यात जास्त घसरलं. नव्या कामाचा अंधार फक्त software चा नाही, माणसाचा नियम आहे; software मध्ये फक्त **प्रत्येक** काम नवं असतं.

विचार करा

1. (derivation) AI आता code चे मोठे भाग minutes मध्ये लिहितो. मग आता estimates बरोबर होतील आणि software वेळेवर बनेल का?

उत्तर: थोडं बरं, पूर्ण नाही. AI ने **लिहिण्याचा** वेळ पाडला; पण estimate चा अंधार लिहिण्यात कमीच होता: तो **समजण्यात** आहे (काय हवंय), **जोडण्यात** (माहोल, दुसरी systems), आणि **तपासण्यात** (कुठली जोडी सुटली). उलट एक नवा धोका आला: AI मुळे “90% दिसणारं” आता तासांत बनतं, म्हणून भ्रम अजून वेगाने बनतो. Founder चा नियम तोच राहील: चालता तुकडा हाच पुरावा. हो, एक बदल खरा आहे: छोटा तुकडा आता **स्वस्त** बनतो, म्हणून प्रयोग जास्त करता येतात, चुकीचा रस्ता लवकर सोडता येतो. AI estimate बरोबर करत नाही; चुकीची किंमत पाडतो.

PART 2 चा शेवट: नकाशा आणि पुढचं पाऊल

जे तुमच्याकडे आता आहे

भाषांतराची शिडी	machine code -> assembly -> languages -> AI; प्रत्येक मजला खालचा लपवतो
तीन गोष्टी	ठेवा (variable), विचारा (if), पुन्हा करा (loop): प्रत्येक program यांचंच जाळं
दोन भाषांतरकार	compiler (आधी, वेगवान) vs interpreter (सोबत, आरामाचा): चुकीची किंमत vs बदलाचा वेग
LANGUAGES	प्रत्येक भाषा चार सौद्यांची जोडणी; "कुठली best" नाही, "कुठल्या कामाला कुठली"
OS	machine चा मालक: पाळ्या, भिंती, दरबानी; मालक जकात वसूल करतो (platform tax)
PROCESS	file झोपलेली recipe; process जिवंत copy आपल्या state सह; restart = नवी स्वच्छ state
फुकटचा खेळ	दरवाजा / freemium / तुम्ही-माल / open source: पैसा नेहमी दुसरीकडून
BUG	लिहिलं vs हवं ची फट; जोड्यांचा स्फोट; मोजणी आहे, हलगर्जीपणा नाही
TESTING	तपासणंही code आहे; किती, हा business प्रश्न: चुकीची किंमत काय
VERSION CONTROL	प्रत्येक बदलाचा photo + परत जायचं button = हिमतीची व्यवस्था (Git, GitHub)
ENVIRONMENT	software = code + माहोल; म्हणून "माझ्या machine वर चालत होतं" (Docker, deploy)
ESTIMATE	प्रत्येक software पहिल्यांदा बनतंय; इलाज माप नाही, छोटे चालते तुकडे (MVP)

एक परीक्षा, स्वतःसाठी

पुस्तक न उघडता, बोलून उत्तरं द्या:

1. iPhone ची app Android वर का चालत नाही?
2. Google Android फुकट का वाटतो?
3. "Bug-free software" चं वचन खोटं का आहे?
4. Software बनवून घेताना पैसा कशावर द्यायचा: वचनावर की कशावर? आणि का?

अडलात तर: 1 -> 3.5, 2 -> 3.5/3.7, 3 -> 4.1/4.2, 4 -> 4.5.

Part 3 मध्ये काय आहे

आतापर्यंतची पूर्ण गोष्ट **एका** machine च्या आत होती. पण तुमचा प्रत्येक message दुसऱ्या machine कडे जातो: शहरापलीकडे, समुद्रापलीकडे, अर्ध्या second मध्ये. कसा? Machines एकमेकींना शोधतात कशा? रस्त्यात कोण-कोण बसलं आहे? “Public रस्त्यावर private बोलणं” (तुमचा password!) कसं वाचतं? आणि internet चा मालक शेवटी कोण? Part 3: MACHINES चं जाळं.

PART 2 चा MINI-GLOSSARY

agile	छोट्या चालत्या तुकड्यांची पद्धत (4.5)
assembly	numbers च्या नावांची भाषा, machine जवळची (3.1)
binary/executable	भाषांतर झालेली, थेट चालणारी file (3.3)
branch / merge	प्रयोगाची copy / ती परत जोडणं (4.3)
bug	लिहिलं आणि हवं यांच्यामधली फट (4.1)
CI	प्रत्येक बदलावर सगळे tests आपोआप (4.2)
code	कडक व्याकरणात लिहिलेली recipe (3.2)
commit	बदलाचा photo + "का" ची नोंद (4.3)
compiler	पूर्ण भाषांतर आधी करणारा (3.3)
condition (if)	अटीवर रस्ता निवडणं (3.2)
container/Docker	code + माहोल एका डब्यात बंद (4.4)
crash	program चं अचानक मरणं (4.1)
debugging	bug शोधून दुरुस्त करणं (4.1)
deploy	code खऱ्या server पर्यंत पोहोचवणं (4.4)
DevOps	deploy आणि माहोलाची व्यवस्था-पेशा (4.4)
driver	hardware शी बोलणारा OS चा तुकडा (3.5)
edge case	कोपऱ्यातली दुर्मिळ परिस्थिती (4.1)
environment	recipe भोवतीची पूर्ण दुनिया (4.4)
file system	OS ची files ची व्यवस्था (3.5)
freemium	फुकट अर्ध, पूर्ण विकलं जातं (3.7)
Git / GitHub	version control चं हत्यार / त्याचा अड्डा (4.3)
interpreter	ओळी-ओळीने सोबत चालणारा दुभाषी (3.3)
loop	एक काम पुन्हा पुन्हा; code मधला leverage (3.2)
loss leader	फुकट चव, विक्री दुसरीकडे (3.7)
machine code	numbers ची भाषा जी CPU थेट समजतो (3.1)
merge conflict	दोन बदलांची टक्कर (4.3)
MVP	पहिलं छोटं चालतं रूप (4.5)
open source	उघडा code, सगळ्यांचं सामायिक हत्यार (3.7)
OS	machine चा मालक: पाळी, भित, दरबान (3.5)
platform (tax)	दरवाजाचा मालक आणि त्याची जकात (3.5)
process / state	जिवंत copy / तिची आत्ताची पूर्ण अवस्था (3.6)
regression	नव्या बदलाने जुनं काही तुटलं (4.2)
scheduling	CPU च्या पाळ्या वाटणं (3.5)
syntax (error)	कडक व्याकरण (त्याची चूक) (3.2)
tech stack	project च्या tools/भाषांची पूर्ण जोडणी (3.4)
unit test	छोटी आपोआप चालणारी तपासणी (4.2)
variable	नावाचा खण (3.2)
version control	बदलांच्या इतिहासाची व्यवस्था (4.3)

THE MACHINE

PART 3 OF 5: MACHINES चं जाळं

पाच PARTS चा नकाशा

PART 1	पैसा आणि MACHINE	झाला
PART 2	MACHINE शी बोलणं [तुम्ही इथे आहात]	झाला
PART 3	MACHINES चं जाळं	network, address, internet
PART 4	DATA आणि आठवणी	storage, database, शोधणं
PART 5	जगाला SERVE करणं	scale, cloud, खर्च, पूर्ण जोड

आधी Part 2 ची परीक्षा

पाच प्रश्न, Part 2 मधले. आधी स्वतः उत्तर द्या, मग खाली बघा. जिथे अडाल, तिथला chapter number सोबत आहे; तिथे परत जा.

1. iPhone ची app Android वर का चालत नाही? (3.5)
2. Google Android फुकट का वाटतो? (3.5, 3.7)
3. मोठ्या software मध्ये bugs नक्की का असतात? (4.1)
4. "माझ्या machine वर तर चालत होतं" चा खरा अर्थ काय? (4.4)
5. Software बनवताना पैसा कशावर द्यायचा: वचनावर की कशावर? (4.5)

उत्तर: 1. App ही OS साठी लिहिलेली असते, machine साठी नाही; मालक (OS) वेगळा, म्हणून app चालत नाही. 2. विकून कमावण्यापेक्षा **मालक** होणं मोठा खेळ: प्रत्येक phone Google च्या search, ads आणि Play Store चा दरवाजा होतो. 3. परिस्थितींची मोजणी स्फोटासारखी वाढते; जी जोडी लिहिणाऱ्याच्या डोक्यात आलीच नाही, तीच कधीतरी एखाद्या user ला भेटते. ही मोजणी आहे, हलगर्जीपणा नाही. 4. Software = code + त्याचा भोवतीचा माहोल (OS, libraries, settings). Code तोच, माहोल वेगळा = निकाल वेगळाच. 5. चालत्या तुकड्यावर. वचन नाही, demo. प्रत्येक तुकडा चालवून दाखवला पाहिजे.

या part मध्ये काय आहे

आतापर्यंतची सगळी गोष्ट एका machine च्या आत होती. पण तुमचा प्रत्येक message दुसऱ्या machine कडे जातो: शहरापलीकडे, समुद्रापलीकडे, अर्ध्या second मध्ये. कसा?

आठ chapters: machines एकमेकींना कशा शोधतात (address), message कसा प्रवास करतो (packets), दोघांनी नियम आधीच का ठरवावे लागतात (protocols), नावांचे numbers कसे होतात (DNS), public रस्त्यावर private गोष्ट कशी लपते (encryption), internet म्हणजे नक्की काय, आणि त्याचे नियम कोण बनवतं.

हे तुमच्या business decision मध्ये कुठे येईल: तुमचं प्रत्येक product internet वर चालेल. त्याचा वेग, खर्च, सुरक्षा, हे सगळं इथल्या समजुतीवर उभं आहे. आणि एक मोठं धंद्याचं गुपित इथे लपलं आहे: networks मध्ये “n squared” नावाची एक अडचण परत परत येते, आणि जो ती सोडवतो तो दरवाजाचा मालक होतो. UPI हे त्याचं जिवंत उदाहरण आहे; या part मध्ये भेटेल.

SECTION 5: MACHINES चं आपसात बोलणं

आठ chapters. एका machine पासून जगभर पसरलेल्या जाळ्यापर्यंत.

हे तुमच्या business decision मध्ये कुठे येईल: “App slow का आहे,” “server चा खर्च किती येईल,” “data safe आहे का,” “हे feature offline चालेल का”: हे सगळे प्रश्न network च्या समजुतीशिवाय सुटत नाहीत. आणि जो founder network समजतो, तो हे पण बघतो की दरवाजे (platforms) कसे बनतात: internet च्या इतिहासात जो जो “सगळे एकमेकांशी बोलू शकतील” असा रस्ता बनवतो, तो तो श्रीमंत होतो. या section मध्ये तो pattern चार वेळा दिसेल.

Chapter 5.1 [SPINE]: एक machine पुरी पडत नाही

Part 1 आणि 2 नी तुम्हाला एक शक्तिशाली machine दिली: universal machine, जिच्यावर कुठलीही recipe चालते. मग प्रश्न असा: एकच machine पुरी का पडत नाही? WhatsApp तुमच्या phone वर आहे ना?

नाही. तुमच्या phone वर WhatsApp चा फक्त **दरवाजा** आहे. विचार करा: तुम्ही message पाठवला आणि मित्राचा phone बंद आहे. तरी message पोहोचतो, तो phone चालू करताच. म्हणजे message मध्ये कुठेतरी **थांबला** होता. कुठे? तुमच्या phone वर नाही (पाठवला, गेला). त्याच्या phone वर नाही (बंद होता). म्हणजे मध्ये **तिसरी** machine आहे, जिच्यावर message वाट बघत बसला होता.

ती तिसरी machine WhatsApp च्या company ची आहे, आणि अशा machines लाखोनी आहेत. का लागतात? तीन कारणं, तिन्ही तुम्ही स्वतः काढू शकता:

Data सगळ्यांचा एकत्र लागतो. तुमचा bank balance तुमच्या phone वर ठेवला तर? Phone हरवला, पैसा गेला. आणि दुकानदाराला कसं कळेल की तुमच्याकडे पैसे आहेत? जो data दोघांना लागतो, तो दोघांच्या मध्ये, तिसऱ्या जागी राहावा लागतो. गावची सोसायटी आपली वही घरी नाही, सोसायटीच्या office मध्ये ठेवते; तेच कारण.

एक machine crores लोकांना serve करू शकत नाही. एक CPU, एका second मध्ये billion पावलं (Part 1). वाटून बघा: 10 million लोक आले तर प्रत्येकाला किती? काम वाटावं लागतं, खूप machines मध्ये. एक cook लग्नाची पंगत वाढू शकत नाही; kitchen ची फौज लागते.

Machines पडतात. वीज जाते, disk जळते. सगळं एकाच जागी ठेवलं आणि ती जागा पडली, तर सगळं गेलं. प्रती (copies) वेगळ्या जागी लागतात.

म्हणून जग दोन भागांत विभागलं गेलं: तुमच्या हातातली **मागणारी** machine, जिचं नाव **client**, आणि company च्या building मधली **देणारी** machine, जिचं नाव **server** (serve करणारी, म्हणून). Zomato वर हे उघड दिसतं: तुमचा phone (client) Zomato च्या server ला मागणी पाठवतो; server order ठेवतो, restaurant च्या screen ला (दुसरा client) दाखवतो, delivery वाल्याच्या phone ला (तिसरा client) पाठवतो. तीन clients, एक server; कोणीच कोणाला थेट जोडलेलं नाही.

आणि आता सगळ्यात मोठा प्रश्न: या machines एकमेकींशी **बोलणार** कशा? दोन machines मध्ये तार ओढा, current चा क्रम पाठवा (Part 1: signal), bits गेले. दोन झाल्या. पण crores machines? प्रत्येकीला प्रत्येकीशी तार जोडली तर? गणित करा: 100 machines ना 4,950 तारा लागतील; 10

million machines ना... जमणारच नाही. या अडचणीचं नाव **n squared problem**: संख्या वाढली की जोड्या संख्येच्या **वर्गाने** वाढतात. हे नाव लक्षात ठेवा; या part मध्ये हा राक्षस परत परत येईल, आणि प्रत्येक वेळेस कोणीतरी त्याला हरवून श्रीमंत होईल.

उपाय तुम्ही रोजच्या जगात बघितला आहे: रस्ता. प्रत्येक घरापासून प्रत्येक घरापर्यंत रस्ता नसतो; गल्ली असते, चौक असतो, highway असतो. तसंच machines चं: तुमची machine जवळच्या चौकाला जोडलेली, चौक मोठ्या चौकाला, असं जोडत जोडत सगळं जग. जोडलेल्या machines च्या या जाळ्याचं नाव **network**.

इथे लोक काय चुकीचं समजतात

"माझा message थेट मित्राच्या phone ला जातो." कधीच नाही. प्रत्येक message आधी company च्या server ला जातो, मग मित्राला. Post सारखंच: शेजारच्या घरचं पत्रही पेटी -> post office -> घर असंच फिरतं. (End-to-end encryption असेल तर company **वाचू** शकत नाही, पण रस्ता त्यांच्याच मधून जातो: Chapter 5.6 मध्ये उघडेल.) म्हणूनच "फुकट" apps च्या company कडे एवढी ताकद असते: सगळा traffic त्यांच्या चौकातून जातो.

MAP वर

रुपयाचा रस्ता: तुम्ही महिना भरला (Level 1 पासून पैसा घुसला) -> telecom company (Jio/Airtel) ला; ती रस्ता देते. App company आपल्या server चा खर्च cloud ला देते (Part 5). आणि हे बघा: ज्या business कडे **server** आहे, त्याच्याकडे **data** आहे; ज्याच्याकडे data, त्याच्याकडे ताकद. Client बनवणं सोपं आणि स्वस्त झालं आहे; server ची बाजू हीच खरी मालमत्ता. तुम्ही जो business बनवाल, त्यात "server वर काय राहणार" हा प्रश्न म्हणजे "मालमत्ता काय असणार" हा प्रश्न.

स्वतः बघा (5 मिनिटं)

Phone airplane mode वर टाका. आता apps उघडा: WhatsApp chats दिसतात (जुना data, phone वर साठवलेला), पण नवे messages येत नाहीत. Google Maps चा नकाशा अर्धवट उघडतो, search चालत नाही. Camera पूर्ण चालतो. आता तुम्हाला प्रत्येक app चे दोन भाग दिसतील: जो तुमच्या हातात आहे, आणि जो कोणा-दुसऱ्याच्या machine वर आहे. ही रेषा बघायला शिकणं म्हणजे network समजणं.

विचार करा

1. (derivation) 100 लोकांची वस्ती आहे. प्रत्येकाने प्रत्येकाशी थेट बोलायचं ठरवलं तर 4,950 जोड्या लागतात. एक चौक (server) बनवला तर फक्त 100. पण आता एक नवी किंमत आली; ती कुठली? (Chapter 0.1 ची आठवण: वही ठेवणाऱ्यावर काय प्रश्न होता?)

उत्तर: विश्वास. सगळं बोलणं एका चौकातून जातं, म्हणजे चौकाचा मालक सगळं बघतो, थांबवू शकतो, बदलू शकतो. N^2 सुटला, पण सत्ता एका जागी एकवटली. म्हणून हा pattern परत परत दिसेल: सोपा रस्ता = मधला मालक. आणि त्याच्याविरुद्धचे प्रयोगही (blockchain वगैरे) याच दुखण्यावर उभे आहेत. Technology चं प्रत्येक design एक सौदा असतो: इथे सुविधा vs सत्ता. जो हे बघतो, तो बातम्या नाही, धंदा वाचतो.

Chapter 5.2 [SPINE]: प्रत्येक machine ला एक address

Network बनलं, रस्ते झाले. आता post office चा जुना प्रश्न उभा राहतो: पत्र पोहोचवायचं तर पत्ता लागतो. “राम ला द्या” ने काम होत नाही; राम लाखो आहेत. Machines च्या जगातही तेच: message पाठवायचा तर त्या machine चा नेमका address लागतो.

Address कसा असावा? Part 1 ने उत्तर आधीच दिलं आहे: machine ला फक्त numbers कळतात. म्हणून machine चा address एक **number** आहे. त्याचं नाव **IP address** (Internet Protocol चा address). साधं रूप: चार numbers, प्रत्येक 0 ते 255 मधला (ओळखीचं वाटलं? 8 bits = 256 स्थिती, Chapter 1.4):

142.250.183.14	(हा Google च्या एका server चा address)
192.168.1.5	(हा तुमच्या घरातल्या phone चा असू शकतो)

आता एक गणित करा. चार numbers, प्रत्येक 256 प्रकारचा: एकूण किती address? $256 \times 256 \times 256 \times 256 = 4 \text{ billion+}$ (430 कोटी). 1980 मध्ये वाटलं “हे कधीच संपणार नाहीत.” आज जगात billions phones, TVs, गाड्या, घड्याळं... address **संपले**. हा खरोखर घडलेला प्रसंग आहे: internet च्या designers नी जग केवढं होईल हे कमी अंदाजलं. (उपायही आला: नवा लांब address, **IPv6**, ज्यात address संपूच शकत नाहीत; जुन्या चार-number रूपाचं नाव **IPv4**, आणि ते अजून सगळीकडे आहे.)

संपत आलेले address जगाने कसे पुरवले? इथून एक सुंदर युक्ती निघाली, जी तुमच्या घरात आहे: तुमच्या WiFi **router** ला एकच **public IP** मिळतो (building ला एक post-box). घरातल्या सगळ्या devices ना router आतल्या आत आपले छोटे **private IP** देतो (खोली क्रमांक 1, 2, 3...). बाहेरून सगळे एकच दिसतात; आत router हिशोब ठेवतो की कुठलं उत्तर कुठल्या device चं. एक address, दहा machines. Building चा watchman जसा: पत्र building च्या नावावर येतं, watchman flat बघून पोहोचवतो.

हे रोजच्या बातम्यांशी असं जोडलेलं आहे: cyber crime च्या बातमीत “police नी IP address वरून आरोपी शोधला” हे वाक्य असतं. आता तुम्हाला ते वाचता येतं: प्रत्येक connection च्या मागे एक public IP असतो, आणि telecom company कडे वही असते की त्या वेळेस तो IP कोणाला दिला होता (Chapter 0.1 चा ledger, परत!). म्हणूनच **VPN** नावाची गोष्ट विकली जाते: ती तुमचा खरा IP लपवते, दुसऱ्या देशातला दाखवते. सुरक्षा आणि लपाछपी, दोन्ही एका address भोवती फिरतात.

इथे लोक काय चुकीचं समजतात

”IP address म्हणजे माझी कायमची ओळख.” नाही. तुमचा public IP telecom company रोज बदलू शकते (भाड्याने दिलेला number आहे, मालकीचा नाही; भाड्याच्या खोलीसारखा). आणि एका IP मागे शंभर लोक असू शकतात (router!). म्हणून IP फक्त “त्या वेळेचा, त्या जागेचा” पत्ता आहे, Aadhaar card नाही. दोन्ही दिशांनी चूक होते: लोक घाबरतात “माझा IP दिसला, सगळं संपलं” (नाही), आणि निश्चितही असतात “IP बदलतो म्हणजे मी लपलो” (नाही: वही आहे ना).

MAP वर

कोण कमावतं: address **स्वतःच** मालमत्ता झाली. IPv4 addresses संपले, म्हणून जुना साठा असणाऱ्यांनी ते विकायला काढले: एक IPv4 address आज हजारो रुपयांना जातो; companies कडे millions addresses चे blocks आहेत, त्यांची किंमत billions मध्ये. 1990 मध्ये फुकट वाटलेले numbers आज सोनं झाले, गावाबाहेरच्या जमिनीसारखे: शहर वाढलं, भाव फुटले. धंद्याची शिकवण: जिची मोजदाद **ठरलेली** आहे आणि मागणी वाढणार आहे, ती गोष्ट लवकर धरून ठेवा. (Domains मध्ये हे परत येईल, Chapter 5.5.)

स्वतः बघा (5 मिनिटं)

Phone च्या Settings मध्ये WiFi उघडा, जोडलेल्या network वर tap करा: तिथे “IP address” दिसेल, साधारण 192.168.x.x असा. हा तुमचा **private** address. आता browser मध्ये search करा: “what is my IP”. वेगळा number दिसेल! हा तुमच्या router चा **public** address. दोन address, दोन जगं: आत आणि बाहेर. Watchman-system तुम्ही स्वतः बघितली.

विचार करा

1. (derivation) Postman “Mumbai, Andheri, XYZ building, flat 402” असा पत्ता तुकड्या-तुकड्यांनी वापरतो: आधी शहर, मग भाग, मग building. IP address पण असाच तुकड्यांनी वाचला जातो (आधी मोठा भाग, मग आतला). सगळे routers जगातल्या **सगळ्या** machines ची यादी का ठेवत नाहीत; हे design का निवडलं?

उत्तर: कारण यादी billions नोंदींची होईल आणि **रोज** बदलेल; प्रत्येक चौकात ती ठेवणं आणि ताजी राखणं अशक्य. तुकड्यांचं design म्हणजे: प्रत्येक router ला फक्त एवढंच माहीत लागतं की “हा भाग त्या दिशेला.” Mumbai चा postman flat 402 जाणत नाही; फक्त Andheri कुठे ते जाणतो. काम वाटलं गेलं; कोणा-एकाला सगळं जाणावं लागत नाही. या design-तत्त्वाचं नाव hierarchy (शिडी), आणि ते technology त सगळीकडे आहे: DNS मध्ये (5.5), company च्या रचनेत, AI च्या आतही. जिथे scale मोठा, तिथे यादी नाही, **शिडी** असते.

Chapter 5.3 [DEPTH]: Message चे तुकडे करणं

(DEPTH chapter. वगळला तरी गोष्ट पुढे जाईल, पण हा लहान आहे, आणि यानंतर “packet” आणि “network slow” चे खरे अर्थ उघडतात.)

Address झाला. आता message पाठवायचा: समजा 3 MB चा photo (Part 1: म्हणजे ~25 million bits). सोपा वाटणारा रस्ता: सगळे bits एकासाथ, एक लांबच लांब प्रवाह म्हणून पाठवा.

हे design वाईट आहे. का, ते truck च्या उदाहरणाने स्वतः काढा. समजा Mumbai-Pune रस्त्यावरून एक 3 km लांब truck जातोय:

1. तो जात असताना दुसरं **कोणीच** रस्ता वापरू शकत नाही. एक मोठा message = बाकी सगळ्यांची थांबलेली रांग.
2. Truck मध्ये कुठे बिघाड झाला तर? **सगळं** परत पाठवा. 3 MB मधला शेवटचा bit चुकला तरी पूर्ण 3 MB परत.
3. आणि रस्ता एकच असेल, तर तो पडला की सगळं थांबलं.

मग design तुम्ही करा. रस्त्याची व्यवस्था कशी हवी?

उत्तर: **message चे लहान तुकडे करा**. प्रत्येक तुकड्यावर लिहा: कोणाला जायचं (address), कोणाकडून आला, आणि तुकडा क्रमांक (1 of 2000, 2 of 2000...). अशा तुकड्यांचं नाव **packet**, आणि या पूर्ण व्यवस्थेचं नाव **packet switching** (1960s चा शोध, आणि internet ची खरी पायाभरणी). आता प्रत्येक तुकडा **स्वतंत्र** प्रवास करतो, आणि फायदे आपोआप येतात:

रस्ते वाटले जातात: तुमच्या तुकड्यांच्या मधून दुसऱ्यांचे तुकडेही जातात; कोणी रस्ता अडवत नाही. तुकडे **वेगवेगळ्या** रस्त्यांनीही जाऊ शकतात: एक highway ने, एक गावातून; जो मोकळा, त्या क्षणी तो. एक तुकडा हरवला तर **फक्त तोच** परत मागवायचा. आणि पोहोचल्यावर क्रमांकाप्रमाणे लावून message परत जोडला जातो. लग्नाचं सामान पाठवायचं तर 3 km चा एक trailer नाही; वीस tempo पाठवता, प्रत्येकावर यादी चिकटवून. एक tempo बिघडला तर तेवढाच परत मागवता.

“हरवलेला परत मागवा, क्रम लावा, खात्री द्या” हे काम करणाऱ्या नियमांचं नाव **TCP**. आणि “परत मागवायचं नाही; जो पोहोचला तो पोहोचला, **वेग** महत्वाचा” या नियमांचं नाव **UDP**. दोन्ही नावं बातम्यांमध्ये येतात; आता अर्थ तुमच्याकडे आहे. File download होताना network 2 seconds गेलं:

download **थांबतो**, मग तिथूनच पुढे. कारण file ला TCP: प्रत्येक तुकडा पोहोचलाच पाहिजे; किंमत म्हणून वेळ. Video call मध्ये network गेलं: आवाज robotic होतो, चेहरा अडकतो, पण call **चालू** राहतो. कारण call ला UDP-सारखे नियम: जुना तुकडा परत मागवून काय उपयोग? तो क्षण तर गेला.

एक गोष्ट लक्षात ठेवा: तुम्हाला वाटतं तुम्ही “connection” उघडलं आहे, एक pipe, एक धागा. प्रत्यक्षात **धागा नाही**. फक्त तुकडे आहेत, स्वतंत्र उड्या मारणारे. “Connection” हा भास आहे जो दोन्ही टोकांचं software तुम्हाला दाखवतं: तुकडे मोजून, हरवलेले मागवून, क्रम लावून. हा भास आहे हे समजलं की network च्या सगळ्या विचित्र गोष्टी सोप्या होतात: robotic आवाज, अडकलेली file, “reconnecting...”

इथे लोक काय चुकीचं समजतात

“Net slow आहे म्हणजे speed कमी.” अर्धवट. दोन वेगळ्या गोष्टी असतात: एका second मध्ये **किती** तुकडे जातात (bandwidth) आणि **एका** तुकड्याला जाऊन-येऊन किती वेळ (latency, Chapter 2.3 वाली). रस्त्याची उपमा: bandwidth म्हणजे रस्त्याला lanes किती; latency म्हणजे प्रवासाला वेळ किती. आठ-lane रस्त्यावरही Pune दोन तास दूरच आहे. Movie ला bandwidth लागते; game आणि video call ला latency मारते. म्हणून “100 Mbps” plan असूनही game मध्ये “lag” येतो. दुकानदार एक number विकतो; खेळ दोन numbers चा आहे.

MAP वर

Company case: packet switching ने जुन्या telephone जगाला हरवलं. Telephone कडे “एक call = एक राखीव line” असं महाग design होतं; packets नी तीच तार हजारो लोकांत वाटली. जो रस्ता स्वस्त करतो तो जिंकतो: pattern परत. आणि packets च्या रस्त्यावरची यंत्रं (routers) बनवणारी **Cisco** एका काळी जगातली सगळ्यात मौल्यवान company होती. रस्ता जिथे नवीन बनतो, तिथे यंत्रं विकणारा पहिला श्रीमंत होतो. (AI मध्ये आज **Nvidia** तेच करते आहे: नवा रस्ता, यंत्रं विकणारी.)

स्वतः बघा (5 मिनिटं)

पुढच्या video call ला network खराब झालं की **लक्ष द्या**: आवाज robotic का होतो? Software हरवलेले packets भरून काढायचा प्रयत्न करतंय. चेहरा का “अडकतो” आणि मग उडी मारतो? मधले frames (packets) गेले; software ने ते सोडले आणि नव्यावर उडी मारली. तुम्ही आता बिघाड नाही, **design** बघताय: वेगासाठी सोडलेला बरोबरपणा.

विचार करा

1. (derivation) Live cricket streaming मध्ये एक packet हरवला. परत मागवावा का? TCP की UDP-विचार? आणि bank transaction मध्ये? दोन उत्तरं वेगळी का?

उत्तर: Cricket: मागवू नका. तो क्षण गेला; परत मागवलेला तुकडा जुना असेल, आणि त्याची वाट बघणं म्हणजे पूर्ण stream उशिरा. प्रेक्षकाला 2 seconds जुनं perfect चित्र नको; आत्ताचं थोडं खराब चालेल. Bank: मागवा, आणि पक्कं पोहोचेपर्यंत काहीही पुढे जाऊ देऊ नका. एक रुपया चुकला तरी चालणार नाही; वेळ लागला तरी चालेल. नियम असा निघतो: जिथे **क्षण** महत्त्वाचा तिथे वेग जिकतो; जिथे **बरोबरपणा** महत्त्वाचा तिथे किंमत म्हणून वेळ. तुमच्या product मधलं प्रत्येक feature या दोन रंगांत वाटून बघा; तंत्र-निवड आपोआप सोपी होते.

Chapter 5.4 [SPINE]: नियमांवर आधीच सहमती

Address आहे, packets आहेत. पण अजून एक प्रश्न लपला आहे, आणि तो सगळ्यात खोल आहे.

Phone call ची सुरुवात आठवा: “Hello?” “हां, बोला.” “मी रमेश बोलतोय...” हे शब्द माहिती देत नाहीत; हा **क्रम** आहे. कोण आधी बोलणार, समोर कोणी आहे हे कसं कळणार, “ऐकू येतंय का” चा इशारा काय. दोघांनी हा क्रम **आधीच** मानलेला असतो, म्हणून call चालतो. एक जपानी क्रमाने बोलला आणि दुसरा मराठी क्रमाने, तर शब्द पोहोचतील पण बोलणं होणार नाही.

Machines मध्ये हा प्रश्न अजून टोकाचा, कारण machine कडे “समजून घेणं” नाही (Part 2: exact लागतं). दोन machines ना बोलायचं तर **सगळं** आधी ठरलेलं लागतं: कोण सुरुवात करणार? Message चा ढाचा काय? “पोहोचला” कसं कळवणार? चूक झाली तर कोण, किती वेळा परत पाठवणार?

अशा आधीच-ठरलेल्या नियमांच्या यादीचं नाव **protocol**. आणि internet म्हणजे काय, याचं अर्थ उत्तर हेच आहे: **internet ही वस्तू नाही, नियमांची यादी आहे**. जो कोणी ते नियम पाळतो, त्याची machine जाळ्यात सामील होते. परवानगी कोणाची नाही, fee कोणाला नाही. नियम public आहेत, दरवाजा सगळ्यांना उघडा. बाजाराच्या आठवडी हाटासारखं: जागेचा नियम, वेळेचा नियम, वजन-मापाचा नियम सगळ्यांना माहीत; कोणीही येऊन दुकान लावतो, हाटाचा मालक कोणी नाही.

आणि नियम एका थरावर नाहीत, **शिडीवर** आहेत (abstraction, परत!): सगळ्यात खाली “तारेवर current कसा” चे नियम; त्यावर “address आणि packets” चे (IP); त्यावर “हरवलेलं मागवा” चे (TCP); आणि सगळ्यात वर “app ला काय हवं” चे. Website मागवण्याच्या नियमांचं नाव तुम्ही रोज बघता: **HTTP**. Browser च्या पट्टीतलं “https://...” म्हणजे “ही बोलणी HTTP च्या नियमांनी होणार” एवढंच. (Email चे नियम: SMTP. या नियम-शिडीचं नाव **protocol stack**.) प्रत्येक थर फक्त आपलं काम जाणतो; खालचा कसा चालतो हे त्याला माहीत नसतं.

आता धंद्याची नजर लावा, कारण इथे या part चं सगळ्यात मोठं उदाहरण आहे: **UPI हे protocol आहे, app नाही**. NPCI ने नियम बनवले: पैसे मागण्याचा ढाचा काय, banks नी कसं बोलायचं, “झालं” कसं कळवायचं. नियम **खुले** ठेवले: म्हणून GPay, PhonePe, Paytm, कुठलीही bank, सगळे एकमेकांशी चालतात. तुम्ही GPay वरून PhonePe वाल्याला पैसे पाठवता आणि विचारही करत नाही: दोन स्पर्धक companies च्या apps मध्ये पैसा बिनधोक वाहतो, कारण protocol समान आहे. निकाल जग बघतंय: महिन्याला **10 billion+** (1,000 कोटी+) व्यवहार. नियम बनवणारा (NPCI) स्वतः मोठं app नाही, पण सगळ्या खेळाचा पाया तो आहे.

जिथे protocol खुला, तिथे कोणीही दुकान उघडतो: email चे नियम खुले, म्हणून हजारो email companies. जिथे नियम एका company चे, तिथे तिची हुकूमत: WhatsApp चे नियम Meta चेच, म्हणून WhatsApp ला जोडणारं दुसरं app बनूच शकत नाही. खुले नियम = स्पर्धा = ग्राहकाला स्वस्त. बंद नियम = किल्ला. दोन्हीचे धंदे चालतात; कुठला खेळ खेळतोय हे कळायला हवं.

इथे लोक काय चुकीचं समजतात

"Internet चा मालक कुठलीतरी company आहे (Google? Microsoft?)." नाही. Internet कोणाचीच मालमत्ता नाही, कारण तो नियमांचा समूह आहे, आणि नियम सगळ्यांकडे आहेत. Google च्या **सेवा** (search, YouTube) त्याच्या आहेत; **रस्ता** त्याचा नाही. हा फरक महत्त्वाचा: सेवा बंद होऊ शकते; रस्ता बंद करायला जगभरचे नियम बदलावे लागतील. (नियम कोण बनवतं, हे Chapter 5.8 मध्ये.)

MAP वर

N squared चा राक्षस परत आला होता; बघितलंत का? UPI च्या आधी प्रत्येक app ला प्रत्येक bank शी **वेगळी** जोडणी बांधावी लागली असती: 50 apps x 300 banks = 15,000 जोड्या. Protocol ने त्या 350 केल्या (प्रत्येकाने फक्त नियमांशी जोडायचं). **जो n-squared ला protocol ने मारतो, तो पूर्ण बाजाराचा पायाभूत थर बनतो.** तुमच्या 4-level नकाशावर: जिथे प्रत्येकाला प्रत्येकाशी जोडावं लागतंय आणि सगळे मेटाकुटीला आलेत, तिथे protocol ची जागा **रिकामी** आहे. अशी जागा दिसली तर नीट बघा: ती billions ची आहे.

स्वतः बघा (5 मिनिटं)

Browser मध्ये कुठलीही website उघडा आणि पट्टीकडे लक्ष द्या: https:// ने सुरुवात. आता एक जुनी गोष्ट: काही वर्षांपूर्वी फक्त http:// असायचं (s नाही). आता browser http च्या site ला "Not secure" म्हणतो. एक अक्षर (s) म्हणजे एक पूर्ण protocol-बदल, आणि तो जगभर का घडला हे Chapter 5.6 सांगेल. आज फक्त ते अक्षर दिसतंय का बघा.

विचार करा

1. (derivation) UPI सारखा खुला protocol बनवण्यात NPCI ला (आणि banks ना) काय मिळतं? त्यांनी बंद, आपल्या मालकीची system का नाही बनवली, जशी Visa/Mastercard ची होती?

उत्तर: बंद system मध्ये प्रत्येक व्यवहारावर fee मिळते (Visa चा खेळ: दुकानदाराकडून 1-2%). खुल्या system मध्ये **वापर** स्फोटासारखा वाढतो: UPI फुकट ठेवला म्हणून चहावाल्यापासून भाजीवाल्यापर्यंत सगळे आले, आणि 10 वर्षांत रोखीचा देश digital झाला. NPCI सरकारी-सहकारी आहे; तिला fee पेक्षा **व्याप्ती** हवी होती. आणि banks ना देशाचा सगळा पैसा system मध्ये आला हे मिळालं. धंद्याची शिकवण: fee-प्रति-व्यवहार हा एक खेळ; पूर्ण बाजाराची पायाभरणी दुसरा. दुसरा खेळ मोठा, पण त्याला लांब श्वास लागतो, आणि कमाई थेट नाही, वरच्या थरांतून येते (data, सेवा, दरवाजा). Google ने Android मध्ये हेच केलं होतं, आठवतं? Pattern एकच.

Chapter 5.5 [SPINE]: Numbers ऐवजी नावं

Address झाले, नियम झाले. पण एक अडचण राहिली: server चा address 142.250.183.14 आहे. तुम्ही “google.com” लिहिता. हे जमतं कसं? आणि मुळात असं का?

कारण सोपं आहे: माणसांना numbers लक्षात राहत नाहीत, नावं राहतात (मित्रांचे phone numbers पाठ केलेत का? नावं save केली). पण machines ना numbers लागतात. म्हणजे मध्ये एक **phonebook** लागते: नाव द्या, number घ्या.

आता design तुम्ही करा. जगभरच्या crores websites साठी phonebook कशी बनवाल?

पहिला विचार: एक मोठी वही, एका जागी. Chapter 0.1 ची घंटा वाजली का? तेच प्रश्न: (1) वही ठेवणाऱ्यावर सगळ्यांचा विश्वास लागेल, (2) जगभरचे प्रश्न एका जागी आले तर ती जागा मरेल, (3) ती जागा पडली तर **सगळं** internet “हरवेल” (रस्ते चालू, पण पत्ते नाहीत).

म्हणून design वेगळं निवडलं, आणि ते तुम्ही post-पत्त्यात आधीच बघितलं (Chapter 5.2): **शिडी**. नाव उलटं वाचा: google.com म्हणजे “com च्या मोठ्या वहीत, google ची नोंद.” प्रत्येक थराची वही वेगळी: सगळ्यात वरची वही फक्त सांगते की “.com” ची यादी कोण सांभाळतो; ती वही सांगते “google.com” ची माहिती कुठे; तिथून number मिळतो. कोणा-एकाकडे सगळं नाही; प्रत्येकाकडे आपला थर. या पूर्ण व्यवस्थेचं नाव **DNS** (Domain Name System), आणि website च्या नावाचं नाव **domain**. गावच्या पत्त्यासारखं: सरपंच गाव जाणतो, पोस्टमन गल्ली, शेजारी घर; कोणी एकटा पूर्ण देश जाणत नाही.

आणि वेगासाठी एक युक्ती: उत्तर मिळालं की तुमचा phone आणि जवळचे servers ते **लक्षात** ठेवतात (थोडा वेळ). पुन्हा पुन्हा शिडी चढावी लागत नाही. या लक्षात-ठेवण्याचं नाव **cache**; Part 5 मध्ये हा शब्द मोठा होणार आहे.

आता सगळ्यात सुंदर भाग: नाव आणि number **वेगळे** असल्यामुळे एक स्वातंत्र्य मिळतं. Website ची machine बदला, शहर बदला, cloud बदला: number बदलेल, पण नाव तेच राहतं. ग्राहकाला कळतही नाही. दुकानाची इमारत बदलली; पाटी तीच राहिली. म्हणून नाव (domain) ही धंद्याची **मालमत्ता** आहे आणि machine भाड्याची वस्तू. (Domain विकत घेणं: registrar कडे, GoDaddy वगैरे, वर्षाला थोडी fee.)

ही boring वाटणारी यंत्रणा केवढी महत्त्वाची आहे, हे 2016 मध्ये जगाने बघितलं: DNS ची सेवा देणाऱ्या एका मोठ्या company वर (Dyn) हल्ला झाला; millions hacked devices नी तिला प्रश्नांनी बुडवलं. निकाल: Twitter, Netflix, Spotify “बंद” झाले, अर्ध्या USA साठी. Servers चालू होते, रस्ते चालू होते,

पण **phonebook** उघडत नव्हती; कोणी कोणाला शोधू शकत नव्हतं. पायाभूत गोष्टी अशाच असतात: दिसत नाहीत; पडल्या की सगळं दिसतं.

इथे लोक काय चुकीचं समजतात

"Website म्हणजे domain." नाही. Domain फक्त **पाटी** आहे; दुकान (server, code, data) वेगळीकडे आहे. म्हणून: domain विकत घेतला म्हणजे website झाली नाही (पाटी आली, दुकान नाही). आणि website "बंद पडली" तर कारण पाटी असू शकते (domain ची fee भरली नाही: वर्षाकाठी हजारो धंदे हे विसरून बंद पडतात!) किंवा दुकान असू शकतं (server पडला). दोन वेगळ्या गोष्टी, दोन वेगळी bills, दोन वेगळे प्रश्न.

MAP वर

Domain हा इतिहासातला सगळ्यात स्वस्त सत्ता-कब्जा ठरला: 1990s मध्ये काही दूरदर्शी लोकांनी साधी नावं (business.com, cars.com) कवडीमोलात घेऊन ठेवली; पुढे त्यांचे भाव कोटींमध्ये फुटले (business.com ~345 million dollars ना विकला गेला). आजही खेळ चालू आहे: चांगली नावं लोक आधीच घेऊन बसतात (squatting). Chapter 5.2 ची शिकवण परत: ठरलेली मोजदाद + वाढती मागणी = धरून ठेवा. आणि तुमच्यासाठी एक व्यवहारी ओळ: धंद्याची कल्पना पक्की नसली तरी तिचा domain 800 रुपयांत आधी घेऊन ठेवा; कल्पना पक्की होईपर्यंत नाव गेलेलं असतं.

स्वतः बघा (5 मिनिटं)

आपल्या phone ची contact-यादी उघडा. हीच तुमची DNS आहे: नावं वर, numbers आत. मित्राने SIM बदलला तरी तुम्ही नावच वापरता; number तुम्हाला माहीतच नसतो. आता कल्पना करा contact-यादी उडाली: सगळे numbers चालू आहेत, पण तुम्ही कोणालाच phone लावू शकत नाही. DNS पडणं म्हणजे **हेच**, जगभरासाठी. नावं ही आठवण आहे; numbers फक्त रस्ते.

विचार करा

1. (derivation) Domain वर्षाला फक्त 800-1500 रुपये का? एवढी महत्त्वाची मालमत्ता एवढी स्वस्त का? (विचार करा: किंमत कशाने ठरते, आणि इथे scarcity नेमकी कशात आहे?)

उत्तर: कारण **नोंदणी** स्वस्त आहे: वहीत एक ओळ लिहिणं, जिचा खर्च जवळजवळ शून्य (Part 1: numbers लिहिणं फुकट). Scarcity ओळीत नाही, **नावात** आहे: cars.com एकच असू शकतो. म्हणून बाजार दोन थरांचा झाला: नोंदणी fee सगळ्यांना सारखी (स्वस्त), पण चांगलं नाव ज्याच्याकडे आहे तो वाटेल ती किंमत मागतो: हीच scarcity ची खरी जागा. शिकवण पुन्हा तीच (Chapter 0.2): किंमत मेहनतीवर नाही, दुर्मिळतेवर उभी असते. आणि दुर्मिळता कधीकधी फक्त “पहिले पोहोचणं” एवढीच असते.

Chapter 5.6 [SPINE]: Public रस्त्यावर private बोलणं

आता एक प्रश्न जो तुमच्या खिशाला थेट भिडतो. Chapter 5.3 ने सांगितलं: तुमचा message तुकड्यांनी, **दुसऱ्यांच्या** यंत्रांमधून प्रवास करतो: telecom चे routers, मधले carriers, कोण-कोण. आणि त्या रस्त्याने तुम्ही काय पाठवता? Bank चा password. Card चा number. खाजगी photo.

Postcard आठवा: postman वाचू शकतो, कारण लिफाफा नाही. तुमचे packets पण **असेच** आहेत, काही केलं नाही तर: रस्त्यातला प्रत्येकजण वाचू शकतो. मग उपाय? लिफाफा लागतो. पण कागदी नाही; **गणिताचा**. या लिफाफा-गणिताचं नाव **encryption**.

पहिला विचार सोपा आहे: एक गुप्त किल्ली ठरवा (समजा: प्रत्येक अक्षर 3 ने पुढे सरकवा). मी कुलूप लावतो, तू उघड. मधला माणूस फक्त कचरा बघतो. हे जुनं तंत्र आहे (Caesar पासून World War पर्यंत), आणि ते चालतं... एक भयानक अडचण सोडून: **किल्ली त्याच्यापर्यंत पोहोचवायची कशी?** रस्ता तर public आहे! किल्ली रस्त्याने पाठवली तर मधलाच ती उचलेल. किल्लीसाठी भेटावंच लागेल... पण internet वर तुम्ही Amazon ला कधी भेटलात का? हा प्रश्न 1970s पर्यंत न-सुटणारा मानला जायचा.

मग एक कल्पना आली जिच्यावर आज सगळं online जग उभं आहे: **किल्ल्या दोन** असलेली कुलूप-यंत्रणा (नाव: **public key encryption**):

एक किल्ली कुलूप लावते (public किल्ली). ही तुम्ही जगाला वाटता, ओरडून सांगता: “माझी कुलूप-किल्ली ही घ्या!”

दुसरी किल्ली उघडते (private किल्ली). ही **फक्त** तुमच्याकडे. कधीच, कोणालाच जात नाही.

आता जादू बघा: Amazon आपली public किल्ली सगळ्यांना देतो. तुम्ही तुमचा card number त्या किल्लीने कुलूपबंद करता आणि पाठवता. रस्त्यातले सगळे बघतात: कचरा. उघडू फक्त Amazon शकतो, कारण उघडणारी किल्ली फक्त त्याच्याकडे. **किल्ली पाठवायची गरजच संपली.** बाहेर पत्रपेटीसारखं आहे: तोंड सगळ्यांसाठी उघडं (कोणीही पत्र टाकू शकतो), पण दार उघडायची चावी फक्त postman कडे. आणि उलटा रस्ता (कुलूप- किल्लीवरून उघड-किल्ली काढणं) इतका अवघड आहे की जगातल्या सगळ्या machines मिळूनही हजारो वर्षांत जमणार नाही (Part 1 ची आठवण: मोठे numbers; इथे ते ढाल बनले आहेत).

Browser मधला **https** आणि कुलूप-icon म्हणजे हेच: तुमच्या आणि त्या site च्या मध्ये ही यंत्रणा चालू आहे. (“ही site खरीच ती आहे” हे प्रमाणित करणाऱ्या दाखल्याचं नाव **certificate**: कुलूप-icon मागचा कागद.) आणि WhatsApp चं “**end-to-end encrypted**” म्हणजे एक पाऊल पुढे: किल्ल्या फक्त

तुमच्या आणि मित्राच्या phone वर; **मधली company** पण वाचू शकत नाही. रस्ता त्यांचा, लिफाफा तुमचा.

हे फक्त सुरक्षा नाही; **आर्थिक पाया** आहे. UPI च्या प्रत्येक व्यवहारात ही यंत्रणा काम करते: तुमचा PIN कधीच उघडा प्रवास करत नाही. लोक online card/PIN टाकायला तयार झाले **कारण** हे गणित त्यांच्या बाजूने उभं आहे. Encryption नसतं तर e-commerce नसता, UPI नसता, online banking नसती. भारताची digital economy एका अशा गणिती कल्पनेवर उभी आहे जिचं patent कोणी कधीच घेतलं नाही.

इथे लोक काय चुकीचं समजतात

”कुलूप-icon दिसतो म्हणजे site विश्वासाची.” **नाही**. कुलूप एवढंच सांगतं की **रस्ता** सुरक्षित आहे: मधला कोणी वाचू शकत नाही. पण पलीकडचा माणूस चोर असू शकतो! Scammer पण https site बनवतो (खर्च: 0 रुपये). Sealed लिफाफ्यात धमकीचं पत्रही येऊ शकतं; लिफाफा sealed आहे एवढ्याने पाठवणारा सज्जन ठरत नाही. कुलूप म्हणजे “बोलणं खाजगी आहे”; “समोरचा खरा आहे” हा वेगळा प्रश्न, आणि तो तुम्हालाच सोडवायचा: नाव नीट वाचा (amaz0n?), घाई करणाऱ्यावर शंका घ्या. रस्ता सुरक्षित; प्रवासी तपासा.

MAP वर

कोण कमावतं: certificates देणारा एक छोटा उद्योग आहे, पण खरा पैसा पलीकडे आहे: encryption मुळे जो **विश्वास** बनला, त्यावर trillion-dollar e-commerce उभा राहिला. हे pattern नीट बघा, तुमच्या नकाशासाठी महत्वाचं: **Level 3 ची एक boring, न-दिसणारी यंत्रणा (encryption) Level 2 च्या पूर्ण नव्या बाजाराला (online विक्री) जन्म देते**. पायाभूत थर जे कमावतो त्यापेक्षा, तो **वर काय उभं राहू देतो** त्यात मोठी कमाई असते. AI मध्ये हेच शोधा: AI कुठला नवा विश्वास / नवा बाजार उभा करू देतो?

स्वतः बघा (5 मिनिटं)

Browser मध्ये bank ची site उघडा, पट्टीतल्या कुलूप-icon वर click करा: “Connection is secure” आणि certificate ची माहिती दिसेल: कोणाला दिला, कधी संपतो. आता एखादी http (s नाही) site उघडायचा प्रयत्न करा: browser लाल इशारा देईल “Not secure”. तुम्ही आता तो इशारा **वाचू** शकता: “या रस्त्यावरचे packets उघडे आहेत, postcard सारखे.”

विचार करा

1. (derivation) सरकारांना कधीकधी वाटतं: “encryption मध्ये आम्हाला एक गुप्त मागचा दरवाजा (backdoor) द्या, फक्त गुन्हेगार पकडण्यासाठी.” तंत्रज्ञानाच्या बाजूने यात काय अडचण आहे? (विचार करा: दरवाजा “फक्त चांगल्या लोकांसाठी” असू शकतो का?)

उत्तर: गणिताला हेतू कळत नाही. Backdoor म्हणजे यंत्रणेत एक कमजोरी **मुद्दाम** ठेवणं; ती कमजोरी police ला पण उघडते आणि चोराला पण, एकदा सापडली की. आणि “फक्त सरकारजवळ किल्ली” म्हणजे जगातली सगळ्यात मोठी चोरी-लायक वस्तू एका जागी: त्या जागेवर हल्ले होणारच. म्हणून तंत्रज्ञ म्हणतात: दरवाजा अर्धा उघडा असू शकत नाही; उघडा किंवा बंद. हा वाद (privacy vs surveillance) जगभर चालू आहे, आणि तुम्हाला तो आता **दोन्ही** बाजूंनी समजतो: सरकारची गरज खरी, पण गणिती किंमतही खरी. अशा जागी “सोपं उत्तर सांगणारा” माणूस एक बाजू विकतोय, हे ओळखा.

Chapter 5.7 [SPINE]: Internet म्हणजे नक्की काय

आता सगळे तुकडे जोडता येतात. “Internet” हा शब्द रोज हजार वेळा वापरता; आज त्याला **डोव्यांनी** बघू.

सुरुवात तुमच्या घरातून. तुमचा phone WiFi ने router ला जोडलेला (Chapter 5.2). Router telecom company च्या (Jio/Airtel च्या) जाळ्याला: तुमच्या भागातल्या खांबावरच्या किंवा जमिनीतल्या fibre तारेला. ती तार शहराच्या मोठ्या केंद्राला. तिथून देशभरच्या मोठ्या रस्त्यांना. हे झालं **एका** company चं जाळं. (Internet चा रस्ता विकणाऱ्या अशा company चं नाव **ISP**: Jio, Airtel, BSNL. आणि तुमच्या घरापर्यंतचा शेवटचा तुकडा, **last mile**, हा सगळ्यात महाग असतो: प्रत्येक घराला वेगळी तार!)

पण Jio चा ग्राहक Airtel च्या ग्राहकाला message पाठवतो तेव्हा? दोघांची जाळी वेगळी आहेत. उत्तर: companies नी आपली जाळी ठरावीक जागी **एकमेकांना जोडली** आहेत (मोठ्या शहरांमधली exchange केंद्रं), आणि तिथे सौदेबाजी आहे: कधी “तू माझा traffic ने, मी तुझा” (बरोबरी; नाव **peering**), कधी लहान जाळं मोठ्याला भाडं देतं. **Internet म्हणजे हेच: हजारो वेगवेगळ्या मालकांची जाळी, एकमेकांना जोडलेली, समान नियमांनी (protocols, Chapter 5.4) बोलणारी.** नावही तेच सांगतं: inter-net, जाळ्यांमधली जोडणी. मालक कोणीच नाही; सहभागी सगळे. ST, railway आणि रिक्शा वेगवेगळ्या मालकांचे असूनही तुम्ही एका प्रवासात तिन्ही वापरता, तसंच.

आणि देशाबाहेर? इथे एक धक्का बसेल: **समुद्राखाली**. जगातला 99% आंतरदेशीय data समुद्रातल्या cables मधून जातो (**undersea cables**): बोटानेच जाड काचेच्या तारा, लाखो km, समुद्रतळाशी पसरलेल्या. (Satellite? 1% पेक्षा कमी: महाग आणि दूर, म्हणजे latency, Chapter 2.3.) भारतात या cables किनाऱ्यावर येतात: Mumbai आणि Chennai ही मोठी “landing” ठिकाणे. एक cable कापली गेली (जहाजाचा नांगर, भूकंप) की पूर्ण देशाचं internet मंद होतं: हे घडलं आहे, पुन्हा घडेल.

एकदा हे दिसलं की “cloud” आणि “WiFi” ची हवाहवाई भाषा उडते: internet म्हणजे काच, तांबं, खांब, समुद्री cables, buildings मधले routers. **वस्तू** आहे, जादू नाही. आणि वस्तू आहे म्हणून तिची मालकी आहे, किंमत आहे, राजकारण आहे: cables कोण टाकतो, exchange कुठे आहे, हे सगळे सत्तेचे प्रश्न.

Jio ची गोष्ट आता पूर्ण नजरेने घ्या. 2016 आधी भारतात data महाग होता कारण जाळं लहान होतं. Jio ने काय केलं: **~30 billion dollars** (2.5 लाख कोटी रुपये) लावून **नवं जाळं** बांधलं: fibre, towers, समुद्री cables चे हिस्से. मग data जवळजवळ फुकट केला (Chapter 3.7: दरवाजा-चाल). निकाल: देश online आला, आणि Jio दरवाजाचा मालक. पण मूळ गोष्ट ही: त्यांनी **सेवा** नाही, **रस्ता** बांधला. रस्ता बांधायला वेळ आणि billions लागतात; म्हणूनच तो कब्जा इतका टिकतो: स्पर्धकाला पुन्हा सगळे खांब उभे करावे लागतील.

इथे लोक काय चुकीचं समजतात

“Internet satellite ने चालतं” (नाही: 99% समुद्री cables). “Cloud म्हणजे हवा” (नाही: cloud म्हणजे दुसऱ्याच्या buildings मधले computers; Part 5 मध्ये पूर्ण). “WiFi म्हणजे internet” (नाही: WiFi फक्त तुमच्या घरातला शेवटचा 10 meter चा बिनतारी तुकडा; पुढे सगळं तार आहे). तिन्ही चुका एकाच मुळातून: internet दिसत नाही, म्हणून लोक त्याला हवा समजतात. आता तुम्हाला तो दिसतो: काच आणि तांबं.

MAP वर

रुपयाचा रस्ता, पूर्ण साखळी: तुमचे महिन्याचे 300 (Level 1 पासून घुसले) -> Jio (ISP) -> Jio cable-companies ना आणि यंत्रं विकणाऱ्यांना (Nokia, Ericsson, Cisco) -> ते chip वाल्यांना (TSMC!). आणि उलटी दिशाही आहे: Google/Meta आता स्वतः समुद्री cables टाकतात (2Africa, Blue-Raman: भारताला जोडणाऱ्या), कारण भाडं महाग पडतं आणि रस्ता आपल्या हातात हवा वाटतो. जेव्हा सेवा-company रस्ता विकत घेते, तेव्हा समजा: **रस्ता हीच खरी सत्ता होती.** (AI मध्ये आज हेच: model-companies chips आणि data-centers विकत घेतायत. Pattern ओळखा.)

स्वतः बघा (5 मिनिटं)

Laptop वर उघडा: submarinecablemap.com (फुकट आहे). Mumbai वर zoom करा: दहा-पंधरा cables किनाऱ्याला येताना दिसतील, नावांसह. प्रत्येक रेषा म्हणजे समुद्रतळाशी पडलेली खरी काच, आणि तुमचा कालचा WhatsApp message कदाचित त्याच रेषेने America ला गेला होता. एकदा हा नकाशा बघितला की “internet” शब्द पूर्वीसारखा कधीच वाटणार नाही.

विचार करा

1. (derivation) एक देश आपल्या देशापुरतं internet बंद करू शकतो (हे घडलं आहे). पण पूर्ण जगाचं internet कोणीही बंद करू शकत नाही. Design मधली कुठली गोष्ट हे दोन्ही सांगते?

उत्तर: देशातलं बंद शक्य आहे कारण देशाबाहेर जाणारे रस्ते **मोजके** असतात: किनाऱ्यावरची landing ठिकाणं आणि मोठे ISPs; सरकारने ते मोजके गळे आवळले की देश वेगळा पडतो. पण जगाचं बंद अशक्य, कारण **केंद्रच नाही**: हजारो जाळी, हजारो मालक, कोणतं एक switch नाही. हे अपघाताने नाही, design ने आहे: internet ची पायाभरणी अशा यंत्रणेसाठी झाली जी एक भाग पडला तरी चालेल (packets नव्या रस्त्यांनी जातात, Chapter 5.3). तुमच्या नकाशासाठी शिकवण: विकेंद्रित (decentralized) यंत्रणा मारायला अवघड, पण तिच्यावर हुकूमतही अवघड. केंद्रित यंत्रणा कमवायला सोपी, मारायलाही सोपी. प्रत्येक platform हा सौदा करतो; तुम्हीही कराल.

Chapter 5.8 [DEPTH]: Internet चे नियम कोण बनवतं

(DEPTH chapter. शेवटचा आहे आणि लहान आहे. यानंतर “standard” हा शब्द बातम्यांमध्ये दिसला की त्यामागचा सत्ता-खेळ दिसेल.)

Chapter 5.4 ने सांगितलं: internet म्हणजे नियम. Chapter 5.7 ने सांगितलं: मालक कोणीच नाही. मग एक विचित्र प्रश्न उरला: **नियम आहेत, पण राजा नाही. हे चालतं कसं? नियम बनवतं कोण?**

उत्तर अजब आहे: काही boring संस्था, जगभरचे engineers, आणि खूप सान्या meetings. गावचा आठवडी हाट आठवा: त्याचाही मालक कोणी नसतो, पण पंचायत, व्यापारी मंडळ आणि जुन्या रीती मिळून नियम चालवतात. इथेही तशीच तीन मुख्य नावं:

IETF: internet चे मूळ नियम (TCP, HTTP वगैरे) इथे घडतात. पद्धत सुंदर आहे: **कोणीही** एक प्रस्ताव लिहू शकतो; त्याला **RFC** म्हणतात. चर्चा होते; प्रस्ताव दोन गोष्टींनी जिंकतो: सहमती आणि **चालणारा code**. सदस्यत्व नाही, fee नाही, देशाचा veto नाही. 1969 पासून हे असंच चालतं.

ICANN: नावं आणि numbers ची वाटणी (domains, IP blocks) सांभाळणारी संस्था. .com कोण चालवेल, नवे प्रकार (.shop, .ai) कधी येतील, हे इथे ठरतं.

W3C आणि browser-companies: web चे नियम (HTML वगैरे). आणि इथे एक खरं सत्य: कागदावर नियम समिती बनवते, पण **व्यवहारात** नियम ते होतात जे मोठे browsers (Chrome!) खरोखर लागू करतात. Google च्या browser कडे 60%+ वाटा आहे; म्हणजे Google च्या निर्णयांना नियमांची ताकद आपोआप येते.

चित्र असं: नियम खुले आहेत, प्रक्रिया लोकशाहीसारखी, **पण** ताकद सारखी वाटलेली नाही: जिच्याकडे users जास्त, तिच्या आवाजाला वजन जास्त. म्हणून मोठ्या companies या boring meetings मध्ये आपले पगारी engineers पाठवतात: standard च्या table वर बसणं म्हणजे भविष्याचा बाजार आखणं. (सर्वमान्य नियमाचं नाव **standard**.)

हे लांब वाटलं तर एक भारतीय उदाहरण सगळं सांगेल: UPI चे नियम NPCI बनवते. आणि म्हणूनच भारताने “आपला protocol आपण बनवणं” हा राष्ट्रीय खेळ केला आहे; त्याचं नाव **DPI** (Digital Public Infrastructure): UPI, Aadhaar-stack, ONDC. नियम बनवणारा बाजार **आखतो**; नियम घेणारा बाजारात फक्त **खेळतो**.

Standard ची लढाई कशी दिसते, त्याची एक गोष्ट: SMS हाही एक standard होता, telecom वाल्यांचा. WhatsApp आलं आणि SMS ला खाऊन टाकलं: कारण WhatsApp चं “standard” एका company चं होतं, वेगाने बदलता येत होतं, आणि फुकट होतं. मग Google ने SMS चा वारस (RCS) पुढे ढकलला; Apple ने वर्षानुवर्ष तो **नाकारला** (iMessage चा किल्ला!), आणि 2024 मध्ये अखेर घेतला. या पूर्ण लढाईत ग्राहक कुठेच नव्हता; लढाई standards ची होती, कारण जो standard जिंकतो त्याची सेवा “default” होते, आणि default होणं म्हणजे billions ग्राहक फुकट मिळणं.

इथे लोक काय चुकीचं समजतात

“कुठलीतरी जागतिक संस्था (UN सारखी) internet चालवते.” नाही. UN ची एक संस्था (ITU) जुनं telecom सांभाळते, पण internet चे खरे नियम वरच्या खुल्या संस्थांमध्ये घडतात. आणि काही देश (ज्यांना केंद्रित नियंत्रण हवं) हे बदलायचा प्रयत्न सतत करतात: “internet ITU खाली आणा” ही त्यांची मागणी असते. म्हणजे “नियम कोण बनवतं” हा प्रश्न संपलेला नाही; ती एक **चालू** लढाई आहे, आणि तिच्यात भारत एक मोठा आवाज आहे (billions users + स्वतःचं DPI).

MAP वर

तुमच्या 4-level नकाशावर ही शेवटची ओळ: **Level 3 च्या खाली अजून एक मजला आहे: नियम. जो नियम बनवतो, तो Level 3 च्या सगळ्या खेळांची जमीन आखतो.** नियम बनवण्यात पैसा **थेट** नाही (IETF फुकट, NPCI ना-नफा), पण नियमांवर उभे राहणारे बाजार billions चे असतात, आणि नियम बनवणाऱ्याच्या देशाला/company ला त्या बाजारात **घरचं मैदान** मिळतं. म्हणून: बातम्यांमध्ये “standard”, “protocol”, “regulation” हे शब्द दिसले की थांबा आणि वाचा. कंटाळवाणे वाटतात, पण तिथे पुढच्या दशकाची मालकी ठरते आहे. AI मध्ये **हीच** लढाई आता चालू आहे: AI चे नियम कोण बनवणार? (Book 2 मध्ये भेटेल.)

स्वतः बघा (5 मिनिटं)

Google मध्ये search करा: “RFC 1149”. उघडेल तो एक **खरा**, अधिकृत RFC आहे: कबुतरांवरून internet packets पाठवण्याचा (विनोदी) प्रस्ताव, 1 April ला लिहिलेला. आणि लोकांनी तो 2001 मध्ये खरा करूनही दाखवला (9 packets, 1 कबुतर). हे बघण्याचा हेतू: RFC किती खुले आहेत हे जाणवणं: विनोदही चालतो, आणि त्याच पद्धतीने जगाचं सगळं internet ही घडलं आहे. खुली प्रक्रिया अशी दिसते.

विचार करा

1. (derivation) भारताने UPI बनवला आणि तो Singapore, UAE, France अशा देशांमध्येही नेतो आहे. “Protocol निर्यात करण्यात” भारताला काय मिळतं? पैसा तर थेट मिळत नाही. (Chapter 5.4 ची NPCI-शिकवण पुढे न्या.)

उत्तर: जमीन मिळते. ज्या देशात UPI-धर्तीचे नियम लागले, तिथे भारतीय banks, apps आणि कौशल्य (हे चालवता येणारे लोक) घरच्या मैदानावर खेळतात. आणि जागतिक व्यवहारात वजन मिळतं: आज पैसे पाठवण्याचे जागतिक नियम Visa/Mastercard/SWIFT (पश्चिमी) आखतात; प्रत्येक देश जो UPI घेतो, तो त्या एकाधिकाराला थोडा कमजोर करतो आणि नियम-table वर भारताची खुर्ची मोठी करतो. हे Level 3 चं भू-राजकारण आहे: 20व्या शतकात देश तेल आणि बंदरांसाठी लढले; 21व्या मध्ये cables, chips आणि protocols साठी लढतात. तुम्ही बनवलेला नकाशा (4 levels) देशांच्या पातळीवरही तंतोतंत लागू होतो: हीच त्याची शेवटची परीक्षा.

PART 3 चा शेवट: नकाशा आणि पुढचं पाऊल

जे तुमच्याकडे आता आहे

CLIENT / SERVER	मागणारी machine / देणारी machine; मालमत्ता server च्या बाजूला असते
N SQUARED	प्रत्येकाला प्रत्येकाशी जोडणं अशक्य; जो हे चौक/protocol ने सोडवतो तो मालक होतो
IP ADDRESS	machine ची number-पत्ता; public vs private (router = building ची watchman)
PACKETS	message चे numbered तुकडे, स्वतंत्र प्रवास; "connection" हा फक्त भास आहे
TCP vs UDP PROTOCOL	बरोबरपणा vs वेग: file vs video call आधीच ठरलेले नियम; internet म्हणजे वस्तू नाही, नियमांचा समूह (UPI = protocol!)
DNS	नाव -> number ची phonebook; शिडी + cache; domain = पाटी, server = दुकान
ENCRYPTION	public रस्त्यावर गणिती लिफाफा; दोन किल्ल्या (public कुलूप लावते, private उघडते); कुलूप = रस्ता सुरक्षित, माणूस नाही!
INTERNET	हजारो मालकांची जाळी, जोडलेली; 99% समुद्री cables; केंद्र नाही, मालक नाही
NIYAM	IETF/ICANN/W3C + मोठ्या companies; standard बनवणारा बाजार आखतो (भारताचं DPI)

एक परीक्षा, स्वतःसाठी

पुस्तक न उघडता, बोलून उत्तर द्या:

1. WhatsApp message थेट मित्राच्या phone ला का जात नाही?
2. Video call मध्ये आवाज robotic का होतो, पण file download मध्ये file कधीच अर्धवट का येत नाही?
3. कुलूप-icon (https) काय सांगतो आणि काय सांगत नाही?
4. UPI ला "protocol" का म्हणावं, app का नाही? आणि त्याने भारताला काय दिलं?

अडाल तर: 1 -> 5.1, 2 -> 5.3, 3 -> 5.6, 4 -> 5.4/5.8.

Part 4 मध्ये काय आहे

Server ला कोटी लोकांचा data मिळतो. तो ठेवायचा कुठे? एका कोटीमधून एक नोंद शोधायची कशी, डोळ्याच्या पापणी लवण्याच्या आत? दोन लोक एकाच वेळेस एक गोष्ट बदलतात तेव्हा काय होतं? आणि सगळा data उडाला तर? Part 4: DATA आणि आठवणी: database चं जग, जिथे प्रत्येक मोठ्या company ची खरी संपत्ती पडलेली असते.

PART 3 चा MINI-GLOSSARY

cache	उत्तर लक्षात ठेवणं, परत शिडी न चढण्यासाठी (5.5)
certificate	"ही site खरीच ती आहे" चा दाखला (5.6)
client / server	मागणारी / देणारी machine (5.1)
DNS	नाव-ते-number phonebook, शिडी-रचनेची (5.5)
domain	website चं नाव; भाड्याची नोंद, मालमत्ता-पाटी (5.5)
DPI	भारताचा सरकारी-protocol खेळ: UPI, Aadhaar (5.8)
encryption	गणिती लिफाफा; वाचू फक्त किल्लीवाला शकतो (5.6)
end-to-end	किल्ल्या फक्त दोन टोकांवर; मधली company पण आंधळी (5.6)
HTTP / HTTPS	web चा protocol / त्याचं सुरक्षित रूप (5.4, 5.6)
ICANN	नावं आणि numbers वाटणारी संस्था (5.8)
IETF / RFC	मूळ नियम बनवणारी संस्था / खुला प्रस्ताव (5.8)
IP address	machine चा number-पत्ता (IPv4 जुना, IPv6 नवा) (5.2)
ISP	internet-रस्ता विकणारी company: Jio, Airtel (5.7)
last mile	तुमच्या घरापर्यंतचा शेवटचा (महाग) तुकडा (5.7)
latency	एक फेरा किती वेळ; bandwidth पेक्षा वेगळी गोष्ट (5.3)
n squared	जोड संख्येच्या वर्गाने वाढतात ही अडचण (5.1)
network	जोडलेल्या machines चं जाळं (5.1)
packet	message चा numbered, स्वतंत्र तुकडा (5.3)
peering	दोन जाळ्यांची "तू माझा, मी तुझा" सौदेबाजी (5.7)
private key	उघडणारी किल्ली; कधीच कोणाला नाही (5.6)
protocol	आधीच ठरलेले बोलण्याचे नियम (5.4)
public key	कुलूप लावणारी किल्ली; जगाला वाटलेली (5.6)
router	packets पुढे ढकलणारा चौक; घरचा watchman (5.2)
standard	सर्वमान्य नियम; बनवणारा बाजार आखतो (5.8)
TCP / UDP	बरोबरपणा-वाले / वेग-वाले नियम (5.3)
undersea cable	समुद्री काच-तार; जगाचा 99% data इथून (5.7)

THE MACHINE

PART 4 OF 5: DATA आणि आठवणी

पाच PARTS चा नकाशा

PART 1	पैसा आणि MACHINE	झाला
PART 2	MACHINE शी बोलणं	झाला
PART 3	MACHINES चं जाळं	झाला
[तुम्ही इथे आहात]		
PART 4	DATA आणि आठवणी	storage, database, शोधणं
PART 5	जगाला SERVE करणं	scale, cloud, खर्च, पूर्ण जोड

आधी Part 3 ची परीक्षा

पाच प्रश्न, Part 3 मधले. आधी स्वतः उत्तर द्या, मग खाली बघा. अडलात तर सोबतच्या chapter वर परत जा.

1. WhatsApp message थेट मित्राच्या phone ला का जात नाही? (5.1)
2. Video call मध्ये आवाज robotic का होतो, पण download झालेली file कधीच अर्धवट का नसते? (5.3)
3. कुलूप-icon (https) काय सांगतो आणि काय सांगत **नाही**? (5.6)
4. UPI ला "protocol" का म्हणावं, app का नाही? (5.4)
5. Internet चा मालक कोण आहे? (5.7)

उत्तर: 1. प्रत्येक message आधी company च्या server ला जातो, मग मित्राला; मध्ये तिसरी machine आहेच. 2. Call ला UDP-विचार: गेलेला क्षण परत मागवून उपयोग नाही, वेग महत्वाचा; file ला TCP: प्रत्येक तुकडा पोहोचलाच पाहिजे, किंमत म्हणून वेळ. 3. कुलूप सांगतं की **रस्ता** सुरक्षित आहे (मधला कोणी वाचू शकत नाही); पलीकडचा माणूस खरा आहे हे सांगत **नाही**. 4. कारण UPI म्हणजे खुले नियम आहेत, जे पाळणारं कुठलंही app/bank खेळात सामील होतं; NPCI ने n-squared ला protocol ने मारलं. 5. कोणीच नाही: हजारो मालकांची जाळी, समान नियमांनी जोडलेली; सेवांचे मालक आहेत, रस्त्याचा नाही.

या part मध्ये काय आहे

Part 3 ने message रस्त्यावर आणला. आता तो server वर पोहोचला. आणि तिथे जगातल्या प्रत्येक business चा सगळ्यात मोठा खजिना पडलेला आहे: **data**. Crores लोकांची खाती, orders, chats, आठवणी.

सात chapters: हा खजिना ठेवायचा कुठे (database), 10 million मधून एक नोंद डोळ्याच्या पापणीत शोधायची कशी (index), data चे आकार कुठले (shapes), दोघांनी एकाच क्षणी लिहिलं तर काय (transactions), कामाला रांगेत कसं लावायचं (queues), जे पुन्हा पुन्हा लागतं ते जवळ कसं ठेवायचं (cache), आणि सगळं हरवलं तर काय (backup).

हे तुमच्या business decision मध्ये कुठे येईल: तुमच्या business ची खरी मालमत्ता code नाही; **data** आहे (Part 3, Chapter 5.1: ज्याच्याकडे data, त्याच्याकडे ताकद). तो शोधता आला नाही तर धंदा हळू; तो चुकला तर ग्राहक गेला; तो हरवला तर धंदा **मेला**. हा part म्हणजे त्या मालमत्तेची राखण.

SECTION 6: लक्षात ठेवणं

सात chapters. Data ठेवण्यापासून तो वाचवण्यापर्यंत.

हे तुमच्या business decision मध्ये कुठे येईल: “App हळू का आहे” याचं उत्तर 10 पैकी 7 वेळा database मध्ये असतं. “Records जुळत नाहीत” ही तक्रार transactions च्या समजेशिवाय सुटत नाही. आणि “सगळा data गेला” ही बातमी ज्या company ची येते, ती बहुतेक पुन्हा उभी राहत नाही. या section नंतर तुम्ही engineer ला तीन नेमके प्रश्न विचारू शकाल: शोध कशावर चालतो, पैशाच्या नोंदी transaction मध्ये आहेत का, आणि backup **तपासला** आहे का.

Chapter 6.1 [SPINE]: विसरण्याची problem

Part 2 चं एक सत्य आठवा (Chapter 3.6): process मेली की तिची state जाते. जे वाचवायचं ते storage मध्ये लिहावं लागतं. मग सोपं वाटतं: file मध्ये लिहून टाका. संपलं?

दुकानाच्या वहीने सुरुवात करू, कारण file म्हणजे तेच: एक वही. छोट्या दुकानाला वही पुरते. आता दुकान वाढतंय. बघा वही कुठे कुठे तुटते:

नोंदी crores झाल्या. “पाटील यांची उधारी किती?” वही पान-पान चाळावी लागते. 10 million नोंदींची file म्हणजे तेच: उत्तरासाठी सगळं वाचणं.

लिहिणारे अनेक झाले. गल्ल्यावर तुम्ही, मागे भाऊ, online भाचा: तिघे **एकाच** वहीत एकाच वेळेस लिहू लागले तर? नोंदी एकमेकांवर चढतील, काही हरवतील. Files सोबत नेमकं हेच होतं: दोन programs नी एकाच वेळेस लिहिलं की भेळ.

प्रश्न अवघड झाले. “या महिन्यात 500 पेक्षा जास्त उधारी असलेले, गावाबाहेरचे ग्राहक कोण?” वहीत उत्तर **आहे**, पण विखुरलेलं. File पण असंच: data आहे, उत्तर काढायचं म्हणजे स्वतः recipe लिहा.

अर्धवट लिहिताना वीज गेली. नोंद अर्धी लिहिली गेली: “पाटील, 500 जमा...” आणि पुढचं गायब. आता वहीवर विश्वास कसा ठेवायचा?

चारही अडचणी एकत्र बघा, आणि design स्वतः सुचतं: वही **प्रत्येकाच्या हातात देणं** चूक आहे. एक वेगळा, कायम चालणारा program हवा, ज्याचं **एकच** काम आहे: data ठेवणं. सगळे programs त्याला **विनंती** करतील (“ही नोंद ठेव,” “हे शोधून दे”); वहीला हात फक्त तोच लावेल. तो रांग लावेल, अर्धवट नोंदी होऊच देणार नाही, शोध वेगवान ठेवेल.

या program चं नाव **database**. आणि लक्ष द्या: database म्हणजे file नाही; तो एक **server** आहे (Part 3 ची भाषा: देणारी machine), जो दिवसरात्र चालू असतो आणि data चा एकमेव दरबान असतो. सोसायटीचं office आठवा: कागदपत्रं सदस्यांच्या घरी नाही, office मध्ये असतात; पाहिजे ते secretary कडे **मागावं** लागतं, आणि नोंदी तोच करतो. नियम जाचक वाटतात, पण 500 सदस्यांची सोसायटी त्यांच्याशिवाय कोसळेल.

तुमच्या भोवतीचं जग आता या नजरेने बघा: bank चं प्रत्येक खातं एका database मध्ये आहे. Zomato ची प्रत्येक order, WhatsApp चा प्रत्येक group, IRCTC चं प्रत्येक ticket. जगातला जवळजवळ प्रत्येक business शेवटी एका database भोवती बांधलेला आहे, जसं जुनं गाव विहिरीभोवती.

इथे लोक काय चुकीचं समजतात

”Excel आहे ना, तोच आमचा database.” छोट्या कामाला खरंच पुरतो, आणि तिथे तो उत्तम आहे. पण वरच्या चार तुटी आठवा: Excel ची file एकाच वेळेस अनेकांनी लिहिली की भेळ होते, 1 million ओळींवर तो धापा टाकतो, आणि अर्धवट लिहिणं त्याला कळत नाही. हजारो businesses “Excel वर चाललय ना” म्हणत वाढले आणि एका दिवशी orders हरवायला लागल्या. Excel वही आहे; database दरबान आहे. दुकान केवढं आहे यावर निवड ठरते; पण फरक **माहीत** असणं founder चं काम आहे.

MAP वर

Database हा Level 3 च्या आतला एक अख्खा देश आहे: Oracle या एका company ने databases विकून decades राज्य केलं (आजही ~50 billion dollars ची company). मग open source आलं (Chapter 3.7): Postgres, MySQL फुकट झाले, आणि आता cloud companies तेच भाड्याने देतात (Part 5). आणि एक खोल गोष्ट: business विकला जातो तेव्हा खरेदीदार काय विकत घेतो? Building नाही, code ही नाही: **database** विकत घेतो: ग्राहक कोण, काय घेतात, किती देतात. Data हीच मालमत्ता, बाकी सगळं आवरण.

स्वतः बघा (5 मिनिटं)

तुमच्या bank च्या app मध्ये statement उघडा आणि filter लावा: फक्त “debit”, फक्त “या महिन्यातले”. अर्ध्या second मध्ये उत्तर आलं. आता कल्पना करा हेच काम एका कागदी वहीत करायचं: पान-पान, खूण करत. तुम्ही आता database ला काम करताना बघितलंत: crores नोंदींमधून, अटी लावून, क्षणात. पुढचा chapter सांगेल हे **कसं** शक्य आहे.

विचार करा

1. (derivation) भाजीवाल्याला वही पुरते; bank ला database लागतो. मधली रेषा नेमकी कुठे आहे? तीन प्रश्न बनवा, जे कुठल्याही business ला विचारून “वही की database” ठरवता येईल.

उत्तर: (1) **लिहिणारे किती?** एक माणूस: वही चालते. एकाच वेळेस अनेक (दुकानं, apps, branches): database. (2) **नोंदी किती?** हजारांत: काहीही चालेल. Millions मध्ये: शोधासाठी database. (3) **चुकीची किंमत काय?** भाजीची एक नोंद चुकली: 20 रुपये गेले. Bank ची एक नोंद चुकली: विश्वास गेला, license गेलं. जिथे चुकीची किंमत मोठी, तिथे दरबान लागतो. लक्षात घ्या: तिन्ही प्रश्न technology चे नाहीत, **business** चे आहेत. Tech ची निवड नेहमी business च्या प्रश्नांनी सुरू होते; हत्यारांनी नाही.

Chapter 6.2 [SPINE]: 10 million मधून एक शोधणं

मागच्या chapter चं वचन: database crores नोंदींमधून क्षणात शोधतो. आज ते वचन उघडून बघू, कारण आतली युक्ती technology तल्या सगळ्यात सुंदर युक्त्यांपैकी आहे, आणि ती तुम्हाला आधीच येते.

आधी problem नीट मोजा. 10 million (1 कोटी) नोंदी आहेत; “खाते क्रमांक 88231904 दाखवा.” सरळ रस्ता: पहिल्या नोंदीपासून बघत चला. सरासरी अर्ध्या नोंदी तपासाव्या लागतील: 5 million वाचनं, आणि disk वरून (Part 1: बाहेर जाणं महाग) म्हणजे कित्येक seconds, कधी minutes. प्रत्येक ग्राहकाला प्रत्येक प्रश्नाला minutes? धंदा मेला.

आता तुमच्या हातातली युक्ती: **शब्दकोश**. त्यात 1 lakh शब्द असतात. तुम्ही “पैसा” शोधताना पान-पान वाचता का? नाही. मधलं पान उघडता: “नमस्कार” दिसलं. “पैसा” त्याच्या **पुढे** येतो, म्हणजे मागचं अर्थ पुस्तक **फेकून दिलंत**. उरलेल्याच्या मध्ये परत तेच. प्रत्येक पावलाला अर्थ शिल्लक.

हे किती जोरात आकसतं ते मोजा: 10 million -> 5 -> 2.5 -> ... फक्त **~23 पावलांत** एक नोंद उरते. 10 million चा शोध, 23 वाचनांत. Chapter 1.4 मध्ये doubling बघितलं होतं (प्रत्येक switch ने दुप्पट); हे त्याचंच उलटं: प्रत्येक पावलाला अर्थ. दुप्पट जितकं वेगाने वर जातं, अर्थ तितकंच वेगाने खाली येतं.

पण थांबा: ही युक्ती चालली कारण शब्दकोश **लावलेला** (sorted) आहे. Database च्या नोंदी येतात तशा पडलेल्या असतात. मग database काय करतो: मूळ नोंदी जिथे आहेत तिथे ठेवतो, आणि बाजूला एक **लावलेली यादी** बनवतो: फक्त खाते क्रमांक, क्रमाने, आणि प्रत्येकापुढे खूण “पूर्ण नोंद कुठे पडली आहे.” पुस्तकाच्या शेवटची सूची जशी: विषय अक्षर-क्रमाने, पुढे पान क्रमांक. या लावलेल्या यादीचं नाव **index**, आणि database ला विचारलेल्या प्रश्नाचं नाव **query**.

फुकट काहीच नसतं, मग index ची किंमत काय? दोन गोष्टी: **जागा** (यादी वेगळी ठेवावी लागते) आणि **लिहिण्याचा वेग** (नवी नोंद आली की मूळ वहीत पण लिहा आणि यादीतही योग्य जागी घुसवा; याद्या पाच असतील तर पाच ठिकाणी). म्हणून प्रत्येक column वर index लावत नाहीत; जिथे शोध वारंवार होतो तिथेच. दुकानदार उधारीची वेगळी यादी ठेवतो, पण “कोण कुठल्या रंगाचा shirt घेतो” ची यादी ठेवत नाही: यादीचं ओझं शोधाच्या फायद्यापेक्षा मोठं होईल.

आता जगाकडे बघा: Aadhaar system 1.4 billion लोकांमधून तुमची नोंद seconds मध्ये काढते: index. तुमच्या phone ची contact-यादी: नावं लावलेली आहेत, म्हणून शोध instant. आणि Google म्हणजे तर पूर्ण **web चा index**: त्यांचे programs जगाची पानं वाचून एक राक्षसी लावलेली यादी बनवतात (“हा शब्द या-या पानांवर”); तुमचा शोध त्या यादीत होतो, जगात नाही. एका index ने trillion-dollar company उभी आहे.

इथे लोक काय चुकीचं समजतात

”Computer वेगवान आहे, म्हणून शोध वेगवान.” नाही: **रचना** मुळे शोध वेगवान आहे. Index नसलेल्या column वर 10 million नोंदींची query द्या: तोच “वेगवान” computer minutes खाईल, app timeout होईल, ग्राहक जाईल. जगातल्या “app हळू आहे” तक्रारींपैकी मोठा हिस्सा म्हणजे कुठेतरी एक query विना-index चालते आहे. Machine ला दोष; गुन्हा रचनेचा.

MAP वर

रुपयाचा रस्ता: जो शोध वेगवान करतो तो कमावतो. प्रत्येक marketplace चं हृदय शोध आहे (Amazon वर वस्तू, Naukri वर उमेदवार); शोध 1 second हळू झाला की विक्री मोजून घटते (Chapter 2.3: Google/Amazon ची मोजमापं). म्हणून database tune करणारे consultants रोजंदारीवर लाखो घेतात, आणि “search जलद करा” एवढ्याच कामाच्या companies आहेत (Elasticsearch, Algolia). आणि आठवा: Google ने index नीट बनवला, एवढ्यावर जगातली सगळ्यात मोठी दुकानदारी उभी राहिली.

स्वतः बघा (5 मिनिटं)

Phone च्या contacts मध्ये कुठलंही नाव शोधा: instant, कारण नावांवर index आहे. आता gallery त जुना एखादा **विशिष्ट** photo शोधा (मागच्या दिवाळीचा तो एक): scroll, scroll, scroll... कारण “photo मधलं दृश्य” यावर index नाही; तुम्ही 5,000 नोंदी सरळ चाळताय. Index असणं आणि नसणं यातला फरक तुम्ही आता हाताने अनुभवलात. (आणि Google Photos “beach” शोधून देतं, कारण त्यांनी AI ने photos चे **वर्णन-index** बनवले आहेत: Book 2 ची झलक.)

विचार करा

1. (derivation) Index इतका जादुई आहे तर प्रत्येक column वर का लावू नये? आणि यातून काढा: कुठल्या business ला जास्त indexes परवडतात: जिथे **वाचणं** खूप आणि लिहिणं कमी (news site), की जिथे **लिहिणं** खूप (sensor चा data दर second)?

उत्तर: प्रत्येक index प्रत्येक **लिहिण्याला** महाग करतो: नवी नोंद आली की सगळ्या याद्या update. दहा indexes = एक insert दहा जागी. News site: दिवसातून शंभर लेख लिहिले, millions वेळा वाचले: indexes भरपूर लावा, वाचन राजा आहे. Sensor-data: दर second हजारो नोंदी लिहायच्या, वाचायच्या क्वचित: indexes कमीत कमी, नाहीतर लिहिणं बुडेल. धडा तोच जुना: प्रत्येक रचना एक **सौदा** आहे, आणि सौदा तुमच्या **कामाच्या आकारावर** ठरतो. “Best” रचना नसते; तुमच्या business ला जुळणारी रचना असते.

Chapter 6.3 [SPINE]: आठवणींचे वेगवेगळे आकार

घरातलं सामान तुम्ही कसं ठेवता, कधी लक्ष दिलंय? कपडे कपाटाच्या **खणांत**. किराणा **पिशव्यांत**, आणि मोठ्या पिशवीत छोट्या पिशव्या. किल्ल्या **खुंटीवर**, प्रत्येकीला नावाचा tag. तीन वेगळ्या पद्धती, आणि तुम्ही त्या विचार न करता निवडता, कारण **सामानाचा आकार पद्धत ठरवतो**.

Data चंही तसंच आहे. तीन मुख्य आकार:

खण (table). ओळी आणि रकाने. प्रत्येक ओळ = एक नोंद; प्रत्येक रकाना = एक गुण. Bank ची passbook हीच आहे: तारीख, तपशील, रक्कम, शिल्लक. जिथे नोंदी **एकसारख्या** आहेत आणि त्यांची **बेरीज-वजाबाकी, जोड-तुलना** करायची आहे, तिथे table राजा. Table शी बोलायची खास भाषा Part 2 मध्ये भेटली होती: **SQL** (“जिथे उधारी > 500, ती नावं दे”). Tables वाल्या database चं कुळ: Oracle, MySQL, **Postgres**.

पिशवी (document). काही नोंदी एकसारख्या नसतात; त्यांच्या आत पोट-नोंदी असतात. एक Instagram post बघा: मजकूर, photos ची यादी, आणि comments; प्रत्येक comment च्या आत replies, likes. हे खणांत बसवायचं म्हणजे ओढाताण; ही तर पिशवीत-पिशवी रचना आहे. अशा आकाराचं नाव **document**, आणि कुळ: MongoDB.

खुंटी (key-value). सगळ्यात सोपा आकार: एक नाव (key), त्यापुढे एक value. प्रश्न एकच विचारता येतो: “या नावाचं काय आहे?” पण म्हणूनच वीज-वेगवान. OTP ठेवायला (phone number -> “482913, 5 मिनिटांत संपेल”), login sessions ठेवायला. कुळ: Redis.

आता महत्त्वाचं वाक्य: **आकार कामावरून ठरतो, fashion वरून नाही**. एकाच Zomato च्या आत तिन्ही आहेत: पैशाचे व्यवहार tables मध्ये (हिशोबाला खण लागतात), restaurant चे menus documents मध्ये (प्रत्येकाची रचना वेगळी), आणि “आत्ता कोण login आहे” खुंटीवर (वेगासाठी). योग्य आकार निवडला की सगळं सोपं; चुकीचा निवडला की प्रत्येक दिवस ओढाताण, जशी किराणा कपाटाच्या खणांत ठेवण्याची.

(चौथा आकार जन्माला येतोय: AI साठी **vector**, जिथे “अर्थाने जवळचं” शोधता येतं. Book 2 मध्ये पूर्ण भेटेल.)

इथे लोक काय चुकीचं समजतात

दोन टोकांच्या चुका. पहिली: “SQL जुनं झालं, नवं Mongo-वगैरे वापरू.” हिशोब, पैसा, नोंदी-जुळणी: इथे tables चा 50 वर्षांचा राजेशाही अनुभव आहे; जुनं म्हणजे कालबाह्य नाही, जुनं म्हणजे **तावून-सुलाखलेलं** (Part 2 ची COBOL-गोष्ट आठवा). दुसरी: “एकच database सगळ्याला पुरेल.” लहान असताना खरंच पुरतो (आणि तेव्हा एकच ठेवावा!); वाढल्यावर आकार वेगळे होतात. चूक निवडीची नाही; निवड **का** केली हे माहीत नसण्याची असते.

MAP वर

आकारा-आकारावर companies उभ्या आहेत: tables (Oracle चं जुनं साम्राज्य, आता Postgres चं फुकट राज्य), documents (MongoDB, ~2 billion dollars वार्षिक धंदा), खुंटी (Redis). आणि आता AI च्या vector आकारासाठी नव्या companies चा सुळसुळाट: नवा आकार जन्मला की त्याच्याभोवती नवा बाजार जन्मतो, प्रत्येक वेळेस. तुमच्यासाठी खूण: जेव्हा data चा **नवा आकार** दिसायला लागतो, तेव्हा त्याच्या राखणदारीचा धंदा अजून रिकामा असतो.

स्वतः बघा (5 मिनिटं)

तुमच्या घरात एक फेरी मारा आणि तीन आकार शोधा: खण कुठे (कपाट, मसाल्याचा डबा), पिशवीत-पिशवी कुठे (किराणा, प्रवासाची bag), खुंटी-tag कुठे (किल्ल्या, चार्जर). प्रत्येक जागी विचारा: ही पद्धत **का**? खण = एकसारख्या गोष्टी, तुलना हवी. पिशवी = आत-आत रचना. खुंटी = एका नावावरून लगेच उचलायचं. Data चे आकार तुम्हाला आधीच येतात; फक्त नावं नवीन होती.

विचार करा

1. (derivation) एका bank ने खाती chat-सारख्या document आकारात ठेवली तर काय बिघडेल? (विचार करा: bank चे रोजचे प्रश्न कुठले असतात?)

उत्तर: Bank चे प्रश्न जोड-तुलनेचे असतात: “सगळ्या खात्यांची बेरीज जुळते का,” “या branch चे आजचे सगळे व्यवहार,” “500 पेक्षा जास्त शिल्लक असलेले.” हे प्रश्न खणांवर (tables) क्षणात चालतात, कारण प्रत्येक नोंदीचा ढाचा सारखा आहे आणि रकाने थेट जोडता येतात. Document आकारात प्रत्येक उत्तरासाठी पिशव्या उघडत बसावं लागेल, हळू आणि चुकांना जागा. आणि उलटंही तितकंच खरं: chat ला tables मध्ये कोंबणं म्हणजे प्रत्येक reply साठी नव्या खणांची ओढाताण. धडा: आधी **प्रश्न** लिहा, मग आकार निवडा. Data कसा दिसतो यापेक्षा, त्याला काय **विचारणार** हे design ठरवतं.

Chapter 6.4 [DEPTH]: दोघं एकाच क्षणी लिहितात तेव्हा

(DEPTH chapter. पण जर तुमच्या business मध्ये पैसा किंवा मोजकं सामान असेल: seats, stock, appointments, तर हा chapter तुमच्यासाठी SPINE च आहे.)

एक खरा प्रसंग: सिनेमाचं शेवटचं ticket शिल्लक आहे. तुम्ही आणि अजून कोणी, दोन वेगळ्या शहरांत, **एकाच क्षणी** “Book” दाबता. दोघांच्याही screens नी आत्ताच दाखवलं होतं: “1 seat शिल्लक.” काय व्हायला हवं? आणि काय होऊ **शकतं**?

आत काय घडतं ते हळू करून बघा. दोन्ही विनंत्या server वर पोहोचल्या. दोन्ही processes नी database ला विचारलं: “seat 14 रिकामी?” दोघांना उत्तर मिळालं: “हो” (कारण दोघांनी **एकाच वेळेस** विचारलं, कोणीच अजून लिहिलं नव्हतं). मग दोघांनी लिहिलं: “seat 14 माझी.” दोन तिकिटं, एक खुर्ची. सिनेमागृहात भांडण.

या घोट्याचं नाव **race** (शर्यत): दोन कामं एकाच data वर धावली, आणि निकाल कोण-आधी-पोहोचला या नशिबावर ठरला. Part 2 आठवा: OS पाळ्या इतक्या वेगाने फिरवतो की “एकाच वेळेस” रोजची गोष्ट आहे. मग इलाज?

पहिला इलाज तुम्ही रोज बघता: **रांग लावा**. लग्नात आहेर लिहिणारी वही एकच असते आणि धरणारा एकच; म्हणून नोंदी कधी चढत नाहीत. Database त याचं नाव **lock**: seat 14 ला हात लावण्याआधी कुंडी लावा; काम संपेपर्यंत दुसरा **थांबतो**. पहिल्याचं booking पूर्ण, कुंडी उघडली, दुसऱ्याला आता खरं उत्तर मिळतं: “seat गेली.” भांडण संपलं; किंमत: थोडी वाट.

पण अजून एक खोल problem उरली आहे. पैसे पाठवणं म्हणजे **दोन** नोंदी: तुमच्या खात्यातून 500 वजा, त्याच्या खात्यात 500 जमा. पहिली नोंद झाली आणि **त्याच क्षणी** वीज गेली तर? पैसे तुमच्याकडून गेले पण कोणाला मिळालेच नाहीत: system नेच चोरी केली. अर्धवट लिहिणं हे न-लिहिण्यापेक्षा वाईट आहे.

म्हणून database एक करार देतो, आणि हा करार या chapter चं हृदय आहे: **“ही कामं एक गट्टा आहेत: एकतर सगळी होतील, नाहीतर एकही नाही.”** या गट्ट्याचं नाव **transaction**. वजा+जमा एका transaction मध्ये बांधा; मध्ये वीज गेली तर database स्वतः अर्ध काम **उलटं फिरवतो** (rollback), जणू काही झालंच नव्हतं. रजिस्ट्रीची उपमा: घर विकणं म्हणजे पैसे देणं + नाव बदलणं; registrar office दोन्ही एकत्रच करतं. “पैसे दिले पण नाव जुनंच” ही अवस्था कायद्याने **अस्तित्वातच** येऊ दिली जात नाही.

आता तुमचा रोजचा अनुभव नव्याने वाचा: tatkal booking च्या वेळेस “payment झालं पण ticket नाही मिळालं; पैसे 3 दिवसांत परत” हे घडतं, तेव्हा तुम्ही transaction चा rollback अनुभवता: गट्टा पूर्ण होऊ शकला नाही, म्हणून system ने उलटं फिरवलं. चिडचिड होते, पण उलटी बाजू बघा: rollback **नसता** तर पैसे जाऊन ticket ही नाही, आणि कोणालाच पत्ता नाही.

इथे लोक काय चुकीचं समजतात

“Computer ला काय, एकाच वेळेस हजार कामं करतो.” करतो; पण **एकाच वहीवर** हजार हात एकाच वेळेस चालू शकत नाहीत; शेवटी नोंद एकच आहे. कुठेतरी रांग लागतेच: प्रश्न फक्त हा की रांग **design** केलेली आहे (lock, transaction) की नशिबावर सोडलेली (race). आणि दुसरी चूक: “असं क्वचितच होत असेल.” Flash sale, tatkal, exam form ची शेवटची तारीख: तुमच्या business चा सगळ्यात **महत्त्वाचा** क्षण हाच असतो जेव्हा सगळे एकदम येतात. Race नेमकी तेव्हाच धावते जेव्हा चूक सगळ्यात महाग असते.

MAP वर

हे जमवणारे लोक Level 3 मधले सगळ्यात महाग engineers आहेत: payment systems, stock exchanges, booking engines. कारण सूत्रात टाका: चुकीची किंमत प्रचंड (पैसा थेट), आणि जमणारे कमी (scarcity). NPCI च्या UPI ची रोजची **~600 million** transactions प्रत्येक वेळेस हा करार पाळतात: वजा-जमा नेहमी गळ्यात. एक race घोटाळा राष्ट्रीय बातमी होईल. जिथे “बरोबर असणं” हाच माल आहे, तिथे हा chapter म्हणजे पूर्ण business.

स्वतः बघा (5 मिनिटं)

पुढच्या वेळेस UPI payment करताना जाणीवपूर्वक बघा: “Processing...” चा क्षण. त्या क्षणात दोन banks च्या दोन databases मध्ये एक गट्टा बांधला जातोय: इथून वजा, तिथे जमा, दोन्हींची खात्री, मगच तुम्हाला हिरवी खूण. आणि कधी “pending” दिसलं तर आता घाबरू नका: गट्टा अर्धा अडकलाय आणि system तो पूर्ण करेल किंवा उलटा फिरवेल; तिसरी अवस्था करारानेच बंद आहे.

विचार करा

1. (derivation) दोन ATMs मधून, एकाच खात्यातून, एकाच क्षणी 800-800 काढण्याचा प्रयत्न झाला; खात्यात 1,000 आहेत. Race न होता हे कसं थांबवलं जातं, ते वरच्या दोन हत्यारांनी (lock, transaction) क्रमाने सांगा.

उत्तर: दोन्ही विनंत्या एकाच खात्याच्या नोंदीवर येतात; database त्या नोंदीला lock लावतो, म्हणजे एक विनंती **आधी** पूर्ण चालते: transaction सुरू -> शिल्लक तपासा (1,000) -> 800 वजा -> नोंद 200 -> गट्टा पूर्ण -> कुंडी उघडा. आता दुसरी विनंती आत येते, आणि तिला **ताजी** शिल्लक दिसते: 200. “पुरेसे पैसे नाहीत,” नकार. दोघांना 800 कधीच मिळत नाहीत, कारण “तपासणं आणि वजा करणं” हा गट्टा अर्ध्यात कोणी घुसू शकत नाही. हीच रचना seats, stock, appointments सगळीकडे: **मोजकी वस्तू + अनेक हात = lock + transaction**. तुमच्या business मध्ये मोजकं काय आहे, हे आत्ताच लिहून ठेवा; तिथेच ही रचना लागेल.

Chapter 6.5 [DEPTH]: कामाला रांगेत लावणं

(DEPTH chapter. लहान आहे, आणि यानंतर “हे काम आत्ता, ते नंतर” ही design ची नजर मिळते: ती founder ची नजर आहे.)

Zomato वर order पडली. आता किती कामं करायची आहेत मोजा: पैसे घ्या, order नोंदवा, restaurant ला कळवा, delivery वाल्याला शोधा, तुम्हाला SMS पाठवा, email पाठवा, loyalty points द्या, हिशोबाच्या वहीत नोंद करा. आठ कामं.

सोपं design: सगळी आठ कामं **आत्ता** करा, मग user ला “order झाली” दाखवा. काय बिघडेल? User थांबलेला आहे: आठ कामं x प्रत्येकाचा वेळ = फिरणारं चक्र, आणि Chapter 2.3 ची आठवण: उशीर = गेलेला ग्राहक. आणि वाईट: SMS पाठवणारी system **पडली** असेल तर? आठपैकी एक काम अडलं म्हणून पूर्ण order अडली? SMS साठी धंदा थांबवायचा?

आता हॉटेलचा waiter बघा; त्याला हे design आधीच येतं. तो तुमची order घेतो, kitchen च्या खिडकीत **चिठ्ठी लावतो**, आणि लगेच पुढच्या table कडे जातो. तो स्वयंपाक होईपर्यंत उभा राहत नाही. चिठ्ठ्या रांगेत लागतात; cooks आपल्या वेगाने त्या उचलतात. Waiter चं काम: मागणी **नोंदवणं**. Kitchen चं काम: ती **पूर्ण करणं**. दोघांचे वेग वेगळे, आणि म्हणूनच दोघांना जोडणारी **रांग** मध्ये आहे.

Software मध्ये या रांगेचं नाव **queue**, आणि या पद्धतीचं नाव **async** (आत्ता नोंदवा, नंतर करा). Order च्या आठ कामांना आता दोन रंगांत वाटा:

आत्ताच झालीच पाहिजेत: पैसे घेणं आणि order नोंदवणं (हे मागच्या chapter चा transaction-गट्टा). हे झालं की user ला हिरवी खूण: “Order झाली!” वेळ: 1-2 seconds.

नंतर चालतील: SMS, email, points, हिशोब. यांच्या चिठ्ठ्या रांगेत टाकल्या. वेगळे कामगार-programs (workers) त्या सावकाश उचलतात. SMS system पडली? चिठ्ठ्या रांगेत **साठून राहतात**; system उठली की निघतात. User ला पत्ताही नाही; त्याची order कधीच अडली नाही.

ही रचना तुमच्या रोजच्या अनुभवात उघड दिसते: WhatsApp वर message पाठवताना **घड्याळ-icon** दिसतं (तुमच्या phone च्या रांगेत आहे), मग एक tick (server च्या रांगेत पोहोचला), मग दोन (मित्राला मिळाला). “पाठवलं” म्हणजे “पोहोचलं” नाही; “रांगेत नोंदवलं.” आणि म्हणूनच network नसतानाही तुम्ही message “पाठवू” शकता: रांग वाट बघते. Diwali च्या मध्यरात्री सगळ्यांचे greetings उशिरा पोहोचतात ते म्हणूनच: रांगा भरलेल्या; पण **हरवत** कोणीच नाही.

इथे लोक काय चुकीचं समजतात

“चांगलं system म्हणजे सगळं realtime.” उलटं आहे: चांगलं system म्हणजे **नेमकं** काय realtime हवं हे माहीत असलेलं. सगळंच आत्ता करायचं तर सगळ्यांची ताकद सगळ्या क्षणी लागते: खर्च टोकाचा, आणि एक भाग पडला की सगळं पडतं. सरकारी office जेव्हा “अर्ज घेतला, पावती घ्या, निकाल टपालाने येईल” म्हणतं, तेव्हा ते (नकळत) उत्तम queue design आहे: घेणं वेगवान, करणं सावकाश. चीड येते ती रांग **किती लांब** आहे याची; रांग **असण्याची** नाही.

MAP वर

रुपयाचा रस्ता: रांगा चालवणं हा स्वतः infra-धंदा आहे (Kafka, RabbitMQ, cloud च्या queue-सेवा). पण तुमच्यासाठी मोठी गोष्ट खर्चाची आहे: “आत्ताच” ची ताकद **सगळ्यात महाग** असते; “नंतर चालेल” ची स्वस्त, कारण ती सावकाश, रात्री, स्वस्त machines वर चालवता येते. म्हणून product design करताना प्रत्येक feature ला विचारा: हे user समोर उभं राहून व्हायला हवं, की चिठ्ठी लावून? या एका प्रश्नाने server चं bill दुप्पट-अर्धं होतं. (Part 5 मध्ये हा हिशोब पूर्ण भेटेल.)

स्वतः बघा (5 मिनिटं)

Phone airplane mode वर टाका आणि कोणाला तरी WhatsApp message लिहा. घड्याळ-icon दिसेल: तुमची चिठ्ठी रांगेत आहे, वाट बघते आहे. आता airplane mode काढा: tick, मग दोन ticks, आपोआप. तुम्ही आता एक queue पूर्ण चक्र चालताना बघितली: नोंदवलं, थांबलं, जमलं तेव्हा झालं, हरवलं काहीच नाही.

विचार करा

1. (derivation) Cinema ticket booking system design करताय. ही कामं दोन रंगांत वाटा: seat अडवणं, पैसे घेणं, ticket चा QR बनवणं, SMS पाठवणं, “आवडेल असे shows” सुचवणं. कुठलं आत्ता, कुठलं रांगेत? आणि एक काम **मुद्दाम** अवघड आहे: QR कुठे टाकाल?

उत्तर: Seat अडवणं + पैसे: आत्ताच, एका transaction-गठ्यात (Chapter 6.4: मोजकी वस्तू + अनेक हात). SMS आणि सुचवणं: रांगेत, उशीर चालतो. QR मधली गोम: user ला ticket **लगेच** दिसायला हवं (theatre ला निघालाय!), म्हणजे “नंतर चालेल” म्हणता येत नाही; पण QR बनवणं म्हणजे फक्त एक चित्र काढणं, त्यात मोजकं काही अडवायचं नाही, म्हणून transaction मध्ये कोंबूही नये. उत्तर: transaction पूर्ण होताच, रांगेच्या **सगळ्यात पुढच्या** टोकाला, आणि screen वर “ticket तयार होतंय...” ची दोन seconds. धडा: “आत्ता vs नंतर” हे दोनच रंग नाहीत; “किती नंतर चालेल” ही एक **मोजपट्टी** आहे, आणि design म्हणजे प्रत्येक कामाला तिच्यावर योग्य जागी ठेवणं.

Chapter 6.6 [DEPTH]: जे पुन्हा पुन्हा लागतं ते जवळ ठेवा

(DEPTH chapter. “Cache” हा शब्द Part 3 मध्ये (DNS) आणि Part 1 मध्ये (मंडईची फेरी) डोकावून गेला होता. आज त्याचा पूर्ण chapter, कारण Part 5 च्या खर्चाच्या हिशोबाचा हा पाया आहे.)

एक प्रसंग: India-Pakistan match चालू आहे. 100 million लोक दर second ला score विचारतायत. आणि score आहे database मधल्या **एका** नोंदीत. 100 million queries, एकाच नोंदीवर, दर second? कुठलाही database जळून जाईल.

पण थांबा: 100 million लोकांना उत्तर **एकच** द्यायचंय: “IND 287/4.” प्रश्न एक, उत्तर एक, विचारणारे 100 million. मग database पर्यंत 100 million वेळा जाण्यात अर्थ काय? **एकदा** जा, उत्तर लिहून घ्या, आणि पुढच्या सगळ्यांना **लिहिलेलं** दाखवा. Score बदलला की नवं आणा.

हीच ती युक्ती जिचं नाव **cache**: महाग रस्त्याने मिळवलेलं उत्तर जवळ ठेवणं, पुन्हा-पुन्हा लागणार म्हणून. तुमच्या घरात ती आधीच आहे: विहीर लांब आहे म्हणून घरात **माठ** भरून ठेवता; प्रत्येक घोटाला विहिरीवर जात नाही. Cook सकाळी **एकदाच** मंडई करतो (Chapter 2.3); दुकानदार गल्ल्याजवळ **चिल्लर** ठेवतो, तिजोरी मागच्या खोलीत. प्रत्येक वेळेस रचना तीच: लांबचं महाग, जवळचं स्वस्त, म्हणून वापरातलं जवळ आणा.

Machine च्या जगात ही रचना **प्रत्येक थरावर** आहे, एकात एक: तुमच्या phone मध्ये (app च्या images साठवलेल्या: airplane mode त chats दिसतात त्या इथूनच), server च्या समोर (score सारखी तयार उत्तरं, Redis सारख्या खुंटी-store मध्ये: Chapter 6.3), database च्या आत (गरम नोंदी RAM च्या मेजावर: Chapter 2.5), आणि CPU च्या आतही. Part 1 चं सूत्र आठवा: आतलं nanosecond, बाहेरचं millisecond, फरक 1 million पट. Cache म्हणजे तो फरक **खिशात घालणं**.

पण फुकट काहीच नाही. Cache ची किंमत एका शब्दात: **शिळेपणा**. माठातलं पाणी कालचं असू शकतं. Score cache केला आणि wicket पडली: काही seconds लोकांना जुना score दिसतो. आता design चा खरा प्रश्न आला: **किती जुनं चालेल?** Cricket score: 2-3 seconds चालतं, कोणाचं काही बिघडत नाही, आणि खर्च 100 million पर्तींनी वाचतो. News चा मथळा: minute चालेल. Bank balance दाखवणं: काही seconds कदाचित; पण **व्यवहार करताना** शिल्लक तपासणं: शिळं **कधीच** चालणार नाही; तिथे थेट database, transaction च्या कुंडीसह (Chapter 6.4). एकाच app मध्ये दोन्ही असतात, आणि कुठे कुठलं हे **ठरवणं** म्हणजेच design.

इथे लोक काय चुकीचं समजतात

”Data नेहमी ताजाच हवा.” ऐकायला छान; प्रत्यक्षात 90% कामांत काही seconds चा शिळेपणा कोणाच्या लक्षातही येत नाही, आणि तिथेच खर्च 10-100 पट घटतो. उलटी चूकही आहे: cache केलं आणि **विसरले**: जुनी किंमत दाखवली गेली, ग्राहकाने जुन्या भावात order केली, भांडण. Cache “कधी रिकामा करायचा” (invalidation) हा software मधल्या सगळ्यात कुप्रसिद्ध अवघड प्रश्नांपैकी आहे. म्हणून नियम: cache लावताना **त्याच क्षणी** लिहा: हे उत्तर किती जुनं चालेल, आणि शिळं दिसलं तर काय बिघडेल. उत्तर “काहीच नाही” असेल तरच लावा.

MAP वर

रुपयाचा रस्ता: Hotstar cricket च्या दिवशी crores एकाच वेळेस बघतात; ते cache शिवाय अशक्य आणि परवडणारंही नाही: एक उत्तर बनतं, crores ना वाटलं जातं; प्रति-प्रेक्षक खर्च जवळजवळ शून्य (leverage चा निखळ अवतार). आणि हीच कल्पना जगभर पसरवलेली एक अख्खी industry आहे: **CDN** (Cloudflare, Akamai): तुमच्या site च्या प्रती जगभरच्या शहरांमध्ये cache करून ठेवणारी. Part 5 मध्ये ती पूर्ण भेटेल. खूण लक्षात ठेवा: **जो एकच उत्तर लाखोंना विकतो, तो cache वर उभा आहे.**

स्वतः बघा (5 मिनिटं)

कुठलंही जड app (Instagram/YouTube) उघडा, एक post/video बघा, बंद करा. आता airplane mode लावून पुन्हा उघडा: आता बघितलेलं बरंचसं **दिसतं**, नवं काही येत नाही. जे दिसलं तो तुमच्या phone मधला cache आहे: लांबून आणलेलं, जवळ ठेवलेलं. आणि आता तुम्हाला तुमच्या phone चं “storage भरलं: cache साफ करा” हे बटणही कळलं: ती जवळ ठेवलेली उत्तरं फेकणं, जागेसाठी.

विचार करा

1. (derivation) तुमच्या दुकानाच्या app मध्ये दोन आकडे दिसतात: वस्तूची **किंमत** आणि “फक्त 2 शिल्लक!” हा **stock**. कुठला cache कराल, कुठला नाही, आणि का? (Chapter 6.4 ची आठवण ठेवा.)

उत्तर: किंमत: cache करा (minute-दोन minutes); ती क्वचित बदलते, आणि बदलल्यावर cache रिकामा करायचा नियम ठेवा. Stock **दाखवणं:** थोडं शिळं चाललं (“2 शिल्लक” प्रत्यक्षात 1 झालं असेल: फार तर उत्सुकता वाढेल). पण stock **वजा करणं**, म्हणजे order घेताना: कधीच cache नाही; ते Chapter 6.4 चं मोजक्या वस्तूचं युद्ध आहे: थेट database, lock, transaction. एकाच आकड्याचे दोन वापर, दोन नियम. हीच या part ची एकत्रित शिकवण: **दाखवणं स्वस्त आणि शिळं चालतं; बदलणं महाग आणि ताजंच लागतं.** हे वाक्य design च्या अर्ध्या चुका थांबवतं.

Chapter 6.7 [SPINE]: Data हरवला तर?

आधी एक सत्य जे कोणी सांगत नाही: **disks मरतात**. नक्की मरतात. प्रत्येक hard disk/SSD चं एक आयुष्य आहे; datacenter मध्ये रोज कुठली ना कुठली मरते. आणि disk एकटी मारेकरी नाही: building ला आग, पूर, चुकून `delete` दाबणारा माणूस (सगळ्यात मोठा मारेकरी हाच), आणि ransomware: तुमचाच data कुलूपबंद करून खंडणी मागणारे चोर.

आता तुमच्या business चा विचार करा: Part 4 ने आत्तापर्यंत सांगितलं की data हीच मालमत्ता. मग प्रश्न “disk मेली तर” नाही; “disk मेली तेव्हा” आहे. उत्तर माणसाला हजारो वर्ष माहीत आहे: सोनं एका जागी ठेवत नाहीत. थोडं घरात, बाकी locker मध्ये; महत्त्वाच्या कागदपत्रांच्या प्रती माहेरी. **प्रती (copies)**. पण प्रतींमध्ये दोन वेगळ्या जाती आहेत, आणि त्या मिसळणं ही जीवघेणी चूक आहे:

जिवंत प्रत (replication). दुसरी machine, जिच्यावर प्रत्येक नोंद त्याच क्षणी पुन्हा लिहिली जाते. मुख्य मेली, ही लगेच पुढे येते: धंदा थांबत नाही. दुकानाचा भाऊ जसा: गळ्यावरचा आजारी पडला तरी दुकान उघडं. **पण**: चुकून delete केलंत, तर भाऊ **तेही** इमाने इतबारे पुन्हा करतो. जिवंत प्रत चुकीपासून वाचवत **नाही**; ती चूकही copy करते.

काळ-प्रत (backup). काल रात्रीचा (किंवा दर तासाचा) पूर्ण photo, वेगळ्या जागी ठेवलेला, आणि मुख्य म्हणजे **मागे वळता येणारं**: “शुक्रवार रात्रीसारखं करून दे.” चुकून delete, ransomware, software च्या bug ने बिघडवलेला data: या सगळ्यांवर उत्तर फक्त हे. जुनी रोजकीर्द जशी: आजची वही जळाली तरी कालची शिल्लक नोंदलेली आहे.

दोन्ही लागतात, कारण शत्रू वेगळे आहेत: machine मरण्यावर replication; माणूस/चोर चुकण्यावर backup. आणि ठेवायचं कसं, याचा जगमान्य नियम लक्षात ठेवण्याइतका सोपा आहे: **3-2-1**. एकूण **3** प्रती; **2** वेगळ्या प्रकारच्या जागांवर; **1** दुसऱ्या **गावात** (building जळाली तरी गाव जळत नाही).

आणि आता या chapter ची सगळ्यात महत्त्वाची ओळ, अधोरेखित करा: **जो backup परत आणून बघितला नाही, तो backup नाहीच**. जगात रोज कुठेतरी हे नाटक घडतं: “backup आहे ना” म्हणत सगळे निश्चित; संकट आलं; backup उघडला; तो रिकामा/अर्धा/जुनाट निघाला: घेणारी recipe महिन्यांपूर्वी चुपचाप बिघडली होती, आणि कोणी **तपासलंच** नव्हतं. Fire drill जसं: नळी आहे म्हणून खात्री नाही; पाणी **येतं** का हे दर महिन्याला बघावं लागतं. Restore ची rehearsal हा backup चा भाग आहे, extra नाही.

तुमच्या स्वतःच्या आयुष्यात हे सगळं आधीच आहे: phone हरवला, नवा घेतला, Google Photos मधून सगळे photos परत आले. ते replication + backup तुमच्यासाठी कोणीतरी चालवत होतं, महिन्याच्या 130 रुपयांत (Chapter 2.1 चं भाडं). आता तेच तुमच्या business साठी जाणीवपूर्वक उभं करायचं आहे.

इथे लोक काय चुकीचं समजतात

”Cloud मध्ये आहे, म्हणजे कायमचं सुरक्षित.” Cloud म्हणजेही कोणाच्यातरी building मधल्या disks च (Part 5); त्या मरतात, accounts बंद होतात, आणि ransomware तुमच्या laptop वरून cloud च्या sync मध्येही चढतो (चूक copy होते: replication ची जात!). आणि दुसरी, engineers चीही चूक: “replication आहे ना, मग backup कशाला”: आता तुम्हाला माहीत आहे, त्या दोन वेगळ्या जाती आहेत, वेगळ्या शत्रूंसाठी. दोन्ही प्रश्न विचारायला शिका: “मुख्य मेली तर किती वेळात उभे?” आणि “चुकून पुसलं तर किती मागे जाता येतं?”

MAP वर

रुपयाचा रस्ता: backup/recovery ही स्वतः एक industry आहे (Veeam, Druva: ~billions revenue), आणि विम्यासारखी विकली जाते: रोज खर्च वाटतो, एका दिवशी company **वाचवते**. आकडे बोलतात: ज्या छोट्या companies चा data मोठ्या प्रमाणात जातो, त्यांतल्या बहुतेक वर्षभरात बंद पडतात; data गेला की Level 2 च मरतो, Level 3 ची चूक असूनही. आणि उलट बाजू: ransomware टोळ्या खंडणी **धंदा** म्हणून चालवतात; त्यांचा पूर्ण खेळच “यांच्याकडे तपासलेला backup नसेल” या bet वर आहे. तुमचा backup म्हणजे त्यांच्या धंद्याला तुमचा नकार.

स्वतः बघा (5 मिनिटं)

तुमच्या स्वतःच्या आयुष्याचा 3-2-1 तपासा: तुमचे photos/documents किती जागी आहेत? Phone एकच जागा असेल, तर आज रात्री Google Photos/Drive चालू करा: 15 मिनिटांत तुमचं 3-2-1 चालू होईल. आणि एक पाऊल पुढे: Drive उघडून एक जुनी file **परत आणून** बघा; आता तुम्ही जगातल्या त्या मोजक्या लोकांत आहात ज्यांनी आपला restore **तपासला** आहे.

विचार करा

1. (derivation) तुमचा backup रोज रात्री 2 वाजता होतो. दुपारी 3 वाजता disk मेली. काय हरवलं? आणि यातून business चा एक निर्णय काढा: backup **किती वेळाने** घ्यावा हे कशावरून ठरतं?

उत्तर: रात्री 2 ते दुपारी 3 मधलं **सगळं:** 13 तासांच्या orders, नोंदी, chats. म्हणजे backup ची वारंवारता हा technology चा नाही, **business** चा निर्णय आहे: “आपल्याला किती तासांचं काम हरवणं परवडतं?” भाजी-दुकान: एक दिवस चालेल, रोज रात्री पुरे. UPI: एक second ही नाही; म्हणून तिथे प्रत्येक नोंद त्याच क्षणी अनेक जागी जाते (replication) आणि वर backups वेगळे. या “किती हरवणं चालेल” च्या मोजपट्टीवर तुमचा business कुठे आहे, हे एका ओळीत लिहा; तुमच्या engineer शी होणारं अर्ध बोलणं तिथून सुरू होईल. (आणि Part 4 इथे संपतो त्याच वाक्यावर ज्याने सुरू झाला: data हीच मालमत्ता; बाकी सगळं तिची राखण.)

PART 4 चा शेवट: नकाशा आणि पुढचं पाऊल

जे तुमच्याकडे आता आहे

DATABASE	data चा एकमेव दरबान: वेगळा, कायम चालणारा program; file नाही, server आहे
INDEX	लावलेली यादी + खुणा; 10 million मधून ~23 पावलांत; किंमत: लिहिणं हळू, जागा जास्त
QUERY / SQL आकार	database ला विचारलेला प्रश्न / tables ची भाषा खण (table), पिशवी (document), खुंटी (key-value); आकार कामावरून, fashion वरून नाही
TRANSACTION	"सगळं किंवा काहीच नाही" चा गठा; अर्धवट अवस्था अस्तित्वातच नाही (rollback)
LOCK / RACE	मोजकी वस्तू + अनेक हात = कुंडी; नाहीतर शर्यत आणि दोन तिकिटं एक खुर्ची
QUEUE / ASYNC	आत्ता नोंदवा, नंतर करा; waiter ची चिठ्ठी; पडलेली system = साठलेली रांग, हरवलेलं काही नाही
CACHE	महाग उत्तर जवळ ठेवणं; किंमत: शिळेपणा; दाखवणं शिळं चालतं, बदलणं ताजं च लागतं
BACKUP vs REPLICATION 3-2-1	काळ-प्रत (चुकीपासून वाचवते) vs जिवंत प्रत (machine मरण्यापासून; पण चूकही copy करते) 3 प्रती, 2 प्रकारच्या जागा, 1 दुसऱ्या गावात; आणि: न तपासलेला backup = backup नाहीच

एक परीक्षा, स्वतःसाठी

पुस्तक न उघडता, बोलून उत्तरं द्या:

1. Excel आणि database मध्ये खरा फरक काय? (नुसतं "मोठा" चालणार नाही; तीन तुटी सांगा.)
2. 10 million नोंदींमधून शोध 23 पावलांत कसा होतो?
3. पैसे पाठवताना "वजा झाले पण जमा नाही झाले" ही अवस्था का अस्तित्वात येऊ शकत नाही?
4. Backup असूनही company का बुडू शकते? (दोन कारणं.)

अडलात तर: 1 -> 6.1, 2 -> 6.2, 3 -> 6.4, 4 -> 6.7.

Part 5 मध्ये काय आहे

आता तुमच्याकडे पूर्ण machine आहे: recipe चालते (Part 1-2), जग जोडलेलं आहे (Part 3), आठवण पक्की आहे (Part 4). शेवटचा प्रश्न उरला, आणि तो थेट पैशाचा आहे: **हे सगळं लाखो लोकांसाठी, एकाच वेळेस, परवडणाऱ्या खर्चात कसं चालवायचं?** Cloud म्हणजे नक्की काय आणि त्याचं bill कसं वाचायचं, traffic अचानक दहापट झाला तर काय, जगभर वेगवान कसं राहायचं (CDN), आणि शेवटी: एक खरं product उघडून त्यात या पूर्ण पुस्तकाचा नकाशा बघणं. Part 5: जगाला SERVE करणं. शेवटचा आणि सगळ्यात मोठा part.

PART 4 चा MINI-GLOSSARY

3-2-1	प्रतींचा नियम: 3 प्रती, 2 जागा-प्रकार, 1 दूर (6.7)
async	आत्ता नोंदवा, नंतर करा (6.5)
backup	काळ-प्रत; मागे वळण्याची सोय (6.7)
cache	महाग उत्तर जवळ ठेवणं; शिळेपणाची किंमत (6.6)
database	data चा दरबान-server; file नाही (6.1)
document	पिशवीत-पिशवी आकार (Instagram post) (6.3)
index	लावलेली यादी + खुणा; शोधाचं इंजिन (6.2)
invalidation	cache कधी रिकामा करायचा हा अवघड प्रश्न (6.6)
key-value	खुंटी-tag आकार; OTP, sessions (6.3)
lock	कुंडी: एका वेळेस एक हात (6.4)
query	database ला विचारलेला प्रश्न (6.2)
race	दोन कामांची शर्यत; निकाल नशिबावर (6.4)
replication	जिवंत प्रत; चूकही copy करते (6.7)
restore	backup परत आणणं; न तपासलेला = नाहीच (6.7)
rollback	अर्धा गढा उलटा फिरवणं (6.4)
SQL	tables शी बोलायची भाषा (6.3)
staleness	शिळेपणा: cache मधलं जुनं उत्तर (6.6)
table	खण-आकार: ओळी x रकाने; हिशोबाचा राजा (6.3)
transaction	सगळं-किंवा-काहीच-नाही चा गढा (6.4)
queue / worker	कामांची रांग / ती उचलणारा program (6.5)

THE MACHINE

PART 5 OF 5: जगाला SERVE करणं

पाच PARTS चा नकाशा

PART 1	पैसा आणि MACHINE	झाला
PART 2	MACHINE शी बोलणं	झाला
PART 3	MACHINES चं जाळं	झाला
PART 4	DATA आणि आठवणी	झाला
[तुम्ही इथे आहात : शेवटचा part]		
PART 5	जगाला SERVE करणं	scale, cloud, खर्च, पूर्ण जोड

आधी Part 4 ची परीक्षा

पाच प्रश्न, Part 4 मधले. आधी स्वतः उत्तर द्या, मग खाली बघा.

1. Excel आणि database मध्ये खरा फरक काय? (6.1)
2. 10 million नोंदींमधून शोध ~23 पावलांत कसा? (6.2)
3. "वजा झाले पण जमा नाही" ही अवस्था का येऊ शकत नाही? (6.4)
4. "दाखवणं शिळं चालतं, बदलणं ताजंच लागतं" चा अर्थ? (6.6)
5. Backup आणि replication मध्ये फरक काय? (6.7)

उत्तर: 1. Excel वही आहे, database दरबान: अनेक हात, crores नोंदी, अवघड प्रश्न, अर्धवट लिहिणं: चारही ठिकाणी वही तुटते. 2. लावलेली यादी (index) + प्रत्येक पावलाला अर्ध फेकणं (शब्दकोशाची युक्ती). 3. कारण transaction: दोन्ही नोंदी एक गट्टा; अर्धा झाला तर database स्वतः उलटा फिरवतो (rollback). 4. Score/किंमत दाखवणं cache मधून चालतं (काही seconds जुनं चालेल); पण stock वजा करणं, पैसे हलवणं: थेट database, कुंडीसह. 5. Backup = काल-रात्रीची काळ-प्रत, चुकीपासून वाचवते; replication = जिवंत प्रत, machine मरण्यापासून वाचवते पण चूकही copy करते. दोन्ही लागतात.

या part मध्ये काय आहे

तुमच्याकडे आता पूर्ण machine आहे: recipe चालते (Part 1-2), जग जोडलेलं आहे (Part 3), आठवण पक्की आहे (Part 4). हा part शेवटचे दोन दरवाजे उघडतो:

Section 7: हे सगळं लाखो लोकांसाठी, एकाच वेळेस, परवडणाऱ्या खर्चात कसं चालतं: scale, cloud, वेग, आणि bill.

Section 8: सगळं जोडून: एक खरं product उघडून त्यात पूर्ण पुस्तक बघणं, काय बनवणं सोपं-अवघड, spec कसा लिहायचा, गोष्टी कशा तुटतात, कोण जिंकतं-मरतं, आणि शेवटी Book 1 ची परीक्षा.

हे तुमच्या business decision मध्ये कुठे येईल: हा पूर्ण part च business आहे. इथे technology आणि पैसा एकाच टेबलावर बसतात: server चं bill, ग्राहकाचा वेग, पडणं-सावरणं, आणि “मी काय बनवू, काय विकत घेऊ, काय टाळू” हे founder चे रोजचे निर्णय.

SECTION 7: खूप लोकांना SERVE करणं

नऊ chapters. एका user पासून crores पर्यंत, आणि त्याच्या bill पर्यंत.

हे तुमच्या business decision मध्ये कुठे येईल: “Viral झालो आणि app पडलं” ही startup ची सगळ्यात महाग गोष्ट आहे: संधी दारात आली आणि दार तुटलं. आणि उलटी बाजू: “चाललं, पण bill ने मारलं.” हा section दोन्हीपासून वाचवतो: गर्दी झेलणं, कधीही बंद न पडणं, वेगवान राहणं, आणि हे सगळं करताना खर्चाचा meter वाचता येणं.

Chapter 7.1 [SPINE]: एका user पासून लाखांपर्यंत

तुमचं app चाललं. 100 users: सगळं छान. एका दिवशी कुठल्या celebrity ने उल्लेख केला, आणि संध्याकाळपर्यंत 100,000 लोक दारात. Server धापा टाकतो, app “loading...” वर अडकतं, नवे users वैतागून जातात. हा क्षण प्रत्येक startup च्या स्वप्नात आणि दुःस्वप्नात एकाच वेळेस असतो.

आता design तुम्ही करा. गर्दी दहापट झाली; रस्ते किती?

पहिला रस्ता: मोठी machine घ्या. जास्त CPU, जास्त RAM. नाव: **vertical scaling** (उभं वाढणं). हलवायाची उपमा: दिवाळीत मागणी दहापट झाली; हलवाई स्वतः दहापट वेगाने काम करायचा प्रयत्न करतो. सोपं आहे (काहीच बदलावं लागत नाही), पण दोन भिंती आहेत: एक, जगातली सगळ्यात मोठी machine ही **एक** मर्यादा आहे; तिच्यापुढे रस्ता बंद. दोन, दुप्पट ताकदीची machine दुप्पट नाही, चौपट महाग असते. आणि तिसरी, सगळ्यात वाईट: ती **एकच** आहे; ती मेली तर सगळं मेलं.

दुसरा रस्ता: अनेक machines लावा. दहा सामान्य servers, प्रत्येक गर्दीचा दहावा हिस्सा घेतो. नाव: **horizontal scaling** (आडवं वाढणं). हलवाई आता दहा counters उघडतो, प्रत्येकावर एक माणूस. हाच खरा खेळ आहे, कारण भिंत नाही: अजून गर्दी? अजून counters. पण दोन नवे प्रश्न जन्मतात, आणि दोन्हीची उत्तरं तुम्ही आधीच शिकला आहात:

प्रश्न 1: कुठला ग्राहक कुठल्या counter वर? दारात एक वाटप करणारा लागतो: आलेली प्रत्येक विनंती जो मोकळ्या server कडे पाठवतो. नाव: **load balancer**: मिठाई दुकानाचा token देणारा माणूस, ST stand चा “platform 4 वर जा” सांगणारा फळा. साधी कल्पना, पण आता पूर्ण system चं दार हाच आहे.

प्रश्न 2: state कुठे? Part 2 आठवा (Chapter 3.6): process ची state. User ची cart server क्रमांक 3 च्या memory त ठेवली, आणि पुढची विनंती load balancer ने server 7 कडे पाठवली: cart गायब! उत्तर Part 4 ने दिलं आहे: state servers वर ठेवायचीच नाही; ती database/cache मध्ये (सामायिक जागी) ठेवायची, आणि servers **रिकामे** (stateless) ठेवायचे: कुठलाही server, कुठलीही विनंती, उत्तर तेच. Counters वरचे कारागीर बदलले तरी चालतं, कारण order ची वही एकाच गळ्यावर आहे.

हे खऱ्या जगात रोज दिसतं: IRCTC tatkal च्या क्षणी, Hotstar world cup च्या दिवशी हजारो servers उभे केले जातात; match संपला की कमी. आणि जिथे हे जमलं नाही तिथे काय होतं तेही तुम्ही बघितलं आहे: concert ची tickets, exam चे forms: “site पडली.”

इथे लोक काय चुकीचं समजतात

”मोठा server घेतला की प्रश्न मिटला.” Vertical scaling हे वेदना- शामक आहे, इलाज नाही: भिंत जवळ येते आणि किंमत उडते. आणि उलटी चूक: पहिल्या दिवसापासून “crores users साठी” बांधत बसणं: ते Chapter 4.5 चं उलटं पाप आहे: users 50 आणि architecture NASA चं. शहाणा क्रम: सुरुवात सोपी (एक server ही चालेल), पण **stateless ठेवा**, म्हणजे गर्दी आली की आडवं वाढणं फक्त “अजून counters” इतकं सोपं राहतं. तयारी design मध्ये; खर्च गरजेच्या वेळी.

MAP वर

रुपयाचा रस्ता: scale झेलता येणं हे startup चं जगणं-मरणं आहे: “viral झालो आणि पडलो” म्हणजे दारात आलेली संधी परत गेली, आणि संधी क्वचित दोनदा दार वाजवते. म्हणून गुंतवणूकदार विचारतात: “हे दहापट users झेलेल का?” आणि म्हणूनच cloud (Chapter 7.5) जिंकलं: “अजून counters” एका click वर मिळू लागले. Level 3 च्या या एका कौशल्याने Level 2 ची कुठली स्वप्नं जगतात-मरतात हे ठरतं.

स्वतः बघा (5 मिनिटं)

पुढच्या मोठ्या sale च्या दिवशी (Big Billion Days / Prime Day) किंवा result च्या दिवशी लक्ष द्या: apps थोडी हळी, कधी रांग, कधी “try again”. आता तुम्हाला का ते दिसतं: मागे कुठेतरी counters कमी पडतायत, load balancer रांगा लावतोय. आणि जेव्हा सगळं गुळगुळीत चालतं, तेव्हा आता तुम्हाला माहीत आहे: ते आपोआप नाही; कोणीतरी हे chapter जगून दाखवलं आहे.

विचार करा

1. (derivation) हलवायाकडे रोजची 100 ताटं जातात; दिवाळीत 1,000 लागणार. दोन रस्ते: (अ) दहापट मोठी कढई, (ब) दहा सामान्य कढया + वाटप करणारा. कुठला निवडाल आणि तुमच्या निवडीत “state” कुठे आहे?

उत्तर: (ब). मोठ्या कढईची भिंत लवकर येते (एवढी मोठी कढई बनतच नाही), ती महाग, आणि ती फुटली तर दिवाळी गेली. दहा कढया: भिंत नाही, एक बिघडली तरी नऊ चालू. आणि state?

Recipe आणि हिशोब. जर प्रत्येक कढईवाल्याची स्वतःची recipe आणि स्वतःची उधार-वही असेल, तर दहा counters म्हणजे दहा वेगळी दुकानं: भेळ. Recipe एक (लिहिलेली, सगळ्यांना समान: Part 2!) आणि गल्ला/वही एक (सामायिक database: Part 4!), कारागीर रिकामे (stateless): मगच दहा कढया एक दुकान राहतं. Scale चं पूर्ण रहस्य या एका वाक्यात आहे: काम वाटा, आठवण एकत्र ठेवा.

Chapter 7.2 [SPINE]: कधीही बंद न होणारी machine

Part 4 चं सत्य आठवा: disks मरतात, machines मरतात, **नक्की** मरतात. आणि तरी UPI रात्री 3 वाजताही चालतो, Google कधी “आज बंद आहे” म्हणत नाही. दोन्ही गोष्टी एकत्र कशा खऱ्या असू शकतात?

उत्तराची पहिली ओळ मनात कोरून घ्या: **मोठ्या systems मध्ये machines मरत नाहीत असं नाही; मरणं गृहीत धरलेलं असतं.** Design चा प्रश्न “मेली तर?” नाही; “मेली तेव्हा ग्राहकाला कळू नये” हा आहे.

तीन हत्यारं, तिन्ही तुमच्या ओळखीची:

प्रत्येक भागाच्या प्रती (redundancy). दोन servers जिथे एक पुरला असता; database ची जिवंत प्रत (Part 4 चं replication); दोन वेगळे रस्ते (Part 3: packets नवा रस्ता घेतात). दवाखान्याचा generator: वीज असते तेव्हा तो रिकामा उभा असतो, आणि तो “वाया” जात नाही; तो विमा आहे. विमानाला दोन engines असतात ती स्पर्धेसाठी नाहीत.

मरण ओळखणं (health checks). दर काही seconds नी प्रत्येक server ला विचारलं जातं: “जिवंत आहेस?” उत्तर थांबलं की तो रांगेतून काढला जातो. Load balancer (मागचा chapter) हेच करत असतो: ग्राहक फक्त जिवंत counters कडेच पाठवले जातात.

आपोआप वळवणं (failover). मुख्य मेला की प्रत **आपोआप** पुढे येते: माणसाची वाट न बघता, seconds मध्ये. दुकानाचा भाऊ गल्ल्यावर बसतो; ग्राहकाला कळतही नाही मालक आजारी आहे.

आता उपलब्धतेची मोजपट्टी शिका, कारण ती **विकली-विकत घेतली** जाते. भाषा “nines” ची आहे: वर्षभरात system किती टक्के वेळ चालू होतं?

99%	(two nines)	वर्षात ~3.7 दिवस बंद
99.9%	(three nines)	वर्षात ~8.8 तास बंद
99.99%	(four nines)	वर्षात ~53 मिनिटं बंद
99.999%	(five nines)	वर्षात ~5 मिनिटं बंद

आणि नियम: **प्रत्येक पुढचा nine साधारण दहापट महाग.** कारण प्रत्येक पायरीला अजून प्रती, अजून जागा (दुसरं गाव: building जळते!), अजून आपोआप-वळवण्याची यंत्रणा, अजून सराव.

शेवटचा तुकडा: बदल. Software रोज बदलतं (Part 2), आणि बहुतेक मरणं **बदलानंतर** येतात. मग नव version चढवताना दुकान बंद? नाही: दहा counters आहेत ना (मागचा chapter): एक-एक करून बदला; नऊ चालू, एक नवा होतोय; नवा बिघडला तर लगेच जुन्यावर परत. दुकान उघडं ठेवून रंगकाम.

इथे लोक काय चुकीचं समजतात

”मोठ्या companies कडे जादुई, कधी-न-बिघडणारे servers असतात.” उलटं: त्यांच्याकडे **रोज** शेकडो machines मरतात, आणि कोणाला पत्ता लागत नाही; हीच कला आहे. आणि founder ची चूक: “आपण छोटे आहोत, आपल्याला हे नको.” अर्थ खरं: five nines नको (दहापट खर्च आठवा); पण “database ची प्रत आहे का, आणि ती **पुढे येते** का हे तपासलंय का” हा प्रश्न पहिल्या ग्राहकापासून लागू आहे. Part 4 ची ओळ इथेही: न तपासलेला failover म्हणजे failover नाहीच.

MAP वर

उपलब्धता ही थेट विक्रीची गोष्ट आहे: cloud companies करारात लिहून देतात “इतके nines देऊ, कमी पडलो तर पैसे परत” (या कराराचं नाव **SLA**). आणि उलट्या बाजूने मोजा: Amazon साठी एक minute बंद = lakhs चं नुकसान; UPI साठी एक तास बंद = राष्ट्रीय बातमी. म्हणून सूत्रात: जिथे बंद पडण्याची किंमत प्रचंड, तिथे nines विकत घेतले जातात, आणि ते **बांधता** येणारे engineers सगळ्यात महागड्यांमध्ये आहेत (scarcity: हे chapter वाचणं सोपं; पहाटे 3 ला जगवणं वेगळं).

स्वतः बघा (5 मिनिटं)

आठवा: तुमचं bank app किंवा UPI शेवटचं कधी “down” होतं? क्वचित, आणि झालं तरी minutes. आता तुमच्या घरची वीज आठवा: महिन्यात किती वेळा जाते? तुमच्या खिशातली digital-पैशाची यंत्रणा तुमच्या भिंतीतल्या विजेपेक्षा जास्त भरवशाची झाली आहे: आणि ती जादूने नाही, या chapter च्या तीन हत्यारांनी. (आणि आता downdetector.com माहीत करून घ्या: जगात कोण-कोण आता पडलंय याची जिवंत यादी. मोठे पण पडतात; रोज.)

विचार करा

1. (derivation) तुमच्या नव्या business साठी hosting निवडताय: 99.9% वाला plan स्वस्त आहे, 99.99% वाला लक्षणीय महाग. निर्णय **कशावरून** कराल? (Hint: Part 4 च्या backup-वारंवारतेचा निर्णय आठवा.)

उत्तर: तिथल्यासारखाच हा business प्रश्न आहे, tech नाही: “एक तास बंद पडण्याची **माझ्या** धंद्याला किंमत काय?” Blog/माहिती-site: 8 तास/वर्ष बंद कोणाला कळणारही नाही: 99.9% पुरे, वाचलेले पैसे marketing ला द्या. Payment/appointment घेणारा धंदा: बंद तास = गेलेले ग्राहक + गेलेला विश्वास: महाग plan स्वस्त पडतो. आणि एक गोम: सगळी साखळी मोजा: तुमचं app 99.99% पण त्यात जोडलेली SMS सेवा 99% असेल, तर ग्राहकाला दिसणारी उपलब्धता साखळीतल्या **कमजोर** दुव्याची असते. Nines ची खरेदी नेहमी साखळीने, दुव्याने नाही.

Chapter 7.3 [SPINE]: दिसणारा भाग, न दिसणारा भाग

Restaurant मध्ये दोन जगं असतात: **dining hall** (टेबलं, मेनू, सजावट: ग्राहक जे बघतो) आणि **kitchen** (चुली, साठा, हिशोब: जिथे खरं काम होतं). दोन्हींची कामं वेगळी, माणसं वेगळी, नियम वेगळे. ग्राहक kitchen मध्ये जात नाही; waiter दोन्हींना जोडतो.

Software चं **नेमकं** असंच विभाजन आहे, आणि हे दोन शब्द तुम्ही आता रोज ऐकाल:

Frontend = dining hall. जे तुमच्या **हातातल्या** machine वर चालतं: screens, buttons, रंग, animations. App चं रूप. हे client वर चालतं (Part 3 ची भाषा), म्हणजे **ग्राहकाच्या** phone/browser वर.

Backend = kitchen. जे **server** वर चालतं: नियम, हिशोब, database शी बोलणं, पैसे हलवणं. ग्राहकाला कधीच थेट दिसत नाही; फक्त त्याचे निकाल दिसतात.

विभाजन का? कारण दोघांचे **स्वभाव** वेगळे आहेत. Frontend वारंवार बदलतं (नवा रंग, नवी रचना: रोज प्रयोग); backend सांभाळून बदलतं (पैशाचे नियम रोज बदलत नाहीत: Part 2 ची compiler/interpreter-निवड आठवा: बदलाचा वेग vs चुकीची किंमत). दुकानाचं शोकेस रोज सजवता; तिजोरीची खोली सहसा हलवत नाही.

आणि आता या chapter ची सगळ्यात महत्त्वाची ओळ; ती security ची पहिली वीट आहे: **frontend वर कधीही विश्वास ठेवायचा नाही**. का? Frontend ग्राहकाच्या machine वर चालतं, आणि ग्राहकाच्या machine वर **त्याचं** राज्य आहे: तिथलं काहीही बदलता येतं. Part 1 आठवा: app म्हणजे numbers; आणि आपल्याच phone वरचे numbers चलाख माणूस बदलू शकतो.

म्हणजे: app ने दाखवली किंमत 500, चलाख माणसाने आपल्या बाजूला ती 5 करून “order” पाठवली, तर? **चांगलं backend** म्हणतं: मला काय फरक पडतो तू काय दाखवत होतास; **माझ्या** वहीत किंमत 500 आहे, मी 500 चीच नोंद करणार. किंमत, शिल्लक, परवानगी, नियम: प्रत्येक तपासणी kitchen मध्ये **पुन्हा** होते. Waiter ने कितीही सांगितलं “याने पैसे दिलेत,” गल्ला स्वतः मोजतो. Frontend ची तपासणी सोयीसाठी असते (ग्राहकाला लगेच “चूक” दाखवायला); **खरी** तपासणी backend वरच.

इथे लोक काय चुकीचं समजतात

दोन उलट्या चुका. पहिली: “App सुंदर दिसतंय, म्हणजे product तयार.” Chapter 4.5 चा “90% चा भ्रम” इथेच जन्मतो: dining hall सजला; kitchen अजून रिकामं आहे. दुसरी: “frontend म्हणजे हलकं काम.” नाही: हजार प्रकारच्या screens वर, थरथरत्या network वर, अधीर माणसाच्या बोटांखाली गुळगुळीत चालणं ही स्वतःची खोल विद्या आहे. कामं **वेगळी** आहेत; लहान-मोठी नाहीत. (Hiring च्या भाषेत हे शब्द: frontend developer, backend developer, आणि दोन्ही करणारा full-stack.)

MAP वर

रुपयाचा रस्ता: teams याच रेषेने वाटलेली असतात, पगारही. आणि founder साठी एक नकाशा-खूण: frontend च्या जगात हत्यारं फुकट आणि AI प्रचंड वेगवान झालाय: दिसणारं बनवणं **स्वस्त** होतंय (Chapter 0.4 ची आठवण: जे सगळ्यांना जमतं ते स्वस्त होतं). Scarcity मागे सरकते आहे: बरोबर, सुरक्षित, न पडणारं kitchen बांधणाऱ्यांकडे. दिसणाऱ्याची किंमत घसरते; न दिसणाऱ्याची चढते. हे वाक्य या पुस्तकाच्या पहिल्या part पासून चालत आलं आहे, आणि इथे ते hiring-निर्णय बनतं.

स्वतः बघा (5 मिनिटं)

कुठल्याही app मध्ये order/booking करताना जाणीवपूर्वक बघा: किंमत **कुठे-कुठे** दिसते? यादीत, cart मध्ये, आणि शेवटच्या “confirm” वर. शेवटचा आकडा नेहमी एक क्षण उशिरा येतो: ते backend **पुन्हा** मोजतंय; frontend च्या आकड्यावर त्याचा विश्वास नाही. आणि कधी “किंमत बदलली आहे, पुन्हा तपासा” असा संदेश आला असेल: तो हाच chapter जिवंत: वहीने waiter ला दुरुस्त केलं.

विचार करा

1. (derivation) एका शाळेने exam चं app बनवलं; वेळ वाचवायला उत्तरांची तपासणी **app मध्येच** (frontend वर) ठेवली: प्रश्न, चार पर्याय, आणि बरोबर उत्तरही app च्या आतच. काय होईल, आणि दुरुस्ती काय?

उत्तर: बरोबर उत्तर app च्या आत म्हणजे ते विद्यार्थ्यांच्या **machine वर** आहे: numbers उघडून वाचता येतात (Part 1: सगळं numbers च आहे). कोणीतरी ती काढेल आणि वाटेल; परीक्षाच संपली. दुरुस्ती: प्रश्न frontend वर, पण विद्यार्थ्यांचं उत्तर server ला जाणार आणि **तपासणी kitchen मध्ये:** बरोबर उत्तर फक्त backend च्या वहीत. नियम एका ओळीत: **जे गुपित आहे किंवा जे पैसे/गुण ठरवतं, ते ग्राहकाच्या machine वर कधीच पाठवायचं नाही.** हा नियम मोडून जगात रोज कुठेतरी परीक्षा, sale, आणि game लुटली जाते.

Chapter 7.4 [SPINE]: दोन programs आपसात बोलतात कसे

Zomato ला delivery वाल्याचा रस्ता नकाशावर दाखवायचा आहे. नकाशा बनवणं म्हणजे उपग्रह, गाड्या, जगाचं survey: decades चं काम. Zomato ते करणार? अर्थात नाही: **Google Maps कडून मागणार**. पण थांबा: मागणार **कसं**? Google च्या office ला फोन? माणसासाठी Google ने screen बनवली आहे; पण इथे माणूस नाही, Zomato चा **program** दुसऱ्या company च्या **program** शी बोलणार आहे.

Part 3 मधून उत्तराचा सांगाडा तुमच्याकडे आहेच: दोन machines बोलतात तेव्हा **आधीच ठरलेले नियम** लागतात (protocol). आता तीच कल्पना एक मजला वर न्या: दोन **programs** बोलतात तेव्हा ठरलेले **प्रश्न आणि उत्तरांचे साचे** लागतात:

”असा प्रश्न विचारा: ‘रस्ता दे: इथून-तिथे.’ उत्तर असं येईल: वळणांची यादी, अंतर, वेळ. चुकीचा पत्ता दिलात तर असा नकार येईल.”

अशा ठरलेल्या प्रश्नोत्तर-साच्यांच्या यादीचं नाव **API**. हॉटेलचं **menu card** हीच कल्पना आहे: हे-हे मागता येतं (यादी), असं मागायचं (क्रमांक सांगा), हे मिळेल (ताट), आणि जे menu त नाही ते मागता येत नाही: kitchen त घुसायची परवानगी नाही, गरजही नाही. API म्हणजे program चं menu card: **आतलं न दाखवता, ठरलेल्या सेवा देणारा दरवाजा**. (Chapter 0.4 चं abstraction, त्याच्या सगळ्यात व्यापारी रूपात.)

तुम्ही रोज दहा APIs मधून जाता, न कळत: app मध्ये “Login with Google” (तुमच्या app ने Google चा API वापरला), weather app (हवामान खात्याचा API), प्रत्येक UPI app (NPCI चा API: Part 3 मध्ये protocol म्हणून भेटला; apps साठी तो API च आहे), payment चं Razorpay-पान (दुकानाने पैसे घेण्याचं पूर्ण kitchen भाड्याने घेतलं: फक्त API जोडला).

आणि इथे एक करार-धर्म आहे: API हा **शब्द दिलेला** असतो. हजार apps तुमच्या API वर बांधलेली आहेत; तुम्ही साचा बदललात (प्रश्नाचं रूप, उत्तराचं रूप), तर ती हजार apps **एका क्षणात** तुटतात. म्हणून जुना साचा चालू ठेवून नवा शेजारी उघडतात (versioning: menu बदललं तरी जुन्या ग्राहकांची थाळी बंद करत नाहीत, आधी सांगून मगच).

इथे लोक काय चुकीचं समजतात

”API म्हणजे engineers चा कुठलातरी बारीक technical शब्द.” नाही: API हा **धंद्याचा दरवाजा** आहे. API देणं म्हणजे जाहीर करणं: “माझ्या दुकानावर तुम्ही **तुमची** दुकानं बांधा.” आणि API **वापरणं** म्हणजे दुसऱ्याच्या जमिनीवर भिंत बांधणं: सोय प्रचंड, पण जमीनमालक भाडं वाढवू शकतो. हे खरोखर घडलं: Google Maps ने 2018 मध्ये API चे भाव अनेक पटींनी वाढवले; रातोरात हजारो apps चं महिन्याचं bill उडालं, काही धंदे बदलावे लागले. दुसऱ्याच्या API वर उभा असलेला business हा **भाडेकरू** आहे; भाडेकरूने करार वाचावा.

MAP वर

रुपयाचा रस्ता: “API विकणं” ही स्वतः एक अर्थव्यवस्था आहे. Razorpay/ Stripe म्हणजे “पैसे घेण्याचं kitchen, API ने भाड्याने”: प्रत्येक व्यवहारावर हिस्सा. Twilio: SMS चं kitchen. Google Maps: नकाशाचं. आणि आता AI: OpenAI/Claude चा API म्हणजे “विचार करण्याचं kitchen, भाड्याने” (Book 2). तुमच्या 4-level नकाशावर बघा: API हे Level 3 च्या आतल्या दुकानांचं **वाणिज्य-रूप** आहे: जो एक कठीण गोष्ट सोपा दरवाजा लावून विकतो, तो हजारो धंद्यांचा पुरवठादार होतो: लोणी सगळे बनवत नाहीत; डेअरीकडून घेतात.

स्वतः बघा (5 मिनिटं)

तुमच्या कुठल्याही app मध्ये हे शोधा: “Login with Google” चं बटण, हवामानाचा तुकडा, नकाशाचा तुकडा, payment चं पान जे क्षणभर वेगळं दिसतं (Razorpay/PayU चा रंग). प्रत्येक तुकडा म्हणजे **दुसऱ्या** company चं kitchen, API ने जोडलेलं. एक app मोजा: आत किती companies चे APIs दिसतात? पाच-सात सहज निघतील. आजचं software म्हणजे स्वतःचं थोडं + APIs ची जोडणी.

विचार करा

1. (derivation) तुमचा business आहे: शहरातल्या 200 tiffin-सेवांना ग्राहक मिळवून देणारं app. दोन प्रश्न: (अ) तुम्ही कुठले APIs **वापरा**ल (विकत घ्याल)? (ब) पुढे-मागे तुम्ही कुठला API **द्या**ल (विकाल)?

उत्तर: (अ) पैसे: Razorpay-सारखा (स्वतःचं payment-kitchen बांधणं म्हणजे वेडेपणा); नकाशा/ अंतर: Maps चा; SMS/WhatsApp सूचना: Twilio-सारखा; Login: Google चा. तुमची खरी मेहनत फक्त **तुमच्या** वेगळेपणावर: tiffin-सेवांची यादी, subscriptions, जुळवणी. (ब) मोठे झालात की तुमच्याकडे एक दुर्मिळ गोष्ट जमते: शहरातल्या tiffin-सेवांचा जिवंत data (कोण, कुठे, किती जागा शिल्लक). “Corporate canteen apps ना tiffin-पुरवठा जोडून देणारा API” हा तुमचा माल होऊ शकतो: तुम्ही भाडेकरूचे जमीनमालक होता. धडा: सुरुवातीला APIs **घेऊन** वेळ वाचवा; मोठेपणी API **देऊन** platform व्हा. (Chapter 3.5 चा जकात-नाका, आता तुमच्या बाजूने.)

Chapter 7.5 [SPINE]: विकत घेण्याऐवजी भाड्याने

2000 साली startup सुरू करायची म्हणजे: servers **विकत** घ्या (lakhs चे), त्यांच्यासाठी थंड खोली, वीज-backup, आणि ते सांभाळणारी माणसं. Users येण्याआधी हा खर्च; आणि अंदाज चुकला तर दोन्ही बाजूंनी मार: कमी घेतले तर गर्दीत पडलात, जास्त घेतले तर रिकामे servers पैसे खातायत.

मग एक गोष्ट घडली जिने हे सगळं बदललं. Amazon (दुकान!) ने स्वतःसाठी प्रचंड machine-यंत्रणा बांधली होती; दिवाळी-गर्दीसाठी बांधलेली क्षमता एरवी **रिकामी** असायची. कोणाला तरी सुचलं: ही रिकामी क्षमता **भाड्याने** दिली तर? 2006 मध्ये त्यांनी दार उघडलं, आणि आज ते दुकान (AWS) मूळ दुकानापेक्षा जास्त नफा कमावतं.

या भाड्याच्या जगाचं नाव **cloud**. Part 3 मध्ये वचन दिलं होतं ते आज फेडू: **cloud म्हणजे हवा नाही; cloud म्हणजे दुसऱ्याच्या building मधल्या लाखो machines, ज्या तुम्ही minutes मध्ये, click वर, भाड्याने घेता, आणि वापराल तेवढंच bill भरता.** मोठे तीन मालक: AWS (Amazon), Azure (Microsoft), Google Cloud.

भाडं का जिंकलं, ते तुमच्या रोजच्या शहाणपणातच आहे: लग्नासाठी hall **भाड्याने** घेता, विकत नाही; रोजच्या प्रवासाला गाडी विकत घेता, पण एका trip साठी Ola. नियम: **गरज स्थिर असेल तर विकत; गरज वर-खाली होणारी असेल तर भाड्याने.** आणि startup ची गरज म्हणजे शुद्ध वर-खाली: आज 100 users, उद्या (कदाचित) 100,000, परवा (कदाचित) परत 500. Cloud ने तीन गोष्टी दिल्या:

आधी पैसे नकोत. Servers विकत घ्यायचा डोंगर-खर्च गेला; भाडं वापराबरोबर वाढतं. (हिशोबाच्या भाषेत: CapEx चं OpEx झालं: एकरकमी गुंतवणुकीचं रूपांतर मासिक खर्चात.)

लवचीकपणा. Chapter 7.1 चे “अजून counters” आता एक click आहेत: match च्या दिवशी हजार servers, रात्री परत शंभर. ह्याचं नाव **elastic**: रबरासारखं ताणणं-आकसणं.

जग दारात. Mumbai, Singapore, Frankfurt: प्रत्येक खंडात त्यांची buildings (**regions**) तयार आहेत. तुमचं app जगभर पोहोचवायला जहाज नको; पुढच्या chapter चा विषय.

पण भाड्याचं जग गुलाबी नाही; दोन काटे लक्षात ठेवा. एक: भाडं **कधीच थांबत नाही**, आणि वाढत्या वापराबरोबर bill असा वाढतो की एक दिवस “स्वतःचं घेतलं असतं तर स्वस्त पडलं असतं” हा हिशोब खरा होतो: Dropbox सारख्या मोठ्या, **स्थिर** वापराच्या companies खरंच cloud सोडून स्वतःच्या machines

वर परतल्या आणि crores वाचवले. दोन: **lock-in**: एका मालकाच्या खास सेवांवर खोल बांधलं की दुसरीकडे जाणं महाग होतं; भाडेकरूला घर बदलणं सोपं नसतं (Chapter 7.4 चा जमीनमालक-धडा, इथे पूर्ण इमारतीच्या आकारात).

इथे लोक काय चुकीचं समजतात

”Cloud स्वस्त असतं.” नेहमी नाही. Cloud **लवचीक** असतं; तेच त्याचं खरं मोल. छोट्या/वर-खाली गरजेला ते स्वस्त **पडतं**; मोठ्या स्थिर गरजेला महाग. आणि दुसरी चूक: “cloud मध्ये टाकलं की सुरक्षित/सांभाळलेलं.” मालक building, वीज, machine सांभाळतो; तुमचा data, तुमचे passwords, तुमचा backup (Part 4!) ही **तुमचीच** जबाबदारी राहते. भाड्याच्या घरात मालक भिंती देतो; कुलूप तुम्हीच लावायचं.

MAP वर

रुपयाचा रस्ता: cloud हा platform-जकातीचा (Chapter 3.5) सगळ्यात भव्य अवतार आहे: जगातल्या **लाखो** businesses चं महिन्याचं भाडं तीन मालकांकडे वाहतं; AWS एकटा वर्षाला ~100 billion dollars. तुमच्या 4-level नकाशावर: Level 3 चं हे तळमजल्याचं दुकान इतकं मोठं झालं की देश जागे झाले: “आपला data आपल्या देशात राहावा” (data localization) या नियमांनी India त cloud-buildings चा पाऊस पडतोय. Level 3 चा निर्णय Level-सरकार पर्यंत: pattern ओळखीचा आहे (Part 3, Chapter 5.8).

स्वतः बघा (5 मिनिटं)

तुम्ही स्वतः cloud चे भाडेकरू आहात: Google Photos/Drive चं महिन्याचं 130 रुपयांचं भाडं (Chapter 2.1) म्हणजे Google च्या building मधल्या disks वरची तुमची खोली. आणि एक शोध करा: search करा “AWS pricing calculator” आणि उघडून फक्त **बघा**: CPU तासाला, GB महिन्याला, data बाहेर GB ला: सगळ्याचे दर लिहिलेले आहेत, भाजी-मंडईच्या फळ्यासारखे. पुढच्या-पुढच्या chapter मध्ये (7.8) हेच दर वाचून bill समजायला शिकू.

विचार करा

1. (derivation) दोन businesses: (अ) exam-निकाळांची site: वर्षातले 360 दिवस जवळजवळ शून्य traffic, निकालाच्या 5 दिवशी देशभर स्फोट. (ब) कारखान्याचं आतलं hisheb-software: रोज त्याच 200 कर्मचार्यांचा सारखा वापर, वर्षभर. कोणी भाड्याने, कोणी विकत, आणि का?

उत्तर: (अ) Cloud चा जन्मच जणू यासाठी: 360 दिवस जवळजवळ शून्य भाडं, स्फोटाच्या दिवशी हजार machines, नंतर परत शून्य. विकत घेणं म्हणजे वर्षभर रिकाम्या राक्षसाला खाऊ घालणं. (ब) गरज स्थिर आणि माहीत: इथे “विकत” (किंवा साधं एक server भाड्याने, पण lock-in शिवाय) स्वस्त पडू शकतं; लवचीकपणाची गरजच नाही, मग त्याचं premium का भरा? नियम एका ओळीत: **वर-खाली होणारी गरज = भाड्याचं premium फायद्याचं; सपाट गरज = premium वाया.** आणि खरा शहाणपणा: दर 6 महिन्यांनी हा हिशोब पुन्हा करणं, कारण business वाढला की उत्तर बदलतं (Dropbox ची गोष्ट).

Chapter 7.6 [DEPTH]: सगळे एकदम आले तर काय तुटतं

(DEPTH chapter. पण तुमच्या आयुष्यात कधीही tatkal, flash sale, निकालाचा दिवस किंवा viral क्षण येणार असेल, तर हा तुमच्या कामाचा आहे.)

Chapter 7.1 ने शिकवलं: गर्दी आली की counters वाढवा. मग प्रश्न संपला? नाही. एक जुनी म्हण नव्या रूपात: **साखळीची ताकद तिच्या सगळ्यात कमजोर दुव्याइतकीच**. लग्नाची जेवणाची रांग आठवा: ताटं भरपूर, वाढपी भरपूर, पण **पोळ्यांचा** counter एकच: सगळी रांग तिथेच अडते. Counters दहा केले, पण सगळे एकाच database ला विचारतात (Part 4): database अडला, तर दहा रिकामे counters फक्त सुंदर दिसतात.

या अडणाऱ्या जागेचं नाव **bottleneck** (बाटलीची मान: बाटली केवढीही मोठी, ओतण्याचा वेग मान ठरवते). आणि गर्दीच्या दिवसाची पहिली विद्या: **अंदाजाने नाही, मोजून** bottleneck शोधणं. मोठ्या systems मध्ये प्रत्येक भागावर मीटर लावलेले असतात (या मोजमापांचं नाव **metrics**): कुठला भाग किती भरलाय, कुठे रांग वाढतेय. Founder च्या भाषेत: dashboard. मीटरशिवाय गर्दीचा दिवस म्हणजे अंधारात आग विझवणं.

Bottleneck सापडला; आता हत्यारं, क्रमाने:

- 1. रुंद करा.** पोळ्यांचे counters वाढवा: database ची प्रत (Part 4 चं replication वाचनासाठी), अजून servers. सरळ, पण नेहमी शक्य नसतं (पैशाची वही एकच असते: Chapter 6.4).
- 2. रांग लावा.** Part 4 चं queue-हत्यार: सगळं आत्ताच नको (SMS नंतर!). आणि ग्राहकालाही **दिसणारी** रांग: BookMyShow ची “You are in queue” ची पायरी म्हणजे design, अपयश नाही: आत गोंधळ होण्यापेक्षा दारावर शिस्त.
- 3. दार आवळा.** मंदिरात दर्शनाला एका वेळेस मोजकेच सोडतात; आत चेंगराचेंगरी होण्यापेक्षा बाहेर रांग बरी. Systems मध्ये याचं नाव **rate limit**: “एका user ला minute ला इतक्याच विनंत्या.” चलाख programs (bots) आणि वेडी गर्दी दोन्हींपासून हेच वाचवतं.

आणि आता ती गोष्ट जी गर्दीच्या दिवसाला खरी भयानक बनवते: **cascade** (कोसळत जाणं). साखळी बघा: एक भाग **हळा** झाला (मेला नाही, हळा!) -> त्याची वाट बघणारे तिष्ठले -> त्यांनी कंटाळून **पुन्हा** प्रयत्न केला (retry) -> म्हणजे गर्दी दुप्पट -> हळा भाग अजून हळा -> अजून retries... एकाच्या खोकल्याने पूर्ण रांगेला ताप. जगातले मोठे outages बहुतेक असेच: कोणी मारलं नाही; system स्वतःच्या retries खाली चिरडली. इलाजाचं नाव मजेदार आहे: **circuit breaker** (घरातल्या MCB वरून): एखादा

भाग सतत अपयशी दिसला, तर त्याला काही क्षण **विचारणं थंबवा**; त्याला श्वास द्या, ग्राहकाला “थोड्या वेळाने” सांगा. आजारी माणसाला दर मिनिटाला “बरा झालास?” विचारत राहणं हाच त्याचा आजार वाढवतं.

इथे लोक काय चुकीचं समजतात

”Traffic दुप्पट होणार, servers दुप्पट केले, तयारी झाली.” Bottleneck **हलतो**: servers दुप्पट केले की अडचण database वर जाते; ती सोडवली की payment-API वर (Chapter 7.4 चा भाडेकरू: त्यांचंही एक rate limit आहे!); ती सोडवली की SMS वर. गर्दीची तयारी म्हणजे एक आकडा दुप्पट करणं नाही; **पूर्ण साखळी** चालवून बघणं. मोठ्या companies निकालाच्या/sale च्या आधी नकली गर्दी घालून सराव करतात (load test): लग्नाआधीची पंगत-rehearsal.

MAP वर

रुपयाचा रस्ता: गर्दीचा क्षण हाच धंद्याचा **सुगीचा** क्षण असतो: sale चा तास, निकालाचा दिवस, match ची रात्र. तिथे पडणं म्हणजे वर्षाची कमाई त्याच दाराशी सोडणं; म्हणून मोठ्या e-commerce companies सण-sale आधी महिने फक्त हीच तयारी करतात. आणि उलटं: “आमची site sale च्या दिवशीही पडत नाही” ही ओळ थेट जाहिरातीत विकली जाते: भरवसा हाच माल (Part 3 च्या encryption-धड्याची आठवण: विश्वास बाजार उभा करतो).

स्वतः बघा (5 मिनिटं)

पुढच्या वेळेस कुठल्या app वर “You are in queue” किंवा “Please try after some time” दिसलं, तर आता ते **वाचा**: पहिलं म्हणजे दारावरची शिस्त-रांग (हत्यार 2), दुसरं बहुधा rate limit किंवा circuit breaker (हत्यार 3, किंवा आजारी भागाला दिलेला श्वास). आधी हे “errors” वाटायचे; आता design दिसतंय. आणि सगळ्यात गोड: आता तुम्हाला कळतं की हे संदेश **नसणारी** आणि सरळ कोसळणारी site कुठल्या पायरीवर आहे.

विचार करा

1. (derivation) तुमच्या नव्या course-वेबसाईटवर उद्या सकाळी 10 ला admission उघडणार: 50,000 जागांसाठी अंदाजे 500,000 लोक येतील. वरची तीन हत्यारं **तुमच्या** case मध्ये नेमकी कशी लावाल? आणि एक खास प्रश्न: जागा-वाटप कुठल्या part च्या कुठल्या हत्याराने चालेल?

उत्तर: दार आवळा: 10 च्या ठोक्याला सगळ्यांना आत न सोडता दारावर दिसणारी रांग (queue page): आत मोजके, बाकी क्रमांकासह वाट बघत. रुंद करा: फक्त-वाचण्याची पानं (माहिती, फॉर्मचं रूप) आधीच cache/CDN वर (Chapter 6.6, पुढचा chapter): ती गर्दी मूळ system ला शिवूच नये. रांग लावा: भरलेले फॉर्म queue त; पावतीं लगेच (“अर्ज मिळाला, क्रमांक हा”), प्रक्रिया मागे सावकाश. आणि जागा-वाटप: ती **मोजकी वस्तू** आहे: Chapter 6.4 चं lock + transaction, database च्या वहीवर; तिथे मात्र रांग नशिबावर नाही, कुंडीने. चार parts ची हत्यारं एका सकाळसाठी: पुस्तक असं एकत्र येतं.

Chapter 7.7 [SPINE]: गोष्टी हळू का वाटतात

तुम्ही app उघडलं आणि पूर्ण एक second काहीच झालं नाही. “फक्त” एक second: पण Google-Amazon च्या मोजमापांनी (Chapter 2.3) शिकवलंय की हा second थेट पैसा आहे. आज तो second उघडून बघू: तो जातो कुठे?

एक विनंती धरा: तुम्ही दूरच्या (America तल्या) एका site चं पान मागितलं. घड्याळ लावा:

नाव -> number (DNS, 5.5)	~30 ms
"बोलूया?" ची सुरुवातीची देवाण-घेवाण (नमस्कार + कुलूप-करार, 5.4/5.6: India-America चे 2-3 फेरे!)	~200 ms
तुमची विनंती जाऊन उत्तर येणं	~150 ms
server चा विचार (DB, 6.2)	~100 ms
पान मोठं आहे: photos वगैरे येणं	~400 ms
screen वर रंगवणं (frontend, 7.3)	~100 ms

	~1 second

आता सगळ्यात महत्त्वाची गोष्ट **दिसते**: server ने फक्त 100 ms खाल्ले. उरलेला जवळजवळ सगळा वेळ **रस्त्यावर** गेला: फेऱ्यांमध्ये. आणि प्रत्येक फेरीची किंमत **अंतराने** ठरते (Part 1 चं भौतिक सत्य: प्रकाशाचा वेगही मर्यादा आहे; India-America फेरा ~200 ms च्या खाली जाऊच शकत नाही). म्हणून वेगाची दोन मोठी हत्यारं:

हत्यार 1: अंतर मारा. उत्तर जर Pune तूनच मिळालं तर फेरा 200 ms चा 20 ms होतो. म्हणजे प्रती **जगभर** ठेवा: Chapter 6.6 चा cache, आता जागतिक आकारात. या धंद्याचं नाव **CDN** (Cloudflare, Akamai): जगभरच्या 300+ शहरांमध्ये त्यांची छोटी godowns; तुमच्या site चे photos, videos, न-बदलणारी पानं तिथे आधीच ठेवलेली; ग्राहकाला जवळच्या godown मधून मिळतं. Hotstar ची match crores लोक बघतात ती अशीच: प्रत्येक frame Mumbai-Delhi-Chennai च्या godowns मधून; मूळ server पर्यंत कोणीच जात नाही. Amul चं दूध प्रत्येक गावात पहाटे पोहोचलेलं असतं; कोणी Anand ला जात नाही.

हत्यार 2: फेऱ्या मारा. जे अंतर मारता येत नाही (तुमचा खरा हिशोब मूळ server वरच होणार), तिथे **फेऱ्यांची संख्या** कमी करा: एका फेरीत जास्त सामान (दहा प्रश्नांऐवजी एक एकत्र प्रश्न), लागणारं आधीच मागवून ठेवणं, दुसऱ्या पानाचं सामान पहिल्या पानासोबतच. मंडईची यादी करून **एकाच** फेरीत सगळं आणणारा cook (2.3) विरुद्ध प्रत्येक कांद्यासाठी वेगळी फेरी मारणारा.

आणि एक तिसरं, चोरटं हत्यार: **वाटणं**. ग्राहकाची घड्याळ त्याच्या **डोक्यात** असते. पान येईपर्यंत रिकामा पांढरा screen विरुद्ध लगेच दिसणारा सांगाडा + “होतंय” ची हालचाल: दुसरं दुप्पट वेगवान **वाटतं**, वेळ तोच असूनही. Restaurant मध्ये लगेच पाणी-पापड आल्यावर जेवणाची वाट सुसह्य होते. हे फसवणूक नाही; हा माणसाच्या अनुभवाचा design आहे: पण फक्त खऱ्या वेगाच्या **जोडीला**, बदली म्हणून नाही.

इथे लोक काय चुकीचं समजतात

”App हळू आहे: मोठा/वेगवान server घ्या.” वरचा हिशोब पुन्हा बघा: server चे 100 ms दुप्पट वेगवान केले तर second चा फायदा 50 ms: कोणाला कळणारही नाही. खरे खाणारे रस्त्यावर आहेत: अंतर, फेरे, आणि पानाचं **वजन** (Chapter 2.1: photo = हजारपट text; अर्ध जड पान = अर्धा वेळ). म्हणून वेगाचं काम **मोजण्याने** सुरू होतं (कुठे जातोय second?), अंदाजाने नाही. Chapter 7.6 चा धडाच: मीटरशिवाय इलाज नाही.

MAP वर

रुपयाचा रस्ता: CDN ही स्वतः एक industry आहे (Cloudflare: billions revenue), आणि तिचं मूळ हेच: **अंतर हे पैशात मोजता येतं**. आणि तुमच्या India-first नकाशासाठी खास ओळ: तुमचा ग्राहक 4G वर, स्वस्त phone वर, गावात आहे: त्याच्यासाठी पानाचं वजन आणि फेरे शहरी WiFi पेक्षा दसपट महाग आहेत. जो founder “माझ्या office च्या WiFi वर तर झकास चालतं” वर थांबतो (Chapter 4.4 चा माहोल-धडा!), तो आपला खरा बाजार हरवतो. India त जिंकलेली apps (WhatsApp, UPI apps) हलक्या वजनासाठी **वेड्या** होत्या, हा योगायोग नाही.

स्वतः बघा (5 मिनिटं)

दोन प्रयोग: (1) एखादी परदेशी, कमी-प्रसिद्ध site उघडा आणि मग कुठलीही मोठी Indian site: फरक **जाणवेल**: तो अंतराचा + CDN चा आहे. (2) Phone च्या settings मधून थोडा वेळ “data saver” लावून नेहमीची apps वापरा: कुठली app तरीही चपळ राहते ते बघा: तिच्या team ने हा chapter जगलेला आहे.

विचार करा

- (derivation) तुमचं tiffin-app (7.4 वालं) गावांमध्ये न्यायचंय. वेगासाठी तीन बदल सुचवा: एक CDN-वाला, एक फेऱ्यावाला, एक वजनावाला. (आणि एक traps-प्रश्न: order **देण्याची** क्रिया CDN वर ठेवता येईल का?)

उत्तर: CDN: menu चे photos, app चं न-बदलणारं सामान जवळच्या godown मधून (एकदा ठेवलं की crores वेळा फुकट-जवळ). फेऱ्या: app उघडताच लागणारं सगळं (आजचा menu + शिल्लक subscription + पत्ता) **एका** एकत्र उत्तरात; दहा वेगळे प्रश्न नकोत. वजन: photos दाबलेले (Chapter 2.1 ची WhatsApp-युक्ती), पहिलं पान photo-शिवाय ही वाचता येईल असं. आणि trap: **नाही**. Order देणं म्हणजे बदल घडवणं: मोजकी वस्तू, ताजी वही, transaction (6.4, 6.6 चा नियम: “दाखवणं शिळं चालतं, बदलणं ताजंच लागतं”). CDN फक्त **दाखवण्याच्या** प्रती वाटतो; **बदल** नेहमी मूळ वहीकडेच जातो. दोन्हींची गल्लत केली की जुना menu आणि दुहेरी orders: दोन्ही chapters एकत्र समजले याचीच ही परीक्षा होती.

Chapter 7.8 [SPINE]: हे सगळं चालवायचा खर्च किती

Part 1 पासून एक गाणं वाजतंय: software ची **copy** फुकट (Chapter 1.6), म्हणून leverage चा राजा. आज त्या गाण्याची दुसरी ओळ, जी पहिल्या यशानंतर प्रत्येक founder ला रात्री जागवते: copy फुकट आहे, पण **चालवणं** फुकट नाही. दर महिन्याला cloud चं bill येतं, आणि ते वाचता न येणं म्हणजे अंधारात सही करणं (Section 2 च्या opener ची भविष्यवाणी).

चला bill उघडू. कितीही लांब असलं तरी त्याच्या **मोठ्या तीन** ओळी याच असतात, आणि तिन्ही तुम्ही या पुस्तकात शिकला आहात:

1. Compute: विचाराचं भाडं. CPU/RAM किती वेळ वापरले (Chapter 3.6: process चालू = meter चालू). दर: machine च्या आकाराप्रमाणे, **तासाला**. हे भाडं खातं कोण: प्रत्येक विनंतीमागचा हिशोब, आणि (Book 2 ची चाहूल) AI चालवायचं असेल तर खास GPU-machines: त्यांचे दर सामान्यांच्या दहा-वीसपट.

2. Storage: आठवणीचं भाडं. किती GB, किती महिने (Chapter 2.1: जे मोजलं जातं तेच विकलं जातं). दर: **GB-महिन्याला**. स्वस्त वाटतं: आणि आहेही: पण data फक्त **वाढतो** (कोण delete करतं?), म्हणून ही ओळ दर महिन्याला थोडी जाड होत जाते: हळू वाढणारं वजन.

3. बाहेर जाणारा data: दरवाजाची जकात. आणि हा तिसरा, सगळ्यात **कपटी**: आत येणारा data बहुधा फुकट; तुमच्या servers मधून जगाकडे **बाहेर** जाणाऱ्या प्रत्येक GB ला दर (नाव: **egress**). Text ला काही वाटत नाही; पण Chapter 2.1 चं प्रमाण आठवा: video = लाखपट. प्रत्येक user ने बघितलेला प्रत्येक video = बाहेर गेलेले MBs = जकात. म्हणून video-app चा egress हाच त्याचा राक्षस, आणि म्हणूनच CDN (7.7) नुसता वेगाचा नाही, **पैशाचा** सौदा आहे: एकदा godown त ठेवा, CDN च्या स्वस्त दराने वाटा.

आता या तीन ओळी धंद्याच्या भाषेत आणा; याचं नाव **unit economics**: **एका** user मागे महिन्याला खर्च किती, आणि कमाई किती? हलवायला ताटामागचा खर्च पाठ असतो; तुम्हाला user-मागचा हवा. आणि इथे Part 1 चं जुनं भाकीत (Chapter 2.1 चा विचार-प्रश्न) पूर्ण वर्तुळ फिरून परत येतं: text-app चा user-खर्च जवळजवळ शून्य: फुकट users पोसता येतात; video-app चा खर्च खरा आणि जड: पहिल्या दिवसापासून कमाईचा रस्ता लागतो. **Product चा आकार त्याच्या business model चा आकार ठरवतो**: तेव्हा हे वाक्य theory होतं; आता तुम्ही ते bill च्या तीन ओळींमध्ये बोट ठेवून दाखवू शकता.

आणि एक इशारा, अनुभवाने लिहिलेला: **bill shock**. पहिली viral रात्र येते, सकाळी founder आनंदाने dashboard उघडतो: users दहापट! आणि मग mail उघडतो: bill **तीसपट**. का तीस, दहा का नाही? कारण गर्दीत सगळं महाग-रूप घेतं: जादा machines (7.1), जादा egress, आणि कुठलातरी विसरलेला भाग (एक न-दाबलेला video, एक loop मधली query) गर्दीने राक्षस बनवलेला. इलाज तोच जुना: **मीटर आणि कुंपण**: रोजच्या खर्चाचा alert (“दिवसाला X च्या वर गेलं तर मला कळवा”) पहिल्या दिवशी लावायचा: घराच्या विजेचं bill महिन्याने कळतं; cloud चं **रोज** बघता येतं, ही सोय न वापरणं गुन्हा आहे.

इथे लोक काय चुकीचं समजतात

“Software बनलं की चालवणं फुकट: copies फुकट असतात ना!” आता तुम्हाला पूर्ण चित्र आहे: copy (recipe ची) फुकट; **serve** करणं (compute + storage + egress) चालू meter. Software चा leverage खरा आहे, पण तो “खर्च शून्य” नाही, “खर्च **user-मागे छोटा आणि घसरणारा**” असा आहे. आणि दुसरी चूक: bill engineer चं काम समजणं. Bill हा business चा आरसा आहे: कुठलं feature महाग आहे, कुठला user तोट्याचा आहे, हे तिथे लिहिलेलं असतं: founder ने महिन्यातून एकदा स्वतः तीन मोठ्या ओळी वाचणं म्हणजे दुकानदाराने गल्ला स्वतः मोजणं.

MAP वर

रुपयाचा रस्ता, पूर्ण साखळी, शेवटच्या वेळेस: तुमचा user 300 रुपये देतो (Level 1) -> तुमचं app (Level 2) -> cloud चं bill: compute/ storage/egress (Level 3 ची जकात) -> cloud मालक chip-वाल्यांना (TSMC पर्यंत: Part 1 चं वर्तुळ पूर्ण). आणि AI आलं की या साखळीत एक जाड नवी ओळ चढते: GPU चं भाडं: Book 2 चा अर्धा हिशोब तिथेच आहे. जो founder ही साखळी वाचू शकतो, तो किंमत **ठरवू** शकतो; जो वाचू शकत नाही, तो “फुकट देऊ, बघू पुढे” म्हणत बुडतो.

स्वतः बघा (5 मिनिटं)

तुमच्या phone चं गेल्या महिन्याचं data-वापराचं पान उघडा (Settings -> Data usage): कुठल्या app ने किती GB खाल्ले? YouTube/Instagram वर, WhatsApp खाली: तेच 1 : हजार : लाख चं प्रमाण. आता उलट्या बाजूने विचार करा: तुम्ही जे **उतरवलं**, ते कोणाच्या तरी bill मध्ये **egress** म्हणून चढलं आहे. तुमचं रोजचं scroll म्हणजे कोणाच्या तरी meter ची फिरती चकती: आता ती चकती तुम्हाला दिसते.

विचार करा

1. (derivation) तुमच्या app मध्ये दोन features सारखेच लोकप्रिय: (अ) मित्रांशी text-gappa, (ब) 24-तासांनी गायब होणारे video-status. पैसे संपत आलेत; एक feature तात्पुरतं बंद करायचंय. Bill च्या तीन ओळींनी ठरवा: कुठलं, आणि का?

उत्तर: (ब) बंद करा. तीन ओळी वाचा: (अ) text: compute चिमूट, storage चिमूट (Chapter 2.1: 1 KB!), egress चिमूट: लाखो users जवळजवळ फुकट. (ब) video: storage जड (प्रत्येक status MBs मध्ये, रोज नवा), आणि egress राक्षसी: प्रत्येक बघणं = बाहेर जाणारे MBs x बघणाऱ्यांची संख्या. एकच feature, पण user-मागचा खर्च हजारपट. आणि खोल धडा: दोन्ही “सारखीच लोकप्रिय” दिसतात **वापराच्या** आकड्यात; **खर्चाच्या** आकड्यात ती एकाच जातीची नाहीतच. Founder ची नजर म्हणजे प्रत्येक feature ला दोन्ही आकड्यांनी बघणं: प्रेम किती, आणि पोट किती खातं.

Chapter 7.9 [SPINE]: Part 0 चा दावा, तपासलेला

Chapter 0.5 मध्ये आपण एक दावा लिहून ठेवला होता:

दावा: Machine समजल्यावर तुम्ही कुठल्याही business कडे बघून सांगू शकाल की त्यात technology ने पैसा कुठे बनेल, काय बनवणं स्वस्त आहे, काय महाग, आणि AI त्यात कुठे घुसेल.

आज तो दावा **तपासायचा** आहे: कुठल्याही “tech” business वर नाही; मुद्दाम एका साध्या, जुन्या धंद्यावर: **गल्लीतलं किराणा दुकान**. काका एकटे चालवतात; वही, तराजू, उधारी, आणि रोजची डोकेदुखी. चला, पुस्तक घेऊन दुकानात शिरू.

काकांच्या डोकेदुख्या ऐका आणि प्रत्येकीखाली पुस्तकातलं हत्यार लिहा:

”**उधारीची वही सांभाळता सांभाळता जीव जातो; कोणाची किती, कधीची, आठवत नाही.**” ही Part 4 ची Chapter 6.1 वाली वही आहे: नोंदी वाढल्या, प्रश्न अवघड झाले. हत्यार: छोटा database (आजच्या जगात एक तयार app). आणि “वसुलीची आठवण आपोआप जावो”: Chapter 6.5 ची रांग: आठवडाभराने आपोआप message. पैसा कुठे बनला: बुडणारी उधारी कमी = थेट खिशात.

”**महिन्याअखेरीस कुठला माल संपला, कुठला पडून सडला, हे आधी कळत नाही.**” हा data आहे जो **आधीच** वहीत आहे; प्रश्न विचारायचं हत्यार नाही (Chapter 6.2 ची query!). रोजची विक्री नोंदली की “काय संपत आलंय” ही यादी आपोआप. आणि एक पाऊल पुढे: गेल्या तीन महिन्यांच्या pattern वरून “पुढच्या आठवड्यात किती लागेल” चा अंदाज: हा Level 4 चा दरवाजा (Book 2), पण त्याची **जमीन** (नोंदलेला data) Level 3 नेच बनते. AI इथे घुसेल: आणि तो **तेव्हाच** घुसू शकेल जेव्हा आधी नोंदी digital झाल्या असतील. (AI ची भूक data आहे; उपाशी AI निरुपयोगी.)

”**UPI आल्यापासून पैसे थेट खात्यात येतात, बरं झालं.**” झालेलंच आहे: काका आधीच Level 3 चे ग्राहक आहेत (Part 3 चा protocol, Chapter 6.4 चे transactions त्यांच्यासाठी कोणीतरी चालवतंय). खूण: technology धंद्यात **थेंबाथेंबाने** घुसते, आणि सगळ्यात आधी तिथे जिथे दुखणं सगळ्यात तीव्र (पैशाची देवाणघेवाण).

”**Online-order वालं app बनवून देतो म्हणतो एकजण, दीड लाख मागतोय.**” आता तुमची Part 2 + 5 ची नजर: काय **विचाराल**? Chalta tukda दाखवा (4.5); कुठले APIs वापरणार, स्वतः काय बनवणार (7.4: payment/नकाशा विकत, फक्त जोडणी); चालवायचा खर्च काय (7.8: याची गिन्हाईकं 200; cloud चं bill चिमूटभर असायला हवं); आणि data कुठे, backup काय (6.7). चार प्रश्नांत दीड लाखाची किंमत उघडी पडते: इथे **बनवण्यासारखं** फार नाहीच: तयार विटा जोडायच्या आहेत.

आता मागे सरका आणि बघा काय घडलं: एका दुकानावर पुस्तकाची पाचही parts बोलली. आणि सूत्र (0.2) शेवटचं लावा: काकांचं दुकान: size लहान (एक गल्ली), scarcity कमी (शेजारी दुसरं दुकान), leverage जवळजवळ शून्य (काका झोपले = दुकान बंद). पण **हजार** दुकानांची उधारी-वसुली सोपी करणारं app: size मोठा, आणि leverage पूर्ण (recipe एकदा, हजार दुकानं). म्हणजे पैसा **काकांच्या** दुखण्यात आहे, पण मोठा पैसा ते दुखणं **हजारदा** सोडवण्यात आहे. हीच Level 2 -> Level 3 ची उडी, आतून बघितलेली.

इथे लोक काय चुकीचं समजतात

”आधी technology शिका/बनवा, मग बघू कुठे लावायची.” उलटा क्रम आहे, आणि आता तुमच्याकडे पुरावा आहे: आपण दुकानात **दुखण्यांपासून** सुरुवात केली, आणि प्रत्येक दुखण्याने आपलं हत्यार स्वतः निवडलं. Technology-आधी-माणसं हा क्रम उलटला (चकाचक app, गरज कोणालाच नाही) की Chapter 0.3 ची विहीर आठवा: दोरी सुंदर, विहीर कोरडी. प्रश्न नेहमी तोच: **कोणाचं, कुठलं दुखणं, किती तीव्र?** उत्तर Level 1 मध्ये; हत्यारं Level 3-4 मध्ये.

MAP वर

हा chapter स्वतःच नकाशाची परीक्षा होती. पण एक शेवटची खूण: लक्षात घ्या की काकांच्या दुकानात आपल्याला **नवीन शोध** एकही लागला नाही: सगळी हत्यारं तयार होती (apps, UPI, cloud, APIs): फुकट किंवा स्वस्त (Chapter 3.7 चं वरदान). म्हणजे आजच्या जगात संधीची व्याख्या बदलली आहे: **हत्यारं शोधणं हा खेळ संपला; दुखणं नेमकं ओळखून हत्यारं जोडणं हा खेळ उरला.** आणि त्या खेळात तुमच्याकडे आता दोन्ही डोळे आहेत: दुखण्याचा (Level 1-2) आणि हत्याराचा (Level 3-4).

स्वतः बघा (5 मिनिटं)

हा chapter तुमच्या हातात एक **चौकट** देऊन जातो; ती कागदावर लिहा: (1) या धंद्यात रोज **पुन्हा-पुन्हा** काय होतं? (recipe-संधी, 1.3) (2) **वही** कुठे आहे आणि ती कागदी का digital? (6.1) (3) **मोजकी वस्तू** कुठली? (6.4) (4) **वाट** कुठे बघावी लागते? (6.5) (5) पैसा **कुठून कुठे** वाहतो? (0.3). पाच प्रश्न: कुठल्याही धंद्याचा X-ray.

विचार करा

- (derivation) आता दावा **तुम्ही** तपासा: तुमच्या ओळखीचा एक धंदा निवडा (शेतकरी, डॉक्टरचा दवाखाना, पूजा-साहित्याचं दुकान, tuition class: कुठलाही). वरची पाच-प्रश्नांची चौकट चालवा आणि **एक** जागा शोधा जिथे technology ने सगळ्यात आधी पैसा बनेल.

उत्तर: उत्तर तुमचं-तुमचं येईल; चौकट बरोबर चालली की ते **आपोआप** दिसतं: बहुधा ते वही-दुखणं असेल (नोंदी, उधारी, appointments) किंवा वाट-दुखणं (रांगा, आठवणी, follow-up), कारण तिथे recipe स्पष्ट, data छोटा, आणि तयार हत्यारं स्वस्त. आणि एक तपासणी: तुमचं उत्तर “अमुक app बनवू” ने सुरू होत असेल तर चौकट पुन्हा चालवा; ते “यांचं तमुक दुखणं रोजचं आहे” ने सुरू व्हायला हवं. दावा तपासला: जर हे जमलं, तर Part 0 चं वचन या पुस्तकाने पाळलं आहे: आणि श्रेय पुस्तकाचं नाही, तुमच्या जोडण्यांचं आहे.

SECTION 8: सगळं जोडून

सहा chapters: पुस्तकाचा कळस. एक खरं product आतून-बाहेरून, founder चा सोपं-अवघड नकाशा, spec लिहिण्याची कला, गोष्टी तुटण्याच्या खऱ्या गोष्टी, जिंकण्या-मरण्याचे नियम, आणि शेवटी परीक्षा.

हे तुमच्या business decision मध्ये कुठे येईल: आतापर्यंत पुस्तकाने तुम्हाला **भाग** दिले; हा section **जोडणी** देतो: जी नजर एक product उघडून त्यात सगळे भाग एकाच वेळेस बघते. तीच नजर घेऊन तुम्ही बनवाल, विकत घ्याल, आणि नकार द्याल.

Chapter 8.1 [SPINE]: एक खरं product उघडून बघणं

रात्रीचे आठ वाजले आहेत. तुम्ही Zomato उघडता, biryani शोधता, order करता, पैसे देता, आणि 40 मिनिटांनी जेवता. **60 seconds** चा तुमचा हात-भाग. आता तेच 60 seconds पुन्हा, पण या वेळेस पुस्तकाच्या डोळ्यांनी: प्रत्येक पावलावर कंसात chapter-क्रमांक: हा तुमचा पूर्ण अभ्यासाचा एकत्रित नकाशा आहे.

- 1. तुम्ही icon ला हात लावता.** काचेखालचा sensor, current ची साखळी, billions transistors (1.1). App: storage मधून RAM च्या मेजावर चढणारी recipe; जन्मणारी process (2.5, 3.6). एकच phone हजार machines बनतो: universal machine (1.6).
- 2. App “zomato.com” ला हाक मारतं.** नावाचा number होतो: DNS ची शिडी, cache मुळे बहुधा एका फेरीत (5.5). कुलूप-करार: https, दोन किल्ल्या (5.6). तुमची विनंती तुकड्यांत: packets, प्रत्येक स्वतंत्र, TCP जुळवणारा (5.3). रस्ता: WiFi चे 10 meter, मग Jio ची काच, कदाचित समुद्राखालची तार (5.7).
- 3. विनंती Zomato च्या दारात.** दारावर वाटप करणारा: load balancer, मागे रिकामे (stateless) servers चे counters (7.1). प्रत्येक server: cloud मधली भाड्याची machine (7.5), जिच्यावर लिहिलेली recipe कुठल्या भाषेत होती आणि कशी भाषांतरित झाली हे तुम्हाला माहीत आहे (3.1-3.3).
- 4. Kitchen विचार करतं.** तुमच्या भागातली restaurants: database ला query, index मुळे क्षणात (6.1, 6.2). Menu आणि photos: cache आणि CDN मधून, जवळच्या godown मधून (6.6, 7.7): म्हणून यादी इतकी पटकन येते. Photos चं वजन: pixels चे numbers, दाबलेले (2.1, 2.2).
- 5. तुम्ही “Place Order” दाबता.** आता खेळ बदलतो: दाखवणं संपलं, बदलणं सुरू (6.6 चा नियम). किंमत frontend वर दिसली होती; खरी तपासणी kitchen मध्ये (7.3). पैसे: Razorpay चा API (7.4), मागे UPI चा protocol (5.4), आणि दोन banks च्या वह्यांमध्ये एक गट्टा: transaction, वजा-जमा एकत्र किंवा काहीच नाही (6.4).
- 6. हिरवी खूण.** आणि त्याच क्षणी चिठ्ठ्या रांगेत: restaurant च्या screen ला, delivery शोधणाऱ्याला, तुमच्या SMS ला (6.5). तुम्हाला वाट लागली नाही; रांगा मागे चालतायत. Delivery वाला नकाशावर सरकतो: Maps चा API (7.4), त्याच्या phone वरून दर काही seconds नी येणारे छोटे packets.

7. आणि पडद्यामागे, सतत. प्रत्येक भागाच्या प्रती जिवंत: failover तयार (7.2). रात्रभर backup चालेल (6.7). मीटर फिरतोय: compute, storage, egress (7.8). आणि पैशाची साखळी वाहते आहे: तुमचे 300 -> restaurant -> Zomato -> cloud/APIs -> chips: Level 1 पासून TSMC पर्यंत (0.3, 7.8).

हे सगळं: **60 seconds मध्ये, प्रत्येक वेळेस, crores लोकांसाठी.** आणि आता या chapter चा खरा मुद्दा: हे **कोणी एकाने** बांधलेलं नाही. Transistor वाल्याला biryani माहीत नाही; Zomato च्या engineer ला silicon माहीत नाही. प्रत्येक थर खालच्या थरावर **विश्वास** ठेवतो आणि फक्त आपलं काम करतो: abstraction (0.4) ही फक्त युक्ती नाही; ती **सहकाराची** पद्धत आहे: लाखो अनोळखी माणसांनी थरा-थरांत वाटून घेतलेलं एक काम. मंडईची भाजी तुमच्या ताटात येते तेव्हाही हेच घडतं: शेतकरी-वाहतूक-मंडई-दुकान-घर: कोणी एक “जेवण बनवत” नाही. Machine चं जगही तसंच पिकतं.

इथे लोक काय चुकीचं समजतात

“एवढं सगळं एका order मागे? मग हे बांधणं अशक्यच.” उलट निष्कर्ष काढा: हे सगळं **तयार** आहे: थर बांधलेले आहेत, दारं (APIs) उघडी आहेत, भाडं परवडणारं आहे. नवा founder 60 seconds ची ही साखळी **पहिल्या दिवशी** वापरू शकतो; त्याला ती **बांधावी** लागत नाही. अशक्य वाटणारं चित्र हीच तुमची सगळ्यात मोठी सोय आहे: तुमचं काम फक्त वरचा, तुमचा, वेगळा थर.

MAP वर

हा chapter च नकाशा आहे; पण एक शेवटचं बोट: वरच्या 7 पावलांमध्ये **किती companies** कमावल्या ते मोजा: telecom, cloud, CDN, payment, Maps, database-सेवा, chips... एक biryani, पंधरा दुकानं. Level 3 म्हणजे एक company नाही; ती एक **बाजारपेठ** आहे, आणि प्रत्येक थराच्या फटीत एक धंदा मावतो. तुमची फट कुठली?

स्वतः बघा (5 मिनिटं)

आज कुठलंही **एक** app वापरताना (order, ticket, payment: काहीही) मनातल्या मनात ही 7 पावलं चालवा: हात-DNS-दार-kitchen-बदल-रांगा- मीटर. पहिल्यांदा हळू जाईल; तिसऱ्या वेळेस आपोआप दिसेल. ज्या दिवशी app “आपोआप उघडताना” तुम्हाला थर दिसायला लागतील, त्या दिवशी हे पुस्तक तुमच्या डोक्यातून बोटांत उतरलं.

विचार करा

1. (derivation) आता तीच trace **तुम्ही** लिहा: UPI ने मित्राला 500 पाठवण्याची. कमीत कमी सहा पावलं, प्रत्येकावर chapter-क्रमांक. (मग खालचं उत्तर बघा.)

उत्तर (सांगाडा): (1) बोट-PIN: switches, आणि PIN कधीच उघडा प्रवास करत नाही: encryption (1.1, 5.6). (2) app -> तुमच्या bank चं दार: DNS, https, packets (5.5, 5.6, 5.3). (3) NPCI चा protocol: दोन banks ची भाषा एक (5.4). (4) दोन वहांमध्ये गठ्ठा: lock + transaction, वजा-जमा एकत्र (6.4). (5) हिरवी खूण तुम्हाला; SMS/नोंदी रांगेतून (6.5). (6) मागे: प्रती, failover, backup (7.2, 6.7); मीटर (7.8); आणि यंत्रणा 99.99% जागी (7.2): म्हणून पहाटे तीनला चहावाल्याकडेही हे चालतं. सहा पावलांत अर्ध पुस्तक: आणि आता ते **तुम्ही** लिहिलंत, वाचलं नाहीत.

Chapter 8.2 [SPINE]: काय बनवणं सोपं, काय अवघड

Founder ची सगळ्यात महाग चूक कुठली माहीत आहे? Feature ची किंमत न ओळखणं: सोपं समजून अवघडात उडी, किंवा अवघड समजून सोप्याला नकार. आज तो नकाशा बनवू: पुस्तकातल्या हत्यारांनीच.

सोपं (आठवडे): जे तयार थरांची फक्त जोडणी आहे. Forms, याद्या, नोंदी, dashboards: म्हणजे “वही + दाखवणं” (6.1 + 7.3). Login, payment, नकाशा, SMS: विकत मिळतं (7.4 चे APIs). असं product म्हणजे लग्नाचा मांडव: बांबू, कापड, खुर्च्या सगळं भाड्याने; कारागिरी फक्त **जोडणीत**. AI ने (Book 2) ही जोडणी अजून वेगवान केली आहे: सोपं आता **अजून** सोपं आहे.

मध्यम (महिने): जिथे तुमचा स्वतःचा थर बांधावा लागतो. दोन बाजूंची बाजारपेठ (ग्राहक + दुकानदार दोघांना पटवणं), नाजूक जुळवाजुळव (delivery वाल्यांचं वेळापत्रक), offline-आधी design (गावाकडचं network: 7.7). इथे तयार तुकडे **पुरत नाहीत**; इथे तुमची खरी मेहनत आणि तुमचा खरा **किल्ला** (वेगळेपण) दोन्ही असतात.

अवघड (वर्ष, फौज): जिथे भौतिकशास्त्र आणि मोजक्या वस्तूंची शर्यत एकत्र येतात. Crores लोकांसाठी **एकाच क्षणी** बरोबर असणं: tatkal-स्केलचं seat-वाटप (6.4 x 7.6), स्वतःचं payment-network (5.4 च्या protocol चं स्वतःचं रूप), realtime video crores ना (2.1 चं वजन x 7.8 चं egress), स्वतःचे AI models शून्यातून. इथे “बनवू का?” चा उत्तर बहुधा “विकत घ्या/भाड्याने घ्या” असतं: दूध लागतं म्हणून कोणी म्हैस पाळत नाही.

आणि आता या नकाशाची **मुख्य फसवणूक**, जी प्रत्येक नव्या founder ला एकदा तरी लुटते: **दिसणं आणि असणं यांचा संबंध उलटा आहे.** “WhatsApp सारखं app” ऐकायला सोपं: दिसतं साधं ना! पण आत: crores च्या रांगा (6.5), end-to-end किल्ल्या (5.6), 99.99% जागेपण (7.2): जगातल्या सगळ्यात अवघड यादीत. उलट “दुकानाची उधार-वही app” ऐकायला किरकोळ, पण खरोखर आठवड्यांचं काम आणि खरा पैसा (7.9 चे काका). **Demo सोपा, production अवघड** (4.5 चा 90%-भ्रम); आणि “थोडंसं वेगळं हवं” ही ओळ ऐकली की कान टवकारा: platform च्या वाटेवरून उतरलात की किंमत दहापट (4.1 च्या जोड्या).

इथे लोक काय चुकीचं समजतात

“Spec मधली एक ओळ = कामाची एक ओळ.” आता तुमच्याकडे मोजपट्टी आहे: ओळ वाचून विचारा: ही **जोडणी** आहे (सोपं), **नवा थर** आहे (मध्यम), की **मोजक्या वस्तूंची शर्यत/भौतिकशास्त्र** आहे (अवघड)? “Users एकमेकांना दिसतील” ही सात शब्दांची ओळ: पण ती realtime असेल तर (7.6, 7.7)

महिन्यांचं दार उघडते. Engineer “हे अवघड आहे” म्हणतो तेव्हा तो बहुधा टाळत नाही; तो हीच मोजपट्टी वाचतोय: आता तुम्ही त्याच्याशी **त्याच** भाषेत बोलू शकता.

MAP वर

रुपयाचा रस्ता: सोपं झालं म्हणजे **स्वस्त** झालं म्हणजे तिथे गर्दी आणि स्पर्धा (0.2 चं सूत्र: scarcity घसरली). नुसती जोडणी असलेली products आता AI सुद्धा बनवतो: तिथे किल्ला नाही. किल्ला दोन जागी उरतो: **मध्यम** पट्ट्यात (तुमची खास जुळवाजुळव, तुमचा data: 6.1) आणि **वाटपात** (8.5 मध्ये येतंय). म्हणून धोरण एका ओळीत: सोप्याने **सुरुवात** करा (वेग!), पण खंदक मध्यम पट्ट्यात खणा; अवघडाला API ने हात लावा, फौजेने नाही.

स्वतः बघा (5 मिनिटं)

तुमच्या phone मधली तीन apps घ्या आणि प्रत्येकीला विचारा: हिचा **अवघड** भाग कुठला असेल? Zomato: वेळेवर delivery ची जुळवाजुळव. WhatsApp: crores realtime + किल्ल्या. UPI app: पैशाची शुद्धता. आणि मग गंमत बघा: प्रत्येकीचा अवघड भाग **दिसत** नाही (0.4 चं जुनं सत्य, शेवटच्या वेळेस): screen वर सगळ्या सारख्याच “सोप्या” दिसतात.

विचार करा

1. (derivation) दोन ideas, दोन्ही “booking”: (अ) डॉक्टरांसाठी appointment-वही: रुग्ण वेळ निवडतो, आठवण जाते. (ब) “सगळ्या शाळांचे निकाल एकाच क्षणी, live” दाखवणारी site. कुठली सोपी, कुठली अवघड, आणि **नेमकं का?** (पुस्तकातले chapter-क्रमांक द्या.)

उत्तर: (अ) सोपी-ते-मध्यम: वही + दाखवणं (6.1, 7.3), आठवणी रांगेतून (6.5); मोजकी वस्तू (वेळेची जागा) आहे, पण शर्यत छोटी: एका दवाखान्यात एका क्षणी दोघं-चौघं भांडतील (6.4 पुरतं). (ब) अवघड: “एकाच क्षणी” + “सगळे” = 7.6 चा राक्षस (निकालाचा दिवस!), वाचनाचा महापूर CDN/cache ने झेलावा लागेल (6.6, 7.7), आणि चुकीचा निकाल दाखवणं परवडत नाही म्हणजे शिळेपणाची मोजपट्टी कडक (6.6). दोन्हीत “booking/निकाल” हे शब्द साधेच; फरक **कोण, किती, एकाच क्षणी का** या तीन प्रश्नांत. Feature च्या शब्दांना नाही, या तीन प्रश्नांना किंमत असते.

Chapter 8.3 [SPINE]: Spec कसा दिसतो

तुम्ही कधीतरी कोणाला तरी (माणसाला किंवा AI ला) म्हणाल: “मला असं app बनवून हवंय.” त्या क्षणी जे कागदावर द्याल, त्याचं नाव **spec** (specification: नेमकं-काय-हवं ची लिहिलेली मागणी). आणि Part 2 पासूनचं सत्य आठवा (3.1): machine ला **exact** लागतं, आणि बनवणाऱ्या माणसालाही. अर्धवट spec = Chapter 4.1 ची फट = “मला हे नको होतं” चा महाग शोध.

घर बांधणाऱ्याला “छान घर बांधा” म्हणत नाही; नकाशा देतो. पण spec म्हणजे 100 पानं नाहीत. चांगला spec **सात प्रश्नांची** उत्तरं आहे: आणि सातही तुम्ही या पुस्तकात शिकला आहात:

- 1. कोणासाठी, कुठलं दुखणं?** (Level 1 पासून सुरुवात: 0.3, 7.9.) एक ओळ: “कोण, रोज काय सहन करतं.”
- 2. मुख्य प्रवास, पावला-पावलाने.** “User हे करतो -> त्याला हे दिसतं -> मग हे.” (1.3 ची recipe-भाषा: तुम्हीच recipe चे पहिले लेखक.)
- 3. मोजकी वस्तू आणि कडक नियम कुठले?** पैसे, जागा, stock: जिथे शर्यत आणि गठ्ठे लागतील (6.4). ही ओळ लिहिली नाहीत, तर ती **चूक होऊन** भेटते.
- 4. कुठला data कायमचा, आणि किती हरवणं चालेल?** वही कुठली (6.1), backup चा भूक-प्रश्न (6.7 चा “किती तास परवडतात?”).
- 5. आत्ता काय, नंतर काय चालेल?** हिरव्या खुणेआधी काय झालंच पाहिजे; रांगेत काय जाऊ शकतं (6.5).
- 6. किती लोक, कुठून, किती वेगात?** शंभर की लाख (7.1); शहर की गाव (7.7); सामान्य दिवस की स्फोटाचे क्षण (7.6).
- 7. काय बनवायचं नाही?** सगळ्यात ताकदवान ओळ. “यात हे **नसेल**” हे लिहिणं म्हणजे 4.1 च्या जोड्यांचा गुणाकार थांबवणं आणि 4.5 चा छोटा तुकडा राखणं.

बघा कसा दिसतो: **टिफिन-सेवेचा spec, नऊ ओळींत:**

कोणासाठी: शहरात एकटे राहणारे; रोजच्या जेवणाची भ्रांत.
 प्रवास: यादीतून सेवा निवड -> महिन्याची वर्गणी -> रोज
 दुपारी डबा; "उद्या नको" रात्री 10 पर्यंत सांगता येतं.
 मोजकं + नियम: सेवेची रोजची डबे-क्षमता (संपली की
 नवीन वर्गणी बंद); पैसे आधी, महिन्याचे.
 Data: वर्गण्या + रोजचे बदल + पैसे; एक तासाहून जास्त
 नोंदी हरवणं चालणार नाही.
 आत्ता vs नंतर: वर्गणी + payment आत्ता; रोजची आठवण,
 सेवेला यादी: रांगेतून.
 किती: पहिल्या वर्षी एक शहर, ~2,000 वर्गणीदार; रोज
 सकाळी 9-11 गर्दीचा फुगा.
 नाही: delivery-मुलांचं app नाही (सेवा स्वतः बघेल),
 रोजची सुट्टी-निवड नाही (फक्त "उद्या नको"), ratings
 नाहीत (व1 मध्ये).

नऊ ओळी: पण बनवणाऱ्याच्या डोक्यातले शंभर प्रश्न मिटले, आणि तुमच्या नकळत तुम्ही database ची
 रूपरेषा (4), queue ची यादी (5), आणि scale ची तयारी (6) लिहून दिलीत: **spec म्हणजे पुस्तकाची
 चौकट, गद्यात.**

इथे लोक काय चुकीचं समजतात

दोन टोकं. "तोंडी सांगितलं, कळेल त्याला": Chapter 3.1 उलटा वाचा: माणूसही अर्धवट ऐकतो, आणि AI
 तर **नेमकं** तेच बनवतो जे लिहिलंय, जे **हवं होतं** ते नाही (1.3: machine निर्णय चालवते). दुसरं टोक:
 "100 पानांचा दस्तऐवज": तो कोणी वाचत नाही, आणि पहिल्या चालत्या तुकड्याने (4.5) तो शिळा होतो.
 Spec **जिवंत** असतो: छोटा, नेमका, प्रत्येक तुकड्यागणिक वाढणारा. आणि सोनेरी नियम: प्रत्येक
 ओळीला विचारा "ही ओळ चुकली तर काय तुटेल?": उत्तर "काही नाही" असेल तर ओळ हटवा.

MAP वर

रुपयाचा रस्ता: spec लिहिता येणं ही **founder ची** सगळ्यात स्वस्त आणि सगळ्यात परताव्याची skill
 झाली आहे: कारण AI च्या जमान्यात बनवणं स्वस्त होतंय (3.2 ची भविष्यवाणी: व्याकरण स्वस्त, साफ
 विचार महाग), म्हणजे **मागणी नेमकी सांगता येणं** हीच उरलेली scarcity. सात प्रश्नांचा हा साचा तुमच्या
 दुसऱ्या मेंदूत (वहीत) ठेवा: पुढच्या प्रत्येक कल्पनेला आधी हे नऊ-ओळी रूप द्या; मगच कोणाशी बोला.
 एक तासाची ही सवय लाखांच्या गैरसमजांची विमा-पॉलिसी आहे.

स्वतः बघा (5 मिनिटं)

वरचा टिफिन-spec झाकून ठेवा आणि **तुमच्या** डोक्यातल्या कुठल्याही एका कल्पनेचा सात-प्रश्नी spec लिहा: नऊ-दहा ओळीच. अडाल तिथे खूण करा: अडलात **म्हणजे** तिथेच तुमच्या कल्पनेतलं धुकं आहे: spec लिहिणं हे बनवणाऱ्यासाठी नाही, आधी **तुमच्यासाठी** असतं: धुकं शोधायचं यंत्र.

विचार करा

1. (derivation) वरच्या टिफिन-spec मध्ये एक महत्वाचं भोक मुद्दाम ठेवलं आहे: एक अशी रोजची घटना जिचं उत्तर spec देत नाही. शोधा. (Hint: Chapter 4.1 चा चष्मा लावा: “नेहमीचा प्रवास” सोडून कडेला काय घडू शकतं?)

उत्तर: असे एकाहून जास्त सापडतील: हीच 4.1 ची शिकवण: पण सगळ्यात टोचणारं: **payment झालं पण डबा आलाच नाही, तर?** (सेवा बंद पडली, डबेवाला आजारी, पाऊस.) पैसे परत? पुढच्या महिन्यात जमा? कोणाची जबाबदारी? हे spec मध्ये नाही: म्हणजे ते **कोणीतरी दुसरं** ठरवणार: बनवणारा engineer, ऐन वेळी, तुमच्या business चा नियम त्याच्या अंदाजाने लिहीत. Edge case म्हणजे तांत्रिक बारकावा नाही; तो **business चा नियम** आहे जो लिहायचा राहिला. Spec चं खरं काम: आनंदाचा प्रवास नाही, **दुखऱ्या फांद्या** आधी लिहिणं.

Chapter 8.4 [DEPTH]: खऱ्या आयुष्यात गोष्टी कशा तुटतात

(DEPTH chapter. पण यातल्या गोष्टी खऱ्या आहेत, आणि founder म्हणून तुमच्या टेबलावर कधीतरी अशीच एक रात्र येईल.)

पुस्तकभर आपण तुटण्याचे तुकडे शिकलो: bugs (4.1), माहोल (4.4), cascade (7.6), मरणान्या disks (6.7). आज तीन **खऱ्या** गोष्टी: जगातल्या सगळ्यात हुशार teams च्या. हेतू टिंगल नाही; हेतू हा: तुटणं **कसं दिसतं** आणि शहाणी माणसं त्यातून **काय** करतात.

गोष्ट 1: स्वतःचंच दार बंद (Facebook, 2021). एका routine बदलात एक चुकीची आज्ञा गेली, आणि Facebook ने जगाच्या नकाशावरून **स्वतःचे पत्ते** पुसले (Part 3 ची भाषा: त्यांचे रस्ते जगाच्या route-वहीतून गायब). WhatsApp, Instagram: 6 तास जग-बंद. आणि कहर: engineers चे **आतले** दरवाजेही त्याच system वर होते: दुरुस्ती करणाऱ्यांना building त शिरता येईना. धडा 1: **बदल हाच सगळ्यात मोठा धोका** (बहुतेक मोठे मृत्यू deploy नंतर येतात: 7.2 चं एक-एक-counter शहाणपण याचसाठी). धडा 2: दुरुस्तीचा रस्ता **वेगळा** ठेवा: आग विझवायची नळी जळणाऱ्या घरातच ठेवू नका.

गोष्ट 2: एक typo, अर्ध internet (Amazon S3, 2017). एक engineer, एक नेहमीची आज्ञा, एक टंकलेखनाची चूक: हेतूपेक्षा **खूप जास्त** servers बंद केले गेले. आणि cascade (7.6): S3 वर जग टेकलेलं होतं, म्हणून हजारो websites एकदम लंगड्या. धडा: चूक **माणसाची** नव्हती; **व्यवस्थेची** होती: इतकी घातक आज्ञा एका typo इतकी सोपी का होती? कुंपण का नव्हतं ("इतकं बंद करायचं? खात्री आहे?")? शहाण्या systems मध्ये मोठ्या विनाशाचे दरवाजे **मुद्दाम जड** केलेले असतात.

गोष्ट 3: Backup होते; restore नव्हतं (GitLab, 2017). रात्रीची गडबड, थकलेला engineer, चुकीच्या server वर delete. ठीक: backup आहे ना? उघडले: पाच जागी प्रती ठेवायची व्यवस्था होती, आणि **पाचही** कुठल्या ना कुठल्या कारणाने अधुन्या/रिकाम्या निघाल्या. 6 तासांचा data कायमचा गेला. Part 4 ची ओळ त्यांच्याच रक्ताने लिहिलेली आहे: **न तपासलेला backup म्हणजे backup नाहीच** (6.7). आणि एक गोष्ट त्यांनी जगाला शिकवली: त्यांनी हे सगळं **जाहीर, live** सांगितलं: लपवलं नाही: आणि जग रागावण्याऐवजी त्यांच्यावर विश्वासाने वाढलं.

तिन्ही गोष्टींतून एकच पद्धत जन्मते, आणि तिचं नाव शिका: **postmortem** (घटनेनंतरचं शवविच्छेदन: पण system चं, माणसाचं नाही). शहाण्या companies प्रत्येक मोठ्या तुटण्यानंतर एक दस्तऐवज लिहितात: काय घडलं (मिनिटा-मिनिटाने), **कुठली व्यवस्था** कमी पडली, आणि पुन्हा होऊ नये म्हणून कुठली **व्यवस्था** बदलली. आणि पहिला नियम: **blameless** (दोषारोप-मुक्त): "कोण चुकलं" विचारायचं नाही; "चूक इतकी **सोपी** का होती" विचारायचं. कारण साधं आहे: माणसाला शिक्षा केली की पुढच्या वेळेस

माणूस चूक **लपवतो**: आणि लपलेली चूक पुन्हा येते, मोठी होऊन. विमान-उद्योग जगात सगळ्यात सुरक्षित याच एका संस्कृतीने: प्रत्येक अपघाताचा जाहीर अभ्यास, दोषारोपाशिवाय; म्हणून प्रत्येक अपघात **शेवटचा** त्या-प्रकारचा.

इथे लोक काय चुकीचं समजतात

”मोठ्या companies कडे असं होत नसेल; आपल्याकडे झालं म्हणजे आपण कच्चे.” वरच्या तीन गोष्टी पुन्हा वाचा: जगातल्या **सगळ्यात** मोठ्या teams च्या आहेत. फरक तुटण्या-न-तुटण्यात नाही; फरक **नंतर** काय होतं यात आहे: लपवणं की शिकणं. आणि founder साठी आरसा: तुमच्या team मध्ये वॉर्ट बातमी तुमच्यापर्यंत **लवकर** येते का उशिरा? उशिरा येत असेल, तर तुमची team कच्ची नाही; तुमची **संस्कृती** महाग आहे: लोक तुम्हाला घाबरून चुका लपवतायत (4.5 ची प्रामाणिक-team खूण).

MAP वर

रुपयाचा रस्ता: विश्वास हा माल आहे (Part 3 पासूनचं गाणं), आणि तुटण्याच्या क्षणी तो **सगळ्यात जास्त** विकला-विकत घेतला जातो. GitLab लपवू शकले असते; त्यांनी उघडलं आणि ग्राहक **वाढले**. उलट ज्या companies तुटणं लपवतात (विशेषतः data-चोरी!), त्या उघडकीस आल्यावर दुप्पट बुडतात: एकदा तुटण्याने, एकदा लपवण्याने. नियम: **तुटणं माफ होतं; लपवणं नाही**. आणि हो: postmortem ची सवय फुकट आहे: पहिल्या दिवसापासून, एका-माणसाच्या company तही.

स्वतः बघा (5 मिनिटं)

तुमच्या स्वतःच्या (कुठल्याही: दुकान, नोकरी, घर) आयुष्यातली एक जुनी, बोचणारी चूक घ्या. तिच्यावर blameless postmortem चे तीन प्रश्न चालवा, कागदावर: (1) नेमकं काय घडलं, क्रमाने? (2) कुठली **व्यवस्था** (सवय, तपासणी, आठवण) असती तर हे टळलं असतं? (3) ती व्यवस्था आज लावता येईल का? “कोण चुकलं” हा प्रश्न यादीत **नाही** हे जाणीवपूर्वक बघा: आणि लक्षात घ्या, या तीन प्रश्नांनी चूक आठवताना टोचणं कमी आणि उपयोग जास्त होतो. हीच पद्धतीची ताकद.

विचार करा

1. (derivation) तुमच्या टिफिन-app (8.3) च्या पहिल्या महिन्यात एक रात्र: चुकीच्या यादीमुळे 200 पैकी 60 जणांना डबे गेलेच नाहीत. Blameless postmortem चा सांगाडा लिहा: घडलं-काय, व्यवस्था-कुठली- कमी, बदल-कुठला. (मग उत्तर बघा.)

उत्तर (सांगाडा): घडलं: रात्री 10 च्या “उद्या नको” बदलांची रांग (6.5) सकाळच्या यादी-निर्मितीनंतर रिकामी झाली: यादी जुनी गेली (6.6 चा शिळेपणा, चुकीच्या जागी!). व्यवस्था कमी: यादी बनल्यावर तिची **तपासणी** नव्हती (एकूण डबे = वर्गणीदार - उद्या-नको: हा साधा ताळा कोणी मोजला नाही: 4.2 ची जाणीवपूर्वक- तपासणी); आणि सेवेकडून “मला 140 च दिसतायत” चा इशारा यायला रस्ताच नव्हता. बदल: यादी बनण्याची वेळ रांग रिकामी झाल्या- **नंतर**; ताळ्याची आपोआप तपासणी, न जुळल्यास माणसाला गजर; आणि प्रत्येक सकाळी सेवेला “आजचे एकूण” ची पावती. बघा: उत्तरात एकही “कोण चुकलं” नाही: तीन नव्या **व्यवस्था** आहेत. हीच सवय, हाच धंदा टिकवते.

Chapter 8.5 [SPINE]: एक technology जिंकते, दुसरी मरते: का?

पुस्तकभर मेलेल्यांची थडगी दिसली: Nokia (1.6), जुनं telephone-जग (5.3), SMS (5.8). आणि प्रत्येक वेळेस एक अस्वस्थ प्रश्न मागे उरला: मेलेले **वाईट** नव्हते. Nokia चे phones दणकट होते; SMS सगळीकडे चालायचा. मग “चांगलं product जिंकतं” हे वाक्य खोटं आहे का?

अर्थ खोटं आहे: आणि तो अर्धा भाग founder साठी सोन्याचा आहे. जिंकण्या-मरण्याचे खरे नियम पाच आहेत; पाचही तुम्ही तुकड्या- तुकड्यांत शिकलात; आज ते एका टेबलावर:

1. जाळ्याची ओढ (network effect). WhatsApp का सोडता येत नाही? App प्रेमासाठी नाही; **माणसं** तिथे आहेत म्हणून. प्रत्येक नवा user जुन्या सगळ्यांसाठी किंमत वाढवतो: n -squared, पण या वेळेस तुमच्या **बाजूने** (5.1 चा राक्षस पाळीव झाला की किल्ला होतो). अशा जागी दुसरा येऊन “10% चांगलं” देऊन जिंकू शकत नाही; त्याला **रिकामं जाळं** विकावं लागतं: आणि रिकाम्या मंडईत कोणी दुकान लावत नाही.

2. दरवाजाचा मालक (platform + default). Chapter 3.5 चा जकात-नाका: जो OS/store/browser चा मालक, तो **default** ठरवतो. आणि माणसं default बदलत नाहीत: जे आधीच चालू आहे ते चालू राहतं. RCS वर्षानुवर्ष Apple च्या दारात उभा होता (5.8): चांगला असून; दार मालकाचं होतं.

3. सोडून जाण्याची किंमत (switching cost). Bank बदलणं, सगळे photos हलवणं, team ला नवं tool शिकवणं: शक्य, पण **वेदना**. जो product तुमचा data, तुमच्या सवयी, तुमच्या जोडण्या धरून ठेवतो, त्याला 10% ने हरवता येत नाही; **10x** लागतो.

4. वेळ (timing). खूप लवकर येणं हे उशिरा येण्याइतकंच घातक: video-calling 2000 साली होतं (network नव्हतं); touch-phones iPhone आधी होते (chips महाग होत्या). तुमची कल्पना नाही, तिच्या **खालचे थर** तयार आहेत का, हा प्रश्न (0.3 चा stack: खालचा मजला कच्चा असेल तर वरचा बांधून मरता). Jio-नंतरच्या India त video- first apps चाललीत हा योगायोग नाही: 2016 ने खालचा मजला ओतला.

5. वाटप (distribution). सगळ्यात कडू आणि सगळ्यात खरा: **ग्राहकापर्यंत पोहोचण्याचा रस्ता** ज्याच्याकडे, तो सामान्य product घेऊनही जिंकतो; रस्ता नसलेला उत्कृष्ट product कपाटात मरतो. म्हणून companies “फुकट” वाटतात (3.7), दरवाजे विकत घेतात (WhatsApp: 19 billion!), आणि म्हणून UPI wallets ना हरवू शकला: प्रत्येक bank, प्रत्येक app हा त्याचा **रस्ता** होता (5.4 चा खुला protocol = हजार रस्ते फुकट).

आता पाचही एकत्र वाचा आणि दिसेल: पाचपैकी **एकही** “product किती छान आहे” बदल नाही. Product चांगलं असावंच लागतं: पण ते **प्रवेश-तिकीट** आहे, विजय नाही. विजय जाळं, दरवाजा, खिळे, वेळ आणि रस्ते यांचा आहे. (आणि आता Nokia चा मृत्यू पुन्हा वाचा: Apple ने चांगला phone नाही, **app-store चं जाळं + दरवाजा** आणला: नियम 1 + 2 एकत्र. आता तुम्हाला बातमी नाही, यंत्र दिसतं.)

इथे लोक काय चुकीचं समजतात

“चांगलं बनवलं की जग दार शोधत येईल.” Founder-जगातली सगळ्यात गोड अफू. आता तुमच्याकडे उतारा आहे: कल्पना सुचली की product-प्रश्नाआधी हे विचारायचं: **माझं जाळं काय? माझा रस्ता काय? माझे खिळे काय? थर तयार आहेत का?** चारही कोरे असतील, तर product कितीही सुंदर असो: तुमच्याकडे अजून **धंदा** नाही, फक्त एक छान वस्तू आहे.

MAP वर

तुमच्या 4-level नकाशाची शेवटची जोडणी: हे पाच नियम म्हणजे Level 2 आणि Level 3 च्या **लग्राचे** नियम आहेत: technology (3) बांधते, पण जिंकवतं ते business चं वाटप-जाळं (2). आणि पुढच्या पुस्तकाची चाहूल: AI मध्ये आता हीच लढाई पेटली आहे: models (product) झपाट्याने सारखे होतायत; लढाई जाळं-दरवाजा-वाटपाची होते आहे (कोणाचा AI कुठल्या phone मध्ये default बसणार?). Book 2 वाचताना हे पाच नियम सोबत ठेवा: बातम्या आपोआप उलगडतील.

स्वतः बघा (5 मिनिटं)

तुम्ही स्वतः एक चालतं-बोलतं उदाहरण आहात: विचारा, मी WhatsApp **का** वापरतो? उत्तरांची यादी करा: “सगळे तिथे आहेत” (नियम 1), “आधीपासून आहे” (नियम 2), “सगळे groups/photos तिथे” (नियम 3). “सगळ्यात सुंदर app आहे म्हणून” हे उत्तर यादीत **नसेल**: आणि हेच या chapter चं जिवंत प्रमाणपत्र.

विचार करा

1. (derivation) Electric गाड्यांची शर्यत चालू आहे. दहा वर्षांनी जिंकलेली company कशाने जिंकलेली असेल: सगळ्यात चांगली **गाडी** बनवून, की दुसऱ्या कशाने? पाच नियम लावा.

उत्तर: गाडी प्रवेश-तिकीट; विजय बहुधा **जाव्याचा** असेल: charging चं जाळं (नियम 1+5: जिथे charger तिथे गाडी खपते, जिथे गाड्या तिथे charger परवडतो: दोन्ही बाजूंनी वाढणारी ओढ), बॅटरी- पुरवठ्याची साखळी आणि किंमत (नियम 4: खालचा थर), service-जाळं आणि software-updates चे खिळे (नियम 3), आणि सरकारी नियम/ subsidies चा दरवाजा (नियम 2 चं सरकारी रूप: 5.8 ची आठवण). म्हणूनच Tesla ने आधी **chargers** पेरले आणि China च्या companies बॅटरी-साखळी धरून बसल्या. गाडी दिसते; युद्ध जाव्यांचं आहे. हे उत्तर कुठल्याही “पुढच्या मोठ्या गोष्टीला” लावता येतं: तीच या chapter ची भेट.

Chapter 8.6 [SPINE]: Book 1 ची परीक्षा

(ही अंतिम परीक्षा नाही: ती Book 2 च्या शेवटी आहे, दोन्ही पुस्तकांवर. ही Book 1 ची तपासणी आहे: machine तुमच्या हातात बसली का?)

नियम सोपे: पुस्तक न उघडता, **बोलून** उत्तर द्या: लिहिणं नको, बडबड चालेल. प्रत्येक प्रश्नाखाली उत्तराचा गाभा आहे: आधी स्वतःचं, मग ताडून बघा. 12 पैकी 9 जमले तर machine तुमची झाली; जे अडले त्यांचे chapter-क्रमांक सोबत: तिथे परत जा, लाज न बाळगता: परत जाणं हाच अभ्यास (front matter चं वचन).

1. पैसा फक्त Level 1 वर का घुसतो? (0.1, 0.3)

गरज फक्त माणसाला असते. Level 2/3/4 आपसात देतात तो पैसा खालून चढत आलेला असतो; साखळी नेहमी एका माणसाच्या गरजेवर संपते.

2. Surgeon पेक्षा app-वाला जास्त का कमावू शकतो? (0.2)

कमाई = size x scarcity x **leverage**; तास सूत्रात नाहीत. Operation प्रत्येक वेळेस हाताने; app एकदा बनते, crores वेळा चालते: copy फुकट.

3. एकच machine camera, bank आणि game कशी होते? (1.6)

Recipe सुद्धा numbers आहेत, म्हणून data च्या जागीच ठेवता येतात: नवी recipe = नवं काम, machine तीच: universal machine.

4. “Machine विचार करते” हे वाक्य कुठे तुटतं? (1.3, 1.5)

आत फक्त आधीच-design केलेल्या gates च्या साखळ्या: प्रत्येक input चं output आधीच ठरलेलं. निर्णय **चालतो**; घेतला माणसाने असतो: recipe लिहिणाऱ्याने.

5. Text : photo : video चं वजन-प्रमाण, आणि ते इतकं का महत्वाचं? (2.1)

1 : हजार : लाख. कारण जिथे वजन तिथे खर्च: storage, egress, वेग: product चा आकारच business model चा आकार ठरवतो.

6. iPhone ची app Android वर का चालत नाही? (3.5)

App machine साठी नाही, **मालकासाठी** (OS) लिहिलेली असते; मालक वेगळा, भाषा वेगळी. आणि मालक असणं = दरवाजा + जकात.

7. मोठ्या software मध्ये bugs नक्की का असतात? (4.1)

परिस्थितींच्या जोड्यांचा स्फोट: जी जोडी लिहिणाऱ्याच्या डोक्यात आली नाही, ती कधीतरी कोणाच्या हाती लागते. मोजणी आहे, हलगर्जीपणा नाही; प्रश्न पकडण्याच्या वेगाचा.

8. Internet चा मालक कोण, आणि तरी नियम कोण बनवत? (5.4, 5.7, 5.8)

मालक कोणीच नाही: हजारो जाळी, समान **नियमांनी** (protocols) जोडलेली. नियम खुल्या संस्था (IETF वगैरे) + मोठ्या companies च्या ताकदीने घडतात; standard बनवणारा बाजार आखतो: UPI/DPI हेच भारताचं उत्तर.

9. कुलूप-icon काय सांगतो, काय नाही? (5.6)

रस्ता सुरक्षित: मधला कोणी वाचू शकत नाही. **समोरचा खरा आहे** हे सांगत नाही: scammer कडेही https असतं. रस्ता तपासला; प्रवासी तुम्हीच तपासायचा.

10. “वजा झाले पण जमा नाही” ही अवस्था का अशक्य आहे? (6.4)

Transaction: दोन्ही नोंदी एक गट्टा: पूर्ण किंवा शून्य; अर्धा झाला तर database स्वतः उलटा फिरवतो (rollback). मोजकी वस्तू + अनेक हात = lock + गट्टा.

11. “App हळू आहे” ची तपासणी कुठून सुरू कराल? (6.2, 7.7)

मोजून: second कुठे जातो? बहुधा रस्त्यावर (फेरे, अंतर, वजन) किंवा विना-index query तः server मोठा करणं हे बहुधा चुकीचं आणि महाग उत्तर.

12. Viral होण्याआधी कुठल्या तीन गोष्टी तयार हव्यात? (7.1, 7.6, 7.8)

Stateless servers (आडवं वाढता यावं: “काम वाटा, आठवण एकत्र”), दारावरची शिस्त (queue/rate limit: कोसळण्याऐवजी रांग), आणि खर्चाचा गजर (bill shock चा विमा). संधी दार वाजवते तेव्हा दार तुटू नये.

शेवटचं एक काम, परीक्षेबाहेरचं: Chapter 0.5 मध्ये तुम्ही एका कागदावर लिहिलं होतं: **”मी हे यासाठी शिकतोय की _____.”** तो कागद (photo) आता उघडा. वाचा. उत्तर बदललंय का? बहुतेकांचं बदलतं: सुरुवातीला ते “tech समजावं म्हणून” असतं; आता त्यात बहुधा “दुखणं-हत्यार-साखळी” ची भाषा आली असेल. तो बदल म्हणजेच हे पुस्तक.

आणि आता पुढचं दार. या पुस्तकाने एक machine दिली जी सांगितलेलं करते: exact recipe, exact निकाल. पण तुम्ही रोज एका machine शी बोलता जी अंदाज करते: तुमच्या अर्धवट वाक्याचा अर्थ लावते, कविता लिहिते, code सुचवते: आणि कधीकधी ठाम आत्मविश्वासाने चुकते. ती machine आतून काय आहे? Recipe कुठे आहे तिच्यात: आणि लिहिली कोणी? “शिकणं” म्हणजे machine साठी नेमकं काय? आणि या सगळ्यात पैसा कुठे वाहतो?

Book 2: The Thinking Machine. भेटू तिथे: आता तुमच्याकडे तिला उघडायची सगळी हत्यारं आहेत.

PART 5 चा शेवट: नकाशा

जे तुमच्याकडे आता आहे (Part 5)

SCALING	उभं (मोठी machine: भिंत) vs आडवं (अनेक + load balancer); मंत्र: काम वाटा, आठवण एकत्र
UPTIME	मरणं गृहीत धरा: प्रती + मरण-ओळख + failover; nines ची मोजपट्टी, प्रत्येक nine दहापट महाग
FRONT / BACK	dining hall vs kitchen; frontend वर कधीही विश्वास नाही: खरी तपासणी kitchen मध्ये
API	program चं menu card; धंद्याचा दरवाजा; वापरणारा भाडेकरू, देणारा जमीनमालक
CLOUD	दुसऱ्याच्या building मधल्या machines, भाड्याने; लवचीकपणा हेच मोल, स्वस्ताई नाही
BOTTLENECK	साखळी = कमजोर दुवा; मीटर लावा; रुंद करा / रांग लावा / दार आवळा; cascade + circuit breaker
LATENCY	second रस्त्यावर जातो: अंतर मारा (CDN) + फेरे मारा + वजन घटवा
खर्च	bill च्या तीन ओळी: compute + storage + egress; unit economics; bill shock चा गजर
दावा-तपासणी जोडणी	पाच-प्रश्नांची चौकट: कुठल्याही धंद्याचा X-ray 60 seconds ची trace; सोपं-मध्यम-अवघड नकाशा; सात-प्रश्नी spec; blameless postmortem; जिंकण्याचे 5 नियम (जाळं, दरवाजा, खिळे, वेळ, वाटप)

Book 1 ची परीक्षा मागच्या chapter मध्ये (8.6) झाली. पुढची दोन पानं संदर्भासाठी आहेत: **पूर्ण पुस्तकाचा नकाशा** (भिंतीवर लावण्यासाठी) आणि **master glossary** (पाचही parts चे शब्द एका जागी). पुस्तक वाचून झाल्यावरही ही दोन पानं तुमच्या टेबलावर जगतील: असा हेतू आहे.

PART 5 चा MINI-GLOSSARY

API	programs चं ठरलेलं प्रश्नोत्तर-menu (7.4)
bottleneck	साखळीतला सगळ्यात अरुंद दुवा (7.6)
cascade	हळा भाग -> retries -> कोसळत जाणं (7.6)
CDN	जगभरच्या godowns मधल्या प्रती (7.7)
circuit breaker	आजारी भागाला श्वास: विचारणं थांबवणं (7.6)
cloud / region	भाड्याच्या machines / त्यांचं शहर-घर (7.5)
compute	CPU-वेळेचं भाडं; bill ची ओळ 1 (7.8)
egress	बाहेर जाणाऱ्या data ची जकात; ओळ 3, कपटी (7.8)
elastic	गरजेप्रमाणे ताणणारं-आकसणारं भाडं (7.5)
failover	मुख्य मेला की प्रत आपोआप पुढे (7.2)
frontend/backend	दिसणारा भाग / kitchen; विश्वास फक्त kitchen वर (7.3)
full-stack	दोन्ही बाजू करणारा developer (7.3)
health check	"जिवंत आहेस?" ची नियमित विचारणा (7.2)
load balancer	दारावरचा वाटप करणारा (7.1)
load test	नकली गर्दीने आधीच सराव (7.6)
lock-in	एका मालकात खोल रुतणं; घर बदलणं महाग (7.5)
metrics	प्रत्येक भागावरचे मीटर (7.6)
network effect	प्रत्येक नवा user जुन्यांची किंमत वाढवतो (8.5)
nines / SLA	उपलब्धतेची मोजपट्टी / तिचं लिखित वचन (7.2)
postmortem	तुटल्यावर व्यवस्थेचं (blameless) विच्छेदन (8.4)
rate limit	दार आवळणं: प्रति-user मर्यादा (7.6)
rolling deploy	एक-एक counter बदलणं, दुकान चालू ठेवून (7.2)
spec	नेमकं-काय-हवं ची सात-प्रश्नी मागणी (8.3)
stateless	server रिकामा; state सामायिक जागी (7.1)
switching cost	सोडून जाण्याची वेदना = खिळे (8.5)
unit economics	एका user मागचा खर्च vs कमाई (7.8)
vertical/horizontal	उभं वाढणं / आडवं वाढणं (7.1)

पूर्ण नकाशा: एक पान, सगळं पुस्तक

(हे पान भिंतीवर लावा. प्रत्येक ओळीमागे एक chapter आहे; अडलात की तिथे परत.)

पैशाचं जग + machine (Parts 1-2):

LEVEL 4 AI / 3 TECH / 2 BUSINESS / 1 गरज: पैसा फक्त
 Level 1 वर घुसतो, वर चढतो (0.1, 0.3)
 कमाई = size x scarcity x leverage; तास नाहीत (0.2)
 लपलेला थर = महाग थर (abstraction, 0.4)
 transistor -> gates -> CPU: numbers वर आधीच लिहिलेली
 recipe, billion पावलं/second (1.1-1.5)
 recipe = numbers => एक machine, सगळी कामं (1.6)
 1 byte = अक्षर; text:photo:video = 1:हजार:लाख (2.1, 2.2)
 RAM = मेज (विसराळू) / storage = गोदाम (पक्कं) (2.5)
 भाषांतर-शिडी: machine code -> languages -> AI (3.1-3.4)
 OS = मालक: पाळ्या, भिंती, दरबान; platform-जकात (3.5)
 फुकट: दरवाजा/freemium/तुम्ही-माल/open source (3.7)
 bugs = जोड्यांचा स्फोट (4.1); tests = तपासणीचा code (4.2)
 Git = रोजकीर्द (4.3); software = code + माहोल (4.4)
 estimate चुकणारच: छोटे चालते तुकडे, MVP (4.5)

जाळ + आठवण (Parts 3-4):

client -> (n-squared -> चौक/protocol!) -> server (5.1)
 IP + router-watchman (5.2); packets, TCP/UDP (5.3)
 internet = नियम, वस्तू नाही; UPI = protocol (5.4)
 DNS = नाव->number ची शिडी + cache (5.5)
 encryption: दोन किल्ल्या; https = रस्ता सुरक्षित,
 माणूस नाही (5.6); 99% data समुद्री काच (5.7)
 standard बनवणारा बाजार आखतो: DPI (5.8)
 database = दरबान (6.1); index = शब्दकोश-युक्ती (6.2)
 आकार: खण/पिशवी/खुंटी (6.3)
 मोजकी वस्तू + अनेक हात = lock + transaction (6.4)
 आत्ता vs रांगेत (6.5); cache: दाखवणं शिळं चालतं,
 बदलणं ताजंच (6.6)
 backup(काळ) != replication(जिवंत); 3-2-1;
 न तपासलेला backup = नाहीच (6.7)

जगाला serve + जोडणी (Part 5):

आडवं वाढा: काम वाटा, आठवण एकत्र (7.1)
मरणं गृहीत धरा: प्रती + failover; nines (7.2)
विश्वास फक्त kitchen वर (7.3); API = menu card (7.4)
cloud = भाड्याचं; लवचीकपणा हेच मोल (7.5)
bottleneck / रांग / दार आवळा; cascade (7.6)
second रस्त्यावर जातो: CDN + फेरे + वजन (7.7)
bill = compute + storage + egress (7.8)
जिकणं = जाळं + दरवाजा + खिळे + वेळ + वाटप (8.5)

कुठल्याही धंद्याचा X-ray (7.9):

1. रोज पुन्हा-पुन्हा काय होतं? (recipe-संधी)
2. वही कुठे, कागदी की digital? (database)
3. मोजकी वस्तू कुठली? (transaction)
4. वाट कुठे बघावी लागते? (queue)
5. पैसा कुठून कुठे वाहतो? (4 levels)

MASTER GLOSSARY: पाचही parts, एका जागी

(शब्द -> एक ओळ -> chapter. Part-निहाय याद्या त्या-त्या part च्या शेवटी आहेत; ही एकत्रित, वारंवार लागणाऱ्यांची.)

abstraction	आतली गुंतागुंत झाकून सोपं handle देणं (0.4)
API	programs चं ठरलेलं प्रश्नोत्तर-menu (7.4)
backend	kitchen: नियम, हिशोब, खरी तपासणी (7.3)
backup / restore	काळ-प्रत / परत आणणं (न तपासलेली = नाहीच) (6.7)
bit / byte	switch ची स्थिती / 8 bits = ~1 अक्षर (1.4)
bottleneck	साखळीतला अरुंद दुवा (7.6)
bug / edge case	लिहिलं-हवं ची फट / कोपऱ्यातली परिस्थिती (4.1)
cache	महाग उत्तर जवळ; किंमत: शिळेपणा (6.6)
CDN	जगभर प्रती: अंतर मारणं (7.7)
client / server	मागणारी / देणारी machine (5.1)
cloud	भाड्याच्या machines; लवचीकपणा हेच मोल (7.5)
compiler/interpreter	भाषांतर आधी / सोबत-सोबत (3.3)
CPU	recipe चालवणारा; fetch-decode-execute (1.3, 2.4)
database	data चा दरबान-server (6.1)
DNS / domain	नाव->number शिडी / website चं नाव (5.5)
deploy / DevOps	code खऱ्या जगात नेणं / त्याची व्यवस्था (4.4)
egress	बाहेर जाणाऱ्या data ची जकात (7.8)
encryption	गणिती लिफाफा; public + private किल्ली (5.6)
environment	code भोवतीचा माहोल (4.4)
failover	प्रत आपोआप पुढे येणं (7.2)
frontend	दिसणारा भाग; विश्वास ठेवायचा नाही (7.3)
Git / commit	बदलांची रोजकीर्द / एक नोंद-photo (4.3)
HTTP / HTTPS	web चे नियम / सुरक्षित रूप (5.4, 5.6)
index / query	लावलेली यादी / विचारलेला प्रश्न (6.2)
IP address	machine चा number-पत्ता (5.2)
latency	फेऱ्याचा वेळ; bandwidth पेक्षा वेगळा (2.3, 5.3)
leverage	काम एकदा, फायदा पुन्हा-पुन्हा (0.2)

load balancer	दारावरचा वाटप करणारा (7.1)
lock / race	कुंडी / नशिबावरची शर्यत (6.4)
loop / variable	पुन्हा-करा / नावाचा खण (3.2)
MVP	पहिलं छोटं चालतं रूप (4.5)
network effect	नवा user जुन्यांची किंमत वाढवतो (8.5)
nines / SLA	उपलब्धतेची मोजपट्टी / लिखित वचन (7.2)
n-squared	जोड्या वर्गाने वाढतात; protocol ने मरतो (5.1)
open source	खुला code; सामायिक हत्यार (3.7)
OS / platform	machine चा मालक / दरवाजा + जकात (3.5)
packet / TCP/UDP	message चा तुकडा / बरोबरपणा vs वेग (5.3)
pixel / sample	photo चा ठिपका / आवाजाचा क्षण (2.2)
process / state	जिवंत copy / तिची आत्ताची अवस्था (3.6)
program / code	machine साठी लिहिलेली recipe (1.3, 3.2)
protocol/standard	आधीच ठरलेले नियम / सर्वमान्य रूप (5.4, 5.8)
queue / async	कामांची रांग / नोंदवा-आत्ता, करा-नंतर (6.5)
RAM / storage	मेज (विसराळू) / गोदाम (पक्कं) (2.5)
rate limit	दार आवळणं (7.6)
replication	जिवंत प्रत; चूकही copy करते (6.7)
rollback	अर्धा गट्टा उलटा फिरवणं (6.4)
scarcity	किती कमी लोकांना जमतं (0.2)
spec	सात-प्रश्नी नेमकी मागणी (8.3)
SQL / table	tables ची भाषा / खण-आकार (6.3)
stack	थरांचा मनोरा: कोण कोणावर (0.3, 3.4)
stateless	server रिकामा; state सामायिक जागी (7.1)
transaction	सगळं-किंवा-काहीच-नाही गट्टा (6.4)
transistor	विजेने दाबला जाणारा switch (1.1)
unit economics	user-मागचा खर्च vs कमाई (7.8)
universal machine	कुठलीही recipe चालवणारी एकच machine (1.6)
version control	बदलांच्या इतिहासाची व्यवस्था (4.3)

पुस्तक संपलं. Machine आता जादू नाही. भेटू Book 2 मध्ये: The Thinking Machine.